NUMBER THEORY

Excellent Frobenius Lifts and Supercongruences

The three Beukers–Vlasenko families
in all dimensions at every odd prime

Complete collected edition

Prepared for Keio
9 November 2026

Abstract. We assemble the complete arguments for excellent-Frobenius-lift truncation supercongruences for the hypercubic, simplicial, and hyperoctahedral families in every dimension and at every odd prime. For every positive cutoff K divisible by p, the normalized truncation quotient descends modulo $p^{2v_p(K)}$, coefficientwise in the full formal power-series ring. The prescribed multipliers of Beukers–Vlasenko, Conjecture 7.5, are included, together with every positive normalized multiplier.

The proof combines a directly constructed compatible orbit module, positive-vertex interpolation, balanced factorial blocks, and support-weighted symmetric Bessel reduction. At the remaining prime 3, an integral two-variable cutoff kernel gives a quadratic bound in the ordinary dimension-polynomial ring. A universal weak descent loses one factor of 3, and scalar residue dimensions recover it at every integer dimension. A separate saturation argument transfers the prescribed congruence to the standard analytic coefficient ring at the primes covered by the published excellent-lift theorem.

The volume retains the earlier independent proofs, modular trace extensions, the elliptic-zero families $27t$, $256t$, and $108t$, the all-level Zagier D arguments, negative-vertex and differential methods, and sharpness results. Exact finite certificates through level five and their complete verification sources accompany the algebraic proofs. The all-level results do not rely on extrapolation from finite calculations. The scope of numerical specialization and the remaining extension problems is stated explicitly.

Research manuscript; internal checking completed, external review pending.
MSC 2020: 11F33, 11B65, 14F30, 33C20.

Contents

1 Global statement, proof dependencies, and exact scope 1

I Positive vertex interpolation: the uniform three-family theorem 6

2 The target and the resulting scope 6

3 Positive vertex interpolation: complete hypercubic and simplicial congruences 7

4 A matrix block argument for the hyperoctahedral family 11

5 Verification, sharpness, and scope 17

II All odd primes for cubes and simplices; the common formal input 19

6 Scope and results 19

7 The excellent Cartier input without group averaging 20

8 All odd primes for cubes and simplices 21

9 The octahedral first level at every odd prime 24

10 Three exact whole-series certificates 26

11 A sharp obstruction for the uncorrected period functional 29

12 The interpolation obstruction and its subsequent resolution 31

13 Verification and relation to the earlier manuscript 31

III Hyperoctahedral descent in arbitrary dimension 32

14 Arbitrary-dimensional descent and independent finite-level results 32

15 The formal excellent Cartier input 33

16 Positive vertices and the exact factorial block 34

17 Support-weighted symmetric reduction in arbitrary dimension 37

18 Mixed-precision descent at the prime three 39

19 The first level 41

20 Finite polynomial certificates for entire series 43

21 Newton certificates uniform in the octahedral dimension 45

22 The fourth level by compact rational Hensel certificates **49**

23 Limits of the auxiliary rows and their resolution **51**

IV All-level three-adic descent **53**

24 Statement and proof architecture **53**

25 The formal excellent Cartier input **54**

26 Positive vertices and exact tails **56**

27 A two-variable kernel and quadratic cutoff divisibility **56**

28 Universal weak descent in the dimension **60**

29 Completion of the formal-series congruence **65**

30 An independent fifth-level certificate in every dimension **66**

31 Verification and exact scope **72**

V Modular Hecke traces and stronger congruences **73**

32 Modular trace methods and their conclusions **73**

33 Modular families of cusp type **77**

34 The Hecke trace **79**

35 Proof of the Main Theorem **80**

36 Proofs of the corollaries **81**

37 Families with an elliptic zero **82**

38 Zagier's case D **88**

39 Limits of the modular method and an elliptic extension **91**

40 Computational verification **92**

VI Alternative elliptic and cusp proofs; exact sharpness **93**

41 Statements, normalization, and scope **93**

42 Elliptic zeros: a rational trace estimate **94**

43 Zagier D at primes congruent to ± 1 modulo five **99**

44 Exact coefficient obstructions to larger exponents 104

45 Relation to the cited literature 108

46 Exact verification and proof dependencies 109

VII Two-cusp descent and complementary Cartier arguments 110

47 Scope and exact statements 110

48 The complete Zagier D congruence 111

49 The first level in every dimension 116

50 A canonical negative-power basis in all three families 117

51 The three-dimensional hyperoctahedral family: a weighted Cartier reduction 119

52 Exact verification and the role of the auxiliary arguments 123

A An independent coefficientwise certificate in dimension four 125

B The original modular verification program 128

C Recorded output of the modular verification program 142

D Complete verification programs 146

E Complete exact certificate program 182

F Complete polynomial arithmetic and individual-certificate source 185

G Complete dimension-uniform verifier 187

H Exact modular input matrices and model-only mode 190

I Compact Hensel roots and complete target certificates 193

J Complete fifth-level certificate source and independent evaluator 195

K Source-to-section preservation map 204

1 Global statement, proof dependencies, and exact scope

This complete edition combines the original four-part manuscript with three subsequent supplements on small primes and hyperoctahedral descent. The resulting formal-series theorem covers the three standard Beukers–Vlasenko families in every dimension at every odd prime. Earlier proofs with more restrictive prime ranges are retained as independent arguments; their local hypotheses are not restrictions on the global theorem below. The modular extensions and their separate prime conditions are retained as well.

1.1 Families, coordinates, and the global theorem

For an integer $n \geq 1$, let g be one of the Laurent polynomials

family	$g(\mathbf{x})$	h
hypercubic	$\prod_{i=1}^{n}(x_i + x_i^{-1})$	2
simplicial	$x_1 + \cdots + x_n + (x_1 \cdots x_n)^{-1}$	$n+1$
hyperoctahedral	$\sum_{i=1}^{n}(x_i + x_i^{-1})$	2.

Write

$$\mathcal{F}(z) = \mathrm{CT}\,\frac{1}{1-zg} = \sum_{j\geq 0} \mathrm{CT}(g^j) z^j = F(u), \qquad u = z^h, \qquad F(u) = \sum_{j\geq 0} a_j u^j,$$

and define the ordinary truncations

$$\mathcal{F}_L(z) = \sum_{0\leq j<L} \mathrm{CT}(g^j) z^j, \qquad F_K(u) = \sum_{0\leq j<K} a_j u^j.$$

The formal excellent lift is

$$\tau(z) = q_0^{-1}(q_0(z)^p), \qquad q_0(z) = z\exp\big(G_0(z)/\mathcal{F}(z)\big),$$

where G_0 is the vertex logarithmic companion of [7, Proposition 9.1]. The orbit-module construction in Proposition 7.1 verifies its integrality and the excellent Cartier relation for every odd prime. Its normalized form is $\sigma(u) = \tau(z)^h$. This is a well-defined member of $u^p(1 + p\mathbb{Z}_p[[u]])$, including the simplicial case $p \mid h$. Every period or truncation series used as a formal denominator has constant term one.

Theorem 1.1 (Global formal truncation congruence)**.** *For each of the three families, every dimension $n \geq 1$, every odd prime p, and every positive integer K divisible by p,*

$$\frac{F_K(u)}{F(u)} \equiv \frac{F_{K/p}(\sigma(u))}{F(\sigma(u))} \pmod{p^{2v_p(K)}\mathbb{Z}_p[[u]]}. \tag{1.1}$$

Consequently, for every pair of positive integers s, m,

$$\begin{aligned} E_{p,s,m}(u) &:= F(u)F_{mp^{s-1}}(\sigma(u)) - F(\sigma(u))F_{mp^s}(u) \\ &\in p^{2s+2v_p(m)}\mathbb{Z}_p[[u]], \end{aligned} \tag{1.2}$$

and in particular

$$\frac{F(u)}{F(\sigma(u))} \equiv \frac{F_{mp^s}(u)}{F_{mp^{s-1}}(\sigma(u))} \pmod{p^{2s}\mathbb{Z}_p[[u]]}. \tag{1.3}$$

All coefficients of these infinite formal series are included.

Proof. The hypercubic and simplicial cases, including $p \mid n+1$ in the latter, are Theorem 6.1(1). For the hyperoctahedral family in dimension $n \geq 3$, Theorem 14.1 gives (1.1) at every prime $p \geq 5$, and Theorem 24.1 gives it at $p = 3$. The dimensions one and two reduce to the corresponding scalar hypercubic positive-vertex matrices, as recorded in those parts. Thus these cases exhaust all dimensions and all odd primes. Substitute $K = mp^s$ into (1.1) and multiply by the formal units $F(u)$ and $F(\sigma(u))$ to obtain (1.2). Division by the appropriate formal units gives (1.3). □

The stronger modulus in (1.2) comes from the single cutoff $K = mp^s$; no assumption $p \nmid m$ is made. The relation

$$\mathcal{F}_{hK}(z) = F_K(z^h) \tag{1.4}$$

shows exactly how this statement meets the original conjecture.

Corollary 1.2 (The prescribed Beukers–Vlasenko truncations)**.** *For every dimension and every odd prime, the formal congruence*

$$\frac{\mathcal{F}(z)}{\mathcal{F}(\tau(z))} \equiv \frac{\mathcal{F}_{hp^s}(z)}{\mathcal{F}_{hp^{s-1}}(\tau(z))} \pmod{p^{2s}\mathbb{Z}_p[[z]]} \qquad (s \geq 1)$$

holds with $h = 2$ for the hypercubic and hyperoctahedral families and $h = n+1$ for the simplicial family. These are the multipliers specified in [7, Conjecture 7.5].

Proof. Use $m = 1$ in Theorem 1.1, substitute $u = z^h$, and apply (1.4) together with $\sigma(u) = \tau(z)^h$. □

In particular, the formerly missing hyperoctahedral sector $p = 3$, $n \geq 4$, $3 \nmid n$, $s \geq 5$ is covered. The theorem is stronger than its $m = 1$ specialization, but the expression "all multipliers" throughout the standard-family arguments means *all normalized multipliers*: the raw lengths are hmp^s. No assertion for every arbitrary raw multiplier is inferred.

1.2 Proof architecture and retained extensions

Family or sector	Prime range	Conclusion and proof
Hypercubic, all $n \geq 1$	Every odd p	All s, m and all coefficients; Theorem 6.1(1).
Simplicial, all $n \geq 1$	Every odd p, including $p \mid n+1$	All s, m and all coefficients; Theorem 6.1(1).
Hyperoctahedral, all $n \geq 1$	Every $p \geq 5$	All s, m and all coefficients; Theorem 14.1 and the scalar reductions.
Hyperoctahedral, all $n \geq 1$	$p = 3$	All s, m and all coefficients; Theorem 24.1 and the scalar reductions.
Thirteen cusp-type modular families	$p \nmid N$, stated Hecke condition	Exponent $(w+1)s$, $w \in \{1, 2\}$; Theorem 32.1.
Elliptic-zero $27t, 256t, 108t$	$p \geq 5$	Exponents $2s, 3s, 3s$, respectively; Theorems 32.2 and 42.3.
Zagier D	Every $p \neq 5$	Exponent $2s$ for all s, m, including $p = 2, 3$; Theorems 32.3 and 48.1.

The original positive-vertex part gives the large-prime proof using the published excellent Cartier theorem. The small-prime supplement constructs a compatible orbit module directly, so no division by a symmetry-group order is used. The next hyperoctahedral supplement controls factorial denominators by the number of differentiated variables and proves descent for every $p \geq 5$. The final supplement establishes an integral two-variable Bessel kernel, a universal coefficientwise descent with one factor of 3 lost, and a dimension residue argument that recovers this factor at every integer dimension. These are algebraic arguments for all indices. The finite certificates at

specified levels are independent checks and alternative proofs of those individual congruences, not premises of an extrapolation to arbitrary levels.

The original modular, elliptic, two-cusp, negative-vertex, differential, and Domb arguments remain included. A universal replacement of $2s$ by $3s$ is false: the exact sharpness statement for three-dimensional hyperoctahedra at $p = 5$ is Theorem 44.2. The stronger $3s$ conclusions in the table have their own hypotheses. The broader elliptic-zero Conjecture 39.1, beyond the proved list, remains an extension problem.

1.3 Transfer to the original analytic coefficient ring

The all-odd-prime theorem is formulated in $\mathbb{Z}_p[[z]]$. For primes satisfying the hypotheses of [7, Theorem 7.3], one can also state the prescribed-truncation congruence in the standard analytic coefficient ring. The following elementary argument makes this transfer explicit; it does not assert that the period $\mathcal{F}$ itself belongs to that ring.

Lemma 1.3 (Saturation of the standard coefficient ring)**.** *Let $P \in \mathbb{Z}_p[z]$ with $P(0) \in \mathbb{Z}_p^\times$, and let R_P be the p-adic completion of $\mathbb{Z}_p[z, 1/P(z)]$. Expansion at zero embeds R_P into $\mathbb{Z}_p[[z]]$, and, for every integer $k \geq 1$,*

$$R_P \cap p^k \mathbb{Z}_p[[z]] = p^k R_P.$$

Proof. For every k, the polynomial P is a unit in $(\mathbb{Z}/p^k\mathbb{Z})[[z]]$. The expansion map

$$(\mathbb{Z}/p^k\mathbb{Z})[z, 1/P] \longrightarrow (\mathbb{Z}/p^k\mathbb{Z})[[z]]$$

is injective: if a fraction has zero expansion, multiplication by its denominator shows that its polynomial numerator is zero modulo p^k. The quotient of R_P by $p^k R_P$ is this localized polynomial ring, since $\mathbb{Z}_p[z, 1/P]$ is Noetherian and its p-adic completion has the same quotients modulo p^k. The compatible expansion maps therefore embed the inverse limit R_P into $\mathbb{Z}_p[[z]]$. Their injectivity modulo p^k gives the stated intersection equality. □

Lemma 1.4 (The prescribed truncations are analytic units)**.** *For each of the three standard families, put $c_j = \mathrm{CT}(g^j)$ and $A(z) = \mathcal{F}_p(z)$. For every prime p and every $r \geq 0$,*

$$\mathcal{F}_{hp^r}(z) \equiv \prod_{i=0}^{r-1} A(z^{p^i}) \pmod{p}, \tag{1.5}$$

where an empty product is one. Consequently these truncations and their compositions with any Frobenius lift are units in a p-adically complete coefficient ring in which A is a unit.

Proof. The Newton polytope Δ is reflexive, and zero is its only interior lattice point. If $0 \leq a < p$ and $b \geq 0$, the congruence $g^{a+pb} \equiv g^a g(\mathbf{x}^p)^b \pmod{p}$ gives

$$c_{a+pb} \equiv c_a c_b \pmod{p}.$$

Indeed an exponent of g^a that can pair with an exponent of $g(\mathbf{x}^p)^b$ is divisible by p. Dividing it by p places it in $(a/p)\Delta \subset \Delta^\circ$, so it must be zero. Iterating this digit relation and summing the indices less than hp^r yields

$$\mathcal{F}_{hp^r}(z) \equiv \mathcal{F}_h(z^{p^r}) \prod_{i=0}^{r-1} A(z^{p^i}) \pmod{p}.$$

In these families a positive constant-term degree is divisible by h, hence $\mathcal{F}_h = 1$. This proves (1.5). Modulo p each factor is $A(z)^{p^i}$; it is therefore a unit whenever A is a unit. Units modulo p lift to units in a p-adically complete ring. A ring endomorphism sends a unit to a unit, which gives the composition assertion. □

Corollary 1.5 (Congruence in the BV3 analytic ring)**.** *Assume that p and the symmetry data satisfy the hypotheses of [7, Theorem 7.3]. Compute the Hasse–Witt polynomials with the initial lift $z \mapsto z^p$, and set*

$$R_0 = \mathbb{Z}_p[z, 1/\widehat{hw^{(1)}(z)}, 1/hw^{(2)}(z)], \qquad \lambda(z) = \frac{\mathcal{F}(z)}{\mathcal{F}(\tau(z))}.$$

For each $s \geq 1$, the prescribed-truncation congruence holds in R_0:

$$\lambda(z) - \frac{\mathcal{F}_{hp^s}(z)}{\mathcal{F}_{hp^{s-1}}(\tau(z))} \in p^{2s} R_0.$$

Proof. The cited theorem and [7, Proposition 7.7 and Theorem 7.15] give $\tau, \lambda \in R_0$ and a Frobenius endomorphism of R_0. The polynomials $hw^{(1)}, hw^{(2)}$ have unit constant terms. Also $A = \mathcal{F}_p \equiv hw^{(1)} \pmod{p}$, so A is a unit of R_0. Lemma 1.4 shows that $\mathcal{F}_{hp^{s-1}}(\tau(z))$ is a unit of R_0. Corollary 1.2, after multiplication by this denominator, says that the element

$$\lambda(z)\mathcal{F}_{hp^{s-1}}(\tau(z)) - \mathcal{F}_{hp^s}(z) \in R_0$$

belongs to $p^{2s}\mathbb{Z}_p[[z]]$. Apply Lemma 1.3 with $P = hw^{(1)}hw^{(2)}$, and then divide by the displayed unit. □

Thus the original analytic-ring interpretation in the published admissible-prime setting follows from the formal theorem. This does not by itself establish an analogous analytic realization at every additional small prime admitted by the formal orbit-module construction. A numerical specialization also requires an evaluation map on the relevant ring. If such a continuous map is given, the ring congruence specializes; identifying a fixed point of the lift and identifying the resulting scalar with a geometric unit root are separate assertions.

1.4 Provenance, verification, and limits

This edition retains the mathematical arguments of the four original manuscripts and incorporates the three subsequent supplements. The global theorem is their explicit combination. The analytic-ring transfer above is recorded here as a short additional argument to clarify the relationship with the original BV3 coefficient ring. Published Cartier, connection, and coordinate results are cited at their points of use; their complete published proofs are not represented as new results of this manuscript.

Three kinds of evidence are distinguished: algebraic proofs for all indices; bounded exact-arithmetic checks, with any historical floating-point checks expressly identified; and finite polynomial certificates whose proved degree and denominator properties imply an entire formal-series congruence. Finite samples alone are never used to infer the global theorem. The manuscript and accompanying calculations have been checked internally; external peer review and proof-assistant formalization have not been completed.

The scope of the global theorem excludes the prime 2, arbitrary raw truncation multipliers, arbitrary Laurent-polynomial families, and unrestricted numerical specialization. These exclusions do not remove any of the prescribed formal truncations of the three-family odd-prime statement. Separately, the modular extension at Zagier D does include $p = 2$, under its own theorem. No claim is made that the open elliptic-zero generalization or every related special-value problem is resolved.

The single `.tex` source contains the mathematical text, references, and embedded program listings of this edition. It requires no companion chapter, bibliography, image, or data file to compile. Standard installed LaTeX packages are required. Computational certificate data used for independent checks are not hypotheses of the algebraic all-level proofs.

1.5 Reading the complete edition

Parts I–IV contain the full three-family argument, including the original restricted-prime proof and the subsequent extensions. Parts V–VII retain the complementary modular and geometric arguments. Repeated formal-input proofs are kept with their local notation so that no proof is replaced by an editorial synopsis. The source-to-section map in Appendix K identifies every retained source section. Earlier code comments and recorded outputs are historical records of those particular computations; current theorem scope is always the scope stated in this introduction.

The algebraic global proof is independent of the optional certificate data. All certificate algorithms needed for reproduction are printed in the appendices, including the new fifth-level generator and an independent evaluator. Large coefficient arrays are reproducible outputs of those programs. They are not external inputs needed to read, compile, or prove the global theorem.

Compile this one source with `latexmk -pdf`, or run `pdflatex` until references stabilize. No separate bibliography, chapter, figure, or data file is needed for compilation.

Part I
Positive vertex interpolation: the uniform three-family theorem

All dimensions, all levels, all multipliers, and all coefficients in the standard full-symmetry prime ranges.

2 The target and the resulting scope

This part proves the uniform ordinary-truncation theorem for the three standard families, including the three-dimensional hyperoctahedral family and every higher dimension. The modular and Zagier D arguments in the other parts are independent extensions. The common input is the excellent Cartier theorem of Beukers and Vlasenko [7, Theorem 7.3], together with its coefficient interpretation immediately after [7, Corollary 3.4].

Let g be one of the following Laurent polynomials in n variables, and put $f = 1 - zg$:

family	$g(\mathbf{x})$	h
hypercubic	$\displaystyle\prod_{i=1}^{n}(x_i + x_i^{-1})$	2
simplicial	$x_1 + \cdots + x_n + (x_1 \cdots x_n)^{-1}$	$n+1$
hyperoctahedral	$\displaystyle\sum_{i=1}^{n}(x_i + x_i^{-1})$	2

Normalize $u = z^h$ and write

$$\mathcal{F}(z) = \mathrm{CT}\,\frac{1}{1-zg} = F(u), \qquad F(u) = \sum_{j\ge 0} a_j u^j, \qquad F_K(u) = \sum_{0\le j<K} a_j u^j.$$

Let $z \mapsto \tau(z)$ be the actual excellent lift of [7, Theorem 7.3]. Its normalized form is $\sigma(u) = \tau(z)^h$. The canonical coordinate formulas in [7, Definition 7.14 and Proposition 9.1] show that this is a series in u, with $\sigma/u^p \in 1 + p\mathbb{Z}_p[[u]]$. All period and truncation series appearing in denominators have constant term one.

Define the cleared defect

$$E_{p,s,m}(u) = F(u)F_{mp^{s-1}}(\sigma(u)) - F(\sigma(u))F_{mp^s}(u). \tag{2.1}$$

The assertion $E_{p,s,m} \in p^{2s}\mathbb{Z}_p[[u]]$ means that *every* coefficient is divisible by p^{2s}, with no bound on its index.

Theorem 2.1 (The three standard families)**.** *Let $n \ge 1$, and use the standard full symmetry groups of [7, Section 7]. Assume that p is an odd prime and*

$p > n$ *for the hypercubic and hyperoctahedral families,* $p > n+1$ *for the simplicial family.*

Then, for every positive integer K divisible by p,

$$\frac{F_K(u)}{F(u)} \equiv \frac{F_{K/p}(\sigma(u))}{F(\sigma(u))} \pmod{p^{2v_p(K)}\mathbb{Z}_p[[u]]}. \tag{2.2}$$

In particular, for all $m, s \ge 1$,

$$E_{p,s,m} \in p^{2s}\mathbb{Z}_p[[u]], \qquad \frac{F(u)}{F(\sigma(u))} \equiv \frac{F_{mp^s}(u)}{F_{mp^{s-1}}(\sigma(u))} \pmod{p^{2s}}.$$

In the original variable, the result covers all truncation lengths hmp^s. The prescribed lengths $2p^s$ for cubes and octahedra and $(n+1)p^s$ for simplices in Conjecture 7.5 are the case $m = 1$.

The cubic and simplicial statements are proved in Theorem 3.1; the octahedral statement in dimension $n \geq 3$ is Theorem 4.1. The low-dimensional octahedral reduction is recorded in that section. The prime conditions of this original proof are retained. Theorems 6.1, 14.1, and 24.1 extend the conclusion to every odd prime by additional arguments, combined in Theorem 1.1.

Family and dimension	Prime range	Conclusion
Hypercubic, $n \geq 4$	$p > n$	All s, m, every coefficient
Simplicial, $n \geq 4$	$p > n+1$	All s, m, every coefficient
Hyperoctahedral, $n = 3$	$p \geq 5$	All s, m, every coefficient
Hyperoctahedral, $n \geq 4$	$p > n$	All s, m, every coefficient

2.1 How the excellent Cartier relation reaches ordinary truncations

For each family, a positive vertex coefficient of $1/f$ has the form

$$\Omega_N(u) = \sum_{j \geq N} B(j,N)u^j, \quad B(j,N) \in \mathbb{Z}, \quad B(j,0) = a_j, \quad B(j,j) = 1.$$

Set $C_N = \Omega_N/F$. The excellent Cartier relation gives

$$C_{pN} - C_N \circ \sigma \in p^{2+2v_p(N)}\mathbb{Z}_p[[u]]. \tag{2.3}$$

The unit triangular matrix B defines integral coefficients $c_{K,N}$ by

$$a_j \mathbf{1}_{j \geq K} = \sum_{N=K}^{j} c_{K,N} B(j,N), \qquad 1 - \frac{F_K}{F} = \sum_{N \geq K} c_{K,N} C_N.$$

The essential new estimates are

$$v_p(c_{K,N}) \geq \max\{0, 2v_p(K) - 2v_p(N)\},$$

and, when $p \mid K$,

$$c_{K,N} \equiv \mathbf{1}_{p|N}\, c_{K/p,N/p} \pmod{p^{2v_p(K)}}.$$

These two estimates let the ordinary tail descend term by term through (2.3) at the required precision. All sums converge u-adically, so the argument covers the entire series. The next two sections prove the estimates without assuming the desired truncation congruence.

3 Positive vertex interpolation: complete hypercubic and simplicial congruences

All congruences of formal series in this section are coefficientwise. Let $n \geq 1$, and consider one of the following two families:

	$g(\mathbf{x})$	h	a_j
hypercubic	$\prod_{i=1}^{n}(x_i + x_i^{-1})$	2	$\binom{2j}{j}^n$
simplicial	$x_1 + \cdots + x_n + (x_1 \cdots x_n)^{-1}$	$n+1$	$((n+1)j)!/(j!)^{n+1}$

Put $f = 1 - zg$, $u = z^h$,

$$\mathcal{F}(z) = \mathrm{CT}(1/f) = F(u), \qquad F(u) = \sum_{j \geq 0} a_j u^j, \qquad F_K(u) = \sum_{0 \leq j < K} a_j u^j.$$

Throughout, p is an odd prime satisfying the symmetry and support-lattice hypotheses of [7, Theorem 7.3]. For the full symmetry groups specified in that paper, these conditions are $p > n$ in the cubic case and $p > n + 1$ in the simplicial case, with the additional odd-prime requirement in dimension one. In particular, $p \nmid h$.

Let $z \mapsto \tau$ be the actual excellent Frobenius lift of that theorem. The normalized lift $u \mapsto \sigma(u) = \tau^h$ is the one characterized by $q(\sigma(u)) = q(u)^p$, where $q(z^h) = q_0(z)^h$ and q_0 is the canonical coordinate of [7, Definition 7.14 and Proposition 9.1]. No coordinate change different from this lift is introduced.

Theorem 3.1 (All levels for hypercubes and simplices)**.** *For either of the above families, and every positive integer K divisible by p,*

$$\frac{F_K(u)}{F(u)} \equiv \frac{F_{K/p}(\sigma(u))}{F(\sigma(u))} \pmod{p^{2v_p(K)}\mathbb{Z}_p[[u]]}. \tag{3.1}$$

Hence, for all integers $m, s \geq 1$,

$$F(u)F_{mp^{s-1}}(\sigma(u)) - F(\sigma(u))F_{mp^s}(u) \in p^{2s}\mathbb{Z}_p[[u]].$$

Equivalently,

$$\frac{F(u)}{F(\sigma(u))} \equiv \frac{F_{mp^s}(u)}{F_{mp^{s-1}}(\sigma(u))} \pmod{p^{2s}}.$$

In the original variable, this proves Conjecture 7.5 for every hypercubic and every simplicial dimension, at all primes in its stated symmetry range. In fact all original truncation lengths hmp^s, $m, s \geq 1$, are covered, not only hp^s.

The proof uses positive vertex coefficient series whose leading terms form an integral triangular basis. It does not use the negative-power canonical basis or any duality between opposite expansions.

3.1 A binomial and multinomial block congruence

Set $r = n + 1$ in the simplicial case and define, for $0 \leq N \leq j$,

$$B(j, N) = \begin{cases} \dbinom{2j}{j-N}^n, & \text{hypercubic,} \\ \dfrac{(rj)!}{(j+(r-1)N)!(j-N)!^{r-1}}, & \text{simplicial.} \end{cases}$$

Put $B(j, N) = 0$ for $N > j$. In both cases these are nonnegative integers with $B(j, 0) = a_j$ and $B(j, j) = 1$.

Lemma 3.2 (Balanced factorial block congruence)**.** *Let p be odd and $l = \lfloor j/p \rfloor$. Then $d_j = a_j/a_l \in \mathbb{Z}_p$, and, for every $1 \leq N \leq l$,*

$$B(j, pN) - d_j B(l, N) \in p^{2+2v_p(N)}\mathbb{Z}_p. \tag{3.2}$$

This lemma itself holds for every positive dimension and every odd prime; the larger prime exclusions in the theorem are used only for the excellent Cartier relation.

Proof. For $a \geq 0$, write

$$U(a) = \prod_{\substack{1 \leq i \leq a \\ p \nmid i}} i.$$

We shall use the exact identity

$$a! = p^{\lfloor a/p \rfloor} \lfloor a/p \rfloor! U(a).$$

Write $j = pl + b$, $0 \le b < p$, and put $H = pN$ and $v = v_p(H) = 1 + v_p(N)$. For the unit index set

$$I = \{i : j - H < i \le j,\ p \nmid i\},$$

one has

$$\sum_{i \in I} \frac{1}{i} \in p^v \mathbb{Z}_p. \tag{3.3}$$

Indeed, the interval has length H, a multiple of p^v. Its residues modulo p^v consist of H/p^v complete copies of the residue classes. Among the unit classes, inversion permutes the residues, and opposite residues cancel because p is odd.

For the cubic case, first omit the power n and write $b(j, N) = \binom{2j}{j-N}$. With $\epsilon = \lfloor 2b/p \rfloor \in \{0, 1\}$, the factorial identity gives

$$d_j^{(1)} = \frac{b(j,0)}{b(l,0)} = p^\epsilon (2l+1)^\epsilon \frac{U(2j)}{U(j)^2} \in \mathbb{Z}_p,$$

and

$$b(j, H) = d_j^{(1)} b(l, N) S, \qquad S = \frac{U(j)^2}{U(j-H)U(j+H)} = \prod_{i \in I} \left(1 + \frac{H}{i}\right)^{-1}.$$

Since each H/i is in $p^v \mathbb{Z}_p$,

$$S \equiv 1 - H \sum_{i \in I} \frac{1}{i} \equiv 1 \pmod{p^{2v}}.$$

Taking the n-th power proves (3.2).

For the simplicial case set $\epsilon = \lfloor rb/p \rfloor$. Then

$$d_j = \frac{a_j}{a_l} = p^\epsilon \frac{(rl+\epsilon)!}{(rl)!} \frac{U(rj)}{U(j)^r} \in \mathbb{Z}_p.$$

The balanced shifts in the factorial denominator give the exact identity

$$B(j, H) = d_j B(l, N) S, \qquad S = \frac{U(j)^r}{U(j + (r-1)H)U(j-H)^{r-1}}.$$

Decomposing the long upper interval into $r - 1$ blocks of length H yields

$$S = \prod_{i \in I} \prod_{k=1}^{r-1} \left(1 + \frac{kH}{i}\right)^{-1}.$$

Therefore

$$S \equiv 1 - H \frac{r(r-1)}{2} \sum_{i \in I} \frac{1}{i} \equiv 1 \pmod{p^{2v}},$$

again by (3.3). All prefactors multiplying $S - 1$ are p-integral, which proves the lemma. □

3.2 An integral triangular descent

The following argument is stated in the notation of the two families, but uses only integrality of B, its diagonal entries one, and Lemma 3.2. For each positive integer K, define integers $c_{K,N}$ by

$$a_j \mathbf{1}_{j \ge K} = \sum_{N=K}^{j} c_{K,N} B(j, N), \qquad j \ge 0, \tag{3.4}$$

and put $c_{K,N} = 0$ for $N < K$. Existence, uniqueness and integrality follow by successive solution in j, since $B(j, j) = 1$.

Lemma 3.3 (Weighted interpolation and descent)**.** *For every $K, N \geq 1$,*

$$v_p(c_{K,N}) \geq \max\{0,\, 2v_p(K) - 2v_p(N)\}. \tag{3.5}$$

If $p \mid K$, then

$$c_{K,N} \equiv \begin{cases} c_{K/p,N/p}, & p \mid N, \\ 0, & p \nmid N \end{cases} \pmod{p^{2v_p(K)}}. \tag{3.6}$$

Proof. Induct on $v = v_p(K)$. If $v = 0$, the weighted bound is integrality and there is no descent assertion. Suppose $v \geq 1$, set $L = K/p$, and assume the weighted bound for L. Let $l = \lfloor j/p \rfloor$. Multiplying the defining identity for L by $d_j = a_j/a_l$ gives

$$a_j \mathbf{1}_{j \geq K} = d_j a_l \mathbf{1}_{l \geq L} = \sum_{N=L}^{l} c_{L,N} d_j B(l, N).$$

By Lemma 3.2, replacing each $d_j B(l, N)$ by $B(j, pN)$ makes an error of valuation at least

$$2 + 2v_p(N) + \max\{0,\, 2(v-1) - 2v_p(N)\} \geq 2v.$$

Thus the candidate coefficient vector, equal to $c_{L,N}$ at index pN and zero at indices not divisible by p, satisfies (3.4) modulo p^{2v}. Successively solving this unit triangular system proves (3.6). For $p \nmid N$, it also gives the weighted bound directly. For $p \mid N$, the induction hypothesis applied to $c_{L,N/p}$ gives the bound $\max\{0, 2(v - v_p(N))\}$, and the error p^{2v} does not weaken it. □

3.3 Positive vertex coefficients and the excellent lift

For the cubic family choose the vertex $\mathbf{v} = (1, \ldots, 1)$. For the simplex choose $\mathbf{v} = (-1, \ldots, -1)$. The coefficient at $\mathbf{x}^{hN\mathbf{v}}$ in the z-adic expansion of $1/f$ is

$$\Omega_N(z) = \sum_{j \geq N} B(j, N) z^{hj}. \tag{3.7}$$

For the cube this follows by choosing $j - N$ negative steps in each of the n independent factors of g^{2j}. For the simplex, the required term of g^{rj} uses the chosen vertex $j + (r-1)N$ times and each of the other $r - 1$ vertices $j - N$ times, giving exactly the displayed multinomial coefficient.

Set

$$C_N(u) = \frac{\Omega_N(z)}{F(u)} \in u^N \mathbb{Z}_p[[u]].$$

The coefficientwise interpretation of the module of second formal derivatives, including in the positive z-adic expansion, is [7, Section 3, the paragraph following Corollary 3.4]. Read [7, Theorem 7.3] at the exponent vector $hN\mathbf{v}$. The left coefficient is $\Omega_{pN}(z)$, and the right coefficient is $\mathcal{F}(z)\Omega_N(\tau)/\mathcal{F}(\tau)$. The error is divisible by $p^2(hN)^2$ in $\mathbb{Z}_p[[z]]$. Since $p \nmid h$ and $F(0) = 1$, division by $F(u)$ gives

$$C_{pN}(u) - C_N(\sigma(u)) \in p^{2+2v_p(N)} \mathbb{Z}_p[[u]]. \tag{3.8}$$

Proof of Theorem 3.1. Summing (3.4) in j, using (3.7), and dividing by $F(u)$, gives the exact identity

$$1 - \frac{F_K(u)}{F(u)} = \sum_{N \geq K} c_{K,N} C_N(u).$$

It converges u-adically because C_N has order at least N. Put $v = v_p(K) \geq 1$ and $L = K/p$. Lemma 3.3 first gives

$$1 - \frac{F_K(u)}{F(u)} \equiv \sum_{N \geq L} c_{L,N} C_{pN}(u) \pmod{p^{2v}}.$$

By (3.8) and (3.5), each summand may be replaced by $c_{L,N}C_N(\sigma(u))$ with error divisible by p^{2v}. Indeed, the error has valuation at least

$$2 + 2v_p(N) + \max\{0, 2(v-1) - 2v_p(N)\} \geq 2v.$$

Both sums still converge: $C_{pN}(u)$ and $C_N(\sigma(u))$ have order at least pN. Consequently

$$1 - \frac{F_K(u)}{F(u)} \equiv \sum_{N \geq L} c_{L,N} C_N(\sigma(u)) = 1 - \frac{F_L(\sigma(u))}{F(\sigma(u))} \pmod{p^{2v}},$$

which is (3.1). Multiplication by the period units gives the cleared-defect and quotient formulations. Finally, $\mathcal{F}_{hM}(z) = F_M(z^h)$ proves the assertion in the original normalization. □

Scope. The proof proves an all-level statement for the ordinary positive truncations, without a restriction on the series coefficient index. It is independent of the earlier first-level and negative-basis arguments. It does not by itself prove the hyperoctahedral case: that positive vertex matrix contains sums of multinomial coefficients, and Lemma 3.2 in its present scalar form does not apply to those sums.

4 A matrix block argument for the hyperoctahedral family

Let $n \geq 3$ and let $p > n$ be prime, so in particular $p \geq 5$. Put

$$g(\mathbf{x}) = \sum_{i=1}^{n} (x_i + x_i^{-1}), \quad f = 1 - zg, \quad u = z^2,$$

and write

$$F(u) = \mathrm{CT}(1/f) = \sum_{j \geq 0} a_j u^j, \qquad F_K(u) = \sum_{j < K} a_j u^j.$$

The normalized excellent lift is denoted by σ, exactly as in the preceding section. Define the positive vertex matrix

$$B(j,N) = (2j)! \sum_{\substack{k_1 + \cdots + k_n = j \\ k_1 \geq N}} \frac{1}{(k_1 - N)!(k_1 + N)! \prod_{i=2}^{n} (k_i!)^2}, \qquad 0 \leq N \leq j. \tag{4.1}$$

Set $B(j,N) = 0$ for $N > j$. These are nonnegative integers, $B(j,0) = a_j$, and $B(j,j) = 1$. The coefficient at x_1^{2N}, with all other exponents zero, in $1/f$ is

$$\Omega_N(z) = \sum_{j \geq N} B(j,N) z^{2j}.$$

Consequently the positive vertex functions

$$C_N(u) = \Omega_N(z)/F(u) \in u^N \mathbb{Z}_p[[u]]$$

satisfy, by the excellent Cartier relation and the positive coefficient interpretation of second formal derivatives,

$$C_{pN}(u) - C_N(\sigma(u)) \in p^{2+2v_p(N)} \mathbb{Z}_p[[u]]. \tag{4.2}$$

Here one reads the coefficient at x_1^{2N} in [7, Theorem 7.3]; see also the paragraph following [7, Corollary 3.4]. Thus the remaining problem is integral interpolation of the ordinary period tail in the C_N.

Theorem 4.1 (All-level hyperoctahedral truncations)**.** *For every $n \geq 3$, every prime $p > n$, and every positive integer K divisible by p,*

$$\frac{F_K(u)}{F(u)} \equiv \frac{F_{K/p}(\sigma(u))}{F(\sigma(u))} \pmod{p^{2v_p(K)}\mathbb{Z}_p[[u]]}.$$

In particular the cleared excellent-lift defect is in $p^{2s}\mathbb{Z}_p[[u]]$ for every normalized $m, s \geq 1$. In the original variable, this includes all truncation lengths $2mp^s$, and hence the prescribed lengths in Conjecture 7.5.

Dimensions one and two. For $n = 1$, the octahedral and cubic Laurent polynomials are identical. For $n = 2$, the substitution $x_1 = y_1y_2$, $x_2 = y_1/y_2$ gives

$$x_1 + x_1^{-1} + x_2 + x_2^{-1} = (y_1 + y_1^{-1})(y_2 + y_2^{-1}).$$

Reading the coefficient at $x_1^{2N}x_2^0$ gives $B(j, N) = \binom{2j}{j-N}^2$. The positive vertex matrix is therefore exactly the cubic matrix in dimension two. Applying the scalar block proof with the octahedral excellent Cartier relation proves the same conclusion at every odd prime, including $p = 3$. Thus the low-dimensional cases introduce no additional exception to the original symmetry range.

The scalar ratio $a_j/a_{\lfloor j/p \rfloor}$ need not give the block congruence used for cubes and simplices. We construct an integral lower triangular row operator in its place.

4.1 An integral analytic unit-factorial expansion

Write $\mathbb{Z}_p\langle X_1, \ldots, X_d\rangle$ for the restricted power series ring: its coefficients tend to zero p-adically as their total degree tends to infinity. For a nonnegative integer a, put

$$U(a) = \prod_{\substack{1 \leq i \leq a \\ p \nmid i}} i, \qquad c = (p-1)!.$$

Lemma 4.2 (Unit-factorial interpolation)**.** *For every $0 \leq r < p$, there is $T_r(X) \in \mathbb{Z}_p\langle X\rangle$ such that, for all integers $\ell \geq 0$,*

$$U(p\ell + r)^{-2} = c^{-2\ell}(r!)^{-2}(1 - 2pH_r\ell + p^2T_r(\ell)), \qquad H_r = \sum_{a=1}^{r} \frac{1}{a}. \tag{4.3}$$

Proof. The unit factorial over one block has the exact paired form

$$Q(k) := \frac{U(p(k+1))}{cU(pk)} = \prod_{a=1}^{(p-1)/2} \left(1 + \frac{p^2k(k+1)}{a(p-a)}\right).$$

Thus $\log Q(X)$ is a sum, indexed by $m \geq 1$, of polynomials of degree at most $2m$ whose coefficients lie in $p^{2m-v_p(m)}\mathbb{Z}_p$. The polynomial $S_d(X)$, equal to $\sum_{0 \leq k < \ell} k^d$ at nonnegative integers $X = \ell$, has coefficient denominators dividing $\operatorname{lcm}(1, \ldots, d+1)$. Indeed, expansion in falling factorials gives

$$k^d = \sum_{a=0}^{d} \left\{ {d \atop a} \right\} k(k-1)\cdots(k-a+1),$$

and the sum of the falling factorial of degree a is the falling factorial of degree $a+1$, divided by $a+1$. Taking this discrete antiderivative term by term in $\log Q$, the m-th contribution therefore has coefficient valuation at least

$$2m - v_p(m) - \lfloor \log_p(2m+1) \rfloor \geq 2.$$

The bound tends to infinity with m, since $p \geq 5$. We obtain a restricted series $A(X) \in p^2\mathbb{Z}_p\langle X\rangle$ such that

$$A(\ell) = \sum_{k=0}^{\ell-1} \log Q(k) = \log(U(p\ell)/c^\ell).$$

Exponentiation converges in the restricted series ring and gives $\exp(-2A) \in 1 + p^2\mathbb{Z}_p\langle X\rangle$. Finally,

$$U(p\ell + r) = U(p\ell)\, r! \prod_{a=1}^{r} (1 + p\ell/a),$$

and

$$\prod_{a=1}^{r} (1 + pX/a)^{-2} \in 1 - 2pH_rX + p^2\mathbb{Z}_p\langle X\rangle.$$

Combining these identities proves the lemma. □

4.2 Symmetric Bessel operators

For an integer $N \geq 0$, put

$$I_N(x) = \sum_{k\geq N} \frac{x^k}{(k-N)!(k+N)!}, \qquad \Phi_N(x) = I_N(x)I_0(x)^{n-1}.$$

For $\theta = x\, d/dx$, one has

$$(\theta^2 - N^2)I_N = xI_N.$$

Use the factorial coefficient lattice

$$\mathcal{H} = \left\{ \sum_{j\geq 0} h_j x^j : (2j)!h_j \in \mathbb{Z}_p \right\}.$$

It is a complete normed $\mathbb{Z}_p$-algebra under the coefficient norm $\sup_j |(2j)!h_j|_p$; multiplication, θ, and multiplication by x preserve its unit ball. In particular, I_N, Φ_N, and all their θ-derivatives belong to $\mathcal{H}$.

Lemma 4.3 (Integral symmetric operator reduction)**.** *Let $P \in \mathbb{Z}_p\langle X_1, \ldots, X_n\rangle$ be symmetric in its n variables, and let $\theta_i = x_i\, \partial/\partial x_i$. There are restricted series $A_0(x), \ldots, A_n(x) \in \mathbb{Z}_p\langle x\rangle$, independent of N, such that*

$$P(\theta_1, \ldots, \theta_n)I_N(x_1) \prod_{i=2}^{n} I_0(x_i)\Big|_{x_1=\cdots=x_n=x} - \sum_{k=0}^{n} A_k(x)\theta^k\Phi_N(x) \in N^2\mathcal{H}. \tag{4.4}$$

The construction is $\mathbb{Z}_p$-linear and bounded for the coefficient norms. If

$$P = \alpha + \beta(X_1 + \cdots + X_n) + p^2R, \qquad R \in \mathbb{Z}_p\langle X_1, \ldots, X_n\rangle,$$

the error in (4.4) belongs to $p^2N^2\mathcal{H}$, and every positive-degree coefficient of every A_k is divisible by p^2. At $N = 0$, (4.4) is an exact identity.

Proof. First take a symmetric polynomial P. Using the equations $\theta_i^2 I_0 = x_i I_0$, and using $\theta_1^2 I_N = x_1 I_N + N^2 I_N$, reduce each differentiated factor to an integral polynomial combination of the factor and its first derivative, modulo N^2. All commutations $\theta_i x_i = x_i(\theta_i + 1)$ have integer

coefficients. After setting all $x_i = x$, symmetry of the reduction with the N^2 terms suppressed gives a combination, with coefficients in $\mathbb{Z}_p[x]$, of

$$E_k = \sum_{\substack{S\subseteq\{1,\dots,n\}\\|S|=k}} \prod_{i\in S} \theta_i f_i(x) \prod_{i\notin S} f_i(x), \quad f_1 = I_N, \quad f_i = I_0 \ (i>1).$$

Here $E_0 = \Phi_N$, $E_1 = \theta\Phi_N$, and

$$\theta E_k = (k+1)E_{k+1} + (n-k+1)xE_{k-1} \pmod{N^2\mathcal{H}}.$$

The correction comes only from differentiating a first derivative of I_N and is N^2 times an integral product of the indicated functions. Solving successively for $E_2, \dots, E_n$ divides only by $2, \dots, n$, which are units because $p > n$. It expresses the result in the form asserted in (4.4), with $A_k \in \mathbb{Z}_p[x]$ independent of N.

Every error lies in $N^2\mathcal{H}$: a coefficient of degree j, multiplied by $(2j)!$, is an integral multinomial coefficient times integral powers of summation indices and coefficients in $\mathbb{Z}_p$. Multiplication by x and differentiation by θ preserve this property. The reduction never decreases a coefficient's p-adic valuation, since its only denominators are products of units from $1, \dots, n$. It is therefore bounded and extends continuously from symmetric polynomials to symmetric restricted series. The output coefficients also tend to zero with degree, giving the asserted restricted A_k. For the affine symmetric polynomial $\alpha + \beta\sum X_i$, take $A_0 = \alpha$, $A_1 = \beta$, and all other $A_k = 0$; this is exact for every N. Apply the bounded linear construction to p^2R for the last assertion. The exactness at $N = 0$ is immediate from the original differential equations. □

4.3 The lower triangular row operator

Lemma 4.4 (Matrix block congruence with a boundary factor)**.** *For each integer $j \geq 0$, put $L = \lfloor j/p \rfloor$. There exist $D_{j,i} \in \mathbb{Z}_p$, $0 \leq i \leq L$, such that*

$$a_j = \sum_{i=0}^{L} D_{j,i} a_i, \tag{4.5}$$

$$B(j, pN) - \sum_{i=0}^{L} D_{j,i} B(i, N) \in p^{2+2v_p(N)} \mathbb{Z}_p \quad (N \geq 1), \tag{4.6}$$

$$D_{j,i} a_i \in p^2 \left(\frac{L!}{i!} \right)^2 \mathbb{Z}_p \quad (0 \leq i < L). \tag{4.7}$$

Proof. Write $j = pL + b$, $0 \leq b < p$, and $\epsilon = \lfloor 2b/p \rfloor$. In (4.1), decompose $k_i = p\ell_i + r_i$, $0 \leq r_i < p$. There is a carry d, $0 \leq d \leq n-1$, such that

$$\sum_i r_i = b + pd, \qquad \sum_i \ell_i = L - d.$$

Terms with $d > L$ are absent. Let $H = pN$. For the first factorial pair, one has

$$\frac{U(k_1)^2}{U(k_1 - H)U(k_1 + H)} \equiv 1 \pmod{p^{2+2v_p(N)}}.$$

This is the symmetric unit-factorial block calculation of Lemma 3.2, applied to the interval of length H ending at k_1. The corresponding factorial summand is a p-integer and the displayed quotient is a unit, so replacing that quotient by one preserves the claimed modulus term by term.

For each d, define the symmetric restricted series

$$P_{b,d}(X_1, \ldots, X_n) = \sum_{\substack{0 \leq r_i < p \\ \sum r_i = b + pd}} \prod_{i=1}^{n} (r_i!)^{-2} (1 - 2pH_{r_i} X_i + p^2 T_{r_i}(X_i)).$$

Symmetry of the residue-index set implies

$$P_{b,d} = \alpha_{b,d} + \beta_{b,d}(X_1 + \cdots + X_n) + p^2 R_{b,d}, \qquad R_{b,d} \in \mathbb{Z}_p\langle X_1, \ldots, X_n \rangle.$$

In particular, the coefficients of the linear terms are all equal; no division by the period a_L is used. The exact factorial identity and Lemma 4.2 now give

$$B(j, pN) \equiv p^{\epsilon} U(2j)(2L + \epsilon)! \sum_{0 \leq d \leq \min(n-1, L)} p^{2d} c^{-2(L-d)} \, [x^{L-d}] \\ P_{b,d}(\theta_1, \ldots, \theta_n) I_N(x_1) \prod_{i=2}^{n} I_0(x_i) \Big|_{x_1 = \cdots = x_n = x} \pmod{p^{2+2v_p(N)}}. \tag{4.8}$$

For $N = 0$, this is an exact equality. Indeed, after taking the p-parts of the factorials, the power of p is $2d + \epsilon$, their numerator is $(2L + \epsilon)!$, and their denominator is $(\ell_1 - N)!(\ell_1 + N)! \prod_{i>1} (\ell_i!)^2$, which verifies all factors in (4.8).

Apply Lemma 4.3 to each $P_{b,d}$, writing its output as

$$A_{d,k}(x) = \sum_{a \geq 0} \alpha_{d,a,k} x^a, \qquad 0 \leq k \leq n.$$

For $a > 0$, all $\alpha_{d,a,k}$ are in $p^2 \mathbb{Z}_p$. The reduction error is in $p^2 N^2 \mathcal{H}$. Multiplying its coefficient of degree $L - d$ by $(2L + \epsilon)!$ preserves this divisibility, since $(2L + \epsilon)!/(2(L - d))!$ is an integer. As $[x^i]\Phi_N = B(i, N)/(2i)!$, define

$$D_{j,i} = p^{\epsilon} U(2j) \frac{(2L + \epsilon)!}{(2i)!} \sum_{d=0}^{\min(n-1, L-i)} p^{2d} c^{-2(L-d)} \sum_{k=0}^{n} \alpha_{d, L-d-i, k} \, i^k. \tag{4.9}$$

The convention $i^0 = 1$ applies also at $i = 0$. All terms are p-integral. Equations (4.5) and (4.6) follow from the exactness at $N = 0$ and the just-proved error bound for $N \geq 1$.

If $i < L$, each summand in the inner expression of (4.9) contains p^2: either $d \geq 1$, or $d = 0$ and $L - d - i > 0$. Therefore

$$D_{j,i} \in p^2 \frac{(2L+\epsilon)!}{(2i)!}\mathbb{Z}_p.$$

Finally,

$$a_i = \frac{(2i)!}{(i!)^2} \sum_{k_1+\cdots+k_n=i} \binom{i}{k_1,\ldots,k_n}^2.$$

The sum is an integer, and

$$\frac{(2L+\epsilon)!}{(i!)^2} = \frac{(2L+\epsilon)!}{(L!)^2}\left(\frac{L!}{i!}\right)^2$$

is an integer multiple of $(L!/i!)^2$. This proves (4.7). □

4.4 Descent across the truncation boundary

Define integers $c_{K,N}$, zero for $N < K$, by

$$a_j \mathbf{1}_{j\geq K} = \sum_{N=K}^{j} c_{K,N} B(j,N).$$

The diagonal entries of B are one, so these integers exist uniquely.

Lemma 4.5 (Octahedral positive interpolation)**.** *For all positive K, N,*

$$v_p(c_{K,N}) \geq \max\{0, 2v_p(K) - 2v_p(N)\}.$$

If $p \mid K$, then

$$c_{K,N} \equiv \begin{cases} c_{K/p,N/p}, & p \mid N, \\ 0, & p \nmid N \end{cases} \pmod{p^{2v_p(K)}}.$$

Proof. Induct on $v = v_p(K)$, with integrality as the base case. Let $v \geq 1$, $M = K/p$, and assume the weighted bound for $c_{M,N}$. Consider $\sum_{N\geq M} c_{M,N} B(j, pN)$. For $j < K$, it is zero, as required. For $j \geq K$, put $L = \lfloor j/p \rfloor \geq M$. Lemma 4.4, together with the induction hypothesis, gives

$$\begin{aligned} \sum_{N=M}^{L} c_{M,N} B(j,pN) &\equiv \sum_{i=0}^{L} D_{j,i} \sum_{N=M}^{i} c_{M,N} B(i,N) \\ &= \sum_{i=M}^{L} D_{j,i} a_i = a_j - \sum_{i<M} D_{j,i} a_i \pmod{p^{2v}}. \end{aligned}$$

The errors in the first congruence have valuation at least

$$2 + 2v_p(N) + \max\{0, 2(v-1) - 2v_p(N)\} \geq 2v.$$

For $i < M \leq L$, the integer $L!/i!$ is divisible by M. The boundary estimate (4.7) thus puts each omitted term in $p^2M^2\mathbb{Z}_p \subseteq p^{2v}\mathbb{Z}_p$. The candidate coefficient vector, supported on multiples of p, therefore solves the defining triangular system for $c_{K,N}$ modulo p^{2v}. Unit triangular inversion proves the descent congruence. The weighted bound follows from it exactly as before: for $p \nmid N$ use the zero candidate, and for $p \mid N$ apply the induction hypothesis to $c_{M,N/p}$. □

Proof of Theorem 4.1. The defining triangular identity gives the exact expansion

$$1 - \frac{F_K(u)}{F(u)} = \sum_{N \geq K} c_{K,N} C_N(u).$$

Let $v = v_p(K) \geq 1$ and $M = K/p$. Lemma 4.5 gives

$$1 - \frac{F_K(u)}{F(u)} \equiv \sum_{N \geq M} c_{M,N} C_{pN}(u) \pmod{p^{2v}}.$$

Use (4.2) and the weighted bound on $c_{M,N}$ to replace C_{pN} by $C_N \circ \sigma$ at the same precision. The result is

$$1 - \frac{F_K(u)}{F(u)} \equiv \sum_{N \geq M} c_{M,N} C_N(\sigma(u)) = 1 - \frac{F_M(\sigma(u))}{F(\sigma(u))} \pmod{p^{2v}}.$$

All sums are legitimate coefficientwise, since their summands have order at least N, respectively pN. Multiplication by the period units and the identity $\mathcal{F}_{2M}(z) = F_M(z^2)$ finish the proof. □

5 Verification, sharpness, and scope

5.1 What has been closed

Theorem 2.1 supplies the full lower bound that was missing for each of the targets in the table of Section 2. It proves the ordinary positive truncation statement for the actual excellent lift, with no restriction on the coefficient index and no upper bound on s. In particular the proof for the three-dimensional octahedral family is uniform in $p \geq 5$ and does not stop at a fixed level.

The theorem establishes the exponent $2s$. A uniform exponent $3s$ already fails for the three-dimensional octahedral family: exact integer evaluation gives

$$[u^{125}]E_{5,3,1} = a_{125} - a_{25} \equiv 2 \cdot 5^6 \pmod{5^7}.$$

Thus this coefficient has valuation $6 = 2s$, which is less than $3s = 9$. Any previously established matching upper bounds for a coefficient can now be combined with Theorem 2.1 to give optimality in their precise parameter ranges.

This original argument has the prime range displayed in its theorem; the complete edition has the larger range of Theorem 1.1. The elliptic-zero and Zagier D arguments are retained in Parts V–VII. They are separate results and are not dependencies of this theorem. Within the three standard families and the displayed prime conditions, the theorem covers every dimension, level, multiplier, and coefficient.

5.2 Why a scalar octahedral block is insufficient

Here is an exact counterexample to using the cubic scalar block argument unchanged for octahedra. In dimension three, take $p = 5$, $j = 10$, $L = 2$, and $N = 1$. Then

$$\begin{aligned} a_{10} &= 25\,989\,269\,017\,140, & a_2 &= 90, \\ B(10,5) &= 20\,650\,661\,328, & B(2,1) &= 28. \end{aligned}$$

The ratio $a_{10}/a_2 = 288\,769\,655\,746$ is integral, but

$$B(10,5) - (a_{10}/a_2)B(2,1) = -8\,064\,899\,699\,560 \equiv 15 \pmod{25}.$$

Its 5-adic valuation is one. The matrix block lemma avoids this failed scalar assertion and controls the necessary sum over lower rows, including the boundary terms.

5.3 Exact arithmetic accompanying the proofs

Appendix D contains the standalone verification programs in full, using integer, rational, and modular polynomial arithmetic. The general proofs above do not depend on the number of coefficients tested. The computations check signs, indices, normalizations, and the actual constructions used in the proof.

- `verify_positive_interpolation.py` checks the balanced factorial block congruence, the weighted triangular descent, and independently constructed excellent-lift defects for cubes and simplices. Its report records eight block cases (12,949 matrix entries), eleven interpolation cases, and seven direct-defect cases. The pure factorial tests intentionally include some primes outside the geometric theorem's range; these are not reported as extensions of the theorem.
- `verify_octahedral_interpolation.py` checks the restricted unit-factorial and symmetric-operator construction, the resulting row matrix, and the interpolation descent. At $p = 5$ and precision p^6, it constructs every row through $j = 145$ in dimensions three and four, checking 4,118 shifted columns and 4,118 boundary terms in total. It also checks five interpolation cases up to column 300 and three independently constructed actual-BV defects, including $s = 3$ in dimensions three and four. Exact parameters and precisions are specified in the embedded program, which regenerates its JSON report.
- The separate program `certificate_octa4_p5_s2.py` constructs the actual lift modulo 625 from negative vertex polynomials and checks a finite polynomial identity of degree at most 2819. Appendix A proves why that identity certifies every coefficient at $n = 4, p = 5, s = 2$. This provides a different exact route to one case of the general theorem.

All output reports distinguish bounded validation from a theorem about infinitely many coefficients. The independently certified whole-series case is labelled as such because its proof includes a finite polynomial identity with an explicit denominator that is a unit.

Part II
All odd primes for cubes and simplices; the common formal input

The orbit module, ramified normalization, scalar descent, independent first levels, and the period-functional obstruction.

6 Scope and results

For the cube, simplex, and octahedron put respectively

$$g=\prod_{i=1}^{n}(x_i+x_i^{-1}),\qquad g=x_1+\cdots+x_n+(x_1\cdots x_n)^{-1},\qquad g=\sum_{i=1}^{n}(x_i+x_i^{-1}).$$

Let $f=1-zg$, $h=2,n+1,2$, $u=z^h$, and

$$F_0(z)=\mathrm{CT}(1/f)=F(u),\quad F(u)=\sum_{j\geq 0}a_ju^j,\quad F_K(u)=\sum_{j<K}a_ju^j.$$

We also write

$$\mathcal{F}(z)=F_0(z),\qquad \mathcal{F}_L(z)=\sum_{0\leq j<L}\mathrm{CT}(g^j)z^j.$$

The canonical excellent lift is denoted by $\tau(z)$ and its normalized form by $\sigma(u)=\tau(z)^h$. Define

$$E_{p,s,m}=F(u)F_{mp^{s-1}}(\sigma(u))-F(\sigma(u))F_{mp^s}(u).$$

Every assertion below is coefficientwise in the indicated formal-series ring. No arbitrary parameter specialization is asserted.

Theorem 6.1 (Odd-prime scope of this addendum)**.** *For every dimension $n\geq 1$ and every odd prime p:*

1. *In the cube and simplex families, for every $p\mid K$,*

$$\frac{F_K}{F}\equiv\frac{F_{K/p}\circ\sigma}{F\circ\sigma}\pmod{p^{2v_p(K)}\mathbb{Z}_p[[u]]}.$$

   Thus $E_{p,s,m}\in p^{2s}\mathbb{Z}_p[[u]]$ for all $m,s\geq 1$. The simplex statement includes $p\mid n+1$.

2. *In the octahedral family, $E_{p,1,1}\in p^2\mathbb{Z}_p[[u]]$.*

3. *The following additional octahedral statements hold:*

$$\begin{aligned}n=4:&\quad E_{3,2,1}\in 81\mathbb{Z}_3[[u]],\quad E_{3,3,1}\in 729\mathbb{Z}_3[[u]],\\ n=5:&\quad E_{5,2,1}\in 625\mathbb{Z}_5[[u]].\end{aligned}$$

   The last three statements are proved by exact finite polynomial certificates whose complete program is included in Appendix E.

The octahedral conclusions of this part are the independent first-level argument and three exact certificates. The all-level strengthening at every odd prime is supplied by Parts III and IV. The finite certificates here are not used to infer untested levels or general multipliers. The orbit-module argument below is a new verification using the general results of [7, Sections 2–5]; it is not an application of [7, Theorem 7.3] after dropping its stated hypotheses. The argument is restricted to $R=\mathbb{Z}_p[[z]]$. Its analytic continuation to other base rings is not needed or claimed here.

7 The excellent Cartier input without group averaging

Proposition 7.1 (Formal excellent input from an orbit module)**.** *Let g be the hypercubic, simplicial, or hyperoctahedral Laurent polynomial, with vertex coefficients one, and let p be an odd prime. Put $R = \mathbb{Z}_p[[z]]$ and $f = 1 - zg$. Let $F_0(z)$ and $G_0(z)$ be the period and logarithmic companion given by the vertex-coefficient formulas of BV3, Proposition 9.1. Then*

$$q_0(z) = z\exp(G_0(z)/F_0(z)) \in z + z^2\mathbb{Z}_p[[z]].$$

The lift $\tau = q_0^{-1}(q_0(z)^p)$ is integral and satisfies

$$\mathscr{C}_p(1/f) - \frac{F_0(z)}{F_0(\tau)}\frac{1}{f^\tau} \in p^2\mathscr{F}_2^\tau.$$

In particular, no exclusion $p \mid n$ for the octahedron, or $p \mid n+1$ for the simplex, is needed for this formal input.

Proof. Write Γ for the support lattice and H for the full monomial symmetry group. Its order will never be inverted. The support lattice is $\mathbb{Z}^n$ in the simplicial and octahedral cases; for a cube its index is 2^{n-1}. Thus its index is prime to p in all three cases. The symmetry group preserves the reflexive polytope degree $d(u) = \deg_\Delta(u)$ and the support lattice. For an H-orbit $O \subset \Gamma$, put

$$S_O = \sum_{u\in O} x^u, \qquad d(O) = d(u)\ (u \in O), \qquad L = \bigoplus_O Rz^{d(O)}S_O.$$

Here a Laurent polynomial uses only finitely many orbits. This is a direct construction of a module of polynomials. We do not obtain it by applying an averaging projector to another crystal.

Fix temporarily any lift σ with $z^\sigma = z^p v$, $v \in 1 + pR$. We check BV3, Definition 2.1. The orbit sums have disjoint supports and coefficient one, so L is p-saturated. The same is true of every L^{σ^i}: its orbit coefficient generator is $(z^{\sigma^i})^{d(O)}$, a power of z times a unit, which is a non-zero-divisor modulo p. Multiplication by f preserves invariance and admissibility, hence $fL \subset L$.

The subset $p\Gamma$ is H-stable. An orbit is either disjoint from $p\Gamma$ or has the form pO' for one orbit O'; the equality uses the p-saturation of the support lattice. Since $d(pu) = pd(u)$,

$$\mathscr{C}_p(z^{d(pO')}S_{pO'}) = z^{pd(O')}S_{O'} = \left(\frac{z^p}{z^\sigma}\right)^{d(O')}(z^\sigma)^{d(O')}S_{O'} \in L^\sigma.$$

The image of any other orbit is zero. This proves compatibility without dividing by either $|H|$ or $|O|$.

Form the Dwork crystal $M = \widehat{\Omega}_{L,f}(\Delta^\circ)$ directly, as in BV3, Definitions 2.6 and 2.8. We do not identify it with the invariant part of an ambient completed module; in particular no exactness of an invariant functor is being assumed. In all three families, $\Delta \cap \Gamma$ consists of the origin and the vertices. Reflexivity then implies that the first two levels have bases

$$M(1) = R/f, \qquad M(2) = R/f^2 \oplus Rzg/f^2.$$

Indeed $2\Delta^\circ \cap \Gamma = \Delta \cap \Gamma$ and the vertices form one orbit. The pair $1/f, zg/f^2$ is an extended basis, since $1/f = (1 - zg)/f^2$ and the change of basis has determinant one. These levels are p-saturated in the ambient integral formal expansion: multiplying an expansion by f or f^2 reduces the claim to divisibility of the numerator coefficients. This also shows that the two basis vectors remain independent modulo p.

The Hasse–Witt computations of BV3, Lemma 7.2, now apply to this explicit compatible module. Their coefficient extraction uses a single vertex coefficient of a polynomial known to be

constant on the vertex orbit; it does not average over that orbit. With the initial lift $z^\sigma = z^p$, the first determinant has constant term one, and

$$\left.\frac{\det HW^{(2)}(M)}{p}\right|_{z=0} = -\frac{1}{p}\binom{2p-2}{p-1} \in \mathbb{Z}_p^\times.$$

The latter computation reads the coefficient of $z^p x^{pv}$ at one vertex v in $(zg)F^{(2)}$; the extreme vertex forces the unique contribution $(-z)^p\binom{2p-2}{p-1}$. The normalized determinant modulo p is independent of the lift. Consequently both Hasse–Witt conditions hold over R for every lift under consideration. The general BV3, Corollary 5.9 and Corollary 4.3, therefore give

$$M = M(2) \oplus \mathscr{F}_2 M, \qquad \mathscr{C}_p(M) \subset M^\sigma(2) + p^2 \mathscr{F}_2 M^\sigma.$$

Only $p > 2$ is required by these general results.

For completeness, the remaining canonical-coordinate argument uses these statements rather than a group-order hypothesis. The derivation $\theta = z\,d/dz$ preserves M. Thus the rank-two quotient has the connection and Cartier matrices of BV3, Propositions 7.6 and 7.7. The positive vertex coefficient calculation of Proposition 9.1 gives exactly F_0 and $F_0 \log z + G_0$ as its two solutions. The same coefficient calculation at exponents $p^s v$ gives $A(0) = B(0) = 0$, the constant Cartier matrix $\mathrm{diag}(1, p)$ for the initial lift, and the upper-right coefficient

$$\lambda_1 = \frac{pF_0(z)F_0(z^\sigma)}{W(z^\sigma)}\left(-\frac{1}{p}\log\frac{z^\sigma}{z^p} + \frac{G_0(z)}{F_0(z)} - \frac{1}{p}\frac{G_0(z^\sigma)}{F_0(z^\sigma)}\right),$$

where $W = F_0^2 + F_0\theta G_0 - (\theta F_0)G_0$. The rank-one reduction modulo p implies $\lambda_1 \in pR$. The determinant identity $\det\Lambda = pW/W^\sigma$ and $W(0) = 1$ give $W \in 1 + zR$ by the same iteration as BV3, Proposition 7.9. For $z^\sigma = z^p$, the displayed formula and the Dieudonne–Dwork lemma yield $q_0 \in z + z^2R$. Its formal inverse is integral, so $\tau = q_0^{-1}(q_0^p)$ defines a lift with $\tau/z^p \in 1 + pR$. Reapply the compatible-module construction with this lift. The displayed expression has $\lambda_1 = 0$, and the remaining coefficient is F_0/F_0^τ. This proves the required excellent Cartier identity in $p^2\mathscr{F}_2 M^\tau$.

Finally, membership in $\mathscr{F}_2 M$ means that every s-fold Cartier iterate lies in $p^{2s}M^{\sigma^s}$. Mapping the directly constructed module to the ambient Dwork module therefore sends $\mathscr{F}_2 M$ into the ambient $\mathscr{F}_2$. Only this inclusion is needed; no converse assertion about invariants or completed modules is used. □

Remark 7.2 (Normalization even when $p \mid h$). Let $h = 2$ for cubes and octahedra and $h = n+1$ for simplices. The explicit vertex formulas give $F_0(z) = F(z^h)$ and $G_0(z) = G(z^h)$, so $q_0(z) = zQ(z^h)$. For every hth root of unity ζ over an algebraic extension of $\mathbb{Q}_p$, uniqueness of the formal inverse gives $\tau(\zeta z) = \zeta^p\tau(z)$. It follows that $\tau(z) = z^p V(z^h)$ with $V \in 1 + p\mathbb{Z}_p[[u]]$. Consequently $\sigma(u) = \tau(z)^h = u^p V(u)^h$ is integral even when $p \mid h$; no hth root is extracted. The positive-vertex Cartier error is divisible by $p^2(hN)^2$, which is at least as strong as $p^{2+2v_p(N)}$. The scalar block descent in the next section therefore applies to cubes and simplices at every odd prime.

8 All odd primes for cubes and simplices

This section reproduces the scalar interpolation argument, with its external input replaced by Proposition 7.1. This makes the new prime range explicit and records all algebraic steps needed for it. The coefficients are $a_j = \binom{2j}{j}^n$ for cubes and $a_j = ((n+1)j)!/(j!)^{n+1}$ for simplices. Throughout p is any odd prime.

Theorem 8.1. *For these two families and every K divisible by p,*

$$\frac{F_K(u)}{F(u)} \equiv \frac{F_{K/p}(\sigma(u))}{F(\sigma(u))} \pmod{p^{2v_p(K)}\mathbb{Z}_p[[u]]}. \tag{8.1}$$

8.1 A binomial and multinomial block congruence

Set $r = n + 1$ in the simplicial case and define, for $0 \le N \le j$,

$$B(j,N) = \begin{cases} \binom{2j}{j-N}^n, & \text{hypercubic,} \\ \dfrac{(rj)!}{(j+(r-1)N)!(j-N)!^{r-1}}, & \text{simplicial.} \end{cases}$$

Put $B(j,N) = 0$ for $N > j$. In both cases these are nonnegative integers with $B(j,0) = a_j$ and $B(j,j) = 1$.

Lemma 8.2 (Balanced factorial block congruence)**.** *Let p be odd and $l = \lfloor j/p \rfloor$. Then $d_j = a_j/a_l \in \mathbb{Z}_p$, and, for every $1 \le N \le l$,*

$$B(j,pN) - d_j B(l,N) \in p^{2+2v_p(N)}\mathbb{Z}_p. \tag{8.2}$$

This lemma itself holds for every positive dimension and every odd prime; the excellent Cartier input is now supplied by Proposition 7.1.

Proof. For $a \ge 0$, write

$$U(a) = \prod_{\substack{1 \le i \le a \\ p \nmid i}} i.$$

We shall use the exact identity

$$a! = p^{\lfloor a/p \rfloor} \lfloor a/p \rfloor ! U(a).$$

Write $j = pl + b$, $0 \le b < p$, and put $H = pN$ and $v = v_p(H) = 1 + v_p(N)$. For the unit index set

$$I = \{ i : j - H < i \le j,\ p \nmid i \},$$

one has

$$\sum_{i \in I} \frac{1}{i} \in p^v \mathbb{Z}_p. \tag{8.3}$$

Indeed, the interval has length H, a multiple of p^v. Its residues modulo p^v consist of H/p^v complete copies of the residue classes. Among the unit classes, inversion permutes the residues, and opposite residues cancel because p is odd.

For the cubic case, first omit the power n and write $b(j,N) = \binom{2j}{j-N}$. With $\epsilon = \lfloor 2b/p \rfloor \in \{0,1\}$, the factorial identity gives

$$d_j^{(1)} = \frac{b(j,0)}{b(l,0)} = p^\epsilon (2l+1)^\epsilon \frac{U(2j)}{U(j)^2} \in \mathbb{Z}_p,$$

and

$$b(j,H) = d_j^{(1)} b(l,N) S, \qquad S = \frac{U(j)^2}{U(j-H)U(j+H)} = \prod_{i \in I} \left(1 + \frac{H}{i}\right)^{-1}.$$

Since each H/i is in $p^v\mathbb{Z}_p$,

$$S \equiv 1 - H \sum_{i \in I} \frac{1}{i} \equiv 1 \pmod{p^{2v}}.$$

Taking the n-th power proves (8.2).

For the simplicial case set $\epsilon = \lfloor rb/p \rfloor$. Then

$$d_j = \frac{a_j}{a_l} = p^\epsilon \frac{(rl+\epsilon)!}{(rl)!} \frac{U(rj)}{U(j)^r} \in \mathbb{Z}_p.$$

The balanced shifts in the factorial denominator give the exact identity

$$B(j,H) = d_j B(l,N) S, \qquad S = \frac{U(j)^r}{U(j+(r-1)H)U(j-H)^{r-1}}.$$

Decomposing the long upper interval into $r-1$ blocks of length H yields

$$S = \prod_{i\in I}\prod_{k=1}^{r-1}\left(1+\frac{kH}{i}\right)^{-1}.$$

Therefore

$$S \equiv 1 - H\frac{r(r-1)}{2}\sum_{i\in I}\frac{1}{i} \equiv 1 \pmod{p^{2v}},$$

again by (8.3). All prefactors multiplying $S-1$ are p-integral, which proves the lemma. □

8.2 An integral triangular descent

The following argument is stated in the notation of the two families, but uses only integrality of B, its diagonal entries one, and Lemma 8.2. For each positive integer K, define integers $c_{K,N}$ by

$$a_j \mathbf{1}_{j\geq K} = \sum_{N=K}^{j} c_{K,N} B(j,N), \qquad j \geq 0, \tag{8.4}$$

and put $c_{K,N} = 0$ for $N < K$. Existence, uniqueness and integrality follow by successive solution in j, since $B(j,j) = 1$.

Lemma 8.3 (Weighted interpolation and descent)**.** *For every $K, N \geq 1$,*

$$v_p(c_{K,N}) \geq \max\{0,\, 2v_p(K) - 2v_p(N)\}. \tag{8.5}$$

If $p \mid K$, then

$$c_{K,N} \equiv \begin{cases} c_{K/p,N/p}, & p \mid N, \\ 0, & p \nmid N \end{cases} \pmod{p^{2v_p(K)}}. \tag{8.6}$$

Proof. Induct on $v = v_p(K)$. If $v = 0$, the weighted bound is integrality and there is no descent assertion. Suppose $v \geq 1$, set $L = K/p$, and assume the weighted bound for L. Let $l = \lfloor j/p \rfloor$. Multiplying the defining identity for L by $d_j = a_j/a_l$ gives

$$a_j \mathbf{1}_{j\geq K} = d_j a_l \mathbf{1}_{l\geq L} = \sum_{N=L}^{l} c_{L,N} d_j B(l,N).$$

By Lemma 8.2, replacing each $d_j B(l,N)$ by $B(j,pN)$ makes an error of valuation at least

$$2 + 2v_p(N) + \max\{0,\, 2(v-1) - 2v_p(N)\} \geq 2v.$$

Thus the candidate coefficient vector, equal to $c_{L,N}$ at index pN and zero at indices not divisible by p, satisfies (8.4) modulo p^{2v}. Successively solving this unit triangular system proves (8.6). For $p \nmid N$, it also gives the weighted bound directly. For $p \mid N$, the induction hypothesis applied to $c_{L,N/p}$ gives the bound $\max\{0, 2(v - v_p(N))\}$, and the error p^{2v} does not weaken it. □

8.3 Positive vertex coefficients and the excellent lift

For the cubic family choose the vertex $\mathbf{v} = (1, \dots, 1)$. For the simplex choose $\mathbf{v} = (-1, \dots, -1)$. The coefficient at $\mathbf{x}^{hN\mathbf{v}}$ in the z-adic expansion of $1/f$ is

$$\Omega_N(z) = \sum_{j \geq N} B(j, N) z^{hj}. \tag{8.7}$$

For the cube this follows by choosing $j - N$ negative steps in each of the n independent factors of g^{2j}. For the simplex, the required term of g^{rj} uses the chosen vertex $j + (r-1)N$ times and each of the other $r - 1$ vertices $j - N$ times, giving exactly the displayed multinomial coefficient.

Set

$$C_N(u) = \frac{\Omega_N(z)}{F(u)} \in u^N \mathbb{Z}_p[[u]].$$

The coefficientwise interpretation of the module of second formal derivatives, including in the positive z-adic expansion, is [7, Section 3, the paragraph following Corollary 3.4]. Read Proposition 7.1 at the exponent vector $hN\mathbf{v}$. The left coefficient is $\Omega_{pN}(z)$, and the right coefficient is $\mathcal{F}(z)\Omega_N(\tau)/\mathcal{F}(\tau)$. The error is divisible by $p^2(hN)^2$ in $\mathbb{Z}_p[[z]]$. Since $h \in \mathbb{Z}$ and $F(0) = 1$, division by $F(u)$ gives

$$C_{pN}(u) - C_N(\sigma(u)) \in p^{2+2v_p(N)} \mathbb{Z}_p[[u]]. \tag{8.8}$$

Proof of Theorem 8.1. Summing (8.4) in j, using (8.7), and dividing by $F(u)$, gives the exact identity

$$1 - \frac{F_K(u)}{F(u)} = \sum_{N \geq K} c_{K,N} C_N(u).$$

It converges u-adically because C_N has order at least N. Put $v = v_p(K) \geq 1$ and $L = K/p$. Lemma 8.3 first gives

$$1 - \frac{F_K(u)}{F(u)} \equiv \sum_{N \geq L} c_{L,N} C_{pN}(u) \pmod{p^{2v}}.$$

By (8.8) and (8.5), each summand may be replaced by $c_{L,N} C_N(\sigma(u))$ with error divisible by p^{2v}. Indeed, the error has valuation at least

$$2 + 2v_p(N) + \max\{0, 2(v-1) - 2v_p(N)\} \geq 2v.$$

Both sums still converge: $C_{pN}(u)$ and $C_N(\sigma(u))$ have order at least pN. Consequently

$$1 - \frac{F_K(u)}{F(u)} \equiv \sum_{N \geq L} c_{L,N} C_N(\sigma(u)) = 1 - \frac{F_L(\sigma(u))}{F(\sigma(u))} \pmod{p^{2v}},$$

which is (8.1). Multiplication by the period units gives the cleared-defect and quotient formulations. Finally, $\mathcal{F}_{hM}(z) = F_M(z^h)$ proves the assertion in the original normalization. □

9 The octahedral first level at every odd prime

We retain the geometric first-level proof with the new input. It also applies to the other two families, but only the octahedral conclusion is new beyond the preceding section. Write

$$c_j = \mathrm{CT}(g^j), \quad \mathcal{F}(z) = \sum_{j \geq 0} c_j z^j, \quad H_N(z) = \mathrm{CT}(f^{N-1}), \quad \lambda = \mathcal{F}(z)/\mathcal{F}(\tau(z)).$$

In this section $\sigma_p(z) = \tau(z)$ and $\sigma_p(u) = \sigma(u)$ denote the two normalizations. Proposition 7.1 supplies the excellent Cartier identity at every odd prime.

Lemma 9.1 (Second-order geometric approximation)**.** *For the above excellent lift,*

$$\lambda \equiv 2H_p - H_{2p} \pmod{p^2\mathbb{Z}_p[[z]]}.$$

More precisely, write

$$\mathscr{C}_p(f^{2p-1}) = H_{2p}(z) + B_p(z)g(\mathbf{x}).$$

Then

$$\tau \equiv -\frac{B_p(z)}{H_{2p}(z)} \pmod{p^2\mathbb{Z}_p[[z]]}.$$

Proof. All the calculations below may be made in the ring of Laurent series whose coefficients are z-adically convergent integral power series. Put $h(\mathbf{x}) = f^\sigma(\mathbf{x}^p)$. Since $h - f^p$ is divisible by p, the exact identity

$$\frac{1}{f} - \frac{2hf^{p-1} - f^{2p-1}}{h^2} = \frac{(h-f^p)^2}{fh^2}$$

has an integral remainder divisible by p^2. Applying the Cartier operator and using $\mathscr{C}_p(A(\mathbf{x})/h(\mathbf{x})^2) = \mathscr{C}_p(A)/f^\sigma(\mathbf{x})^2$, we obtain

$$\mathscr{C}_p(1/f) \equiv \frac{2f^\sigma \mathscr{C}_p(f^{p-1}) - \mathscr{C}_p(f^{2p-1})}{(f^\sigma)^2} \pmod{p^2}.$$

We justify the two numerator supports explicitly. Let Γ be the lattice spanned by the support of g. Since its index is prime to p, $p\mathbf{u} \in \Gamma$ implies $\mathbf{u} \in \Gamma$. Reflexivity gives integral facet functions L_j with $\Delta = \{L_j \le 1\}$. A lattice point in $(1-1/p)\Delta$ has $L_j \le 0$ for every j, and hence is zero. A lattice point in $(2-1/p)\Delta$ has $L_j \le 1$, and hence belongs to Δ. The only points of $\Delta \cap \Gamma$ are zero and the vertices. Complete symmetry therefore yields the exact identities

$$\mathscr{C}_p(f^{p-1}) = H_p, \qquad \mathscr{C}_p(f^{2p-1}) = H_{2p} + B_p g.$$

For the hypercube, the restriction to Γ is essential; its points have coordinates all even or all odd.

The excellent Cartier relation $\mathscr{C}_p(1/f) \equiv \lambda/f^\sigma \pmod{p^2\mathscr{F}_2^\sigma}$ is in particular a coefficientwise congruence modulo p^2. Multiplying by $(f^\sigma)^2$ gives

$$\lambda(1-\tau g) \equiv 2H_p(1-\tau g) - H_{2p} - B_p g \pmod{p^2}.$$

Since $\mathrm{CT}(g) = 0$, comparison of the coefficients of 1 and g first gives $\lambda \equiv 2H_p - H_{2p}$, and then $\tau H_{2p} \equiv -B_p$. The latter can be divided by the unit H_{2p}, whose constant term is one. □

Theorem 9.2 (The first level at every odd prime)**.** *Set $h = 2$ for the hypercubic and hyperoctahedral families and $h = n+1$ for the simplicial family. At every odd prime,*

$$\frac{\mathcal{F}(z)}{\mathcal{F}(\sigma_p(z))} \equiv \mathcal{F}_{hp}(z) \pmod{p^2\mathbb{Z}_p[[z]]}.$$

Equivalently, in the normalized variable $u = z^h$,

$$\frac{F(u)}{F(\sigma_p(u))} \equiv F_p(u) \pmod{p^2\mathbb{Z}_p[[u]]}.$$

These are the $s = 1$ cases of the specified original truncation lengths; $\mathcal{F}_h = 1$.

Proof. The coefficient of z^j in $2H_p - H_{2p}$ is

$$c_j w_j, \qquad w_j = (-1)^j \left(2\binom{p-1}{j} - \binom{2p-1}{j}\right),$$

with binomial coefficients zero past the top index. If $0 \le j < p$, the product expansion

$$(-1)^j \binom{ap-1}{j} = \prod_{i=1}^{j} \left(1 - \frac{ap}{i}\right) \equiv 1 - apH_j \pmod{p^2}$$

shows that $w_j \equiv 1 \pmod{p^2}$. For $p \le j < 2p$, Lucas's congruence gives $w_j \equiv 1 \pmod{p}$.

The remaining required divisibilities follow directly from the factorial formulas. In the cubic case, $c_{2k} = \binom{2k}{k}^n$, and for $k < p$ this is divisible by p whenever $2k \ge p$. In the octahedral case,

$$c_{2k} = \sum_{k_1+\cdots+k_n=k} \frac{(2k)!}{(k_1!\cdots k_n!)^2},$$

and every denominator is prime to p when $k < p$; hence the same divisibility holds term by term. In the simplicial case, putting $r = n+1$,

$$c_{rk} = \frac{(rk)!}{(k!)^r}, \qquad 0 \le k < p, \qquad v_p(c_{rk}) \ge \left\lfloor \frac{rk}{p} \right\rfloor.$$

Thus in all three cases c_j is divisible by p for $p \le j < \min(2p, hp)$, and in the simplicial case it is divisible by p^2 for $2p \le j < hp$. The other two families have $h = 2$, so there is no latter range. It follows coefficientwise that

$$2H_p - H_{2p} \equiv \mathcal{F}_{hp} \pmod{p^2}.$$

Apply Lemma 9.1 and then the normalized coordinate relation $q_0(z)^h = q(z^h)$. □

10 Three exact whole-series certificates

Here $g = \sum_{i=1}^{n}(x_i + x_i^{-1})$, $u = z^2$ and

$$a_j = \binom{2j}{j} \sum_{k_1+\cdots+k_n=j} \binom{j}{k_1,\ldots,k_n}^2.$$

We use $(n,p,s) = (4,3,2), (4,3,3), (5,5,2)$ and normalized multiplier $m = 1$. In the last case Proposition 7.1 supplies the Cartier input and its rank-two differential quotient, including the integral Wronskian, despite $p \mid n$. For the first two cases the smaller group C_8 also suffices for the published BV input. The following argument is common to the three certificates.

10.1 Vertex polynomials and their derivative estimate

Define

$$b_j = \sum_{k_2+\cdots+k_n=j} \binom{j}{k_2,\ldots,k_n}^2, \qquad A_K(u) = \sum_{j=0}^{K-1} u^j \sum_{\ell=0}^{j} (-1)^\ell \frac{(2K-\ell-1)!\, b_{j-\ell}}{(2K-2j-1)!\,\ell!\,(j-\ell)!^2}.$$

The factorial ratio is an integer multinomial coefficient and $A_K(0) = 1$. Expanding $1/f$ in the chamber with dominant vertex x_1, the negative of the coefficient of x_1^{-2K}, all other exponents being zero, is

$$P_{2K}(u) = u^{-K} A_K(u).$$

Let $\lambda = F(u)/F(\sigma(u))$. Reading this vertex coefficient in the excellent Cartier congruence gives

$$P_{2pK}(u) - \lambda P_{2K}(\sigma(u)) \in p^{2+2v_p(K)} \mathbb{Z}_p((u)). \tag{10.1}$$

The displayed elements have bounded negative support. No claim that every element of the completed coefficient ring is a Laurent series with bounded negative support is needed.

Lemma 10.1. *For $C_K = A_{2K}/A_K$ and $\theta = u\,d/du$,*

$$\theta C_K \in p^{v_p(K)}\mathbb{Z}_p[[u]].$$

Proof. Set $R_K = P_{2K}/F$ and use the normalized canonical coordinate q, with $\partial = q\,d/dq$. The second-order quotient relation [7, Proposition 7.6], read at the same vertex, gives after division by F and passage to q

$$\partial^2 R_K \in p^{2v_p(K)}\mathbb{Z}_p((q)).$$

More explicitly, in normalized coordinates let $\mathcal{L} = \theta^2 - B_u\theta - A_u$ and $H = \theta q/q = W/F^2$. Then $\theta H/H = B_u - 2\theta F/F$ and the exact identity

$$\mathcal{L}(FR) = FH^2\partial^2 R$$

shows that this passage preserves the required precision. The gauge and coordinate factors are integral units by [7, Proposition 7.9 and Corollary 7.11]; the normalization factor 2 is a p-adic unit. For $r = v_p(K)$, integrality of R_K together with $j^2[q^j]R_K \in p^{2r}\mathbb{Z}_p$ implies $j[q^j]R_K \in p^r\mathbb{Z}_p$ for every nonzero integer j. Thus ∂R_K, and then θR_K, belong to $p^r\mathbb{Z}_p((q))$ and $p^r\mathbb{Z}_p((u))$, respectively. Differentiate $C_K = u^K R_{2K}/R_K$; the factors after removing their displayed powers of u are integral units. Every term has the factor p^r, giving the assertion. □

10.2 The scaled Hensel equation

Fix one of the three triples (n, p, s) in Theorem 6.1, and set

$$M = p^{2s}, \quad M_y = p^{2s-1}, \quad N = p^{2s-2}, \quad X = u^p, \qquad \sigma(u) = X(1 + py(u)), \quad y \in \mathbb{Z}_p[[u]].$$

Eliminating λ between (10.1) for $K = N, 2N$ gives

$$(1 + py)^N C_{pN}(u) - C_N(X(1 + py)) \in p^{4s-2}\mathbb{Z}_p[[u]].$$

Divide this expression by pN. Its coefficient of y^r, for $r \geq 1$, has valuation at least

$$r - 1 - v_p(r). \tag{10.2}$$

Indeed Lemma 10.1 implies that the rth Taylor coefficient of $C_N(X(1 + h))$ lies in $(N/r)\mathbb{Z}_p[[u]]$; also $\binom{N}{r} \in (N/r)\mathbb{Z}_p$. Multiplication by $p^r/(pN)$ gives (10.2). The constant coefficient is integral as well, since the actual y solves the preceding congruence and all positive-degree coefficients are integral.

Take $d = 3$ for the two level-two cases, and $d = 6$ for $(n, p, s) = (4, 3, 3)$. The bound (10.2) is at least $2s - 1$ for every $r > d$. Thus the equation for y modulo M_y is a degree-d polynomial. Its linear coefficient has constant term 1, and all nonlinear coefficients are divisible by p.

Here are completely explicit common-denominator numerators for that polynomial. Compute the four vertex polynomials at indices $N, 2N, pN, 2pN$ modulo p^{4s-2} and set

$$a_r = [h^r]A_N(X(1 + h)), \quad b_r = [h^r]A_{2N}(X(1 + h)), \quad a_0 = A_N(X).$$

Define

$$T_0 = b_0, \qquad T_r = b_r a_0^r - \sum_{i=1}^{r} a_i T_{r-i} a_0^{i-1}.$$

Then $T_r/a_0^{r+1} = [h^r]C_N(X(1 + h))$. For $0 \leq r \leq d$ put

$$Q_r = \frac{p^r}{pN}\left(\binom{N}{r} A_{2pN} a_0^{d+1} - A_{pN} T_r a_0^{d-r}\right) \pmod{M_y}. \tag{10.3}$$

The quotients are coefficientwise integral by the preceding argument; the certificate independently checks this exact divisibility. Their common denominator $A_{pN}a_0^{d+1}$ is a unit. Thus

$$\sum_{r=0}^{d} Q_r y^r = 0 \pmod{M_y}, \qquad Q_0(0)=0, \quad Q_1(0)=1, \quad Q_r \in p\mathbb{Z}_p[u]\ (r\ge 2). \tag{10.4}$$

Two roots of this equation have difference multiplied by a unit, because after subtracting their equations the factor is congruent to Q_1 modulo p. Therefore the root modulo M_y is unique.

The program constructs a rational root Y/Z from these Q_r using the finite multivariate Lagrange expansion, with $Z(0)=1$, and then checks the entire polynomial identity

$$\sum_{r=0}^{d} Q_r Y^r Z^{d-r} = 0 \pmod{M_y}. \tag{10.5}$$

This identity and uniqueness, rather than an assumption about the target truncations, prove $y = Y/Z \pmod{M_y}$ and hence $\sigma = X(1+pY/Z) \pmod{M}$.

For clarity, the construction is explicit. Write $A = Q_0$, $B = Q_1$. For a profile $(m_2,\dots,m_d)$ of nonnegative integers, put $m = \sum m_r \ge 1$, $R = \sum r m_r$ and $\ell = R - m + 1$. Its contribution to Y/Z is

$$(-1)^{R+1}\frac{R!}{\ell!\prod m_r!}\frac{A^\ell \prod_{r=2}^{d} Q_r^{m_r}}{B^{R+1}}.$$

Include also the term $-A/B$, and retain only profiles of total coefficient valuation less than $2s-1$. The integer factorial quotient is the usual rooted-plane-tree coefficient in Lagrange inversion. The code checks its integrality. The verified common denominators are B^7, B^{13}, and B^5 for $(n,p,s) = (4,3,2),(4,3,3),(5,5,2)$ respectively. The direct check (10.5) makes the root certification independent of any truncation of the Lagrange expansion.

10.3 The target polynomial identities

The vertex relation at $K = N$ yields

$$\lambda = \frac{(1+py)^N A_{pN}(u)}{A_N(X(1+py))} \pmod{M}.$$

Since $v_p\binom{N}{r} \ge v_p(N) - v_p(r)$, for every $r \ge 2$ the term $\binom{N}{r}(py)^r$ is divisible by M. Hence $(1+h)^N = 1 + Nh \pmod{M}$ on $h \in p\mathbb{Z}_p[[u]]$.

Set $f = F_{p^s}$, $f_0 = F_{p^{s-1}}$ and, for $0 \le r < 2s$, define

$$L_r = A_{pN}(u)\big([h^r]f_0(X(1+h)) + N[h^{r-1}]f_0(X(1+h))\big) - f(u)[h^r]A_N(X(1+h)),$$

where a coefficient with negative index is zero. The target congruence is equivalent to $\sum_{r<2s} L_r h^r = 0 \pmod{M}$. Substitute the already certified $h = pY/Z$ and clear the unit denominator:

$$\boxed{\sum_{r=0}^{2s-1} L_r (pY)^r Z^{2s-1-r} = 0 \pmod{p^{2s}}.} \tag{10.6}$$

Only at this last step do the ordinary period truncations enter the calculation. The program verifies every coefficient of (10.5) and (10.6) exactly.

n	p	s	N	working modulus	$(\deg Y, \deg Z)$	target bound
4	3	2	9	729	(521,520)	1562
4	3	3	81	59049	(14297,14296)	70519
5	5	2	25	15625	(301,300)	506

Both polynomial identities are zero. Multiplying the verified $\lambda F_{p^{s-1}}(\sigma) - F_{p^s}$ congruence by the unit $F(\sigma)$ proves the certificate assertions of Theorem 6.1.

11 A sharp obstruction for the uncorrected period functional

This section concerns the transfer functional used in [6, Section 4], rather than a counterexample to an excellent-lift truncation congruence. All constant terms below are taken in the positive t-adic expansion. Let p be odd, let $s \geq 1$, and put $q = p^s$ and $M = 2q$. For the n-dimensional octahedral polynomial set

$$g_n(\mathbf{x}) = \sum_{i=1}^{n}(x_i + x_i^{-1}), \qquad f = 1 - tg_n, \qquad B_n(j,1) = [x_1^2]g_n(\mathbf{x})^{2j},$$

where the coefficient at every other variable is zero.

Lemma 11.1 (A coefficient of exact first-order precision)**.** *For every $n \geq 1$, every odd prime p, and every $s \geq 1$,*

$$B_n(p^s, 1) \equiv 2(2-n)p^s \pmod{p^{s+1}}.$$

In particular, if $p \nmid n-2$, then $v_p(B_n(p^s,1)) = s$.

Proof. For a composition $k_1 + \cdots + k_n = q$ with $k_i \geq 0$ write $V(\mathbf{k}) = q!/(k_1! \cdots k_n!)$. Pairing the positive and negative steps in each coordinate gives

$$B_n(q,1) = \binom{2q}{q} \sum_{\substack{k_1+\cdots+k_n=q \\ k_1 \geq 1}} V(\mathbf{k})^2 \frac{k_1}{k_1+1}.$$

The identity

$$k_1 V(\mathbf{k}) = q\frac{(q-1)!}{(k_1-1)!k_2! \cdots k_n!}$$

implies $v_p(V(\mathbf{k})) \geq s - v_p(k_1)$. Thus each summand has valuation at least

$$2s - v_p(k_1) - v_p(k_1+1).$$

If $1 \leq k_1 \leq q-2$, at most one of the consecutive integers k_1, k_1+1 is divisible by p, and its valuation is at most $s-1$. Consequently all such summands vanish modulo p^{s+1}. The summand with $k_1 = q$ equals $q/(q+1)$. For $k_1 = q-1$ there are $n-1$ compositions, each contributing $q(q-1)$. Hence

$$\sum V(\mathbf{k})^2 \frac{k_1}{k_1+1} \equiv \frac{q}{q+1} + (n-1)q(q-1) \equiv (2-n)q \pmod{p^{s+1}}.$$

Finally, Lucas's congruence gives $\binom{2q}{q} \equiv 2 \pmod{p}$, which proves the claim. □

Define the period functional and its Cartier compatibility defect by

$$\rho_{M,g_n}(\omega) = \mathrm{CT}(g_n^M \omega), \qquad \mathcal{D}_M(\omega) = \rho_{M,g_n}(\omega) - \rho_{M/p,g_n}(\mathscr{C}_p\omega).$$

No coefficient Frobenius changes g_n, since its coefficients are integers. The Cartier operator on positive t-adic expansions keeps the coefficients of $\mathbf{x}^{p\mathbf{u}}$ and divides their exponent vectors by p.

Proposition 11.2 (Sharpness of the general transfer precision)**.** *There is a fixed second formal derivative $\eta \in \mathscr{F}_2$, independent of s, for which*

$$[t^3]\rho_{M,g_n}(\eta) = [t^3]\mathcal{D}_M(\eta) = 4B_n(p^s,1).$$

Thus, if $p \nmid n-2$, both displayed coefficients have valuation exactly s. Moreover there is a fixed signed-permutation-invariant $\eta_{\mathrm{sym}} \in \mathscr{F}_2$ for which

$$[t^3]\rho_{M,g_n}(\eta_{\mathrm{sym}}) = [t^3]\mathcal{D}_M(\eta_{\mathrm{sym}}) = 8nB_n(p^s,1).$$

If $p \nmid n-2$, the latter valuation is exactly $s + v_p(n)$.

Proof. Put $\theta_i = x_i\partial/\partial x_i$ and choose

$$\eta = \theta_1^2\left(\frac{t^3x_1^{-2}}{f^3}\right), \qquad \eta_{\mathrm{sym}} = \sum_{i=1}^{n}\theta_i^2\left(\frac{t^3(x_i^2+x_i^{-2})}{f^3}\right).$$

These elements lie in the integral Dwork module at odd p: the numerator exponents $\pm 2e_i$ lie in the interior of 3Δ, their t-adic orders satisfy admissibility, and the pole-order-three normalization factor $2! = 2$ is a p-adic unit. Applying Euler derivations preserves this module. Being second derivatives, the displayed elements belong to $\mathscr{F}_2$; equivalently their coefficients have the required squared exponent divisibility. The symmetric element is fixed by every coordinate permutation and coordinate inversion. Its construction uses a sum, with no division by the group order.

Their initial positive expansions are

$$\eta = 4t^3x_1^{-2} + O(t^4), \qquad \eta_{\mathrm{sym}} = 4t^3\sum_i(x_i^2+x_i^{-2}) + O(t^4).$$

Since p is odd, none of the exponent vectors $\pm 2e_i$ is divisible by p. Therefore both Cartier images are $O(t^4)$. Taking constant terms after multiplying by g_n^M, and using signed-permutation symmetry of g_n^M, proves the coefficient identities. The valuation claims now follow from Lemma 11.1. □

The same obstruction applies to the truncation kernel $h = tg_n$. For a lift $\sigma(t) = t^p v(t)$ with $v(0)$ a unit, replace the defect by

$$\rho_{M,h}(\omega) - \rho_{M/p,h^\sigma}(\mathscr{C}_p\omega).$$

The first term has initial coefficient $4B_n(p^s,1)$ (respectively $8nB_n(p^s,1)$) at t^{M+3}, whereas the second has order at least $M+4$. Thus the same exact valuation holds.

For $(n,p) = (4,3)$, even the symmetric example has valuation exactly s, so the raw period-functional Cartier compatibility cannot be uniformly upgraded to p^{s+1} on all second formal derivatives. For $(n,p) = (5,5)$ the symmetric example has valuation $s+1$, which is less than $2s$ for $s \geq 2$. If a factor p^2 is prefixed to the test element, the corresponding valuation is $s + v_p(n) + 2$; this still rules out a uniform p^{2s} conclusion for all sufficiently large s from membership in $p^2\mathscr{F}_2$ alone.

These are operator-level obstructions, not failures of the original supercongruence. The actual excellent Cartier error is a particular element of $p^2\mathscr{F}_2$ and could possess additional cancellation. The calculation shows precisely why such additional information, or a different transfer functional, is needed. It does not prove that no corrected period method can work.

Source of the transfer statement. Beukers–Vlasenko, *Dwork crystals II*, arXiv:1907.10390v3, Section 4, Proposition 4.1, states the period-functional compatibility modulo $p^{v_p(M)}$. Corollary 4.4 applies it to truncation matrices. The propositions above exhibit an explicit sharpness mechanism for this general functional estimate in the octahedral setting.

What the excellent error alone contributes. Suppose $\mathscr{C}_p(\omega) = \lambda\omega^\sigma + p^2\epsilon$ with $\epsilon \in \mathscr{F}_2$. Put $h = tg_n$ and $m = M/p$. Then the exact transfer decomposition is

$$\rho_{M,h}(\omega) - \lambda\rho_{m,h^\sigma}(\omega^\sigma) = \big(\rho_{M,h}(\omega) - \rho_{m,h^\sigma}(\mathscr{C}_p\omega)\big) + p^2\rho_{m,h^\sigma}(\epsilon).$$

The standard period-modulo-m estimate gives the second summand divisibility by p^{s+1}, since $\mathscr{F}_2 \subseteq \mathscr{F}_1$ and $v_p(m) = s-1$. The first summand is only guaranteed to be divisible by p^s by BV2, Proposition 4.1. Thus merely inserting the excellent Cartier identity into that general proof does not prove a gain beyond p^s. The exact coefficient examples above show why a uniform upgrade of the first functional estimate on $\mathscr{F}_2$ fails. They do not exclude cancellations for the distinguished element $\omega = 1/f$.

12 The interpolation obstruction and its subsequent resolution

The original matrix proof in Part I uses $p > n$ in its symmetric Bessel reduction. The support-weighted argument in Part III removes that restriction at $p \geq 5$. At $p = 3$ the normalized leading correction is $1 - 3(X^3 - X) \pmod 9$, which is pointwise $1 \pmod 9$ on integers but not coefficientwise $1 \pmod 9$. The dimension-polynomial argument in Part IV resolves this remaining obstruction at every level and every normalized multiplier.

The three certificates in this part bypass these steps at the stated parameter choices. Merely having an algorithm that constructs a finite target polynomial at each level would not prove its vanishing at unexecuted levels. Their role as individual whole-series proofs is unchanged by the later algebraic theorem.

One elementary source of potential compensation at primes dividing the dimension is the exact identity, for $j \geq 1$,

$$a_j = 2n \ \mathrm{CT}(x_1 g^{2j-1}) \in 2n\mathbb{Z}.$$

Indeed the $2n$ monomials of g contribute equal constant terms by signed-permutation symmetry. This does not by itself control the zero boundary index or the shifted N^2 errors in the row construction.

13 Verification and relation to the earlier manuscript

This part incorporates the first small-prime supplement into the complete edition. It supplies the all-odd-prime cube and simplex results, the common formal orbit-module input, and independent octahedral first-level and certificate arguments. Subsequent parts supply the full octahedral descent.

The new orbit-module and scalar results are symbolic arguments. The three specified octahedral levels are computer-assisted proofs: their reduction to finite polynomial identities is proved above, and the complete exact verifier is printed below. Finite power-series checks used during exploration are not premises of these theorems. The results remain subject to independent mathematical review; no historical priority or formal proof-assistant certification is claimed.

For reproduction, use Python 3 and execute

```
python3 verify_small_prime_certificates.py --dimension 4 --p 3 --s 2
python3 verify_small_prime_certificates.py --dimension 4 --p 3 --s 3
python3 verify_small_prime_certificates.py --dimension 5 --p 5 --s 2
```

The program writes each coefficient payload and a report. An assertion failure invalidates that run; a successful run verifies both the lift root identity and the cleared target identity. The embedded source regenerates the three full payloads and their reports; the independently archived payloads are not needed to compile this single-source edition. No third-party Python packages are needed.

Part III
Hyperoctahedral descent in arbitrary dimension

All primes at least five, the mixed-precision proof at three, and independent certificates through level four.

14 Arbitrary-dimensional descent and independent finite-level results

Put

$$g_n(\mathbf{x}) = \sum_{i=1}^{n}(x_i + x_i^{-1}), \quad f = 1 - zg_n, \quad u = z^2,$$

$$F_n(u) = \sum_{j\geq 0} a_{n,j}u^j, \qquad a_{n,j} = \frac{(2j)!}{(j!)^2} \sum_{k_1+\cdots+k_n=j} \binom{j}{k_1,\ldots,k_n}^2, \qquad F_{n,K}(u) = \sum_{j<K} a_{n,j}u^j.$$

Let σ_n be the normalized excellent lift of the canonical coordinate specified in Section 15. Define

$$E_{n,p,s,m}(u) = F_n(u)F_{n,mp^{s-1}}(\sigma_n(u)) - F_n(\sigma_n(u))F_{n,mp^s}(u).$$

All congruences in this paper are coefficientwise. The normalized length mp^s corresponds to the original length $2mp^s$ in the variable z; normalized $m = 1$ therefore corresponds to the multiplier 2 specified for octahedra in BV3, Conjecture 7.5.

Theorem 14.1 (All-level small-prime descent)**.** *For every dimension $n \geq 3$ and prime p satisfying either*

$$p \geq 5 \qquad \text{or} \qquad (p = 3 \text{ and } 3 \mid n),$$

and every positive integer K divisible by p, one has

$$\frac{F_{n,K}(u)}{F_n(u)} \equiv \frac{F_{n,K/p}(\sigma_n(u))}{F_n(\sigma_n(u))} \pmod{p^{2v_p(K)}\mathbb{Z}_p[[u]]}.$$

In particular $E_{n,p,s,m} \in p^{2s}\mathbb{Z}_p[[u]]$ for every $s, m \geq 1$, with no restriction on the coefficient index.

Theorem 14.2 (Dimension-uniform remaining levels)**.** *For every $n \geq 3$, at $p = 3$ and normalized multiplier $m = 1$,*

$$E_{n,3,s,1} \in 3^{2s}\mathbb{Z}_3[[u]] \qquad (s = 1, 2, 3, 4).$$

The first level is proved algebraically in Section 19. The three higher levels are proved by finite exact polynomial identities and the dimension argument of Sections 21 and 22. They do not constitute an induction over all s.

The following table records the scope of the arguments in this part; the global scope is Theorem 1.1.

Dimension	Prime 3	Every prime $p \geq 5$
$3, 6, 9, 12, \ldots$	Every s, m; every coefficient	Every s, m; every coefficient
$4, 5, 7, 8, 10, 11, \ldots$	$m = 1$, levels $s = 1, 2, 3, 4$; every coefficient	Every s, m; every coefficient

The two all-level methods of this part leave the sector $p = 3$, $n \geq 4$, $3 \nmid n$ untreated. The finite-level theorem supplies normalized $m = 1$ through $s = 4$. The complete all-level, all-multiplier statement in that sector is proved in Part IV; it is not inferred from these tests. Dimensions one and two already have all-odd-prime scalar proofs, since they agree with the corresponding cubic positive vertex matrices. This paper supplies new descent arguments beyond the prime range in the published BV3 theorem; its formal input is checked using the general machinery of that paper, not by silently removing a hypothesis from its Theorem 7.3.

15 The formal excellent Cartier input

Proposition 15.1 (Formal excellent input from an orbit module)**.** *Let $g = g_n$ be the hyperoctahedral Laurent polynomial and let p be an odd prime. Put $R = \mathbb{Z}_p[[z]]$ and $f = 1 - zg$. Let $F_0(z)$ and $G_0(z)$ be the period and logarithmic companion given by the vertex-coefficient formulas of BV3, Proposition 9.1. Then*

$$q_0(z) = z \exp(G_0(z)/F_0(z)) \in z + z^2\mathbb{Z}_p[[z]].$$

The lift $\tau = q_0^{-1}(q_0(z)^p)$ is integral and satisfies

$$\mathscr{C}_p(1/f) - \frac{F_0(z)}{F_0(\tau)} \frac{1}{f^\tau} \in p^2 \mathscr{F}_2^\tau.$$

In particular, no exclusion $p \mid n$ is needed for this formal input.

Proof. Write Γ for the support lattice and H for the full monomial symmetry group. Its order will never be inverted. The support lattice is $\mathbb{Z}^n$, so it is p-saturated. The symmetry group preserves the reflexive polytope degree $d(u) = \deg_\Delta(u)$ and the support lattice. For an H-orbit $O \subset \Gamma$, put

$$S_O = \sum_{u \in O} x^u, \qquad d(O) = d(u)\ (u \in O), \qquad L = \bigoplus_O R z^{d(O)} S_O.$$

Here a Laurent polynomial uses only finitely many orbits. This is a direct construction of a module of polynomials. We do not obtain it by applying an averaging projector to another crystal.

Fix temporarily any lift σ with $z^\sigma = z^p v$, $v \in 1 + pR$. We check BV3, Definition 2.1. The orbit sums have disjoint supports and coefficient one, so L is p-saturated. The same is true of every L^{σ^i}: its orbit coefficient generator is $(z^{\sigma^i})^{d(O)}$, a power of z times a unit, which is a non-zero-divisor modulo p. Multiplication by f preserves invariance and admissibility, hence $fL \subset L$.

The subset $p\Gamma$ is H-stable. An orbit is either disjoint from $p\Gamma$ or has the form pO' for one orbit O'; the equality uses the p-saturation of the support lattice. Since $d(pu) = pd(u)$,

$$\mathscr{C}_p(z^{d(pO')} S_{pO'}) = z^{pd(O')} S_{O'} = \left(\frac{z^p}{z^\sigma}\right)^{d(O')} (z^\sigma)^{d(O')} S_{O'} \in L^\sigma.$$

The image of any other orbit is zero. This proves compatibility without dividing by either $|H|$ or $|O|$.

Form the Dwork crystal $M = \widehat{\Omega}_{L,f}(\Delta^\circ)$ directly, as in BV3, Definitions 2.6 and 2.8. We do not identify it with the invariant part of an ambient completed module; in particular no exactness of an invariant functor is being assumed. In this family, $\Delta \cap \Gamma$ consists of the origin and the vertices. Reflexivity then implies that the first two levels have bases

$$M(1) = R/f, \qquad M(2) = R/f^2 \oplus Rzg/f^2.$$

Indeed $2\Delta^\circ \cap \Gamma = \Delta \cap \Gamma$ and the vertices form one orbit. The pair $1/f, zg/f^2$ is an extended basis, since $1/f = (1 - zg)/f^2$ and the change of basis has determinant one. These levels are p-saturated in the ambient integral formal expansion: multiplying an expansion by f or f^2 reduces the claim

to divisibility of the numerator coefficients. This also shows that the two basis vectors remain independent modulo p.

The Hasse–Witt computations of BV3, Lemma 7.2, now apply to this explicit compatible module. Their coefficient extraction uses a single vertex coefficient of a polynomial known to be constant on the vertex orbit; it does not average over that orbit. With the initial lift $z^\sigma = z^p$, the first determinant has constant term one, and

$$\left.\frac{\det HW^{(2)}(M)}{p}\right|_{z=0} = -\frac{1}{p}\binom{2p-2}{p-1} \in \mathbb{Z}_p^\times.$$

The latter computation reads the coefficient of $z^p x^{pv}$ at one vertex v in $(zg)F^{(2)}$; the extreme vertex forces the unique contribution $(-z)^p\binom{2p-2}{p-1}$. The normalized determinant modulo p is independent of the lift. Consequently both Hasse–Witt conditions hold over R for every lift under consideration. The general BV3, Corollary 5.9 and Corollary 4.3, therefore give

$$M = M(2) \oplus \mathscr{F}_2 M, \qquad \mathscr{C}_p(M) \subset M^\sigma(2) + p^2 \mathscr{F}_2 M^\sigma.$$

Only $p > 2$ is required by these general results.

For completeness, the remaining canonical-coordinate argument uses these statements rather than a group-order hypothesis. The derivation $\theta = z\,d/dz$ preserves M. Thus the rank-two quotient has the connection and Cartier matrices of BV3, Propositions 7.6 and 7.7. The positive vertex coefficient calculation of Proposition 9.1 gives exactly F_0 and $F_0 \log z + G_0$ as its two solutions. The same coefficient calculation at exponents $p^s v$ gives $A(0) = B(0) = 0$, the constant Cartier matrix $\operatorname{diag}(1, p)$ for the initial lift, and the upper-right coefficient

$$\lambda_1 = \frac{pF_0(z)F_0(z^\sigma)}{W(z^\sigma)}\left(-\frac{1}{p}\log\frac{z^\sigma}{z^p} + \frac{G_0(z)}{F_0(z)} - \frac{1}{p}\frac{G_0(z^\sigma)}{F_0(z^\sigma)}\right),$$

where $W = F_0^2 + F_0\theta G_0 - (\theta F_0)G_0$. The rank-one reduction modulo p implies $\lambda_1 \in pR$. The determinant identity $\det\Lambda = pW/W^\sigma$ and $W(0) = 1$ give $W \in 1 + zR$ by the same iteration as BV3, Proposition 7.9. For $z^\sigma = z^p$, the displayed formula and the Dieudonne–Dwork lemma yield $q_0 \in z + z^2R$. Its formal inverse is integral, so $\tau = q_0^{-1}(q_0^p)$ defines a lift with $\tau/z^p \in 1 + pR$. Reapply the compatible-module construction with this lift. The displayed expression has $\lambda_1 = 0$, and the remaining coefficient is F_0/F_0^τ. This proves the required excellent Cartier identity in $p^2\mathscr{F}_2 M^\tau$.

Finally, membership in $\mathscr{F}_2 M$ means that every s-fold Cartier iterate lies in $p^{2s}M^{\sigma^s}$. Mapping the directly constructed module to the ambient Dwork module therefore sends $\mathscr{F}_2 M$ into the ambient $\mathscr{F}_2$. Only this inclusion is needed; no converse assertion about invariants or completed modules is used. □

The sign symmetry gives $q_0(z) = zQ(z^2)$ and $\tau(-z) = -\tau(z)$. Thus $\tau = z^p V(u)$ with $V \in 1 + p\mathbb{Z}_p[[u]]$, and

$$\sigma_n(u) = \tau(z)^2 = u^p V(u)^2.$$

No division by the dimension occurs in this normalization. The corresponding canonical coordinate in u is $q(u) = q_0(z)^2$.

16 Positive vertices and the exact factorial block

Suppress the subscript n when it is fixed. For $0 \le N \le j$ put

$$B(j, N) = (2j)! \sum_{\substack{k_1+\cdots+k_n=j \\ k_1 \ge N}} \frac{1}{(k_1 - N)!(k_1 + N)! \prod_{i>1}(k_i!)^2},$$

and set $B(j,N)=0$ if $N>j$. These are integers, $B(j,j)=1$, and $B(j,0)=a_j$. Let

$$\Omega_N(u)=\sum_{j\geq N}B(j,N)u^j,\qquad C_N=\Omega_N/F\in u^N\mathbb{Z}_p[[u]].$$

The series Ω_N is the coefficient at x_1^{2N}, with all other exponents zero, in the positive expansion of $1/f$. Reading Proposition 15.1 at that exponent and using the coefficient characterization of second formal derivatives gives

$$C_{pN}-C_N\circ\sigma_n\in p^{2+2v_p(N)}\mathbb{Z}_p[[u]]. \tag{16.1}$$

Here the exponent factor is $(2N)^2$ and 2 is a p-adic unit.

The unit triangular system uniquely defines integers $c_{K,N}$ by

$$a_j\mathbf{1}_{j\geq K}=\sum_{N=K}^{j}c_{K,N}B(j,N),\qquad c_{K,N}=0\quad(N<K).$$

Consequently

$$1-F_K/F=\sum_{N\geq K}c_{K,N}C_N. \tag{16.2}$$

This identity converges u-adically. It is the connection between Cartier's coefficient congruence and ordinary truncations.

Use the Bessel series and the factorial coefficient lattice

$$I_N(x)=\sum_{k\geq N}\frac{x^k}{(k-N)!(k+N)!},\quad \Phi_N=I_NI_0^{n-1},\quad \theta=x\frac{d}{dx},\quad \mathcal{H}=\left\{\sum h_jx^j:(2j)!h_j\in\mathbb{Z}_p\right\}.$$

Then $(\theta^2-N^2)I_N=xI_N$ and $(2j)![x^j]\Phi_N=B(j,N)$. The lattice is complete for the factorial coefficient norm; multiplication, x, and θ preserve it. This follows from the integrality of the relevant multinomial factors. For $f_1=I_N$, $f_i=I_0$ for $i>1$, put

$$E_l=\sum_{|S|=l}\prod_{i\in S}\theta f_i\prod_{i\notin S}f_i.$$

Then $E_0=\Phi_N$, $E_1=\theta\Phi_N$, and

$$\theta E_l=(l+1)E_{l+1}+(n-l+1)xE_{l-1}\pmod{N^2\mathcal{H}}.$$

The omitted term comes from differentiating the first factor and is N^2 times an integral product of the displayed factors.

16.1 A balanced unit-factorial estimate at every odd prime

Write $U(a)=\prod_{1\leq i\leq a,\,p\nmid i}i$. For $H=pN$ and $k\geq H$,

$$\frac{U(k)^2}{U(k-H)U(k+H)}=\prod_{\substack{k-H<i\leq k\\ p\nmid i}}(1+H/i)^{-1}\equiv 1\pmod{p^{2+2v_p(N)}}.$$

Indeed, for $v=v_p(H)$ the unit residues in this interval are complete copies modulo p^v. Their reciprocals sum to zero modulo p^v by pairing opposite residues. Expanding the product to first order proves the estimate.

16.2 An integral analytic unit-factorial expansion

Write $\mathbb{Z}_p\langle X_1, \dots, X_d\rangle$ for the restricted power series ring: its coefficients tend to zero p-adically as their total degree tends to infinity. For a nonnegative integer a, put

$$U(a) = \prod_{\substack{1 \le i \le a \\ p \nmid i}} i, \qquad c = (p-1)!.$$

Lemma 16.1 (Unit-factorial interpolation)**.** *For $p \ge 5$ and every $0 \le r < p$, there is $T_r(X) \in \mathbb{Z}_p\langle X\rangle$ such that, for all integers $\ell \ge 0$,*

$$U(p\ell + r)^{-2} = c^{-2\ell}(r!)^{-2}\bigl(1 - 2pH_r\ell + p^2 T_r(\ell)\bigr), \qquad H_r = \sum_{a=1}^{r} \frac{1}{a}. \tag{16.3}$$

Proof. The unit factorial over one block has the exact paired form

$$Q(k) := \frac{U(p(k+1))}{cU(pk)} = \prod_{a=1}^{(p-1)/2} \left(1 + \frac{p^2 k(k+1)}{a(p-a)}\right).$$

Thus $\log Q(X)$ is a sum, indexed by $m \ge 1$, of polynomials of degree at most $2m$ whose coefficients lie in $p^{2m - v_p(m)}\mathbb{Z}_p$. The polynomial $S_d(X)$, equal to $\sum_{0 \le k < \ell} k^d$ at nonnegative integers $X = \ell$, has coefficient denominators dividing $\operatorname{lcm}(1, \dots, d+1)$. Indeed, expansion in falling factorials gives

$$k^d = \sum_{a=0}^{d} \left\{ {d \atop a} \right\} k(k-1)\cdots(k-a+1),$$

and the sum of the falling factorial of degree a is the falling factorial of degree $a+1$, divided by $a+1$. Taking this discrete antiderivative term by term in $\log Q$, the m-th contribution therefore has coefficient valuation at least

$$2m - v_p(m) - \lfloor \log_p(2m+1) \rfloor \ge 2.$$

The bound tends to infinity with m, since $p \ge 5$. We obtain a restricted series $A(X) \in p^2\mathbb{Z}_p\langle X\rangle$ such that

$$A(\ell) = \sum_{k=0}^{\ell-1} \log Q(k) = \log\bigl(U(p\ell)/c^{\ell}\bigr).$$

Exponentiation converges in the restricted series ring and gives $\exp(-2A) \in 1 + p^2\mathbb{Z}_p\langle X\rangle$. Finally,

$$U(p\ell + r) = U(p\ell)\, r! \prod_{a=1}^{r} (1 + p\ell/a),$$

and

$$\prod_{a=1}^{r} (1 + pX/a)^{-2} \in 1 - 2pH_r X + p^2\mathbb{Z}_p\langle X\rangle.$$

Combining these identities proves the lemma. □

16.3 The block formula used in both arguments

For the prime in use, put $c = (p-1)!$ and choose restricted functions W_r interpolating

$$U(p\ell + r)^{-2} = c^{-2\ell}(r!)^{-2} W_r(\ell).$$

The preceding lemma gives them for $p \geq 5$; Section 18 gives the correct version for $p = 3$. Write $j = pL + b$, $0 \leq b < p$, $\epsilon = \lfloor 2b/p \rfloor$. Splitting each $k_i = p\ell_i + r_i$ in the formula for $B(j, pN)$ gives a carry d with

$$\sum r_i = b + pd, \quad \sum \ell_i = L - d, \quad 0 \leq d \leq n - 1.$$

Define

$$P_{b,d}(\mathbf{X}) = \sum_{\substack{0 \leq r_i < p \\ \sum r_i = b + pd}} \prod_i (r_i!)^{-2} W_{r_i}(X_i).$$

The factorial identity $k! = p^{\lfloor k/p \rfloor} \lfloor k/p \rfloor! U(k)$ and the balanced estimate give

$$B(j, pN) \equiv p^{\epsilon} U(2j)(2L + \epsilon)! \sum_{d=0}^{\min(n-1, L)} p^{2d} c^{-2(L-d)} [x^{L-d}] \tag{16.4}$$
$$P_{b,d}(\theta_1, \ldots, \theta_n) I_N(x_1) \prod_{i>1} I_0(x_i) \Big|_{x_i = x} \pmod{p^{2+2v_p(N)}}.$$

At $N = 0$ the equality is exact. The power of p extracted from the factorials is $2d + \epsilon$, and the remaining numerator is $(2L + \epsilon)!$. Each original summand and the modified summand is p-integral, so the balanced replacement is valid term by term. The reduction of this symmetric operator is the only part that previously required $p > n$.

17 Support-weighted symmetric reduction in arbitrary dimension

Fix any dimension $n \geq 3$ and any prime $p \geq 5$. No inequality between p and n will be imposed. We use the unit-factorial interpolation and the positive vertex matrix from the preceding section.

17.1 The precise denominator of scalar reduction

For a monomial in $X_1, \ldots, X_n$, its support is the set of variables with positive exponent. Consider a symmetric restricted series P_k all of whose nonzero monomials have support of size exactly k. Applying its differential operator to $I_N(x_1) \prod_{i>1} I_0(x_i)$, and using

$$\theta_i^2 I_0 = x_i I_0, \qquad \theta_1^2 I_N = (x_1 + N^2) I_N,$$

reduces it, modulo $N^2 \mathcal{H}$, to a combination of E_l for $0 \leq l \leq k$. This support bound holds because reducing one factor cannot differentiate a formerly undifferentiated factor. All reduction coefficients and commutator coefficients are integral.

Set $D_l = l! E_l$. The elementary-derivative recurrence gives

$$D_0 = \Phi_N, \quad D_1 = \theta \Phi_N, \qquad D_{l+1} = \theta D_l - l(n - l + 1) x D_{l-1} \pmod{N^2 \mathcal{H}}.$$

More precisely, the omitted term is N^2 times an integral combination of products of the indicated Bessel factors and their first derivatives. Induction therefore expresses D_l by an integral scalar differential polynomial in x, θ, with an error in $N^2 \mathcal{H}$. Hence expressing E_l loses at most $v_p(l!)$, both in its scalar coefficients and in the error. It does not lose an additional factorial at each recurrence step.

It follows that a symmetric support-k input in $p^k \mathbb{Z}_p \langle \mathbf{X} \rangle$, for $k \geq 2$, has a scalar reduction with all coefficients and its N^2 error divisible by p^2: indeed

$$k - v_p(l!) \geq k - v_p(k!) \geq 2 \qquad (0 \leq l \leq k, p \geq 3, k \geq 2).$$

A support-one input in $p^2 \mathbb{Z}_p \langle \mathbf{X} \rangle$ has the same property, since only E_0, E_1 occur and no denominator is introduced. Constants and a multiple of $\sum_i X_i$ reduce exactly to the corresponding constant and multiple of θ, with no positive power of x.

These statements extend to restricted series: for fixed n, the only denominators are among $1!, \dots, n!$, and all other operations are integral and bounded. Coefficients tending to zero before reduction still tend to zero afterwards. The scalar coefficient series in x are therefore restricted as well.

17.2 Application to the actual unit-factorial operator

For $0 \le r < p$, unit-factorial interpolation gives

$$W_r(X) = 1 - 2pH_rX + p^2T_r(X), \qquad T_r \in \mathbb{Z}_p\langle X\rangle.$$

Evaluation at $\ell = 0$ gives $T_r(0) = 0$, hence $W_r(0) = 1$. For $b < p$ and a carry d, the operator used in the block formula is

$$P_{b,d}(\mathbf{X}) = \sum_{\substack{0\le r_i<p \\ \sum r_i = b+pd}} \prod_i (r_i!)^{-2} W_{r_i}(X_i).$$

After splitting each W_r into its constant term and its positive-degree part, a term with exactly k positive-degree factors has support k and a factor p^k. The symmetry of the residue-index set makes each support component symmetric. For support one, its linear part is an affine symmetric operator and every term of degree at least two has a factor p^2. These are exactly the cases just covered.

Consequently there are restricted scalar coefficient series $A_{d,0}(x), \dots, A_{d,n}(x) \in \mathbb{Z}_p\langle x\rangle$ such that

$$P_{b,d}(\theta_1, \dots, \theta_n) I_N(x_1) \prod_{i>1} I_0(x_i)\Big|_{x_i=x} - \sum_{k=0}^{n} A_{d,k}(x)\theta^k \Phi_N \in p^2 N^2 \mathcal{H}.$$

All positive-degree coefficients of the $A_{d,k}$ lie in $p^2\mathbb{Z}_p$. At $N = 0$ the identity is exact. This conclusion is asserted only for the displayed factorial operators; it need not hold for an arbitrary symmetric input of the form $\alpha + \beta \sum X_i + p^2 R$ when $p \le n$.

17.3 The row operator and the complete descent

Writing $j = pL + b$, $\epsilon = \lfloor 2b/p \rfloor$, and $A_{d,k}(x) = \sum_a \alpha_{d,a,k} x^a$, define

$$D_{j,i} = p^{\epsilon} U(2j) \frac{(2L+\epsilon)!}{(2i)!} \sum_{d=0}^{\min(n-1, L-i)} p^{2d} c^{-2(L-d)} \sum_{k=0}^{n} \alpha_{d,L-d-i,k} i^k.$$

The exact factorial-block decomposition and the preceding reduction give, without any further use of $p > n$,

$$a_j = \sum_{i \le L} D_{j,i} a_i,$$
$$B_n(j, pN) - \sum_{i \le L} D_{j,i} B_n(i, N) \in p^{2+2v_p(N)} \mathbb{Z}_p.$$

The coefficients $D_{j,i}$ are integral. If $i < L$, each summand either has a carry $d \ge 1$ and thus a factor p^2, or has positive x-degree and thus a factor p^2 from the scalar reduction. Therefore

$$D_{j,i} \in p^2 \frac{(2L+\epsilon)!}{(2i)!} \mathbb{Z}_p, \qquad D_{j,i} a_i \in p^2 (L!/i!)^2 \mathbb{Z}_p.$$

Define the integral coefficients $c_{K,N}$ by

$$a_j \mathbf{1}_{j \ge K} = \sum_{N=K}^{j} c_{K,N} B_n(j, N).$$

For completeness, the descent now proceeds exactly as follows. Induct on $v = v_p(K)$. At $v = 0$ integrality suffices. If $p \mid K$, put $M = K/p$. By the induction hypothesis,

$$v_p(c_{M,N}) \geq \max(0, 2(v-1) - 2v_p(N)).$$

Hence every row error in replacing $B_n(j, pN)$ by its D-combination, after multiplication by $c_{M,N}$, has valuation at least $2v$. For $j \geq K$, the omitted boundary terms satisfy

$$\sum_{i<M} D_{j,i} a_i \in p^{2v}\mathbb{Z}_p,$$

because $M \mid L!/i!$. For $j < K$ both triangular sums vanish. Thus the vector supported on multiples of p, with entries $c_{M,N/p}$, solves the defining triangular system for $c_{K,N}$ modulo p^{2v}. Since B_n is unit triangular,

$$c_{K,N} \equiv \mathbf{1}_{p|N} c_{M,N/p} \pmod{p^{2v}}.$$

This also gives $v_p(c_{K,N}) \geq \max(0, 2v_p(K) - 2v_p(N))$, completing the induction.

The exact identity

$$1 - F_K/F = \sum_{N \geq K} c_{K,N} C_N$$

and excellent Cartier's coefficient congruence

$$C_{pN} - C_N \circ \sigma \in p^{2+2v_p(N)}\mathbb{Z}_p[[u]]$$

now imply

$$\frac{F_K(u)}{F(u)} \equiv \frac{F_{K/p}(\sigma(u))}{F(\sigma(u))} \pmod{p^{2v_p(K)}}.$$

Together with Proposition 15.1, this proves that every octahedral dimension $n \geq 3$ satisfies the all-level, all-multiplier, every-coefficient truncation congruence at every prime $p \geq 5$.

18 Mixed-precision descent at the prime three

Throughout this section $3 \mid n$, and $B_n(j, N)$, $a_j = B_n(j, 0)$, I_N, $\Phi_N = I_N I_0^{n-1}$ and $c_{K,N}$ have their usual meanings. We write $v = v_3$.

18.1 The support-weighted reduction

Let $\mathcal{H} = \{\sum h_j x^j : (2j)! h_j \in \mathbb{Z}_3\}$. For a symmetric operator whose monomials involve exactly k distinct variables, reduction by $\theta_i^2 I_N = (x_i + \delta_{i1} N^2) I_N$ uses elementary derivative products E_l only for $0 \leq l \leq k$. Put $R_l = l! E_l$. Suppressing the N^2 terms, their recurrence is

$$R_0 = \Phi_N, \quad R_1 = \theta \Phi_N, \qquad R_{l+1} = \theta R_l - l(n-l+1) x R_{l-1}.$$

Consequently conversion to scalar derivatives loses at most $v(l!)$, including in the error term. If the input of support k is multiplied by 3^k, then for $k \geq 2$ its output and error retain a factor 9, because $k - v(l!) \geq k - v(k!) \geq 2$. This reduction is bounded for fixed n and extends from polynomials to restricted series.

An additional observation will control the coefficient of θ^0. For $l \geq 1$, the scalar differential polynomial expressing R_l has all its positive-x-degree θ^0 coefficients divisible by n. This follows directly from the recurrence above, evaluated on 1. For $l = 0$, a symmetric support-k input contributes a multiple of $\binom{n}{k}$. Since

$$\binom{n}{k} = \frac{n}{k}\binom{n-1}{k-1}, \qquad k - v(k) \geq 2 \quad (k \geq 2),$$

the positive-x-degree scalar constant coefficient from a 3^k support-k input is divisible by $9n$ in $\mathbb{Z}_3$. For a support-one input with coefficients in $9\mathbb{Z}_3$, the same conclusion is immediate. Thus all these good terms have positive-x coefficients divisible by 9, and their θ^0 coefficients are divisible by $9n$.

18.2 Isolating the cubic term

Write $U(a)=\prod_{1\le i\le a,\,3\nmid i} i$ and $c=2$. For $0\le r<3$ there are restricted integral series T_r such that

$$U(3\ell+r)^{-2}=c^{-2\ell}(r!)^{-2}W_r(\ell),\qquad W_r(X)=1-6H_rX-3(X^3-X)+9T_r(X).$$

Indeed, the first term in the discrete antiderivative of the paired unit-factorial logarithm is $\frac{3}{2}(X^3-X)$; all subsequent terms have coefficient valuation at least 3. In exponentiating, every power at least two has valuation at least 2 after its factorial denominator. Multiplication by $\prod_{a\le r}(1+3X/a)^{-2}$ gives the formula.

For $b\in\{0,1,2\}$ and a carry d, put

$$P_{b,d}(\mathbf{X})=\sum_{\substack{0\le r_i<3\\ \sum r_i=b+3d}}\prod_i (r_i!)^{-2}W_{r_i}(X_i),\qquad \alpha_{b,d}=[y^{b+3d}](1+y+y^2/4)^n.$$

Apart from an affine symmetric operator and the single term

$$-3\alpha_{b,d}\sum_i (X_i^3-X_i),$$

all terms are good in the support-weighted sense just proved: support one has a factor 9, and support $k\ge 2$ has a factor 3^k. The exceptional term has the exact reduction

$$\sum_i(\theta_i^3-\theta_i)\prod_i f_i=nx\Phi_N+(x-1)\theta\Phi_N+N^2(\theta I_N)I_0^{n-1},$$

where $f_1=I_N$ and $f_i=I_0$ for $i>1$. It therefore loses only one factor of 3, explicitly and controllably.

18.3 A row operator with two precisions

There are integral coefficients $D_{j,i}$ for $0\le i\le L=\lfloor j/3\rfloor$ with the following properties:

$$\begin{aligned} a_j&=\sum_{i\le L}D_{j,i}a_i,\\ B_n(j,3N)-\sum_{i\le L}D_{j,i}B_n(i,N)&\in 3^{1+2v(N)+\mathbf{1}_{3\nmid j}}\mathbb{Z}_3,\\ D_{j,i}a_i&\in 3^{2+\mathbf{1}_{3\nmid j}}(L!/i!)^2\mathbb{Z}_3\qquad (i<L).\end{aligned}$$

To prove this, use the residue decomposition $j=3L+b$ and the exact factorial-block expression (16.4). Its carry-d term has the prefactor $3^{\epsilon+2d}U(2j)(2L+\epsilon)!c^{-2(L-d)}$, where $\epsilon=\lfloor 2b/3\rfloor$. Apply the preceding support-weighted reduction and the displayed exact cubic reduction, and define $D_{j,i}$ by collecting the coefficients of $\theta^k\Phi_N$ as in the ordinary row construction. The balanced unit-factorial replacement has precision $3^{2+2v(N)}$ and hence is sufficient throughout.

The exceptional error only matters for $d=0$. For $b=1$ one has $\alpha_{1,0}=n\in 3\mathbb{Z}_3$, and for $b=2$ the outer factor 3^ϵ supplies the missing 3. This proves the stated error. At $N=0$ all reductions are exact.

For the boundary bound, first take $i>0$. Write

$$a_i=\frac{(2i)!}{(i!)^2}b_n(i),\qquad b_n(i)=\sum_{k_1+\cdots+k_n=i}\binom{i}{k_1,\dots,k_n}^2.$$

The integer $b_n(i)$ is divisible by n: it counts pairs of words with the same content, and cyclically relabelling all n letters acts freely on such pairs when $i>0$. Good positive-x terms provide 9,

and the exceptional term provides $3\alpha_{b,0}$; the factor $n \mid b_n(i)$ supplies the extra 3. For $b = 1$ its $\alpha_{1,0} = n$, and for $b = 2$ the outer 3 supplies the additional precision. Terms with $d \geq 1$ already have 3^{2d}. After multiplication by a_i, the factorial factor is an integer multiple of $(L!/i!)^2$, exactly as in the original boundary argument.

For $i = 0$ and positive x-degree, use the stronger divisibility of the scalar constant coefficient: good terms have $9n$, and the exceptional one is $-3\alpha_{b,d}nx$. The only remaining terms have $d = L \geq 1$ and x-degree zero. If $L \geq 2$, 3^{2L} already supplies more than the required extra power. If $L = d = 1$, the required 9 is present; for $b = 2$ there is the outer 3; for $b = 1$, the coefficient $\alpha_{1,1} = [y^4](1 + y + y^2/4)^n$ is divisible by 3, since $3 \mid n$ and this polynomial has, modulo 3, only degrees divisible by 3. This completes the boundary estimate.

18.4 The preserved triangular lattice

For $3 \mid n$, Lucas' theorem applied to the factorial summands gives

$$B_n(j, 3N) \equiv a_b B_n(\lfloor j/3 \rfloor, N) \pmod 3, \qquad b = j \bmod 3.$$

Here $a_1 = 2n \equiv 0$ and $a_2 = 6n(2n-1) \equiv 0$ modulo 3. In particular $B_n(j, N) \in 3\mathbb{Z}_3$ when $3 \nmid j$ and $3 \mid N$. Therefore the unit-triangular matrix B_n and its inverse preserve the lattice

$$\{(e_j) : e_j \in 3^{q+\mathbf{1}_{3\nmid j}}\mathbb{Z}_3\}.$$

We prove simultaneously, by induction on $v(K)$, that

$$v(c_{K,N}) \geq 1 + \max\{0, 2v(K) - 2v(N)\},$$

and, if $3 \mid K$,

$$c_{K,N} - \mathbf{1}_{3|N} c_{K/3,N/3} \in 3^{2v(K)+\mathbf{1}_{3\nmid N}}\mathbb{Z}_3.$$

The base case uses $3 \mid a_j$ for $j > 0$ and unit-triangular inversion. Set $M = K/3$. The row operator and the induction hypothesis imply that

$$a_j \mathbf{1}_{j \geq K} - \sum_{N \geq M} c_{M,N} B_n(j, 3N) \in 3^{2v(K)+\mathbf{1}_{3\nmid j}}\mathbb{Z}_3.$$

The boundary terms have this precision because $M \mid L!/i!$ for $i < M \leq L$; the row errors have this precision because

$$1 + 2v(N) + 1 + \max(0, 2v(M) - 2v(N)) \geq 2v(K).$$

The additional power for $3 \nmid j$ is supplied by the row lemma. The preserved lattice gives the asserted refined descent. For $3 \nmid N$ it gives the enhanced weighted bound directly. For $3 \mid N$, the induction bound on $c_{M,N/3}$ supplies the desired bound, and the error of valuation at least $2v(K)$ is stronger than $1 + \max(0, 2v(K) - 2v(N))$.

Finally combine ordinary descent modulo $3^{2v(K)}$, the exact tail expansion, and excellent Cartier's congruence $C_{3N} - C_N \circ \sigma \in 3^{2+2v(N)}\mathbb{Z}_3[[u]]$. This proves the complete all-level, all-multiplier, all-coefficient octahedral truncation congruence at $p = 3$ whenever $3 \mid n$.

19 The first level

We record the geometric first-level argument for completeness. Write

$$c_j = \mathrm{CT}(g^j), \quad \mathcal{F}(z) = \sum_{j \geq 0} c_j z^j, \quad H_N(z) = \mathrm{CT}(f^{N-1}), \quad \lambda = \mathcal{F}(z)/\mathcal{F}(\tau(z)).$$

In this section $\sigma_p(z) = \tau(z)$ and $\sigma_p(u) = \sigma(u)$ denote the two normalizations. Proposition 15.1 supplies the excellent Cartier identity at every odd prime.

Lemma 19.1 (Second-order geometric approximation)**.** *For the above excellent lift,*

$$\lambda \equiv 2H_p - H_{2p} \pmod{p^2\mathbb{Z}_p[[z]]}.$$

More precisely, write

$$\mathscr{C}_p(f^{2p-1}) = H_{2p}(z) + B_p(z)g(\mathbf{x}).$$

Then

$$\tau \equiv -\frac{B_p(z)}{H_{2p}(z)} \pmod{p^2\mathbb{Z}_p[[z]]}.$$

Proof. All the calculations below may be made in the ring of Laurent series whose coefficients are z-adically convergent integral power series. Put $h(\mathbf{x}) = f^\sigma(\mathbf{x}^p)$. Since $h - f^p$ is divisible by p, the exact identity

$$\frac{1}{f} - \frac{2hf^{p-1} - f^{2p-1}}{h^2} = \frac{(h-f^p)^2}{fh^2}$$

has an integral remainder divisible by p^2. Applying the Cartier operator and using $\mathscr{C}_p(A(\mathbf{x})/h(\mathbf{x})^2) = \mathscr{C}_p(A)/f^\sigma(\mathbf{x})^2$, we obtain

$$\mathscr{C}_p(1/f) \equiv \frac{2f^\sigma \mathscr{C}_p(f^{p-1}) - \mathscr{C}_p(f^{2p-1})}{(f^\sigma)^2} \pmod{p^2}.$$

We justify the two numerator supports explicitly. Let Γ be the lattice spanned by the support of g. Since its index is prime to p, $p\mathbf{u} \in \Gamma$ implies $\mathbf{u} \in \Gamma$. Reflexivity gives integral facet functions L_j with $\Delta = \{L_j \le 1\}$. A lattice point in $(1-1/p)\Delta$ has $L_j \le 0$ for every j, and hence is zero. A lattice point in $(2-1/p)\Delta$ has $L_j \le 1$, and hence belongs to Δ. The only points of $\Delta \cap \Gamma$ are zero and the vertices. Complete symmetry therefore yields the exact identities

$$\mathscr{C}_p(f^{p-1}) = H_p, \qquad \mathscr{C}_p(f^{2p-1}) = H_{2p} + B_p g.$$

The excellent Cartier relation $\mathscr{C}_p(1/f) \equiv \lambda/f^\sigma \pmod{p^2\mathscr{F}_2^\sigma}$ is in particular a coefficientwise congruence modulo p^2. Multiplying by $(f^\sigma)^2$ gives

$$\lambda(1-\tau g) \equiv 2H_p(1-\tau g) - H_{2p} - B_p g \pmod{p^2}.$$

Since $\mathrm{CT}(g) = 0$, comparison of the coefficients of 1 and g first gives $\lambda \equiv 2H_p - H_{2p}$, and then $\tau H_{2p} \equiv -B_p$. The latter can be divided by the unit H_{2p}, whose constant term is one. □

Theorem 19.2 (The first level at every odd prime)**.** *Set $h = 2$ for the hyperoctahedral family. At every odd prime,*

$$\frac{\mathcal{F}(z)}{\mathcal{F}(\sigma_p(z))} \equiv \mathcal{F}_{hp}(z) \pmod{p^2\mathbb{Z}_p[[z]]}.$$

Equivalently, in the normalized variable $u = z^h$,

$$\frac{F(u)}{F(\sigma_p(u))} \equiv F_p(u) \pmod{p^2\mathbb{Z}_p[[u]]}.$$

These are the $s = 1$ cases of the specified original truncation lengths; $\mathcal{F}_h = 1$.

Proof. The coefficient of z^j in $2H_p - H_{2p}$ is

$$c_j w_j, \qquad w_j = (-1)^j \left(2\binom{p-1}{j} - \binom{2p-1}{j}\right),$$

with binomial coefficients zero past the top index. If $0 \le j < p$, the product expansion

$$(-1)^j \binom{ap-1}{j} = \prod_{i=1}^{j} \left(1 - \frac{ap}{i}\right) \equiv 1 - apH_j \pmod{p^2}$$

shows that $w_j \equiv 1 \pmod{p^2}$. For $p \le j < 2p$, Lucas's congruence gives $w_j \equiv 1 \pmod{p}$.

The remaining required divisibilities follow directly from the factorial formulas. In the octahedral case,

$$c_{2k} = \sum_{k_1+\cdots+k_n=k} \frac{(2k)!}{(k_1!\cdots k_n!)^2},$$

and every denominator is prime to p when $k < p$; hence the numerator is divisible by p whenever $2k \ge p$. Thus $p \mid c_j$ for $p \le j < 2p$: odd j give zero, and for even $j = 2k$ in this range the numerator $(2k)!$ is divisible by p while every denominator is a unit. It follows coefficientwise that

$$2H_p - H_{2p} \equiv \mathcal{F}_{hp} \pmod{p^2}.$$

Apply Lemma 19.1 and then the normalized coordinate relation $q_0(z)^h = q(z^h)$. □

20 Finite polynomial certificates for entire series

20.1 Vertex polynomials and their derivative estimate

Define

$$b_j = \sum_{k_2+\cdots+k_n=j} \binom{j}{k_2,\ldots,k_n}^2, \qquad A_K(u) = \sum_{j=0}^{K-1} u^j \sum_{\ell=0}^{j} (-1)^\ell \frac{(2K-\ell-1)!\, b_{j-\ell}}{(2K-2j-1)!\,\ell!\,(j-\ell)!^2}.$$

The factorial ratio is an integer multinomial coefficient and $A_K(0) = 1$. Expanding $1/f$ in the chamber with dominant vertex x_1, the negative of the coefficient of x_1^{-2K}, all other exponents being zero, is

$$P_{2K}(u) = u^{-K} A_K(u).$$

Let $\lambda = F(u)/F(\sigma(u))$. Reading this vertex coefficient in the excellent Cartier congruence gives

$$P_{2pK}(u) - \lambda P_{2K}(\sigma(u)) \in p^{2+2v_p(K)} \mathbb{Z}_p((u)). \tag{20.1}$$

The displayed elements have bounded negative support. No claim that every element of the completed coefficient ring is a Laurent series with bounded negative support is needed.

Lemma 20.1. *For $C_K = A_{2K}/A_K$ and $\theta = u\, d/du$,*

$$\theta C_K \in p^{v_p(K)} \mathbb{Z}_p[[u]].$$

Proof. Set $R_K = P_{2K}/F$ and use the normalized canonical coordinate q, with $\partial = q\, d/dq$. The second-order quotient relation [7, Proposition 7.6], read at the same vertex, gives after division by F and passage to q

$$\partial^2 R_K \in p^{2v_p(K)} \mathbb{Z}_p((q)).$$

More explicitly, in normalized coordinates let $\mathcal{L} = \theta^2 - B_u\theta - A_u$ and $H = \theta q/q = W/F^2$. Then $\theta H/H = B_u - 2\theta F/F$ and the exact identity

$$\mathcal{L}(FR) = FH^2\partial^2 R$$

shows that this passage preserves the required precision. The gauge and coordinate factors are integral units by [7, Proposition 7.9 and Corollary 7.11]; the normalization factor 2 is a p-adic unit. For $r = v_p(K)$, integrality of R_K together with $j^2[q^j]R_K \in p^{2r}\mathbb{Z}_p$ implies $j[q^j]R_K \in p^r\mathbb{Z}_p$ for every nonzero integer j. Thus ∂R_K, and then θR_K, belong to $p^r\mathbb{Z}_p((q))$ and $p^r\mathbb{Z}_p((u))$, respectively. Differentiate $C_K = u^K R_{2K}/R_K$; the factors after removing their displayed powers of u are integral units. Every term has the factor p^r, giving the assertion. □

20.2 The scaled Hensel equation

Fix an odd prime p, dimension n, and level $s \geq 2$, and set

$$M = p^{2s}, \quad M_y = p^{2s-1}, \quad N = p^{2s-2}, \quad X = u^p, \qquad \sigma(u) = X(1+py(u)), \quad y \in \mathbb{Z}_p[[u]].$$

Eliminating λ between (20.1) for $K = N, 2N$ gives

$$(1+py)^N C_{pN}(u) - C_N(X(1+py)) \in p^{4s-2}\mathbb{Z}_p[[u]].$$

Divide this expression by pN. Its coefficient of y^r, for $r \geq 1$, has valuation at least

$$r - 1 - v_p(r). \tag{20.2}$$

Indeed Lemma 20.1 implies that the rth Taylor coefficient of $C_N(X(1+h))$ lies in $(N/r)\mathbb{Z}_p[[u]]$; also $\binom{N}{r} \in (N/r)\mathbb{Z}_p$. Multiplication by $p^r/(pN)$ gives (20.2). The constant coefficient is integral as well, since the actual y solves the preceding congruence and all positive-degree coefficients are integral.

For $p = 3$, take $d = 3$ at $s = 2$ and $d = 6$ at $s = 3$. The bound (20.2) is at least $2s - 1$ for every $r > d$. Thus the equation for y modulo M_y is a degree-d polynomial. Its linear coefficient has constant term 1, and all nonlinear coefficients are divisible by p.

Here are completely explicit common-denominator numerators for that polynomial. Compute the four vertex polynomials at indices $N, 2N, pN, 2pN$ modulo p^{4s-2} and set

$$a_r = [h^r]A_N(X(1+h)), \quad b_r = [h^r]A_{2N}(X(1+h)), \quad a_0 = A_N(X).$$

Define

$$T_0 = b_0, \qquad T_r = b_r a_0^r - \sum_{i=1}^{r} a_i T_{r-i} a_0^{i-1}.$$

Then $T_r/a_0^{r+1} = [h^r]C_N(X(1+h))$. For $0 \leq r \leq d$ put

$$Q_r = \frac{p^r}{pN}\left(\binom{N}{r} A_{2pN} a_0^{d+1} - A_{pN} T_r a_0^{d-r}\right) \pmod{M_y}. \tag{20.3}$$

The quotients are coefficientwise integral by the preceding argument; the certificate independently checks this exact divisibility. Their common denominator $A_{pN}a_0^{d+1}$ is a unit. Thus

$$\sum_{r=0}^{d} Q_r y^r = 0 \pmod{M_y}, \qquad Q_0(0) = 0, \quad Q_1(0) = 1, \quad Q_r \in p\mathbb{Z}_p[u] \ (r \geq 2). \tag{20.4}$$

Two roots of this equation have difference multiplied by a unit, because after subtracting their equations the factor is congruent to Q_1 modulo p. Therefore the root modulo M_y is unique.

The program constructs a rational root Y/Z from these Q_r using the finite multivariate Lagrange expansion, with $Z(0) = 1$, and then checks the entire polynomial identity

$$\sum_{r=0}^{d} Q_r Y^r Z^{d-r} = 0 \pmod{M_y}. \tag{20.5}$$

This identity and uniqueness, rather than an assumption about the target truncations, prove $y = Y/Z \pmod{M_y}$ and hence $\sigma = X(1+pY/Z) \pmod{M}$.

For clarity, the construction is explicit. Write $A = Q_0$, $B = Q_1$. For a profile $(m_2, \ldots, m_d)$ of nonnegative integers, put $m = \sum m_r \geq 1$, $R = \sum r m_r$ and $\ell = R - m + 1$. Its contribution to Y/Z is

$$(-1)^{R+1} \frac{R!}{\ell! \prod m_r!} \frac{A^\ell \prod_{r=2}^{d} Q_r^{m_r}}{B^{R+1}}.$$

Include also the term $-A/B$, and retain only profiles of total coefficient valuation less than $2s-1$. The integer factorial quotient is the usual rooted-plane-tree coefficient in Lagrange inversion. The code checks its integrality. The fixed common denominators used below are B^7 and B^{13} at $p=3$, levels 2 and 3, respectively. The direct check (20.5) makes the root certification independent of any truncation of the Lagrange expansion.

20.3 The target polynomial identities

The vertex relation at $K=N$ yields

$$\lambda = \frac{(1+py)^N A_{pN}(u)}{A_N(X(1+py))} \pmod{M}.$$

Since $v_p\binom{N}{r} \geq v_p(N) - v_p(r)$, for every $r \geq 2$ the term $\binom{N}{r}(py)^r$ is divisible by M. Hence $(1+h)^N = 1+Nh \pmod{M}$ on $h \in p\mathbb{Z}_p[[u]]$.

Set $f = F_{p^s}$, $f_0 = F_{p^{s-1}}$ and, for $0 \leq r < 2s$, define

$$L_r = A_{pN}(u)\big([h^r]f_0(X(1+h)) + N[h^{r-1}]f_0(X(1+h))\big) - f(u)[h^r]A_N(X(1+h)),$$

where a coefficient with negative index is zero. The target congruence is equivalent to $\sum_{r<2s} L_r h^r = 0 \pmod{M}$. Substitute the already certified $h = pY/Z$ and clear the unit denominator:

$$\boxed{\sum_{r=0}^{2s-1} L_r (pY)^r Z^{2s-1-r} = 0 \pmod{p^{2s}}.} \tag{20.6}$$

Only at this last step do the ordinary period truncations enter the calculation. The program verifies every coefficient of (20.5) and (20.6) exactly.

If the two identities hold, multiplication by the unit $F(\sigma)$ gives $E_{n,p,s,1} \in p^{2s}\mathbb{Z}_p[[u]]$. Thus a finite exact polynomial certificate proves a statement about the entire series.

21 Newton certificates uniform in the octahedral dimension

The dimension parameter can be handled by integer polynomials rather than by a finite search with an unproved upper bound. This gives the following additional result at the wild prime 3.

Theorem 21.1. *For every octahedral dimension $n \geq 2$, let σ_n be its normalized excellent lift. Then, for $s = 2, 3$,*

$$F_n(u)F_{n,3^{s-1}}(\sigma_n(u)) - F_n(\sigma_n(u))F_{n,3^s}(u) \in 3^{2s}\mathbb{Z}_3[[u]].$$

This is an assertion about all dimensions and every formal-series coefficient at each of the two specified levels. The multiplier is $m=1$; no induction over all levels is asserted here.

We use the negative-vertex and scaled-Hensel certificate construction. Its formal Cartier and derivative inputs are those already established by the integral orbit-module argument. The new ingredient is a finite difference criterion that makes the dependence on n rigorous.

21.1 Integer polynomials in the dimension

Put

$$b_n(j) = \sum_{k_1+\cdots+k_n=j} \binom{j}{k_1,\ldots,k_n}^2, \qquad a_{n,j} = \binom{2j}{j} b_n(j).$$

For each j, $b_n(j)$ is the value at n of a polynomial in $\mathbb{Z}[T]$. To see this, count ordered pairs of length-j words with the same letter multiplicities. If precisely r letters occur, let $d_{j,r}$ be the number of pairs on a fixed r-letter alphabet using every letter. Simultaneous renaming of letters is a free action of S_r, so $r! \mid d_{j,r}$, and

$$b_n(j) = \sum_{r=0}^{j} \binom{n}{r} d_{j,r} = \sum_{r=0}^{j} \frac{d_{j,r}}{r!} n(n-1)\cdots(n-r+1).$$

Thus the assertion concerns ordinary integer coefficients, not merely integer-valued polynomials.

In particular the coefficients of every period truncation and of

$$A_{n,K}(u) = \sum_{j<K} u^j \sum_{\ell=0}^{j} (-1)^\ell \frac{(2K-\ell-1)!\, b_{n-1}(j-\ell)}{(2K-2j-1)!\ell!(j-\ell)!^2}$$

are integer polynomials in n. The factorial ratio is an integer multinomial coefficient.

For $e \geq 1$ set

$$\nu_3(e) = \min\{r \geq 0 : v_3(r!) \geq e\}, \qquad \Delta P(T) = P(T+1) - P(T).$$

Lemma 21.2 (Finite difference criterion)**.** *Let c be an integer.*

1. *If $P(T) \in 3^{-a}\mathbb{Z}_3[T]$, then*
$$\Delta^r P(c) \in 3^{-a} r!\mathbb{Z}_3 \qquad (r \geq 0).$$
2. *If $P \in \mathbb{Z}_3[T]$ and $P(c), \ldots, P(c+\nu_3(e)-1)$ are all divisible by 3^e, then $P(n) \in 3^e\mathbb{Z}_3$ for every integer n.*

The same conclusions hold coefficientwise with an additional polynomial variable u.

Proof. Finite differences of monomials are multiples of $r!$, by the usual Stirling-number formula, and translation by c preserves ordinary integer coefficients. This proves the first assertion. For the second, use the finite Newton expansion

$$P(c+X) = \sum_{r \geq 0} \Delta^r P(c) \binom{X}{r}.$$

For $r < \nu_3(e)$, the difference is an integer combination of the assumed values. For $r \geq \nu_3(e)$ the first assertion makes it divisible by 3^e. Finally $\binom{X}{r}$ is an integer at every integer X, including negative integers. □

21.2 Ordinary-polynomial models for the Hensel coefficients

Fix $s = 2$ or 3, and put

$$k = 2s-1, \quad N = 3^{2s-2}, \quad M = 3^{2s}, \quad M_y = 3^k, \quad M_{\text{work}} = 3^{4s-2}, \quad X = u^3.$$

Let $d = 3$ when $s = 2$, and $d = 6$ when $s = 3$. The four vertex polynomials are evaluated at $N, 2N, 3N, 6N$. Writing

$$a_r = [h^r]A_{n,N}(X(1+h)), \quad b_r = [h^r]A_{n,2N}(X(1+h)),$$

define

$$T_0 = b_0, \qquad T_r = b_r a_0^r - \sum_{i=1}^{r} a_i T_{r-i} a_0^{i-1}$$

and

$$Q_r(n,u) = \frac{3^r}{3N}\left(\binom{N}{r}A_{n,6N}a_0^{d+1} - A_{n,3N}T_r a_0^{d-r}\right), \qquad 0 \le r \le d.$$

Before reduction, these are polynomials satisfying

$$Q_r(T,u) \in 3^{-k}\mathbb{Z}_3[T,u].$$

For every integer dimension $n \ge 2$, their coefficient values are integral. The scaled-Hensel equation is

$$\sum_{r=0}^{d} Q_r(n,u)y_n(u)^r = 0 \pmod{3^k}, \qquad \sigma_n(u) = u^3(1+3y_n(u)).$$

The root is unique because $Q_1(n,0) = 1$ and the nonlinear coefficients are divisible by 3. The omitted Taylor coefficients have valuation at least $r - 1 - v_3(r) \ge k$.

We now construct polynomials

$$\widetilde{Q}_r(T,u) \in \mathbb{Z}_3[T,u]$$

whose values agree with $Q_r(n,u)$ modulo 3^k for every $n \ge 2$. Set

$$D = \nu_3(2k), \qquad d_0 = \nu_3(k), \qquad c = 2.$$

The program computes the D input values at $n = 2, \dots, D+1$ modulo 3^k, and all their differences of orders $0 \le j < D$. It verifies, coefficientwise in u, that

$$\Delta^j Q_r(2,u) \in 3^{\min(k, v_3(j!))}\mathbb{Z}_3[u] \qquad (0 \le j < D). \tag{21.1}$$

For $j \ge D$, Lemma 21.2 gives $\Delta^j Q_r(2,u) \in 3^k\mathbb{Z}_3[u]$ automatically, since the only possible common denominator is 3^k. The finite calculation therefore controls the entire Newton expansion in the dimension variable.

For $0 \le j < d_0$, choose the coefficientwise integer representative $\delta_{r,j}(u)$ of $\Delta^j Q_r(2,u)$ modulo 3^k with coefficients between 0 and $3^k - 1$. The verified divisibility implies that $\delta_{r,j}/j! \in \mathbb{Z}_3[u]$. Define

$$\widetilde{Q}_r(T,u) = \sum_{j<d_0} \frac{\delta_{r,j}(u)}{j!}(T-2)(T-3)\cdots(T-j-1).$$

For $j = 0$ the product is 1. Every discarded Newton coefficient is zero modulo 3^k, either by the calculation or by the factorial bound. For the retained coefficients, replacing the exact difference by $\delta_{r,j}$ changes its value by a multiple of $3^k\binom{n-2}{j}$. Hence

$$\widetilde{Q}_r(n,u) \equiv Q_r(n,u) \pmod{3^k} \qquad (n \ge 2).$$

This argument proves the model for all dimensions; it is not an extrapolation from the computed dimensions.

21.3 Two polynomial identities imply all dimensions

Use the same fixed finite Lagrange expression in the $\widetilde{Q}_r$ at every dimension to construct

$$Y(T,u), Z(T,u) \in \mathbb{Z}_3[T,u].$$

Concretely, let $A = \widetilde{Q}_0$, $B = \widetilde{Q}_1$ and use the universal nonlinear valuation weights $w_r = r-1-v_3(r)$, $2 \le r \le d$. For a profile $(m_2, \dots, m_d)$ with $\sum w_r m_r < k$, put $R = \sum r m_r$, $m = \sum m_r$, and $\ell = R - m + 1$. Besides $-A/B$, its rational-root term is

$$(-1)^{R+1}\frac{R!}{\ell!\prod m_r!}\frac{A^\ell \prod_{r=2}^{d}\widetilde{Q}_r^{m_r}}{B^{R+1}}.$$

The factorial quotient is integral. Clear all these terms with the fixed denominator $Z = B^b$, where $b = 7$ for $s = 2$ and $b = 13$ for $s = 3$. The resulting Y and Z have ordinary 3-integral coefficients in the dimension variable. The program directly verifies the root identity below, so its conclusion does not rely on an unverified truncation of the Lagrange expression.

The two finite polynomials tested are

$$\mathcal{R}(T,u) = \sum_{r=0}^{d} \widetilde{Q}_r(T,u) Y(T,u)^r Z(T,u)^{d-r}$$

and

$$\mathcal{T}(T,u) = \sum_{r=0}^{2s-1} L_r(T,u)(3Y(T,u))^r Z(T,u)^{2s-1-r},$$

where L_r is the target numerator

$$L_r = A_{n,3N}([h^r]F_{n,3^{s-1}}(X(1+h)) + N[h^{r-1}]F_{n,3^{s-1}}(X(1+h))) - F_{n,3^s}[h^r]A_{n,N}(X(1+h)),$$

viewed as an ordinary integer polynomial in $T = n$. Every coefficient of both $\mathcal{R}$ and $\mathcal{T}$ belongs to $\mathbb{Z}_3[T]$.

For each of the $\nu_3(2s)$ consecutive dimensions

$$n = 2, \ldots, \nu_3(2s) + 1,$$

the program checks the entire identities

$$\mathcal{R}(n,u) = 0 \pmod{3^{2s-1}}, \qquad \mathcal{T}(n,u) = 0 \pmod{3^{2s}}.$$

Lemma 21.2, applied to each coefficient of u, proves both identities in every dimension. Also $Z(n,0)$ is a unit for all dimensions, because the modeled linear coefficient agrees modulo 3 with the actual coefficient whose constant term is 1. The root identity and uniqueness give $y_n = Y(n,u)/Z(n,u) \pmod{3^k}$. The target identity consequently gives the asserted excellent-lift truncation congruence. This proves Theorem 21.1.

21.4 Reproducible exact verification

The complete verifier is `certify_all_dimensions_newton.py`; it imports the exact polynomial arithmetic from `verify_dimension_families.py`. Run it with `-s 2` or `-s 3`. The input and output counts are as follows.

s	working modulus	input dimensions	model degree	target dimensions	max. target degree
2	729	15	8	9	1562
3	59049	24	11	15	70519

All difference-integrality checks and all coefficients of both polynomial identities pass. The target modulus is 81 in the first row and 729 in the second row. No formal-series coefficient cutoff is used.

The deterministic SHA-256 certificate identifiers are

$s = 2$	`ca7ba583179e93370023d03b848ddf0f52b63fa39dcc57b3dd9d4c078e79938f`
$s = 3$	`cbe30563e24bebbd768fce3ce9eda9513d18ec87e8fa9aa07814c062e1a83712`

The archived certificate includes the input Hensel coefficients, all Newton differences, the ordinary-model coefficients, the fixed Lagrange profiles, and the target verification reports. The digests identify those data; the proof is the exact algebra above together with the reproducible finite polynomial calculations.

22 The fourth level by compact rational Hensel certificates

The Newton method extends one level further. A compact rational Hensel calculation avoids explicitly expanding the large universal numerator and denominator in the dimension parameter.

Theorem 22.1. *For every octahedral dimension $n \geq 2$,*

$$F_n(u)F_{n,27}(\sigma_n(u)) - F_n(\sigma_n(u))F_{n,81}(u) \in 6561\mathbb{Z}_3[[u]].$$

Together with the earlier first-level argument and Theorem 21.1, this proves the normalized multiplier-1 congruence at levels $s = 1, 2, 3, 4$ in every dimension. It does not give an induction to all levels.

22.1 The dimension model at precision 3^7

Here the parameters in the preceding construction are

$$s = 4, \quad k = 7, \quad N = 729, \quad M = 6561, \quad M_y = 2187, \quad M_{\mathrm{work}} = 4782969 = 3^{14}, \quad d = 9.$$

The Taylor valuation bound $r - 1 - v_3(r)$ is at least 7 for every $r \geq 10$, so a degree-9 scaled Hensel equation suffices. Its coefficients, before reduction, satisfy

$$Q_r(T, u) \in 3^{-7}\mathbb{Z}_3[T, u], \qquad 0 \leq r \leq 9.$$

The four negative-vertex polynomials used are $A_{n,729}$, $A_{n,1458}$, $A_{n,2187}$ and $A_{n,4374}$, all computed modulo 3^{14}. The division by $3N = 3^7$ therefore determines the Hensel coefficients modulo 3^7 without loss of required precision.

We have

$$\nu_3(14) = 30, \qquad \nu_3(7) = 18, \qquad \nu_3(8) = 18.$$

The exact calculation at the 30 input dimensions $n = 2, \ldots, 31$ verifies all the finite-difference conditions

$$\Delta^j Q_r(2, u) \in 3^{\min(7, v_3(j!))}\mathbb{Z}_3[u] \qquad (0 \leq j < 30,\ 0 \leq r \leq 9).$$

For $j \geq 30$, the common-denominator bound and Lemma 21.2 give divisibility by 3^7 automatically. The preceding Newton construction thus supplies fixed polynomials

$$\widetilde{Q}_r(T, u) \in \mathbb{Z}_3[T, u], \qquad \deg_T \widetilde{Q}_r \leq 17,$$

such that

$$\widetilde{Q}_r(n, u) = Q_r(n, u) \pmod{2187} \qquad \text{for every } n \geq 2.$$

This statement has already been proved before the target truncations are tested.

22.2 A universal polynomial numerator exists

The following argument makes the change of rational denominators in individual compact certificates harmless. Put $B = \widetilde{Q}_1$ and define rational functions in T, u by

$$y_0 = 0, \qquad y_{j+1} = -\frac{\widetilde{Q}_0}{B} - \sum_{r=2}^{9} \frac{\widetilde{Q}_r}{B} y_j^r \qquad (0 \leq j < 7).$$

There are fixed polynomials $Y_*(T, u), Z_*(T, u) \in \mathbb{Z}_3[T, u]$ with $y_7 = Y_*/Z_*$. Indeed one may take $Z_* = B^e$, where the successive denominator exponents satisfy

$$e_0 = 0, \qquad e_{j+1} = 9e_j + 1; \qquad e_7 = 597871.$$

Only the existence of these finite polynomials is needed; they are not expanded by the verifier.

For each integer dimension $n \geq 2$, $B(n,u)$ is a formal-series unit, and $\widetilde{Q}_r(n,u) \in 3\mathbb{Z}_3[[u]]$ for $r \geq 2$. These divisibilities are pointwise in the dimension; no divisibility of every ordinary polynomial coefficient in T is asserted. Thus the map on the right of the recurrence contracts differences by a factor of 3 on $\mathbb{Z}_3[[u]]$. Starting from 0, seven iterations agree modulo 3^7 with the unique integral Hensel root. Consequently

$$\frac{Y_*(n,u)}{Z_*(n,u)} = y_n(u) \pmod{2187} \qquad (n \geq 2), \tag{22.1}$$

and $Z_*(n,0)$ is a unit.

Let $L_0(T,u), \ldots, L_7(T,u) \in \mathbb{Z}_3[T,u]$ be the same target polynomials as before, now with $N = 729$, $F_{n,81}$ and $F_{n,27}$. The single fixed polynomial

$$\mathcal{T}_*(T,u) = \sum_{r=0}^{7} L_r(T,u)(3Y_*(T,u))^r Z_*(T,u)^{7-r} \in \mathbb{Z}_3[T,u]$$

will be used for the dimension argument. It is important that this polynomial is fixed independently of n, even though the compact rational functions computed below use different denominators at different dimensions.

22.3 Compact roots at the eighteen testing dimensions

For each $n = 2, \ldots, 19$, the program constructs a small rational root $\widehat{Y}_n/\widehat{Z}_n$ of the degree-9 Hensel equation modulo 2187. Its denominator has unit constant term. At the initial digit, reduce $-Q_0/Q_1$ in $\mathbb{F}_3(u)$ by a polynomial gcd. Suppose A/D is already a root modulo 3^j, and put

$$S = \sum_{r=0}^{9} Q_r A^r D^{9-r} \pmod{3^{j+1}}.$$

Every coefficient of S is divisible by 3^j. Over $\mathbb{F}_3(u)$ the correction is

$$\delta = -\frac{S/3^j}{\overline{D}^9 \overline{Q}_1}.$$

Reduce this fraction by a polynomial gcd and replace A/D by $A/D + 3^j\delta$. Repeating for $j = 1, \ldots, 6$ gives the desired root. At every step all divisions by powers of 3 are checked exactly. The final calculation independently verifies the entire identity

$$\sum_{r=0}^{9} Q_r \widehat{Y}_n^r \widehat{Z}_n^{9-r} = 0 \pmod{2187}.$$

Thus correctness of the final certificate does not depend on merely trusting the gcd algorithm or the digit construction.

The program then verifies every coefficient of

$$\sum_{r=0}^{7} L_r(n,u)(3\widehat{Y}_n)^r \widehat{Z}_n^{7-r} = 0 \pmod{6561} \qquad (n = 2, \ldots, 19). \tag{22.2}$$

No formal-series cutoff is used. The largest of these compact target polynomials has degree 10471.

The actual Hensel polynomial and its ordinary dimension model agree modulo 2187. Uniqueness therefore identifies each compact root with the universal root (22.1) modulo 2187. Multiplication

by 3 improves the precision to 6561 for the variable $h = 3y$ in the target polynomial. Because both denominators are units, (22.2) implies

$$\mathcal{T}_*(n, u) = 0 \pmod{6561} \qquad (n = 2, \dots, 19).$$

Apply Lemma 21.2 coefficientwise to the fixed ordinary polynomial $\mathcal{T}_*(T, u)$. The $18 = \nu_3(8)$ consecutive values imply the same divisibility for every integer dimension $n \geq 2$. By (22.1), clearing and then restoring the unit denominators gives $\lambda_n F_{n,27}(\sigma_n) - F_{n,81} \in 6561\mathbb{Z}_3[[u]]$. Multiplication by the unit $F_n(\sigma_n)$ proves Theorem 22.1.

22.4 Exact implementation and reproducibility

The model calculation uses factorial unit parts and valuations. For an integral factorial ratio, subtract the factorial valuations and multiply the unit parts and their inverses modulo the working prime power; then restore the nonnegative power of 3. This computes the same integer ratio modulo that prime power. The implementation checks nonnegativity of every resulting valuation. NumPy integer matrix operations are exact here: the explicit bound

$$(3^{14})^2 \cdot 4374 < 2^{63}$$

guarantees that no signed 64-bit dot product can overflow. All polynomial arithmetic in the root and target identities uses exact integer arithmetic.

The model stage has a `-model-only` option, so no attempt to expand a large universal numerator is necessary. A complete reproduction consists of

```
python3 certify_all_dimensions_newton_fast.py --s 4 --model-only
python3 compact_hensel.py --s 4 --n \
  2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19
python3 summarize_level4_certificate.py
```

The first script saves the 30 Hensel inputs, their Newton differences, and the ordinary models. The second saves the 18 compact roots and their complete target numerator polynomials. The last script records deterministic identifiers for these already verified data.

The aggregate SHA-256 identifier is

`795a410dff5b06382a482373dc2e2451689b3c643634ac13b66e384ab0ca313c.`

It hashes the model checkpoint identifier together with all eighteen individual compact-certificate identifiers. As before, the proof is the exact algebra and the reproducible polynomial identities; the hash only identifies the archived data.

23 Limits of the auxiliary rows and their resolution

The two arguments in this part do not cover $p = 3$, $n \geq 4$, $3 \nmid n$ at arbitrary levels. Part IV supplies that case, including arbitrary normalized multipliers. The following failures of stronger auxiliary estimates explain why a different argument is needed; they remain valid after the target congruence has been proved.

The obstruction at 3 is concrete. The coefficient $-3(X^3 - X)$ in unit-factorial interpolation cannot be placed in $9\mathbb{Z}_3\langle X\rangle$. Its values on integers do have an additional factor of 3, but ordinary restricted-series coefficient divisibility is stronger than integer-valued divisibility. The mixed-precision proof compensates for this using $3 \mid n$; it does not justify applying the same induction when $3 \nmid n$.

A seemingly stronger alternative row construction is also insufficient. Define the unique exact integral row by

$$B(j,3N)=\sum_{i\le L} D^{\mathrm{ex}}_{j,i}B(i,N),\qquad L=\lfloor j/3\rfloor,$$

including $N=0$. It need not have the individual boundary divisibility used above. For example, in dimension 5, at $j=12, L=4, i=0$,

$$D^{\mathrm{ex}}_{12,0}\equiv 27 \pmod{81},$$

whereas $9(4!)^2$ has 3-adic valuation 4. These are failures of auxiliary proposed estimates, not counterexamples to the target excellent-lift congruence.

The numerical checks of the mixed induction in dimensions $3,6,9,12,15,18$, for $K\le 40$ and $N\le 120$, are consistency checks only. The all-level theorems rest on the written induction. In contrast, the three finite-level dimension conclusions explicitly include exact finite computation as part of their proofs; their finite test bounds are proved by the Newton lemma. No step treats a power-series cutoff or a finite list of dimensions as an unproved substitute for a universal statement.

Part IV
All-level three-adic descent

The integral cutoff kernel, universal weak descent, dimension residues, and an independent fifth-level certificate.

24 Statement and proof architecture

Let

$$g_n(\mathbf{x}) = \sum_{i=1}^{n}(x_i + x_i^{-1}), \qquad f = 1 - zg_n, \qquad u = z^2,$$

and put

$$b_n(j) = \sum_{k_1+\cdots+k_n=j} \binom{j}{k_1,\ldots,k_n}^2, \qquad a_{n,j} = \binom{2j}{j} b_n(j),$$

$$F_n(u) = \sum_{j\geq 0} a_{n,j}u^j, \qquad F_{n,K}(u) = \sum_{0\leq j<K} a_{n,j}u^j.$$

The excellent lift $\sigma_n(u)$ is defined in Section 25. All divisibilities below are coefficientwise in the displayed formal-series or ordinary polynomial ring.

Theorem 24.1 (All levels at the remaining prime)**.** *For every integer $n \geq 3$ and every positive integer K divisible by 3,*

$$\frac{F_{n,K}(u)}{F_n(u)} \equiv \frac{F_{n,K/3}(\sigma_n(u))}{F_n(\sigma_n(u))} \pmod{3^{2v_3(K)}\mathbb{Z}_3[[u]]}.$$

In particular, for every $s, m \geq 1$,

$$\begin{aligned} E_{n,3,s,m}(u) &:= F_n(u)F_{n,m3^{s-1}}(\sigma_n(u)) - F_n(\sigma_n(u))F_{n,m3^s}(u) \\ &\in 3^{2s}\mathbb{Z}_3[[u]]. \end{aligned}$$

When $3 \mid m$, the proof gives the stronger precision $3^{2s+2v_3(m)}$.

The same arithmetic proof works in dimensions one and two; they are also used as residue representatives in the dimension argument. Normalized length $m3^s$ in u is length $2m3^s$ in z. Thus $m = 1$ is the hyperoctahedral multiplier 2 in BV3, Conjecture 7.5 [7].

The proof has four independent mathematical components:

1. An orbit module supplies the formal excellent Cartier congruence at every odd prime, including $3 \mid n$.
2. An integral Bessel kernel proves the uniform quadratic cutoff bound in the ordinary coefficient ring $\mathbb{Z}[T]$.
3. A universal row over $\mathbb{Z}_3\langle T\rangle$ gives descent modulo $3^{2v_3(K)-1}$ coefficientwise in T.
4. The scalar dimensions $0, 1, 2$ recover the missing factor at every integer dimension. The exact tail expansion transfers the result to ordinary truncations.

The level-five certificates are separate checks of entire series. They are not premises of the all-level argument. This part establishes the remaining formal 3-adic hyperoctahedral case. The analytic transfer at the original BV3-admissible primes is stated separately in Corollary 1.5.

25 The formal excellent Cartier input

Proposition 25.1 (Formal excellent input from an orbit module)**.** *Let $g = g_n$ be the hyperoctahedral Laurent polynomial and let p be an odd prime. Put $R = \mathbb{Z}_p[[z]]$ and $f = 1 - zg$. Let $F_0(z)$ and $G_0(z)$ be the period and logarithmic companion given by the vertex-coefficient formulas of BV3, Proposition 9.1 [7]. Explicitly, $F_0(z) = F_n(z^2)$ and, with $H_r = \sum_{a=1}^{r} 1/a$,*

$$G_0(z) = \sum_{j\geq 1} z^{2j} \sum_{k_1+\cdots+k_n=j} (H_{2j} - H_{k_1}) \frac{(2j)!}{\prod_i (k_i!)^2}.$$

Then

$$q_0(z) = z \exp(G_0(z)/F_0(z)) \in z + z^2\mathbb{Z}_p[[z]].$$

The lift $\tau = q_0^{-1}(q_0(z)^p)$ is integral and satisfies

$$\mathscr{C}_p(1/f) - \frac{F_0(z)}{F_0(\tau)} \frac{1}{f^\tau} \in p^2 \mathscr{F}_2^\tau.$$

In particular, no exclusion $p \mid n$ is needed for this formal input.

Proof. Write Γ for the support lattice and H for the full monomial symmetry group. Its order will never be inverted. The support lattice is $\mathbb{Z}^n$, so it is p-saturated. The symmetry group preserves the reflexive polytope degree $d(u) = \deg_\Delta(u)$ and the support lattice. For an H-orbit $O \subset \Gamma$, put

$$S_O = \sum_{u\in O} x^u, \qquad d(O) = d(u)\ (u \in O), \qquad L = \bigoplus_O R z^{d(O)} S_O.$$

Here a Laurent polynomial uses only finitely many orbits. This is a direct construction of a module of polynomials. We do not obtain it by applying an averaging projector to another crystal.

Fix temporarily any lift σ with $z^\sigma = z^p v$, $v \in 1 + pR$. We check BV3, Definition 2.1. The orbit sums have disjoint supports and coefficient one, so L is p-saturated. The same is true of every L^{σ^i}: its orbit coefficient generator is $(z^{\sigma^i})^{d(O)}$, a power of z times a unit, which is a non-zero-divisor modulo p. Multiplication by f preserves invariance and admissibility, hence $fL \subset L$.

The subset $p\Gamma$ is H-stable. An orbit is either disjoint from $p\Gamma$ or has the form pO' for one orbit O'; the equality uses the p-saturation of the support lattice. Since $d(pu) = pd(u)$,

$$\mathscr{C}_p(z^{d(pO')} S_{pO'}) = z^{pd(O')} S_{O'} = \left(\frac{z^p}{z^\sigma}\right)^{d(O')} (z^\sigma)^{d(O')} S_{O'} \in L^\sigma.$$

The image of any other orbit is zero. This proves compatibility without dividing by either $|H|$ or $|O|$.

Form the Dwork crystal $M = \widehat{\Omega}_{L,f}(\Delta^\circ)$ directly, as in BV3, Definitions 2.6 and 2.8. We do not identify it with the invariant part of an ambient completed module; in particular no exactness of an invariant functor is being assumed. In this family, $\Delta \cap \Gamma$ consists of the origin and the vertices. Reflexivity then implies that the first two levels have bases

$$M(1) = R/f, \qquad M(2) = R/f^2 \oplus Rzg/f^2.$$

Indeed $2\Delta^\circ \cap \Gamma = \Delta \cap \Gamma$ and the vertices form one orbit. The pair $1/f, zg/f^2$ is an extended basis, since $1/f = (1 - zg)/f^2$ and the change of basis has determinant one. These levels are p-saturated in the ambient integral formal expansion: multiplying an expansion by f or f^2 reduces the claim to divisibility of the numerator coefficients. This also shows that the two basis vectors remain independent modulo p.

The Hasse–Witt computations of BV3, Lemma 7.2, now apply to this explicit compatible module. Their coefficient extraction uses a single vertex coefficient of a polynomial known to be

constant on the vertex orbit; it does not average over that orbit. With the initial lift $z^\sigma = z^p$, the first determinant has constant term one, and

$$\left.\frac{\det HW^{(2)}(M)}{p}\right|_{z=0} = -\frac{1}{p}\binom{2p-2}{p-1} \in \mathbb{Z}_p^\times.$$

The latter computation reads the coefficient of $z^p x^{pv}$ at one vertex v in $(zg)F^{(2)}$; the extreme vertex forces the unique contribution $(-z)^p\binom{2p-2}{p-1}$. The normalized determinant modulo p is independent of the lift. Consequently both Hasse–Witt conditions hold over R for every lift under consideration. The general BV3, Corollary 5.9 and Corollary 4.3, therefore give

$$M = M(2) \oplus \mathscr{F}_2 M, \qquad \mathscr{C}_p(M) \subset M^\sigma(2) + p^2 \mathscr{F}_2 M^\sigma.$$

Only $p > 2$ is required by these general results.

For completeness, the remaining canonical-coordinate argument uses these statements rather than a group-order hypothesis. The derivation $\theta = z\,d/dz$ preserves M: for an admissible numerator A,

$$\theta\left((r-1)!\frac{A}{f^r}\right) = (r-1)!\frac{\theta A}{f^r} + r!\frac{Azg}{f^{r+1}}.$$

Thus the rank-two quotient has the connection and Cartier matrices of BV3, Propositions 7.6 and 7.7. The positive vertex coefficient calculation of Proposition 9.1 gives exactly F_0 and $F_0 \log z + G_0$ as its two solutions. The same coefficient calculation at exponents $p^s v$ gives $A(0) = B(0) = 0$, the constant Cartier matrix $\mathrm{diag}(1, p)$ for the initial lift, and the upper-right coefficient

$$\lambda_1 = \frac{pF_0(z)F_0(z^\sigma)}{W(z^\sigma)}\left(-\frac{1}{p}\log\frac{z^\sigma}{z^p} + \frac{G_0(z)}{F_0(z)} - \frac{1}{p}\frac{G_0(z^\sigma)}{F_0(z^\sigma)}\right),$$

where $W = F_0^2 + F_0\theta G_0 - (\theta F_0)G_0$. The rank-one reduction modulo p implies $\lambda_1 \in pR$. In the Cartier matrix $\Lambda = (\lambda_i; \mu_i)$, horizontality gives

$$\mu_1 = \theta\lambda_1 + \frac{\theta(z^\sigma)}{z^\sigma}(\lambda_0 + B^\sigma\lambda_1) \in pR,$$

because $\theta(z^\sigma)/z^\sigma \in pR$ and the connection coefficient B is integral. Hence $\det\Lambda/p \in R$. The determinant identity $\det\Lambda = pW/W^\sigma$ and $W(0) = 1$ give $W \in 1 + zR$ by the same iteration as BV3, Proposition 7.9. For $z^\sigma = z^p$, the displayed formula and the Dieudonne–Dwork lemma yield $q_0 \in z + z^2R$. Its formal inverse is integral, so $\tau = q_0^{-1}(q_0^p)$ defines a lift with $\tau/z^p \in 1 + pR$. Reapply the compatible-module construction with this lift. The displayed expression has $\lambda_1 = 0$, and the remaining coefficient is F_0/F_0^τ. This proves the required excellent Cartier identity in $p^2\mathscr{F}_2 M^\tau$.

Finally, membership in $\mathscr{F}_2 M$ means that every s-fold Cartier iterate lies in $p^{2s}M^{\sigma^s}$. Mapping the directly constructed module to the ambient Dwork module therefore sends $\mathscr{F}_2 M$ into the ambient $\mathscr{F}_2$. Only this inclusion is needed; no converse assertion about invariants or completed modules is used. □

The sign symmetry gives $q_0(z) = zQ(z^2)$ and $\tau(-z) = -\tau(z)$. Thus $\tau = z^p V(u)$ with $V \in 1 + p\mathbb{Z}_p[[u]]$, and

$$\sigma_n(u) = \tau(z)^2 = u^p V(u)^2.$$

No division by the dimension occurs in this normalization. The corresponding canonical coordinate in u is $q(u) = q_0(z)^2$.

26 Positive vertices and exact tails

Suppress the subscript n when it is fixed. For $0 \le N \le j$ put

$$B(j,N) = (2j)! \sum_{\substack{k_1+\cdots+k_n=j \\ k_1 \ge N}} \frac{1}{(k_1-N)!(k_1+N)! \prod_{i>1}(k_i!)^2},$$

and set $B(j,N) = 0$ if $N > j$. These are integers, $B(j,j) = 1$, and $B(j,0) = a_j$. Let

$$\Omega_N(u) = \sum_{j \ge N} B(j,N) u^j, \qquad C_N = \Omega_N / F \in u^N \mathbb{Z}_p[[u]].$$

The series Ω_N is the coefficient at x_1^{2N}, with all other exponents zero, in the positive expansion of $1/f$. Reading Proposition 25.1 at that exponent and using the coefficient characterization of second formal derivatives gives

$$C_{pN} - C_N \circ \sigma_n \in p^{2+2v_p(N)} \mathbb{Z}_p[[u]]. \tag{26.1}$$

Here the exponent factor is $(2N)^2$ and 2 is a p-adic unit.

The unit triangular system uniquely defines integers $c_{K,N}$ by

$$a_j \mathbf{1}_{j \ge K} = \sum_{N=K}^{j} c_{K,N} B(j,N), \qquad c_{K,N} = 0 \quad (N < K).$$

Consequently

$$1 - F_K/F = \sum_{N \ge K} c_{K,N} C_N. \tag{26.2}$$

This identity converges u-adically. It is the connection between Cartier's coefficient congruence and ordinary truncations.

26.1 A balanced unit-factorial estimate at every odd prime

Write $U(a) = \prod_{1 \le i \le a,\, p \nmid i} i$. For $H = pN$ and $k \ge H$,

$$\frac{U(k)^2}{U(k-H)U(k+H)} = \prod_{\substack{k-H < i \le k \\ p \nmid i}} (1 + H/i)^{-1} \equiv 1 \pmod{p^{2+2v_p(N)}}.$$

Indeed, for $v = v_p(H)$ the unit residues in this interval are complete copies modulo p^v. Their reciprocals sum to zero modulo p^v by pairing opposite residues. Expanding the product to first order proves the estimate.

27 A two-variable kernel and quadratic cutoff divisibility

Put

$$J(x) = \sum_{j \ge 0} \frac{x^j}{(j!)^2}, \qquad I_N(x) = \sum_{j \ge N} \frac{x^j}{(j-N)!(j+N)!}, \qquad \theta = x\frac{d}{dx}.$$

Then $\theta^2 J = xJ$ and $(\theta^2 - x) I_N = N^2 I_N$. In this section, n is an indeterminate. Define

$$J(x)^n = \sum_{j \ge 0} b_n(j) \frac{x^j}{(j!)^2}, \qquad a_{n,j} = \binom{2j}{j} b_n(j),$$

and

$$B_n(j,N) = (2j)![x^j](I_N(x)J(x)^{n-1}).$$

The matrix B_n is lower triangular with diagonal entries 1. For $K \geq 1$ define $c_{K,N}(n)$, $N \geq K$, by the formal identity

$$(J(x)^n)_{\geq K} = \sum_{N \geq K} c_{K,N}(n) I_N(x) J(x)^{n-1}. \tag{27.1}$$

Equivalently,

$$a_{n,j}\, \mathbf{1}_{j \geq K} = \sum_{N=K}^{j} c_{K,N}(n) B_n(j,N).$$

Theorem 27.1 (Quadratic cutoff divisibility)**.** *For all integers $1 \leq K \leq N$,*

$$\boxed{\frac{N^2}{2nK^2}\, c_{K,N}(n) \in \mathbb{Z}[n].} \tag{27.2}$$

The quotient in (27.2) is a polynomial: in particular, the assertion includes divisibility of $c_{K,N}$ by n.

We prove a stronger statement at the level of the Bessel differential operator. The proof uses the following two elementary integrality facts.

Lemma 27.2 (Integral dimension polynomials)**.** *For every $j \geq 0$, $b_n(j) \in \mathbb{Z}[n]$. Consequently $J(x)^n$ and $J(x)^{1-n}$ belong to the factorial-square series ring*

$$\mathcal{A}_{\mathbb{Z}[n]} = \left\{ \sum_{j \geq 0} q_j \frac{x^j}{(j!)^2} : q_j \in \mathbb{Z}[n] \right\}.$$

The analogous two-variable ring, with denominators $(a!b!)^2$, is also closed under multiplication.

Proof. The binomial expansion of J^n gives

$$b_n(j) = \sum_{r=0}^{j} (n)_r T_{j,r}, \qquad T_{j,r} = \frac{1}{r!} \sum_{\substack{k_1+\cdots+k_r=j \\ k_i \geq 1}} \binom{j}{k_1, \ldots, k_r}^2,$$

where $(n)_r = n(n-1)\cdots(n-r+1)$. The integer in the sum before division by $r!$ counts pairs of ordered partitions of two labelled j-element sets, with corresponding blocks nonempty and of equal size. Simultaneously permuting the r block labels is a free action of the symmetric group. Thus $T_{j,r} \in \mathbb{Z}$. The case $j = r = 0$ has its usual value 1. Closure of the series rings follows because their multiplication constants are squares of binomial coefficients. Substitution of $1-n$ for n gives the second assertion. □

Lemma 27.3 (An integral ballot quotient)**.** *For nonnegative integers a, b,*

$$\frac{a-b}{a+b+1}\binom{2a}{a}\binom{2b}{b} \in \mathbb{Z}. \tag{27.3}$$

Proof. The case $a = b$ is immediate. Fix a prime p and put $h = v_p(a+b+1)$. For $1 \leq r \leq h$, the residues A, B of a, b modulo $q = p^r$ satisfy $A + B = q - 1$. The rth contribution to the valuation of the product of central binomial coefficients is

$$\left\lfloor \frac{2A}{q} \right\rfloor + \left\lfloor \frac{2B}{q} \right\rfloor.$$

It is 1 except when q is odd and $A = B = (q-1)/2$, when it is 0. For odd p, this exception is equivalent to $p^r \mid 2a+1$, and, since $p^r \mid a+b+1$, to $p^r \mid a-b$. For $p=2$ every contribution is 1; when $h > 0$, $a-b$ is then odd. Summing the first h contributions therefore gives at least $\max(0, h - v_p(a-b))$. All remaining contributions are nonnegative. This proves that the numerator in (27.3) contains every prime power in its denominator. □

Put $r(x) = \theta J(x)/J(x)$. This is a well-defined series with zero constant term. Define the divided-difference kernel

$$g(X,Y) = \frac{(r(X)-r(Y))^2}{X-Y}. \tag{27.4}$$

The apparent division is harmless: the numerator is divisible by $(X-Y)^2$ in $\mathbb{Q}[[X,Y]]$.

Lemma 27.4 (Integral Bessel kernel)**.** *The kernel g belongs to the two-variable factorial-square series ring $\mathcal{A}^{(2)}_{\mathbb{Z}}$; explicitly,*

$$(a!b!)^2[X^aY^b]g(X,Y) \in \mathbb{Z} \qquad (a,b \geq 0).$$

Proof. Write

$$A = J(X)J(Y), \qquad W = (\theta_X J(X))J(Y) - J(X)(\theta_Y J(Y)), \qquad D = \frac{W^2}{X-Y}.$$

Both $W/(X-Y)$ and D are formal power series. For $T = \theta_X + \theta_Y$, the Bessel equation gives

$$TW = (X-Y)A.$$

Since $T(X-Y) = X-Y$, it follows that

$$(T+1)D = 2WA = (\theta_X - \theta_Y)A^2.$$

Vandermonde's identity gives

$$J(X)^2 = \sum_{a\geq 0} \binom{2a}{a} \frac{X^a}{(a!)^2}.$$

Comparing coefficients therefore yields the exact formula

$$(a!b!)^2[X^aY^b]D = \frac{a-b}{a+b+1}\binom{2a}{a}\binom{2b}{b}. \tag{27.5}$$

Lemma 27.3 proves $D \in \mathcal{A}^{(2)}_{\mathbb{Z}}$. Finally,

$$g = J(X)^{-2}J(Y)^{-2}D,$$

and the inverse powers lie in the same integral ring by Lemma 27.2 (or by geometric inversion in that ring). □

Theorem 27.5 (Integral cutoff kernel)**.** *Let*

$$h_K(x) = J(x)^{1-n}(J(x)^n)_{\geq K}, \qquad R_K(x) = (\theta^2 - x)h_K(x).$$

Then

$$\frac{((K-1)!L!)^2}{n}[x^{K+L}]R_K(x) \in \mathbb{Z}[n] \qquad (K \geq 1,\ L \geq 0). \tag{27.6}$$

More precisely, the following exact two-variable generating identity holds:

$$\boxed{\sum_{K\geq 1,\, L\geq 0} \frac{[x^{K+L}]R_K(x)}{n} X^L Y^{K-1} = J(X)^{1-n}J(Y)^n(1-(n-1)g(X,Y)).} \tag{27.7}$$

Here division by n denotes an identity in $\mathbb{Q}[n][[X,Y]]$; the proof itself establishes that every coefficient of R_K is divisible by n.

Proof. Summing the cutoffs first gives

$$\sum_{K\geq 1} h_K(x)z^{K-1} = \frac{J(x) - J(x)^{1-n}J(zx)^n}{1-z}.$$

Apply $\theta_x^2 - x$ with z fixed. Set $X = x$, $Y = zx$ and $P = J(X)^{1-n}J(Y)^n$. The operator θ_x becomes $T = \theta_X + \theta_Y$. Using $\theta r = x - r^2$, direct logarithmic differentiation gives

$$(T^2 - X)P = n(Y - X)P + n(n-1)(r(X) - r(Y))^2 P.$$

The term $J(x)$ is annihilated. Thus

$$\sum_{K\geq 1} R_K(X)(Y/X)^{K-1} = nXP\left(1 - (n-1)\frac{(r(X) - r(Y))^2}{X - Y}\right).$$

Since R_K starts in degree at least K, dividing by nX and comparing nonnegative powers of X, Y gives (27.7). Its right side is in $\mathcal{A}^{(2)}_{\mathbb{Z}[n]}$ by Lemmas 27.2 and 27.4. This proves (27.6). □

Proof of Theorem 27.1. Multiply (27.1) by J^{1-n} and apply $\theta^2 - x$. This gives the exact locally finite identity

$$R_K(x) = \sum_{N\geq K} N^2 c_{K,N}(n) I_N(x).$$

For $j = K + L$, Theorem 27.5 implies

$$\frac{(2j)!}{2nK^2}[x^j]R_K(x) = \frac{1}{2}\binom{2j}{j}\binom{j}{K}^2\left(\frac{((K-1)!L!)^2}{n}[x^j]R_K(x)\right) \in \mathbb{Z}[n].$$

Here $j \geq K \geq 1$, so the central binomial coefficient is even. On the other hand,

$$\frac{(2j)!}{2nK^2}[x^j]R_K(x) = \sum_{N=K}^{j} \frac{N^2 c_{K,N}(n)}{2nK^2}\binom{2j}{j-N}.$$

The matrix $\left(\binom{2j}{j-N}\right)_{j,N\geq K}$ is lower triangular over $\mathbb{Z}$, with diagonal 1. Induction on j therefore proves that every quotient $N^2 c_{K,N}(n)/(2nK^2)$ lies in $\mathbb{Z}[n]$. □

Corollary 27.6 (A stronger truncated-convolution divisibility)**.** *Define*

$$d_j(K) = \sum_{k=K}^{j}\binom{j}{k}^2 b_n(k)b_{1-n}(j-k), \qquad d_{K-1}(K) = 0.$$

Then, for every $j \geq K \geq 1$,

$$\frac{d_j(K) - d_{j-1}(K)}{n\binom{j-1}{K-1}^2} \in \mathbb{Z}[n].$$

Proof. The definition gives $h_K(x) = \sum_{j\geq K} d_j(K)x^j/(j!)^2$. Consequently

$$[x^j]R_K(x) = \frac{d_j(K) - d_{j-1}(K)}{((j-1)!)^2}.$$

Apply (27.6) with $L = j - K$. □

3-adic consequence. For every $K, N \geq 1$ with $N \geq K$, the theorem supplies the coefficientwise bound

$$c_{K,N}(n) \in 3^{\max(0,\, 2v_3(K)-2v_3(N))}\mathbb{Z}_3[n].$$

It also supplies an additional factor n. No evaluation at an integer dimension is needed to obtain these statements. This coefficientwise form is the feature required by a residue-class argument in the dimension.

28 Universal weak descent in the dimension

Put

$$J(x) = \sum_{j\geq 0} \frac{x^j}{(j!)^2}, \qquad I_N(x) = \sum_{j\geq N} \frac{x^j}{(j-N)!(j+N)!}, \qquad \theta = x\frac{d}{dx}.$$

For an indeterminate T define

$$B_T(j,N) = (2j)![x^j]I_N J^{T-1}, \qquad a_{T,j} = B_T(j,0).$$

These belong to $\mathbb{Z}[T]$, and $B_T(j,j) = 1$. Indeed, $(j!)^2[x^j]J^T$ is the integer polynomial obtained by counting pairs of words with equal letter multiplicities, and multiplication by I_N gives integer multinomial factors. The inverse triangular system

$$a_{T,j}\mathbf{1}_{j\geq K} = \sum_{N=K}^{j} c_{K,N}(T)B_T(j,N)$$

therefore defines $c_{K,N}(T) \in \mathbb{Z}[T]$.

Theorem 27.1, with T in place of n, proves the following ordinary-polynomial bound for every $1 \leq K \leq N$:

$$N^2 c_{K,N}(T) \in K^2\mathbb{Z}_3[T]. \tag{UW}$$

We use this proved bound throughout the rest of the argument.

28.1 A universal factorial coefficient lattice

Let $A = \mathbb{Z}_3\langle T\rangle$ be the ring of restricted power series with integral coefficients, with its coefficient norm, and put

$$\mathcal{H}_A = \left\{ \sum_{j\geq 0} h_j x^j : (2j)!h_j \in A \right\}.$$

Products, x, and θ preserve $\mathcal{H}_A$ and have norm at most one. For every integer r, $J^{T-r} \in \mathcal{H}_A$. All statements in this subsection are coefficient statements in T; divisibility of values at integer dimensions alone would not suffice.

Lemma 28.1 (A single-slot unit factorial interpolation)**.** *There are $V_r(X) \in 1 + 3\mathbb{Z}_3\langle X\rangle$, for $r = 0, 1, 2$, such that $V_r(0) = 1$ and*

$$U(3\ell + r)^{-1} = 2^{-\ell}(r!)^{-1}V_r(\ell), \qquad U(a) = \prod_{\substack{1\leq i\leq a \\ 3\nmid i}} i.$$

Writing $W_r = V_r^2$, there are restricted integral series $Z_r(X,Y)$ with

$$V_r(X-Y)V_r(X+Y) - W_r(X) = 3Y^2 Z_r(X,Y). \tag{1}$$

Proof. The paired block ratio is

$$\frac{U(3(\ell+1))}{2U(3\ell)} = 1 + \frac{9}{2}\ell(\ell+1).$$

The logarithm and its discrete antiderivative converge in the restricted series ring. The m-th logarithmic term, after discrete antidifferentiation, has coefficient valuation at least

$$2m - v_3(m) - \lfloor \log_3(2m+1) \rfloor \geq 1,$$

and the bound tends to infinity. Here is the denominator estimate used in this step. If an integral polynomial P has degree at most d, write $P(X) = \sum_{a=0}^{d} e_a (X)_a$ with integral e_a. Its discrete antiderivative vanishing at zero is $\sum_{a=0}^{d} e_a (X)_{a+1}/(a+1)$, since $\Delta(X)_{a+1} = (a+1)(X)_a$. Its coefficient denominator therefore loses at most $\lfloor \log_3(d+1) \rfloor$ powers of 3. Apply this with $d = 2m$ to the mth logarithmic term. Exponentiation gives the interpolation for $r = 0$; multiply by $\prod_{a=1}^{r}(1+3X/a)^{-1}$ for the other two residues. The logarithmic interpolant vanishes at $X = 0$. For (1), write $V_r = 1 + 3Z$. The difference on its left has coefficients divisible by 3, is even in Y, and vanishes at $Y = 0$. Division by $3Y^2$ thus remains in the integral restricted series ring. □

28.2 Keeping dimension denominators in one factorial

For a fixed positive integer dimension n, let $f_1 = I_N$ and $f_i = J$ for $i > 1$. For $\ell \geq 0$ set

$$E_\ell = \sum_{|S|=\ell} \prod_{i \in S} \theta f_i \prod_{i \notin S} f_i, \qquad D_\ell = \ell! E_\ell.$$

Their polynomial-dimension continuation is explicit:

$$D_\ell = (T-1)_\ell I_N J^{T-1-\ell} (\theta J)^\ell + \ell (T-1)_{\ell-1} (\theta I_N) J^{T-\ell} (\theta J)^{\ell-1},$$

where the second term is omitted when $\ell = 0$ and $(T)_r$ denotes a falling factorial. In particular $D_\ell \in \mathcal{H}_A$. For $\ell \geq 1$ the exact differential recurrence is

$$\theta D_\ell = D_{\ell+1} + \ell(T-\ell+1) x D_{\ell-1} + N^2 \ell (T-1)_{\ell-1} I_N J^{T-\ell} (\theta J)^{\ell-1}. \tag{2}$$

Together with $\theta D_0 = D_1$, this shows that D_ℓ equals an integral scalar differential operator in x, θ, T applied to $I_N J^{T-1}$, modulo $N^2 \mathcal{H}_A$. No factorial division occurs in this conversion.

An active-slot pattern has R labelled occupied positions. At each position prescribe a residue $r \in \{0,1,2\}$ and either a constant factor or a positive-degree factor of W_r. The pair consisting of residue zero and a constant factor is inactive and is omitted. If precisely k positions have positive-degree factors and the residue sum is $b + 3d$, then

$$R \leq k + b + 3d. \tag{3}$$

Expand each W_r as $1 + 3T_r$, where $T_r(0) = 0$ and $T_r \in \mathbb{Z}_3\langle X \rangle$. The corresponding symmetric operator is the sum over ordered injections of the R positions into the n variables, divided by $R!$, with a factor 3^k and the units $(r!)^{-2}$.

For a monomial in a pattern, reduce each one-variable derivative using $\theta^2 J = xJ$ and $\theta^2 I_N = (x + N^2) I_N$. Modulo $N^2 \mathcal{H}_A$, all single-variable reduction coefficients are integral and independent of the chosen slot. If exactly ℓ of the R positions retain first derivatives, the injection sum is

$$(T-\ell)_{R-\ell} \ell! E_\ell = (T-\ell)_{R-\ell} D_\ell. \tag{4}$$

Thus (2) gives scalar reduction with the *single* denominator $R!$. The same denominator suffices for the error. In fact, a single-variable error can only arise when one of the labelled positions is the distinguished first slot; the other positions contribute $(T-1)_{R-1}$ times an integral product of

Bessel factors and their derivatives. The error from (2) is explicitly integral in the same lattice. All these assertions extend continuously to the restricted series in a fixed pattern.

Here are explicit error and convergence details. There are polynomials $A_q, B_q \in \mathbb{Z}[x]$ and $H_{q,N} \in \mathcal{H}_{\mathbb{Z}_3}$, all of norm at most one, such that

$$\theta^q J = A_q J + B_q \theta J, \qquad \theta^q I_N = A_q I_N + B_q \theta I_N + N^2 H_{q,N}.$$

Start with $A_0 = 1$, $B_0 = H_{0,N} = 0$, and use

$$A_{q+1} = \theta A_q + x B_q, \quad B_{q+1} = A_q + \theta B_q, \quad H_{q+1,N} = \theta H_{q,N} + B_q I_N.$$

If the derivative orders of the R labelled positions are $e_1, \dots, e_R$, the error before division by $R!$ is exactly

$$N^2 (T-1)_{R-1} J^{T-R} \sum_{h=1}^{R} H_{e_h,N} \prod_{q \neq h} \theta^{e_q} J. \tag{4a}$$

There is only one distinguished Bessel factor, so errors from two different slots never multiply. Formula (4a) proves directly that no dimension denominator is hidden in the error. Similarly, iterating (2) gives $D_\ell = P_\ell(x, \theta, T)\Phi_N + N^2 Q_{\ell,N}$ with $P_\ell \in \mathbb{Z}[x, \theta, T]$ in normal order and $Q_{\ell,N} \in \mathcal{H}_A$, both of norm at most one. Consequently the entire ordered-injection reduction is bounded by one before its single division by $R!$.

For fixed R, coefficients of the restricted input tend to zero with the total derivative order; the preceding bounded maps therefore converge to scalar coefficient series in $A\langle x \rangle$ and errors in $\mathcal{H}_A$. The scalar coefficients have degree at most R in T, before this restricted limit. For fixed residue sum $B = b + 3d$, one also has $k \geq R - B$. Thus the norm of the contribution of all patterns of size R, after their prefactor, is bounded by

$$3^{-\epsilon - 2d - k + v_3(R!)} \leq 3^{-\epsilon - 2d - R + B + v_3(R!)},$$

which tends to zero as $R \longrightarrow \infty$. There are only finitely many residue/constant patterns at each R, and their number causes no loss in a nonarchimedean norm. These bounds justify summing over all R, all derivative orders, and the bounded coefficient extractions used below. In particular the resulting dimension dependence lies in the ordinary-coefficient Tate algebra A, not merely in the larger ring of integer-valued functions.

Put $\epsilon = \lfloor 2b/3 \rfloor$. For $k + d \geq 1$,

$$\epsilon + 2d + k - v_3((k + b + 3d)!) \geq 1. \tag{5}$$

Indeed, for $r \geq 1$, $v_3(r!) \leq (r-1)/2$; apply this to the three values of b. The resulting real lower bounds are respectively $(k + d + 1)/2$, $(k + d)/2$, and $(k + d + 1)/2$, and the left side is an integer. For $k = d = 0$, the denominator divides $b!$ with $b < 3$. It follows that the prefactor $3^{\epsilon + 2d + k}$ absorbs the entire dimension denominator $R!$, and retains one factor of 3 whenever $k + d \geq 1$. For fixed residue sum, the lower bound tends to infinity as R tends to infinity. Hence the sum over all patterns converges in the coefficient norm of A, including its scalar coefficients and its N^2 error.

Lemma 28.2 (Universal weak row)**.** *Write $j = 3L + b$, with $0 \leq b < 3$. There are $D_{j,i}(T) \in A$, $0 \leq i \leq L$, such that*

$$a_{T,j} = \sum_{i \leq L} D_{j,i}(T) a_{T,i},$$

$$B_T(j, 3N) - \sum_{i \leq L} D_{j,i}(T) B_T(i, N) \in 3N^2 A,$$

$$D_{j,i}(T) a_{T,i} \in 3(L!/i!)^2 A \qquad (i < L).$$

Proof. Split every index as $k_i = 3\ell_i + r_i$. If the residue carry is d, the outside factorial factor is

$$3^{\epsilon+2d}U(2j)(2L+\epsilon)!2^{-2(L-d)}.$$

After the balanced unit replacement, the operator is

$$P_{b,d} = \sum_{\sum r_i = b+3d} \prod_i (r_i!)^{-2} W_{r_i}(\theta_i)$$

acting on $I_N(x_1)\prod_{i>1} J(x_i)$, followed by diagonal specialization and extraction of $[x^{L-d}]$. Use the active-slot reduction just proved, including the prefactor $3^{\epsilon+2d}$. Its scalar part is integral in A and its error is in $3N^2\mathcal{H}_A$: the case $k = d = 0$ is a constant operator and has no reduction error, and every other case has the factor of 3 in (5).

The balanced unit replacement also has an error in $3N^2A$ coefficientwise in T. This point cannot be inferred merely by interpolating its values at integer dimensions. Instead use (1) at the distinguished first slot. It supplies $3N^2$ times an integral restricted derivative operator. With its residue r_1 fixed, the remaining slots have residue sum $b + 3d - r_1$. Their active patterns have $R \le k + b + 3d - r_1 \le k + b + 3d$ and contribute at most the denominator $R!$, without requiring scalar reduction. The factor $3^{1+\epsilon+2d+k}$ leaves at least one 3 after this denominator, by (5) and its $k = d = 0$ case. The universal factorial lattice makes the remaining coefficient extraction integral. More explicitly, an ordered pattern on these R ordinary slots, with derivative orders e_q, has error numerator

$$(T-1)_R\big(Z_{r_1}(\theta, N)I_N\big)J^{T-1-R}\prod_{q=1}^{R}\theta^{e_q}J,$$

apart from its displayed factors $3N^2$, 3^k, and residue units. This expression is in $\mathcal{H}_A$ with norm at most one before division by $R!$, and the same convergence bounds apply.

Collecting the scalar operators defines the row $D_{j,i}(T)$. At $N = 0$ the balanced identity and all Bessel reductions are exact, so the first identity follows in A. Every row summand has the factor $(2L+\epsilon)!/(2i)!$. If $i < L$, it must either have a positive carry d or positive scalar x-degree. The case $k = d = 0$ has neither; thus (5) supplies one factor of 3. Therefore

$$D_{j,i}(T) \in 3\frac{(2L+\epsilon)!}{(2i)!}A \quad (i < L).$$

Multiplication by $a_{T,i} = (2i)!b_T(i)/(i!)^2$ proves the boundary assertion, since $(2L+\epsilon)!/(L!)^2$ is an integer. □

A finite choice of active patterns for each row. The preceding convergence argument can be shortened for the actual row construction, and the rows can in fact be chosen in $\mathbb{Z}_3[T]$. Make the following truncation *before* scalar reduction. At carry d the coefficient being extracted is $[x^{L-d}]$. Every positive-degree factor of $W_r(\theta) - 1$ contributes at least one power of x: for a J factor this follows from $\theta^a(1) = 0$ for $a > 0$, and for the distinguished I_N factor its initial order is $N \ge 1$ when $N > 0$, whereas at $N = 0$ it is again J. Thus a pattern with $k > L - d$ differentiated slots has zero extracted coefficient, simultaneously for every $N \ge 0$ and every dimension. The same conclusion holds directly in the universal dimension formulation: the remaining power of J has constant term one and cannot lower these orders.

Retain only $k \le L - d$. Then

$$R \le k + b + 3d \le L + b + 2d \le 3L + b = j.$$

There are therefore only finitely many active-slot patterns for a fixed row. The derivative orders within a pattern remain infinite, but their restricted convergence is bounded as above,

and their scalar coefficients have degree at most $R \le j$ in T. Their limit is consequently an ordinary polynomial over $\mathbb{Z}_3$, after the already proved denominator compensation. This gives $D_{j,i}(T) \in \mathbb{Z}_3[T]$ with degree at most j. For the balanced error the fixed distinguished operator need not have positive derivative order; applying the same cutoff to the k differentiated *remaining* slots still gives a zero coefficient when $k > L - d$, and bounds their active union by $R \le j$. Thus even the infinite-R limit is dispensable for each individual row. The weaker A-valued formulation of the lemma was retained to avoid relying on this optional refinement.

28.3 The coefficientwise one-digit loss

Proposition 28.3 (Coefficientwise weak descent)**.** *If $3 \mid K$, $N \ge K$, and $v = v_3(K)$, then*

$$c_{K,N}(T) - \mathbf{1}_{3|N} c_{K/3,N/3}(T) \in 3^{2v-1}\mathbb{Z}_3[T]. \tag{6}$$

Proof. Put $M = K/3$. By (UW), multiplying the universal row error $3N^2A$ by $c_{M,N}(T)$ gives an error in $3M^2A$. For $j \ge K$ the boundary terms with $i < M \le L$ are also in $3M^2A$, because $M \mid L!/i!$. Thus

$$a_{T,j}\mathbf{1}_{j\ge K} - \sum_{N\ge M} c_{M,N}(T) B_T(j, 3N) \in 3M^2A.$$

For $j < K$ both triangular expressions vanish. Unit triangular inversion over $\mathbb{Z}[T]$ now gives (6) in A. Its left side is an ordinary integer polynomial; divisibility in the coefficient norm of A is exactly divisibility of its ordinary coefficients, so the assertion holds in $\mathbb{Z}_3[T]$. □

Lemma 28.4 (The three residue representatives)**.** *For dimensions $n = 0, 1, 2$, every $3 \mid K$ and $N \ge K$ satisfy*

$$c_{K,N}(n) - \mathbf{1}_{3|N} c_{K/3,N/3}(n) \in 3^{2v_3(K)}\mathbb{Z}_3. \tag{7}$$

Proof. At $n = 0$, $a_{0,j} = 0$ for $j > 0$, and hence $c_{K,N}(0) = 0$ for $K \ge 1$ by triangular inversion. At $n = 1, 2$, respectively,

$$B_n(j, N) = \binom{2j}{j - N}^{\rho}, \qquad \rho = 1, 2.$$

For completeness, these two matrices have a one-term row with precision $9N^2$. If $j = 3L + b$ and $\epsilon = \lfloor 2b/3 \rfloor$, the balanced unit-factorial congruence gives

$$B_n(j, 3N) \equiv d_j B_n(L, N) \pmod{9N^2\mathbb{Z}_3},$$

where

$$d_j = \left(3^{\epsilon}(2L+1)^{\epsilon}\frac{U(2j)}{U(j)^2}\right)^{\rho}.$$

At $N = 0$ this is the exact identity $a_{n,j} = d_j a_{n,L}$. The needed balanced estimate is elementary and independent of the weaker universal interpolation above. If $H = 3N$ and $j \ge H$, then

$$\frac{U(j)^2}{U(j-H)U(j+H)} = \prod_{\substack{j-H<i\le j \\ 3\nmid i}} (1 + H/i)^{-1} \equiv 1 \pmod{H^2\mathbb{Z}_3}.$$

Indeed the reciprocal sum over the interval is zero modulo $3^{v_3(H)}$, because its units comprise complete copies of the unit residue classes modulo $3^{v_3(H)}$, paired with their negatives; all terms of order at least two already have H^2. Now put $M = K/3$. By (UW), multiplying the one-term row error by $c_{M,N}(n)$ gives an error in $9M^2\mathbb{Z}_3 = K^2\mathbb{Z}_3$. There are no off-diagonal boundary terms. The same unit-triangular inversion as above proves (7) directly, without induction on the level. □

Theorem 28.5 (Recovery of all levels at 3)**.** *For every integer dimension $n \geq 1$, every $3 \mid K$, and every $N \geq K$,*

$$c_{K,N}(n) - \mathbf{1}_{3|N} c_{K/3,N/3}(n) \in 3^{2v_3(K)}\mathbb{Z}_3. \tag{8}$$

Consequently, using the excellent Cartier coefficient congruence (26.1), the truncation supercongruence holds at $p = 3$ for every dimension, every level, and every multiplier.

Proof. By (6), the polynomial

$$Q(T) = 3^{1-2v_3(K)}\big(c_{K,N}(T) - \mathbf{1}_{3|N} c_{K/3,N/3}(T)\big)$$

belongs to $\mathbb{Z}_3[T]$. By (7), $Q(0), Q(1), Q(2)$ are all zero modulo 3. An ordinary integral polynomial has the same value modulo 3 at integers in the same residue class; thus $Q(n) \equiv 0 \pmod 3$ for every integer n. This proves (8). Combine (8), the weighted value bound supplied by (UW), the exact tail identity, and

$$C_{3N} - C_N \circ \sigma_n \in 3^{2+2v_3(N)}\mathbb{Z}_3[[u]].$$

Taking $K = m3^s$ gives the asserted truncation precision (indeed the stronger $3^{2v_3(K)}$ precision when $3 \mid m$). □

Corollary 28.6 (The dimension Frobenius ideal)**.** *Let $3 \mid K$, let $N \geq K$, and put $v = v_3(K)$. As an ordinary polynomial in the dimension variable, the descent error satisfies*

$$\boxed{c_{K,N}(T) - \mathbf{1}_{3|N} c_{K/3,N/3}(T) \in 3^{2v-1}(T^3 - T)\mathbb{Z}_3[T] + 3^{2v}\mathbb{Z}_3[T].} \tag{9}$$

Proof. The reduction modulo 3 of the integral quotient $Q(T)$ in the preceding proof vanishes at $0, 1, 2$. Over $\mathbb{F}_3$, its three distinct linear factors therefore divide Q:

$$T(T-1)(T-2) = T^3 - T.$$

Lift the polynomial quotient to $\mathbb{Z}_3[T]$ and multiply by 3^{2v-1} to obtain (9). Since $n^3 - n$ is divisible by 3 at every integer dimension, the ideal on the right evaluates into $3^{2v}\mathbb{Z}_3$. This records exactly where the recovered factor of 3 resides. □

28.4 Why the formulation is sharp

The full coefficientwise analogue of (8) is false already at $K = N = 3$. Direct triangular inversion gives

$$c_{1,1}(T) = 2T, \qquad c_{3,3}(T) = 80T - 180T^2 + 120T^3,$$

and hence

$$\frac{c_{3,3}(T) - c_{1,1}(T)}{3} \equiv T^3 - T \pmod 3.$$

The quotient is a nonzero polynomial over $\mathbb{F}_3$ even though it vanishes at all integer dimensions modulo 3. The recovery step therefore proves full descent *at all dimension values*; it does not strengthen (6) to coefficientwise full descent. In particular, it cannot be recycled to prove (UW) without an additional argument.

29 Completion of the formal-series congruence

We spell out the last transfer to ensure that no coefficient cutoff or level cutoff is implicit. Fix a positive integer dimension n and a positive K divisible by 3, and write $M = K/3$ and $v = v_3(K)$. The proved arithmetic descent gives

$$c_{K,N}(n) \equiv \mathbf{1}_{3|N} c_{M,N/3}(n) \pmod{3^{2v}}.$$

Therefore the exact tail identity of Section 26 yields

$$1 - \frac{F_{n,K}}{F_n} \equiv \sum_{R \geq M} c_{M,R}(n) C_{3R} \pmod{3^{2v}}.$$

By the quadratic cutoff theorem,

$$v_3(c_{M,R}(n)) \geq \max\{0, 2v_3(M) - 2v_3(R)\}.$$

The excellent Cartier error in replacing C_{3R} by $C_R \circ \sigma_n$ thus has valuation at least

$$2 + 2v_3(R) + \max\{0, 2v_3(M) - 2v_3(R)\} \geq 2v.$$

Every displayed sum is u-adically locally finite, since C_R starts in degree R. It follows that

$$1 - \frac{F_{n,K}}{F_n} \equiv \sum_{R \geq M} c_{M,R}(n)(C_R \circ \sigma_n) = 1 - \frac{F_{n,M} \circ \sigma_n}{F_n \circ \sigma_n} \pmod{3^{2v}\mathbb{Z}_3[[u]]}.$$

This proves Theorem 24.1. Both F_n and $F_n \circ \sigma_n$ have constant term 1, so multiplying by them preserves this coefficientwise ideal and gives the cross-multiplied statement for $E_{n,3,s,m}$.

Previously remaining sector	Result here	Scope
$p = 3$, $n \geq 4$, $3 \nmid n$, $m = 1$	All $s \geq 5$	Every coefficient; no dimension bound
$p = 3$, $n \geq 4$, $3 \nmid n$, normalized m	All $s, m \geq 1$	Precision $3^{2s+2v_3(m)}$
$p = 3$, $3 \mid n$	Included in the same theorem	No extra argument or exclusion

30 An independent fifth-level certificate in every dimension

The following finite calculation proves an entire formal-series congruence at level five. It is independent of any induction from level five to higher levels. Its theoretical inputs are the formal excellent Cartier identity of Proposition 25.1, the negative-vertex formula, the derivative bound for A_{2K}/A_K, and ordinary-polynomial finite differences in the dimension parameter. No target truncation congruence is used to construct the lift.

Theorem 30.1. *For every hyperoctahedral dimension $n \geq 2$, with the normalized excellent lift σ_n, one has*

$$F_n(u)F_{n,81}(\sigma_n(u)) - F_n(\sigma_n(u))F_{n,243}(u) \in 59049\mathbb{Z}_3[[u]].$$

Thus every coefficient of the normalized multiplier-1 error at $p = 3$, $s = 5$ is divisible by 3^{10}. The dimension is unrestricted.

30.1 The two formal vertex inputs

Here is a justification of the inputs used by the finite verifier. Expand $1/(1 - zg_n)$ in the chamber with dominant vertex x_1. For $u = z^2$, the negative of its coefficient of x_1^{-2K}, with all other exponents zero, is

$$P_{2K}(u) = u^{-K} A_{n,K}(u), \qquad A_{n,K}(0) = 1.$$

Indeed, write $S = \sum_{i=2}^{n}(x_i + x_i^{-1})$. The identity

$$\frac{1}{1 - zg_n} = -\frac{1}{zx_1} \frac{1}{1 - (z^{-1} - S)x_1^{-1} + x_1^{-2}}$$

and the recurrence $U_{m+1}(v/2) = vU_m(v/2) - U_{m-1}(v/2)$ give

$$P_{2K} = z^{-1}\,\mathrm{CT}_{x_2,\dots,x_n}\,U_{2K-1}((z^{-1}-S)/2).$$

Expanding $U_m(v/2) = \sum_{\ell\ge 0}(-1)^\ell\binom{m-\ell}{\ell}v^{m-2\ell}$ and using $\mathrm{CT}\,S^{2r} = \binom{2r}{r}b_{n-1}(r)$ yields exactly the finite multinomial formula for $A_{n,K}$ below. The excellent Cartier identity of Proposition 25.1, and the coefficient characterization of second formal derivatives, give

$$P_{2pK}(u) - \lambda_n(u)P_{2K}(\sigma_n(u)) \in p^{2+2v_p(K)}\mathbb{Z}_p((u)), \qquad \lambda_n = F_n/F_n^{\sigma_n}. \tag{30.1}$$

This reads one vertex coefficient after inverting z; each of the displayed coefficients has bounded negative support. No assertion that all coefficients of a completed ambient ring have bounded negative support is required.

We also need $\theta(A_{2K}/A_K) \in p^{v_p(K)}\mathbb{Z}_p[[u]]$, where $\theta = u\,d/du$. To see this, the rank-two quotient of the same orbit module gives an operator $\mathcal{L} = \theta^2 - B_u\theta - A_u$ with integral coefficients. Reading its relation at the vertex gives $\mathcal{L}(P_{2K}) \in p^{2v_p(K)}\mathbb{Z}_p((u))$. Let $R_K = P_{2K}/F_n$, use the integral canonical coordinate q, and put $\partial = q\,d/dq$, $H = \theta q/q$. The period and logarithmic solution of that quotient imply

$$H = W/F_n^2 \in \mathbb{Z}_p[[u]]^\times, \qquad \theta H/H = B_u - 2\theta F_n/F_n, \qquad \mathcal{L}(F_nR) = F_nH^2\partial^2 R.$$

The canonical-coordinate and Wronskian integrality were established in the proof of Proposition 25.1; the normalization from z to u divides only by the unit 2. Hence $\partial^2 R_K \in p^{2v_p(K)}\mathbb{Z}_p((q))$. If $r = v_p(K)$, the two facts $[q^j]R_K \in \mathbb{Z}_p$ and $j^2[q^j]R_K \in p^{2r}\mathbb{Z}_p$ imply $j[q^j]R_K \in p^r\mathbb{Z}_p$ for every integer j. Thus $\theta R_K \in p^r\mathbb{Z}_p((u))$. Finally differentiate $A_{2K}/A_K = u^K R_{2K}/R_K$: the term from u^K contains K, and the other two terms contain the just-proved factor p^r. After their displayed powers of u are removed, all remaining factors are integral units. This proves the derivative estimate.

30.2 A directly cleared lift equation

Put $\mathcal{C}_K = A_{2K}/A_K$. For a general level $s \ge 2$, set

$$k = 2s-1, \qquad N = 3^{2s-2}, \qquad X = u^3, \qquad \sigma = X(1+3y).$$

The negative-vertex Cartier identities, at indices N and $2N$, give the scaled equation

$$\Phi(y) = \frac{(1+3y)^N\mathcal{C}_{3N}(u) - \mathcal{C}_N(X(1+3y))}{3N} = 0 \pmod{3^k}.$$

For its positive-degree coefficients, the derivative estimate gives $[h^r]C_N(X(1+h)) \in (N/r)\mathbb{Z}_3[[u]]$; also $\binom{N}{r} \in (N/r)\mathbb{Z}_3$. Division by $3N$ therefore makes every positive-degree coefficient of Φ integral. The constant coefficient is integral because the actual integral y satisfies the unscaled equation modulo 3^{4s-2}. Its coefficient of y is a formal-series unit with constant term one; its coefficients of y^r, $r \ge 2$, are divisible by 3. More precisely, for $r \ge 1$ the coefficient has valuation at least $r - 1 - v_3(r)$.

Multiplication by the integral unit $A_{3N}(u)A_N(X(1+3y))$ gives a substantially smaller polynomial calculation than repeatedly differentiating a quotient. Define

$$H(y) = \frac{(1+3y)^N A_{6N}(u)A_N(X(1+3y)) - A_{3N}(u)A_{2N}(X(1+3y))}{3N} = \sum_{r\ge 0} Q_r(u)y^r. \tag{30.2}$$

If

$$a_i = [h^i]A_N(X(1+h)), \qquad b_i = [h^i]A_{2N}(X(1+h)),$$

then these coefficients are calculated by the exact formula

$$Q_r = \frac{3^r}{3N}\left(A_{6N}\sum_{i=0}^{r}\binom{N}{r-i}a_i - A_{3N}b_r\right). \tag{30.3}$$

In particular, $Q_0(0) = 0$, $Q_1(0) = 1$, and $Q_r \in 3\mathbb{Z}_3[[u]]$ for $r \geq 2$.

For completeness, multiplication by the clearing unit changes the Taylor tail bound. Its coefficient of y^i is divisible by 3^i. For $r \geq 1$, a term involving the jth coefficient of Φ, $1 \leq j \leq r$, therefore has valuation at least $r - 1 - v_3(j)$. The term from $j = 0$ has valuation at least r. Consequently

$$v_3(Q_r) \geq r - 1 - \lfloor \log_3 r \rfloor \qquad (r \geq 1). \tag{30.4}$$

At $s = 5$ the parameters are

$$k = 9, \quad N = 6561, \quad M_y = 3^9 = 19683, \quad M = 3^{10} = 59049, \quad M_{\text{work}} = 3^{18} = 387420489.$$

The bound in (30.4) is at least nine for every $r \geq 12$. Hence the degree-11 equation $\sum_{r=0}^{11} Q_r y^r = 0 \pmod{19683}$ is sufficient. Formula (30.3) divides by $3N = 3^9$, so computing its numerator modulo 3^{18} retains all nine required digits.

30.3 Exact scaled Bessel convolutions

The large vertex indices are

$$N = 6561, \quad 2N = 13122, \quad 3N = 19683, \quad 6N = 39366.$$

A dense matrix of multinomial coefficients is unnecessary. Write

$$b_d(j) = \sum_{k_1+\cdots+k_d=j}\binom{j}{k_1,\ldots,k_d}^2, \qquad J_3(x) = \sum_{j\geq 0}\frac{3^j}{(j!)^2}x^j.$$

Legendre's formula gives

$$v_3\left(\frac{3^j}{(j!)^2}\right) = s_3(j) \geq 0,$$

where $s_3(j)$ is the ternary digit sum. Thus $J_3 \in \mathbb{Z}_3[[x]]$ and

$$[x^j]J_3(x)^d = \frac{3^j b_d(j)}{(j!)^2}.$$

The negative-vertex polynomial has the finite expression

$$A_{n,K}(u) = \sum_{j=0}^{K-1}\frac{u^j}{(2K-2j-1)!}\sum_{\ell=0}^{j}\frac{(-1)^\ell(2K-\ell-1)!}{\ell!(j-\ell)!^2}b_{n-1}(j-\ell).$$

Define

$$h_K = \max_{0\leq\ell<K}\{K - \ell - v_3((2K-\ell-1)!) + v_3(\ell!)\}, \qquad \alpha_{K,\ell} = (-1)^\ell 3^{\ell-K+h_K}\frac{(2K-\ell-1)!}{\ell!}.$$

Every $\alpha_{K,\ell}$ is 3-integral by the definition of h_K. The coefficient formula becomes one ordinary convolution:

$$[u^j]A_{n,K} = \frac{3^{K-j-h_K}}{(2K-2j-1)!}[x^j]\left(\sum_{\ell=0}^{K-1}\alpha_{K,\ell}x^\ell\right)J_3(x)^{n-1}, \qquad 0 \leq j < K. \tag{30.5}$$

If the factorial in front is separated into its unit part and its power of 3, the valuation of the prefactor is

$$K - j - h_K - v_3((2K-2j-1)!) = \frac{1 + s_3(2K-2j-1)}{2} - h_K \geq 1 - h_K.$$

Therefore convolution modulo 3^{E+H}, with $H = \max_K h_K$, determines every required vertex coefficient modulo 3^E. At level five $E = 18$ and $H = 10$, so the implementation uses 3^{28}. All divisions by powers of 3 are exact and are checked. Only coefficients below the relevant finite vertex degree are required in (30.5); this finite polynomial calculation imposes no cutoff on the final formal-series assertion.

All convolutions use exact integer multiplication. Coefficients are packed in a base $B = 2^{8w}$ satisfying

$$B > \min(a, b)(m-1)^2,$$

where a, b are the input lengths and m is the modulus. Thus no convolution coefficient can carry into the next packed block. GMP multiplies the resulting nonnegative integers exactly, after which the blocks are reduced modulo m. No floating-point arithmetic or unchecked fixed-width multiplication is used. The scaled formula was also checked against every coefficient of the original factorial formula at several small vertex indices and against the archived complete level-two and level-three vertex certificates.

30.4 The ordinary dimension model

The number $b_n(j)$ counts ordered pairs of length-j words in an n-letter alphabet with the same letter multiplicities. For words using exactly r prescribed letters, simultaneous relabelling by S_r acts freely. Their number is therefore $r!c_{j,r}$ with $c_{j,r} \in \mathbb{Z}$. It follows that $b_n(j) = \sum_r c_{j,r}(n)_r \in \mathbb{Z}[n]$. Every factorial ratio in the vertex formula is an integer multinomial coefficient, so $A_{n,K} \in \mathbb{Z}[n, u]$ as well. Before reduction modulo 3^9, formula (30.3) has the uniform denominator bound

$$Q_r(T, u) \in 3^{-9}\mathbb{Z}_3[T, u].$$

The finite-difference fact needed here is elementary. If $P(T) \in 3^{-a}\mathbb{Z}_3[T]$, then

$$\Delta^r P(c) \in 3^{-a} r!\mathbb{Z}_3,$$

since finite differences of monomials are multiples of $r!$. Consequently, if $P \in \mathbb{Z}_3[T]$ takes values divisible by 3^e at $\nu_3(e)$ consecutive integers, it does so at every integer: in the Newton expansion $P(c + X) = \sum_r \Delta^r P(c)\binom{X}{r}$, the initial differences are integer combinations of those values and the later differences are divisible by 3^e because of $r!$. This holds coefficientwise with an extra polynomial variable u.

Recall $\nu_3(e) = \min\{r : v_3(r!) \geq e\}$. Here

$$\nu_3(18) = 39, \qquad \nu_3(9) = 21, \qquad \nu_3(10) = 24.$$

The calculation uses 39 consecutive input dimensions $n = 2, \ldots, 40$. For every $0 \leq r \leq 11$ and $0 \leq j < 39$ it checks every coefficient of

$$\Delta^j Q_r(2, u) \in 3^{\min(9, v_3(j!))}\mathbb{Z}_3[u]. \tag{30.6}$$

For $j \geq 39$, the ordinary-polynomial denominator bound makes the Newton difference divisible by 3^9 automatically. The retained differences of orders $0, \ldots, 20$, divided by $j!$ and expressed in the falling-factorial basis, give fixed polynomials

$$\widetilde{Q}_r(T, u) \in \mathbb{Z}_3[T, u], \qquad \deg_T \widetilde{Q}_r \leq 20,$$

with

$$\widetilde{Q}_r(n,u) \equiv Q_r(n,u) \pmod{3^9} \qquad \text{for every integer } n \geq 2.$$

All 39 input evaluations of this model are additionally checked against their original coefficients.

Put $B = \widetilde{Q}_1$ and define

$$y_0 = 0, \qquad y_{j+1} = -\frac{\widetilde{Q}_0 + \sum_{r=2}^{11} \widetilde{Q}_r y_j^r}{B} \qquad (0 \leq j < 9).$$

This gives fixed finite polynomials $Y_*, Z_* \in \mathbb{Z}_3[T,u]$ with $y_9 = Y_*/Z_*$. For example one can take $Z_* = B^{235794769}$, using $e_0 = 0$ and $e_{j+1} = 11e_j + 1$. It is unnecessary to expand these enormous universal polynomials. For each integer n, $B(n,u)$ is a unit and every specialized nonlinear coefficient is divisible by 3. The iteration therefore contracts by 3 after specialization and gives

$$Y_*(n,u)/Z_*(n,u) \equiv y_n(u) \pmod{3^9}.$$

This argument uses divisibility at integer dimensions; it does not assume that every ordinary coefficient in the indeterminate T of $\widetilde{Q}_r$ is divisible by 3.

30.5 Twenty-four complete target identities

For each $n = 2, \ldots, 25$, the program constructs a compact rational root Y_n/Z_n with $Z_n(0)$ a unit, and independently verifies

$$\sum_{r=0}^{11} Q_r(n,u) Y_n^r Z_n^{11-r} = 0 \pmod{3^9}.$$

The digit construction is rational Hensel lifting in $\mathbb{F}_3(u)$. It starts with $-Q_0/Q_1$ modulo 3 and corrects one digit at a time. Each residual division by the current power of 3 is checked exactly. Rational cancellation is accelerated, when applicable, by factoring the denominator into powers of $1+u$ and $1-u$ using ternary-digit Taylor translation. The final root identity makes correctness independent of trusting that acceleration.

Let $F = F_{n,243}$, $F^- = F_{n,81}$ and $X = u^3$. Define integral polynomials, ordinary in n, by

$$\begin{aligned} L_r(n,u) = A_{n,3N}(u)\big([h^r]F^-(X(1+h)) + N[h^{r-1}]F^-(X(1+h))\big) \\ - F(u)[h^r]A_{n,N}(X(1+h)), \qquad 0 \leq r \leq 9. \end{aligned}$$

where the coefficient with index -1 is zero. This uses $(1+3y)^N \equiv 1 + 3Ny \pmod{3^{10}}$. At every one of the 24 dimensions, every coefficient of the following finite polynomial is checked:

$$\sum_{r=0}^{9} L_r(n,u)(3Y_n)^r Z_n^{9-r} = 0 \pmod{59049}. \tag{30.7}$$

The largest degree bound among these exact target calculations is 141406. For example, dimension four has rational degrees $(\deg Y_4, \deg Z_4) = (9865, 9856)$ and target degree bound 108664; dimension five has degrees $(13548, 10263)$ and target degree bound 141397.

Now use the single universal ordinary polynomial

$$\mathcal{T}_*(T,u) = \sum_{r=0}^{9} L_r(T,u)(3Y_*(T,u))^r Z_*(T,u)^{9-r} \in \mathbb{Z}_3[T,u].$$

Hensel uniqueness identifies the compact root and the universal root modulo 3^9 at each testing dimension. Multiplication by 3 improves that precision to 3^{10} in the target variable. Since all denominators have unit constant term, (30.7) gives $\mathcal{T}_*(n,u) \equiv 0 \pmod{3^{10}}$ for $n = 2, \ldots, 25$. The finite-difference criterion just proved, applied coefficientwise to this fixed ordinary polynomial, propagates the congruence from $24 = \nu_3(10)$ consecutive dimensions to every $n \geq 2$. Restoring the unit denominators and the negative-vertex expression for the Cartier multiplier proves $\lambda_n F_{n,81}(\sigma_n) - F_{n,243} \in 3^{10}\mathbb{Z}_3[[u]]$. Multiplication by the unit $F_n(\sigma_n)$ proves Theorem 30.1.

30.6 Reproduction and archived data

The complete source programs are printed in Appendix J. They can be saved in `research/octa_level5`. A reproduction which recalculates even previously cached data is

```
python3 scaled_convolution.py
python3 linear_reduce.py
python3 direct_certificate.py --test
python3 uniform_level.py --s 5 --stage all --recompute
```

The scripts import the elementary polynomial and earlier certificate modules in the sibling `octa_new_certificates` directory. The archive contains 39 exact input files, the Newton differences and ordinary model, 24 full compact certificates, and a summary with `power_series_cutoff: null`. The aggregate identifier of the model and the 24 target certificates is

`013f86b4dc5cb189b9be96b71c0eac752cffb93dbf0428d0f04b3f50674b0529.`

The identifier records the archived objects; the proof consists of the exact algebraic reductions and the complete finite polynomial identities, not the identifier itself.

31 Verification and exact scope

The main theorem is an algebraic all-level proof. It depends on the formal excellent Cartier construction in Section 25, the exact cutoff kernel, the universal coefficientwise weak row, and the scalar residue representatives. It does not assume that a finite set of tested levels controls subsequent levels. In particular, the transition from $s = 5$ to arbitrary s uses no extrapolation.

The ordinary-polynomial qualification is essential. Integer-valued polynomials alone would not justify reduction of dimensions modulo 3. Conversely, the final descent is asserted at every integer dimension: its full-precision coefficientwise version in the indeterminate T is false, as the explicit $T^3 - T$ example demonstrates.

The formal base ring of this part is $\mathbb{Z}_3[[u]]$. Corollary 1.5 supplies the transfer to the standard analytic ring at BV3-admissible primes. An analytic realization at every additional small prime, evaluation at a specified fixed point, and identification with a geometric unit root require their own analytic and geometric verification. The prime 2 is outside the Cartier input used here. This manuscript and its exact certificates have been checked internally; they have not undergone external peer review or proof-assistant formalization.

Part V
Modular Hecke traces and stronger congruences

The complete modular framework, thirteen cusp-type families, elliptic zeros, and a primal/dual Laurent proof for Zagier D.

32 Modular trace methods and their conclusions

32.1 Dwork congruences and the excellent lift

For a power series $F(t) = \sum_{n\geq 0} a_n t^n \in 1 + t\mathbb{Z}[[t]]$ and an integer $M \geq 1$ we write $F_M(t) = \sum_{n<M} a_n t^n$ for its truncation. Let p be a prime. A *Frobenius lift* is a power series $t^\sigma \in t\,\mathbb{Z}_p[[t]]$ with $t^\sigma \equiv t^p \pmod p$. When a_n is the constant term of the n-th power of a Laurent polynomial, Mellit–Vlasenko [17] and Beukers–Vlasenko [5, 7] proved that for every Frobenius lift

$$\frac{F(t)}{F(t^\sigma)} \equiv \frac{F_{mp^s}(t)}{F_{mp^{s-1}}(t^\sigma)} \pmod{p^s} \qquad (m, s \geq 1) \tag{32.1}$$

as power series in t. For the lift $t \mapsto t^p$ and the Legendre family this goes back to Dwork [15]. In the situations of interest F satisfies a differential equation with maximal unipotent monodromy at $t = 0$. Let $F(t)\log t + G(t)$ be its second Frobenius solution and let $q(t) = t\exp(G(t)/F(t))$ be the canonical coordinate; its inverse $t(q)$ is the mirror map. Dwork singled out the *excellent* Frobenius lift, which in these terms is

$$t^\sigma = t(\gamma^{p-1} q(t)^p) \qquad ([7, \text{Def. } 7.14]),$$

where γ is the vertex coefficient of the Laurent polynomial; in all cases below $\gamma = 1$, so $t^\sigma(t(q)) = t(q^p)$. Beukers and Vlasenko [7, Thm. 7.3] showed that for completely symmetric Calabi–Yau families $f = 1 - tg(x)$ the excellent lift is the unique Frobenius lift for which the Cartier operator acts on $1/f$ by the scalar $F(t)/F(t^\sigma)$ modulo p^2F_2 rather than only modulo pF_1. They then wrote: *"Our second main objective was to show that (19) holds modulo p^{2s} when $t \mapsto t^\sigma$ is an excellent Frobenius lift. Unfortunately we were unable to prove this for any example of g."* Their (19) is our (32.1), and they formulated the following conjecture (see also the survey [22]).

Conjecture BV (Beukers–Vlasenko [7, Conj. 7.5]). *Congruence (32.1) with the excellent lift holds modulo p^{2s} for $m = 2$ in the hypercubic and hyperoctahedral families and for $m = n + 1$ in the simplicial families.*

Here n is the number of variables, $F(t) = \sum_k \mathrm{CT}(g^k)t^k$, and

$$g_{\text{hypercubic}} = \prod_{i=1}^{n}\Big(x_i + \frac{1}{x_i}\Big), \quad g_{\text{hyperoctahedral}} = \sum_{i=1}^{n}\Big(x_i + \frac{1}{x_i}\Big), \quad g_{\text{simplicial}} = x_1 + \cdots + x_n + \frac{1}{x_1 \cdots x_n}.$$

The primes are those of [7, Thm. 7.3]: $p \nmid \#G \cdot [\mathbb{Z}^n : \Gamma_g]$, with G the symmetry group of g and Γ_g the lattice spanned by its support. Beukers and Vlasenko also observe [7, §8] that for $n = 2$ the hypercubic family is modular: its mirror map satisfies $t(q)^2 = \lambda(q^2)/16$.

Beukers [8] proves first-level congruences modulo p^2 or p^3 for many modular families and asks about their extension to higher levels. The trace inductions in this part supply the higher-level statements under the hypotheses stated below. Fu [16] studies a cohomological criterion for supercongruences associated with rank-two factors and excellent-lift fixed points; that criterion is not an input to any proof in this volume.

32.2 Results

The precise hypotheses are collected in Definition 33.2. Informally, a modular family of *cusp type* consists of a Hauptmodul $t \in q + q^2\mathbb{Z}[[q]]$ for a genus-zero group $\Gamma \supseteq \Gamma_0(N)$ and a holomorphic modular form $\Theta \in 1 + q\mathbb{Z}[[q]]$ of weight $w \in \{1, 2\}$ which has no zeros in the upper half plane and vanishes only at the cusps where t has its pole. Let $F \in 1 + t\mathbb{Z}[[t]]$ be defined by $F(t(q)) = \Theta(q)$ and let $t^\sigma \in t^p\mathbb{Z}[[t]]$ be the excellent lift, $t^\sigma(t(q)) = t(q^p)$.

Theorem 32.1 (Main Theorem)**.** *Let $(N, \Gamma, w, \chi, \Theta, t)$ be a modular family of cusp type and let $p \nmid N$ be a prime for which the Hecke condition (C_p) of Definition 33.2 holds (by Lemma 33.3 this is automatic in all examples of Table 1). Then for all integers $m, s \geq 1$*

$$\frac{F(t)}{F(t^\sigma)} \equiv \frac{F_{mp^s}(t)}{F_{mp^{s-1}}(t^\sigma)} \pmod{p^{(w+1)s}\mathbb{Z}_p[[t]]}.$$

Equivalently, $\Theta(q)\, F_{mp^{s-1}}(t(q^p)) \equiv \Theta(q^p)\, F_{mp^s}(t(q)) \pmod{p^{(w+1)s}\mathbb{Z}[[q]]}$.

family	w	Γ	χ	Θ	t	cusps	r_Θ
Legendre, $\binom{2n}{n}^2$	1	$\Gamma_0(4)$	χ_{-4}	$\eta_2^{10}/\eta_1^4\eta_4^4$	$\eta_1^8\eta_4^{16}/\eta_2^{24}$	$\frac{1}{2}$	$\frac{1}{2}$
Zagier A (Franel)	1	$\Gamma_0(6)$	χ_{-3}	$\eta_2\eta_3^6/\eta_1^2\eta_6^3$	$\eta_1^3\eta_6^9/\eta_2^3\eta_3^9$	$\frac{1}{3}$	1
Zagier B	1	$\Gamma_0(9)$	χ_{-3}	η_1^3/η_3	η_9^3/η_1^3	0	1
Zagier C	1	$\Gamma_0(6)$	χ_{-3}	$\eta_2^6\eta_3/\eta_1^3\eta_6^2$	$\eta_1^4\eta_6^8/\eta_2^8\eta_3^4$	$\frac{1}{2}$	1
Zagier E	1	$\Gamma_0(8)$	χ_{-4}	$\eta_2^{10}/\eta_1^4\eta_4^4$	$\eta_1^4\eta_4^2\eta_8^4/\eta_2^{10}$	$\frac{1}{2}$	1
Zagier F	1	$\Gamma_0(6)$	χ_{-3}	$\eta_1^6\eta_6/\eta_2^3\eta_3^2$	$\eta_2\eta_6^5/\eta_1^5\eta_3$	0	1
$\binom{2n}{n}^3$	2	$\Gamma_0(4)+4$	1	$\eta_2^{20}/\eta_1^8\eta_4^8$	$\eta_1^{24}\eta_4^{24}/\eta_2^{48}$	$\frac{1}{2}$	$\frac{1}{2}$
Apéry (γ)	2	$\Gamma_0(6)+6$	1	$\eta_2^7\eta_3^7/\eta_1^5\eta_6^5$	$\eta_1^{12}\eta_6^{12}/\eta_2^{12}\eta_3^{12}$	$\frac{1}{2}, \frac{1}{3}$	1
Almkvist (η)	2	$\Gamma_0(5)$	$(\frac{\cdot}{5})$	η_1^5/η_5	η_5^6/η_1^6	0	1
Domb (α)	2	$\Gamma_0(6)+3$	1	$\eta_1^4\eta_3^4/\eta_2^2\eta_6^2$	$\eta_2^6\eta_6^6/\eta_1^6\eta_3^6$	$0, \frac{1}{3}$	1
Almkvist–Zudilin (δ)	2	$\Gamma_0(6)+2$	1	$\eta_1^3\eta_2^3/\eta_3\eta_6$	$\eta_3^4\eta_6^4/\eta_1^4\eta_2^4$	$0, \frac{1}{2}$	1
Almkvist (ε)	2	$\Gamma_0(8)+8$	1	$\eta_2^6\eta_4^6/\eta_1^4\eta_8^4$	$\eta_1^8\eta_8^8/\eta_2^8\eta_4^8$	$\frac{1}{2}, \frac{1}{4}$	1
Almkvist (ζ)	2	$\Gamma_0(9)+9$	1	$\eta_3^{10}/\eta_1^3\eta_9^3$	$\eta_1^6\eta_9^6/\eta_3^{12}$	$\frac{1}{3}, \frac{2}{3}$	1

Table 1: Modular families of cusp type, with $\eta_\delta = \eta(\delta\tau)$. The data are those of [8, App. B, C]; "cusps" lists the $\Gamma_0(N)$-cusps in Γc, where Θ vanishes and t has its pole. The coefficients a_n are listed in §33.5.

The hypothesis that Θ vanishes only at cusps excludes several natural families. Our second and third results treat the most prominent ones.

Theorem 32.2. *Let F be one of*

$$F_{27}(t) = \sum_{n\geq 0} \frac{(3n)!}{n!^3} t^n, \qquad F_{256}(t) = \sum_{n\geq 0} \frac{(4n)!}{n!^4} t^n, \qquad F_{108}(t) = \sum_{n\geq 0} \frac{(2n)!\,(3n)!}{n!^5} t^n,$$

that is ${}_2F_1(\frac{1}{3}, \frac{2}{3}; 1; 27t)$, ${}_3F_2(\frac{1}{4}, \frac{1}{2}, \frac{3}{4}; 1, 1; 256t)$ and ${}_3F_2(\frac{1}{3}, \frac{1}{2}, \frac{2}{3}; 1, 1; 108t)$, with $w = 1, 2, 2$ respectively, and let t^σ be the excellent lift for the modular parametrisation of Table 2. Then for every prime $p \geq 5$ and all $m, s \geq 1$

$$\frac{F(t)}{F(t^\sigma)} \equiv \frac{F_{mp^s}(t)}{F_{mp^{s-1}}(t^\sigma)} \pmod{p^{(w+1)s}\mathbb{Z}_p[[t]]}.$$

Here Θ vanishes at an elliptic point of Γ, and the trace of Q_P is a rational function of t with poles at CM points of conductor p. For $s = m = 1$, Theorem 32.2 is due to Beukers [8, Thms. 1.4, 1.13 and Cor. 6.2].

Theorem 32.3. *Let $a_n = \sum_k \binom{n}{k}^2 \binom{n+k}{k}$ be the Apéry numbers for $\zeta(2)$ (Zagier's case D), $F(t) = \sum a_n t^n$, and let t^σ be the excellent lift, $t^\sigma(t(q)) = t(q^p)$ with $t(q) = q\prod_{n\geq 1}(1-q^n)^{5(\frac{n}{5})}$. Then for every prime $p \neq 5$ and all $m, s \geq 1$*

$$\frac{F(t)}{F(t^\sigma)} \equiv \frac{F_{mp^s}(t)}{F_{mp^{s-1}}(t^\sigma)} \pmod{p^{2s}\mathbb{Z}_p[[t]]}.$$

This family lives on $\Gamma_1(5)$, and for $p \equiv \pm 2 \pmod 5$ the Hecke operator involves the diamond operator $\langle 2\rangle$, which does not fix t. Even the case $s = m = 1$ is not covered by [8, Thm. 1.4]; Beukers remarks that computer experiments support it [8, Remark 1.3]. In Theorems 32.2 and 32.3 the modular $t(q)$ is the normalized mirror map of the differential equation of F, as in the parametrizations of [8, Appendices B–C]; hence t^σ is the excellent lift. The exact coordinate normalization is treated in Lemma 36.1; checks [8g] and [9a] are supplementary finite checks.

Corollary 32.4 (Conjecture BV in dimensions two and three)**.** *Conjecture BV holds for the hypercubic and the hyperoctahedral families with $n = 2$ (for every odd prime p), for the hypercubic family with $n = 3$ (for every prime $p \geq 5$), and for the simplicial families with $n = 2$ and $n = 3$ (for every prime $p \geq 5$). For $n = 3$ the hypercubic and simplicial congruences even hold modulo p^{3s}. For every family and dimension explicitly listed in this corollary, the same assertion holds for every original truncation multiplier divisible by 2 in the hypercubic or hyperoctahedral case, or by $n + 1$ in the simplicial case. The exponent is $2s$ in dimension two and $3s$ for the hypercubic and simplicial families in dimension three.*

The three-dimensional hyperoctahedral family has the $2s$ bound of Theorem 2.1; no $3s$ assertion for that family is made here. The same theorem supplies the uniform result in every dimension. Section 39.1 explains why the termwise modular estimate alone does not supply it.

Corollary 32.5 (Beukers's question)**.** *For every family of Table 1, every prime $p \nmid N$ and all $m, s \geq 1$, the excellent lift congruence holds modulo p^{2s} in weight one and modulo p^{3s} in weight two. This covers every family of [8, App. B, C] that satisfies Beukers's Assumptions (I) or (II) with Θ an η-product.*

Corollary 32.6. *For every family of Table 1, $p \nmid N$ and $m, s \geq 1$, $a_{mp^s} \equiv a_{mp^{s-1}} \pmod{p^{(w+1)s}}$.*

Corollary 32.6 is not new. For the Apéry numbers it is Coster's theorem [12] (conjectured in [9]). For the sporadic sequences these two-term supercongruences were conjectured by Osburn, Sahu and Straub [19] and proved through the work of several authors, the last cases in [21, 2]. Here it comes out of the same argument as the power series congruences.

Remark 32.7. For $s = m = 1$, Theorem 32.1 is Beukers's Theorems 1.4 and 1.13 [8] in the η-product cases. Our proof of the case $s = 1$ is a trace version of his argument. It needs only that the zeros of Θ are cusps, not the η-product identity $\Theta(p\tau)\prod_k \Theta(\frac{\tau+k}{p}) = c\,\Theta(\tau)^{p+1}$ of [8, Lemma 3.2].

32.3 Method

Let $P \in \mathbb{Z}[T]$ be a polynomial and put

$$Q_P(\tau) = \frac{\Theta(\tau)\,P(t(p\tau))}{\Theta(p\tau)}, \qquad G_P(z) = \frac{\Theta(z^p)\,P(t(z))}{\Theta(z)} \in \mathbb{Z}[[z]].$$

The function Q_P is modular for $\Gamma_0(Np)$, and its $p+1$ conjugates under Γ are indexed by the Hecke cosets of level p. The conjugate for $\tau \mapsto p\tau$ is Q_P itself. The other p conjugates, for $\tau \mapsto (\tau + k)/p$, add up to $p^{w+1}\chi(p)\,U_p G_P$, where U_p extracts the coefficients of q^{pn}. Hence the trace is

$$\operatorname{Tr}(Q_P) = Q_P + p^{w+1}\chi(p)\,U_p G_P. \tag{32.2}$$

For a family of cusp type the trace has no poles in $\mathbb{H}$, so it is a polynomial $\Pi_P(t) \in \mathbb{Z}[t]$. A local computation at the cusp bounds its degree by $p \deg P + (p-1)r_\Theta$ (Proposition 34.4). Now take $P = F_{mp^{s-1}}$. The induction hypothesis says $G_P \equiv F_{mp^{s-2}}(t(z^p))$ modulo $p^{(w+1)(s-1)}$, so $p^{w+1}U_pG_P \equiv p^{w+1}F_{mp^{s-2}}(t)$ modulo $p^{(w+1)s}$. Thus Q_P is congruent modulo $p^{(w+1)s}$ to a polynomial in t of degree $< mp^s$. Since $Q_P \equiv F(t)$ modulo t^{mp^s}, this polynomial is $F_{mp^s}(t)$, and that is the theorem. In the language of Hecke operators on forms of weight $-w$ the induction is the three-term recursion

$$h_s \equiv p^{w+1}\chi(p)(T_p h_{s-1} - h_{s-2}) \pmod{p^{(w+1)s}}, \qquad h_s := F_{mp^s}(t)/\Theta$$

(Remark 34.5), in the spirit of Atkin–Swinnerton-Dyer congruences.

In the language of Dwork crystals, the excellent lift is the branch $\tau \mapsto p\tau$ of the Hecke correspondence; Beukers–Vlasenko characterise it as the lift for which the off-diagonal Cartier coefficient λ_1 vanishes. The remaining branches carry an explicit factor p^w, and this factor doubles or triples the exponent of (32.1).

Elliptic zeros. For the families of Theorem 32.2, Θ vanishes at an elliptic point e of order k_0, like t^{-r}, and the trace has poles at the Hecke translates of e. We write the trace of Q_P for $P = F_{mp^{s-1}} = \sum_i a_i t^i$ as $\sum_i a_i S_i$, where S_i is the trace of $\Theta(\tau)t(p\tau)^i/\Theta(p\tau)$. The functions S_i satisfy a linear recursion in i whose coefficients are the symmetric functions of the $p+1$ Hecke translates $t \circ \beta$. These are rational functions in t whose denominators are powers of a polynomial $D \equiv \pm 1 \pmod p$, essentially Beukers's polynomial $T_{p,p+1}$ [8]. A Gauss norm estimate on the disc $|t|_p < p^{k_0/(p+1)}$ gives

$$v_p([t^n]S_i) \geq \frac{k_0}{p+1}(n - pi - \lfloor pr \rfloor)$$

(Proposition 37.5). So the coefficients of t^n, $n \geq mp^s$, of the trace come essentially from monomials t^i near the top of the truncation. For those, the base-p digits of i are $p-1$, and by Legendre's formula each such digit contributes $w+1$ factors p to a_i. The two effects balance exactly, and the induction goes through.

Zagier's case D. Here $\Gamma = \Gamma_1(5)$; Θ vanishes at the cusp $\frac{2}{5}$, where t has its pole. For $p \equiv \pm 1 \pmod 5$ the argument for cusp type applies. For $p \equiv \pm 2$ the Hecke correspondence on $X_1(5)$ contains the diamond operator $\langle 2 \rangle$, which maps t to $-1/t$ and Θ to $\pm t\Theta$. The trace of Q_P is then a Laurent polynomial in t, but Q_P is a branch of the trace for the dual family $(t\Theta, -1/t)$, and conversely. We prove the congruence and its dual together, by a joint induction (§38).

32.4 Limits of the termwise modular method

The hyperoctahedral family with $n = 3$ has a modular parametrisation with an elliptic zero on $\Gamma_0(6)+$, with $1/r = 2 < w+1 = 3$. The termwise decay estimate below is insufficient to recover the required cancellation (Section 39.1). The full $2s$ bound follows from Theorem 2.1, and its sharpness is proved in Theorem 44.2. The uniform higher-dimensional argument also comes from Part I; it does not require modular parametrisations of the higher-dimensional periods. The broader elliptic-zero assertion in Conjecture 39.1 remains a separate extension beyond the listed families.

Organisation

§33 sets up notation and hypotheses; §34 proves the trace identity and the polynomiality; §35 proves Theorem 32.1; §36 proves the corollaries; §37 proves Theorem 32.2; §38 proves Theorem 32.3; §39 discusses the limits of the modular method and a broader extension; §40 describes the verification script, which is reproduced in the appendix.

33 Modular families of cusp type

33.1 Notation

Let $\mathbb{H}$ be the upper half plane and $q = e^{2\pi i\tau}$. For $\gamma = \left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right) \in \mathrm{GL}_2^+(\mathbb{Q})$ and a function f on $\mathbb{H}$ put $(f|_w\gamma)(\tau) = \det(\gamma)^{w/2}(c\tau+d)^{-w}f(\gamma\tau)$, so that $(f|_w\gamma_1)|_w\gamma_2 = f|_w(\gamma_1\gamma_2)$ and $f|_w(\lambda I) = \mathrm{sgn}(\lambda)^w f$ for $\lambda \in \mathbb{Q}^\times$; we write $j(\gamma,\tau) = c\tau+d$ and $T = \left(\begin{smallmatrix} 1 & 1 \\ 0 & 1 \end{smallmatrix}\right)$. For a power series $h = \sum h_n q^n$ let $U_p h = \sum h_{pn} q^n$. If $h(z) \in \mathbb{C}[[z]]$ and $q^{1/p} := e^{2\pi i\tau/p}$, $\zeta_p = e^{2\pi i/p}$, then

$$\sum_{k=0}^{p-1} h(\zeta_p^k q^{1/p}) = p\,(U_p h)(q), \qquad U_p(h_1(z^p)h_2(z)) = h_1(q)\,U_p h_2(q). \tag{33.1}$$

For a cusp c and $A \in \mathrm{SL}_2(\mathbb{Z})$ with $A\infty = c$, and a function f which is meromorphic at c (of weight 0, or of weight w), the function $f \circ A$, respectively $f|_w A$, has an expansion $\sum_\nu b_\nu e^{2\pi i\nu\tau}$ with ν in a lattice $\frac{1}{H}\mathbb{Z}$ bounded from below. We write $\nu_A(f) = \min\{\nu : b_\nu \neq 0\}$. Clearly $\nu_A(f_1f_2) = \nu_A(f_1) + \nu_A(f_2)$ and $\nu_A(f_1+f_2) \geq \min(\nu_A(f_1), \nu_A(f_2))$, and ν_A does not change when A is replaced by $\pm AT^x$ or, for f modular for $\Gamma_0(N)$, by γA with $\gamma \in \Gamma_0(N)$.

33.2 Hecke cosets

Let $p \nmid N$ and $M(N,p) = \left\{\left(\begin{smallmatrix} a & b \\ Nc & d \end{smallmatrix}\right) \in M_2(\mathbb{Z}) : ad - Nbc = p\right\}$; let M_p be the set of all integral matrices of determinant p.

Lemma 33.1. (H1) *$M(N,p) = \Gamma_0(N)\left(\begin{smallmatrix} 1 & 0 \\ 0 & p \end{smallmatrix}\right)\Gamma_0(N)$, and $\Gamma_0(N)\backslash M(N,p)$ has the $p+1$ representatives $\beta_\infty = \left(\begin{smallmatrix} p & 0 \\ 0 & 1 \end{smallmatrix}\right)$ and $\beta_k = \left(\begin{smallmatrix} 1 & k \\ 0 & p \end{smallmatrix}\right)$, $0 \leq k \leq p-1$.*

(H2) *The map $\Gamma_0(N)\beta \mapsto \mathrm{SL}_2(\mathbb{Z})\beta$ is a bijection $\Gamma_0(N)\backslash M(N,p) \to \mathrm{SL}_2(\mathbb{Z})\backslash M_p$.*

(H3) *$M(N,p)\gamma = M(N,p)$ for $\gamma \in \Gamma_0(N)$; right multiplication permutes the cosets.*

(H4) *If g is an Atkin–Lehner matrix for $\Gamma_0(N)$, then $g^{-1}M(N,p)g = M(N,p)$, and $\beta \mapsto g^{-1}\beta g$ permutes $\Gamma_0(N)\backslash M(N,p)$.*

(H5) *$\mathrm{adj}(\beta) = p\beta^{-1} \in M(N,p)$ for $\beta \in M(N,p)$.*

Proof. (H1) is standard, see [13, §5.2] and [8, §3]. Note that $\gcd(a,N) = \gcd(d,N) = 1$ holds automatically for $\left(\begin{smallmatrix} a & b \\ Nc & d \end{smallmatrix}\right) \in M(N,p)$. For (H2): by Hermite normal form every element of M_p is $\mathrm{SL}_2(\mathbb{Z})$-equivalent to exactly one $\left(\begin{smallmatrix} a' & b' \\ 0 & d' \end{smallmatrix}\right)$ with $a'd' = p$, $0 \leq b' < d'$. These are the $p+1$ matrices of (H1), so the map is surjective between sets of the same cardinality. (H3) and (H5) are clear. (H4) goes back to Atkin–Lehner [1]; for completeness let $g = \left(\begin{smallmatrix} ea & b \\ Nc & ed \end{smallmatrix}\right)$ with $e \parallel N$ and $\det g = e$. A direct computation gives $g^{-1}\left(\begin{smallmatrix} 1 & 0 \\ 0 & p \end{smallmatrix}\right)g = \left(\begin{smallmatrix} ead - p(N/e)bc & (1-p)bd \\ (p-1)Nac & pead - (N/e)bc \end{smallmatrix}\right) \in M(N,p)$. Since g normalises $\Gamma_0(N)$, (H1) gives $g^{-1}M(N,p)g \subseteq M(N,p)$; the same holds for g^{-1}, which is a scalar multiple of an Atkin–Lehner matrix, so equality holds, and $g^{-1}\Gamma_0(N)\beta g = \Gamma_0(N)g^{-1}\beta g$. □

For a Dirichlet character χ modulo N with values in $\{\pm1\}$ and $\beta = \left(\begin{smallmatrix} a & b \\ Nc & d \end{smallmatrix}\right) \in M(N,p) \cup \Gamma_0(N)$ we put $\chi(\beta) = \chi(d)$. Then $\chi(\gamma\beta) = \chi(\gamma)\chi(\beta) = \chi(\beta\gamma)$ for $\gamma \in \Gamma_0(N)$, because the lower right entries multiply modulo N.

33.3 The hypotheses

Definition 33.2. A *modular family of cusp type* is a tuple $(N, \Gamma, w, \chi, \Theta, t)$ with the following properties.

(F1) $w \in \{1,2\}$. Either $\Gamma = \Gamma_0(N)$ and χ is a character modulo N with values in $\{\pm1\}$ and $\chi(-1) = (-1)^w$; or $\Gamma = \Gamma_0(N) + S$ is an Atkin–Lehner extension and χ is trivial.

(F2) $\Theta \in 1 + q\mathbb{Z}[[q]]$ is a holomorphic modular form of weight w for $\Gamma_0(N)$ with character χ, i.e. $\Theta|_w\gamma = \chi(\gamma)\Theta$ for $\gamma \in \Gamma_0(N)$, and $\Theta|_w g = \varepsilon_g\Theta$ with $\varepsilon_g \in \{\pm 1\}$ for every Atkin–Lehner matrix $g \in \Gamma$.

(F3) $t \in q + q^2\mathbb{Z}[[q]]$ is a Hauptmodul for Γ: it is Γ-invariant, holomorphic on $\mathbb{H}$, meromorphic at the cusps, and induces an isomorphism $X(\Gamma) \to \mathbb{P}^1(\mathbb{C})$.

(F4) (*cusp type*) Θ has no zeros in $\mathbb{H}$, and there is a cusp c such that, for every cusp c', Θ vanishes at c' if and only if $c' \in \Gamma c$, if and only if t has a pole at c'.

(F5) $r_\Theta := \nu_A(\Theta)/(-\nu_A(t)) \le 1$ for $A \in \mathrm{SL}_2(\mathbb{Z})$ with $A\infty = c$. (This is the ratio of the orders of Θ and $1/t$ at c in any local parameter; it does not depend on A.)

For a prime $p \nmid N$ we consider the *Hecke condition*

(C_p) for every $\beta \in M(N,p)$ the cusp βc lies in Γc and has the same width in $\Gamma_0(N)$ as c.

Lemma 33.3. *Let $c = a/s$ with $\gcd(a,s) = 1$ and $e = \gcd(s,N)$. If $\gcd(e, N/e) \in \{1,2\}$, then (C_p) holds for every prime $p \nmid N$; in fact $\beta c \in \Gamma_0(N)c$. The same conclusion (C_p) holds for the family (ζ) of Table 1 ($N = 9$, $c = \frac{1}{3}$).*

Proof. Let $\beta = \left(\begin{smallmatrix} \alpha & b \\ N\gamma & \delta \end{smallmatrix}\right) \in M(N,p)$ and $(a_1, s_1) = (\alpha a + bs, N\gamma a + \delta s)$. Since $\mathrm{adj}(\beta)(a_1,s_1)^{\mathsf{T}} = p(a,s)^{\mathsf{T}}$, the gcd g of a_1, s_1 divides p, and $\beta c = (a_1/g)/(s_1/g)$. As $p \nmid N$ and $\gcd(\delta, N) = 1$ we get $\gcd(s_1/g, N) = \gcd(\delta s, N) = e$. If $\gcd(e, N/e) \in \{1,2\}$, all cusps whose reduced denominator has gcd e with N are $\Gamma_0(N)$-equivalent [13, Prop. 3.8.3], [8, Lemma 3.5]; equivalent cusps have the same width. For $N = 9$ and $e = 3$ there are two such classes, represented by $\frac{1}{3}$ and $\frac{2}{3}$. They are interchanged by the Fricke matrix $w_9 = \left(\begin{smallmatrix} 0 & -1 \\ 9 & 0 \end{smallmatrix}\right)$ (as $w_9(\frac{1}{3}) = -\frac{1}{3}$), which lies in $\Gamma = \Gamma_0(9) + 9$, and both have width 1. □

33.4 The power series F and the excellent lift

Lemma 33.4. *Let $(N, \Gamma, w, \chi, \Theta, t)$ satisfy* (F2), (F3). *Let $q(t) \in t + t^2\mathbb{Z}[[t]]$ be the inverse of $t(q)$, $F(t) := \Theta(q(t)) \in 1 + t\mathbb{Z}[[t]]$, and $t^\sigma(t) := t(q(t)^p)$. Then*

(a) *$F(t(q)) = \Theta(q)$ and $F(t^\sigma(t(q))) = \Theta(q^p)$;*

(b) *$t^\sigma \in t^p + t^{p+1}\mathbb{Z}[[t]]$ and $t^\sigma \equiv t^p \pmod p$;*

(c) *for $L \ge 1$: $F_L(t^\sigma)/F(t^\sigma) \in 1 - a_L t^{pL} + t^{pL+1}\mathbb{Z}[[t]]$.*

Proof. (a) is the definition. (b) $q(t)^p \in t^p + t^{p+1}\mathbb{Z}[[t]]$ and $t(q) \in q + q^2\mathbb{Z}[[q]]$; moreover $t(q^p) \equiv t(q)^p \pmod p$ in $\mathbb{Z}[[q]]$, and substituting $q = q(t)$ gives $t^\sigma \equiv t^p$. (c) $F_L(u)/F(u) = 1 - (F(u) - F_L(u))/F(u) \in 1 - a_L u^L + u^{L+1}\mathbb{Z}[[u]]$; substitute $u = t^\sigma$. □

For all families of Table 1, $t(q)$ is the mirror map of the Picard–Fuchs equation of F [23, 3, 8], so t^σ is the excellent lift in the sense of [7]. This identification is used only in Corollary 32.4; Theorem 32.1 is a statement about the modular $t(q)$.

33.5 The examples

The coefficients a_n of the families in Table 1 are

$\binom{2n}{n}^2$; $\sum_k \binom{n}{k}^3$; $\sum_k (-3)^{n-3k}\frac{n!}{(n-3k)!k!^3}$; $\sum_k \binom{n}{k}^2\binom{2k}{k}$; $\sum_k 4^{n-2k}\binom{n}{2k}\binom{2k}{k}^2$; $\sum_k (-8)^{n-k}\binom{n}{k}\sum_j \binom{k}{j}^3$;
$\binom{2n}{n}^3$; $\sum_k \binom{n}{k}^2\binom{n+k}{k}^2$; $(-1)^n\sum_k (-1)^k\binom{n}{k}^3[\binom{4n-5k-1}{3n} + \binom{4n-5k}{3n}]$; $(-1)^n\sum_k \binom{n}{k}^2\binom{2k}{k}\binom{2n-2k}{n-k}$;
$\sum_k (-3)^{n-3k}\frac{(n+k)!}{k!^4(n-3k)!}$; $\sum_k \binom{n}{k}^2\binom{2k}{n}^2$; $\sum_{k,l}\binom{n}{k}^2\binom{n}{l}\binom{k}{l}\binom{k+l}{n}$,

in the order of the table (binomial coefficients $\binom{x}{y}$ with $x<y$ are 0); see [8, App. B, C], [23, 3]. The hypotheses of Definition 33.2 are checked as follows. Each Θ and t is an η-quotient. By the theorem of Gordon, Hughes and Newman [20, Thm. 1.64] it is modular for $\Gamma_0(N)$ with character $d\mapsto(\frac{(-1)^k\prod\delta^{r_\delta}}{d})$, where k is the weight. An η-quotient has no zeros or poles in $\mathbb{H}$, and Ligozat's formula [20, Thm. 1.65] gives its order at every cusp. The Atkin–Lehner matrix W_e maps η_δ to a constant times $\eta_{e\delta/\gcd(e,\delta)^2}$, so invariance under W_e can be read off the exponents. In every case t has a simple pole on $X(\Gamma)$ and no other poles, so it is a Hauptmodul of the genus-zero group Γ. On $X_0(4)$ the family $\binom{2n}{n}^3$ has a double pole at $\frac12$, where $X_0(4)\to X(\Gamma_0(4)+4)$ ramifies. All these data are verified by checks [1], [2] of the script in the appendix: the identity $F(t(q))=\Theta(q)$, the orders at all cusps, r_Θ, the characters and the Atkin–Lehner symmetries. Condition (C_p) holds for all $p\nmid N$ by Lemma 33.3, since $\gcd(e,N/e)\le 2$ for each cusp in the table except $\frac13$ in level 9.

34 The Hecke trace

Throughout this section $(N,\Gamma,w,\chi,\Theta,t)$ is a modular family of cusp type, $p\nmid N$ satisfies (C_p), and $P\in\mathbb{Z}[T]$ is a nonzero polynomial. For $\beta\in M(N,p)$ put

$$R_\beta(\tau):=p^{w/2}\chi(\beta)\,\frac{\Theta(\tau)}{(\Theta|_w\beta)(\tau)}\,P\big(t(\beta\tau)\big).$$

Since $(\Theta|_w\beta)(\tau)=p^{w/2}j(\beta,\tau)^{-w}\Theta(\beta\tau)$ and Θ has no zeros in $\mathbb{H}$, each R_β is holomorphic on $\mathbb{H}$.

Lemma 34.1. (a) *$R_{\gamma\beta}=R_\beta$ for $\gamma\in\Gamma_0(N)$.*

(b) *$R_\beta(\gamma\tau)=R_{\beta\gamma}(\tau)$ for $\gamma\in\Gamma_0(N)$.*

(c) *$R_\beta(g\tau)=R_{g^{-1}\beta g}(\tau)$ for every Atkin–Lehner matrix $g\in\Gamma$.*
Consequently $\mathrm{Tr}_p(P):=\sum_{[\beta]\in\Gamma_0(N)\backslash M(N,p)}R_\beta$ is well defined and Γ-invariant.

Proof. (a) $\Theta|_w\gamma\beta=\chi(\gamma)\Theta|_w\beta$, $\chi(\gamma\beta)=\chi(\gamma)\chi(\beta)$, $\chi(\gamma)^2=1$ and $t\circ\gamma=t$. (b) We have $\Theta(\gamma\tau)=\chi(\gamma)j(\gamma,\tau)^w\Theta(\tau)$ and $(\Theta|_w\beta)(\gamma\tau)=j(\gamma,\tau)^w(\Theta|_w\beta\gamma)(\tau)$, so $\chi(\beta)\Theta(\gamma\tau)/(\Theta|_w\beta)(\gamma\tau)=\chi(\beta\gamma)\Theta(\tau)/(\Theta|_w\beta\gamma)(\tau)$, and $t(\beta\gamma\tau)$ is the argument of P in $R_{\beta\gamma}$. (c) Here $\chi=1$. Put $\beta'=g^{-1}\beta g\in M(N,p)$ (Lemma 33.1(H4)). Then $\Theta|_w\beta g=\Theta|_w g\beta'=\varepsilon_g\Theta|_w\beta'$, and as in (b) $\Theta(g\tau)/(\Theta|_w\beta)(g\tau)=\varepsilon_g\Theta(\tau)/(\Theta|_w\beta g)(\tau)=\Theta(\tau)/(\Theta|_w\beta')(\tau)$, while $t(\beta g\tau)=t(g\beta'\tau)=t(\beta'\tau)$. The last assertion follows from (H3) and (H4). □

Lemma 34.2. *$R_{\beta_\infty}=Q_P$ with $Q_P(\tau)=\Theta(\tau)P(t(p\tau))/\Theta(p\tau)$, and*

$$\sum_{k=0}^{p-1}R_{\beta_k}=p^{w+1}\chi(p)\,(U_pG_P)(q),\qquad G_P(z)=\frac{\Theta(z^p)\,P(t(z))}{\Theta(z)}\in\mathbb{Z}[[z]].$$

In particular the q-expansion of $\mathrm{Tr}_p(P)$ is $Q_P+p^{w+1}\chi(p)U_pG_P\in\mathbb{Z}[[q]]$.

Proof. $\Theta|_w\beta_\infty=p^{w/2}\Theta(p\tau)$ and $\chi(\beta_\infty)=1$. Next, $(\Theta|_w\beta_k)(\tau)=p^{-w/2}\Theta(\frac{\tau+k}{p})$ and $\chi(\beta_k)=\chi(p)$, so $R_{\beta_k}=p^w\chi(p)\Theta(\tau)P(t(\frac{\tau+k}{p}))/\Theta(\frac{\tau+k}{p})=p^w\chi(p)G_P(\zeta_p^kq^{1/p})$, because $\Theta(\tau)=\Theta((\zeta_p^kq^{1/p})^p)$. Now apply (33.1). Integrality holds because $\Theta\in1+q\mathbb{Z}[[q]]$ is a unit in $\mathbb{Z}[[q]]$. □

Lemma 34.3 (Branches at the cusp)**.** *Let $A\in\mathrm{SL}_2(\mathbb{Z})$ with $A\infty=c$. For $\beta\in M(N,p)$ write $\beta A=A_\beta\beta'$ with $A_\beta\in\mathrm{SL}_2(\mathbb{Z})$ and $\beta'=\left(\begin{smallmatrix}a'&b'\\0&d'\end{smallmatrix}\right)$, $a'd'=p$ (Hermite normal form).*

(i) *Exactly one class $[\beta]$ has $(a',d')=(p,1)$; the other p classes have $(a',d')=(1,p)$.*

(ii) *For every Γ-invariant φ meromorphic at c: $\nu_A(\varphi\circ\beta)=\frac{a'}{d'}\,\nu_A(\varphi)$. Moreover $\nu_A(\Theta|_w\beta)=\frac{a'}{d'}\,\nu_A(\Theta)$.*

Proof. (i) By (H2) and since right multiplication by A permutes $\mathrm{SL}_2(\mathbb{Z})\backslash M_p$, the map $[\beta] \mapsto \mathrm{SL}_2(\mathbb{Z})\beta A$ is a bijection onto $\mathrm{SL}_2(\mathbb{Z})\backslash M_p$. The diagonal (a', d') of the Hermite normal form is an invariant of the class, and M_p has one class with $(p, 1)$ and p classes with $(1, p)$.

(ii) $A_\beta\infty = \beta A\infty = \beta c$. By (C_p) there is $g \in \Gamma$ with $gc = \beta c$, and the widths of c and βc in $\Gamma_0(N)$ are equal, say to h. The matrix $\delta := A_\beta^{-1} g A \in \mathrm{GL}_2^+(\mathbb{Q})$ fixes ∞, so $\delta = \left(\begin{smallmatrix} u & x \\ 0 & v \end{smallmatrix}\right)$ with $uv > 0$. Since g normalises $\Gamma_0(N)$ and $\mathrm{Stab}_{\Gamma_0(N)}(c) = \pm A\langle T^h\rangle A^{-1}$, $\mathrm{Stab}_{\Gamma_0(N)}(\beta c) = \pm A_\beta\langle T^h\rangle A_\beta^{-1}$, conjugation by δ maps $\pm\langle T^h\rangle$ onto itself. As $\delta T^h \delta^{-1} = T^{hu/v}$, this forces $u/v = \pm 1$, hence $u = v$. Thus $A_\beta = u^{-1} g A T^{-x/u}$. For Γ-invariant φ we get $\varphi(\beta A\tau) = \varphi(A_\beta\beta'\tau) = \varphi(A(\beta'\tau - x/u))$ with $\beta'\tau - x/u = \frac{a'}{d'}\tau + \text{const}$, which proves the first claim. Similarly $(\Theta|_w\beta)|_w A = \Theta|_w A_\beta\beta' = \pm\varepsilon_g\,(\Theta|_w A)|_w(T^{-x/u}\beta')$, and $(f|_w T^y\beta')(\tau) = p^{w/2} d'^{-w} f\big(\frac{a'\tau + b'}{d'} + y\big)$; here $\varepsilon_g := \chi(g)$ if $g \in \Gamma_0(N)$. □

Proposition 34.4. *There is a polynomial $\Pi_P \in \mathbb{Z}[T]$ with $\mathrm{Tr}_p(P) = \Pi_P(t)$ and*

$$\deg \Pi_P \le p \deg P + \lfloor r_\Theta(p-1)\rfloor.$$

Proof. Cusps outside Γc. Let $c' \notin \Gamma c$ and $A' \in \mathrm{SL}_2(\mathbb{Z})$ with $A'\infty = c'$. For $\beta \in M(N, p)$ we have $\beta c' \notin \Gamma c$. Indeed, if $\beta c' = gc$ with $g \in \Gamma$, then $c' = \mathrm{adj}(\beta) g c = g\,(g^{-1}\,\mathrm{adj}(\beta) g)\,c \in gM(N, p)c \subseteq \Gamma c$ by (H4), (H5) and (C_p), a contradiction. Write $\beta A' = A'_\beta \beta'$ as in Lemma 34.3. Then $A'_\beta\infty = \beta c'$. Since Θ does not vanish at c' and $\beta c'$, and t has no pole there,

$$R_\beta(A'\tau) = p^{w/2}\chi(\beta)\frac{(\Theta|_w A')(\tau)}{((\Theta|_w A'_\beta)|_w\beta')(\tau)}\,P\big(t(A'_\beta\beta'\tau)\big)$$

is holomorphic at $\tau = i\infty$; the factors $j(A', \tau)^w$ cancel. So each R_β, and hence $\mathrm{Tr}_p(P)$, is holomorphic at c'.

The cusp c. Put $\rho = \nu_A(\Theta) > 0$ and $\kappa = -\nu_A(t) > 0$, so $r_\Theta = \rho/\kappa$. As above, $R_\beta(A\tau) = p^{w/2}\chi(\beta)\,(\Theta|_w A)(\tau)\,((\Theta|_w\beta)|_w A)(\tau)^{-1} P(t(\beta A\tau))$. Since $\nu_A(t) < 0$ we have $\nu_A(P \circ t) = -\kappa \deg P$. Lemma 34.3 then gives

$$\nu_A(R_\beta) = \rho - \tfrac{a'}{d'}\rho - \tfrac{a'}{d'}\kappa \deg P = \begin{cases} -(p-1)\rho - p\kappa \deg P, & (a', d') = (p, 1), \\ (1 - \frac{1}{p})\rho - \frac{1}{p}\kappa \deg P, & (a', d') = (1, p). \end{cases}$$

Hence $\nu_A(\mathrm{Tr}_p(P)) \ge -(p-1)\rho - p\kappa \deg P$.

Polynomiality. By Lemma 34.1 and the two previous steps, $\mathrm{Tr}_p(P)$ is a Γ-invariant function, holomorphic on $\mathbb{H}$ and at all cusps outside Γc, and meromorphic at c. It therefore defines a regular function on $X(\Gamma) \smallsetminus \{c\}$. Since t maps $X(\Gamma)$ isomorphically onto $\mathbb{P}^1$ with $t(c) = \infty$, we get $\mathrm{Tr}_p(P) = \Pi_P(t)$ for some $\Pi_P \in \mathbb{C}[T]$. Comparing ν_A gives $\kappa \deg \Pi_P = -\nu_A(\mathrm{Tr}_p(P)) \le (p-1)\rho + p\kappa \deg P$, which is the degree bound. Finally, the q-expansion of $\Pi_P(t)$ lies in $\mathbb{Z}[[q]]$ by Lemma 34.2, and $t \in q + q^2\mathbb{Z}[[q]]$. Comparing coefficients of $q^0, q^1, \ldots$ inductively shows $\Pi_P \in \mathbb{Z}[T]$. □

Remark 34.5. Let $T_p = U_p + \chi(p) p^{-w-1} V_p$ be the Hecke operator on forms of weight $-w$ and character χ, where $V_p h(q) = h(q^p)$. By (33.1), $U_p G_P = \Theta \cdot U_p(P(t)/\Theta)$, so Lemma 34.2 says $\mathrm{Tr}_p(P)/\Theta = p^{w+1}\chi(p)\,T_p(P(t)/\Theta)$. Proposition 34.4 is therefore the statement that T_p maps $\{P(t)/\Theta : \deg P \le d\}$ into $p^{-w-1}\{P(t)/\Theta : \deg P \le pd + \lfloor r_\Theta(p-1)\rfloor\}$. For a family of cusp type these spaces of weakly holomorphic forms of weight $-w$ consist of forms with poles only at Γc.

35 Proof of the Main Theorem

Fix $m \ge 1$. Put $K_j = mp^j$ for $j \ge 0$ and $F_{K_{-1}} := 0$. For $s \ge 1$ consider

$$(\mathrm{H}_s) \qquad \Theta(q)\,F_{K_{s-1}}(t(q^p)) \equiv \Theta(q^p)\,F_{K_s}(t(q)) \pmod{p^{(w+1)s}\mathbb{Z}[[q]]}.$$

Dividing by the unit $\Theta(q^p)F_{K_{s-1}}(t(q^p))$ of $\mathbb{Z}[[q]]$ and substituting $q = q(t)$ shows that (H_s) is equivalent to the congruence of Theorem 32.1 (Lemma 33.4(a)). We prove (H_s) by induction on s.

Let $s \geq 1$, and assume (H_{s-1}) if $s \geq 2$. Apply Lemma 34.2 and Proposition 34.4 to $P = F_{K_{s-1}}$, a polynomial of degree $< K_{s-1}$:

$$Q := \frac{\Theta(q)\, F_{K_{s-1}}(t(q^p))}{\Theta(q^p)} = \Pi(t) - p^{w+1}\chi(p)\, U_p G, \qquad G(z) = \frac{\Theta(z^p)F_{K_{s-1}}(t(z))}{\Theta(z)}, \tag{35.1}$$

with $\Pi \in \mathbb{Z}[T]$ and $\deg \Pi \leq p(K_{s-1}-1) + \lfloor r_\Theta(p-1) \rfloor \leq K_s - p + (p-1) < K_s$, by (F5).

Claim: $p^{w+1}U_pG \equiv p^{w+1}F_{K_{s-2}}(t(q)) \pmod{p^{(w+1)s}}$. For $s = 1$ both sides are $\equiv 0 \pmod{p^{w+1}}$. For $s \geq 2$, (H_{s-1}) in the variable z reads $\Theta(z)F_{K_{s-2}}(t(z^p)) \equiv \Theta(z^p)F_{K_{s-1}}(t(z)) \pmod{p^{(w+1)(s-1)}}$. Dividing by the unit $\Theta(z)$ gives $G(z) \equiv F_{K_{s-2}}(t(z^p)) \pmod{p^{(w+1)(s-1)}}$. Applying U_p, which preserves congruences, and (33.1) gives $U_pG \equiv F_{K_{s-2}}(t(q))$ modulo $p^{(w+1)(s-1)}$, and the claim follows.

By (35.1) and the claim,

$$Q \equiv \widetilde{\Pi}(t) := \Pi(t) - p^{w+1}\chi(p)F_{K_{s-2}}(t) \pmod{p^{(w+1)s}}, \qquad \widetilde{\Pi} \in \mathbb{Z}[T],\ \deg \widetilde{\Pi} < K_s.$$

On the other hand, Lemma 33.4(c) gives $Q = F(t) \cdot F_{K_{s-1}}(t^\sigma)/F(t^\sigma) \in F(t) + t^{K_s}\mathbb{Z}[[t]]$. Compare coefficients of t^n in $Q \equiv \widetilde{\Pi}(t)$. For $n < K_s$ we get $[t^n]\widetilde{\Pi} \equiv a_n$. For $n \geq K_s$ we get $[t^n]Q \equiv 0$. Hence $Q \equiv F_{K_s}(t) \pmod{p^{(w+1)s}}$, which is (H_s). This completes the induction and the proof of Theorem 32.1. □

Remark 35.1. The proof gives the explicit polynomial congruence

$$F_{mp^s}(t) \equiv \Pi_{F_{mp^{s-1}}}(t) - p^{w+1}\chi(p)F_{mp^{s-2}}(t) \pmod{p^{(w+1)s}},$$

with $\Pi_{F_{mp^{s-1}}}$ the trace polynomial of Proposition 34.4. In the examples the exponent $(w+1)s$ is sharp: for most (p, s, m) the difference in (H_s) has valuation exactly $(w+1)s$ (check [4]).

36 Proofs of the corollaries

36.1 Coefficients

Proof of Corollary 32.6. By Lemma 33.4(b),(c) and $a_0 = 1$, the coefficient of t^{K_s} in

$$Q = F(t)\,\frac{F_{K_{s-1}}(t^\sigma)}{F(t^\sigma)}$$

is $a_{K_s} - a_{K_{s-1}}$. Since $Q \equiv F_{K_s}(t) \pmod{p^{(w+1)s}}$ and $\deg F_{K_s} < K_s$, it is divisible by $p^{(w+1)s}$. □

36.2 Conjecture BV

Lemma 36.1. *Let $F(u) \in 1 + u\mathbb{Z}[[u]]$ be annihilated by a differential operator with maximal unipotent monodromy at 0, with canonical coordinate $q_F(u) \in u + u^2\mathbb{Z}_p[[u]]$ and mirror map t_F, and let $u^\sigma = t_F(q_F(u)^p)$. Let $k \geq 1$ with $p \nmid k$, and $F_0(t) := F(t^k)$. Then:*

(a) *the canonical coordinate of F_0 is $q_0(t) = t\big(q_F(t^k)/t^k\big)^{1/k} \in t + t^2\mathbb{Z}_p[[t]]$;*

(b) *the excellent lift $t^{\sigma_0} = t_0(q_0(t)^p)$ of F_0 satisfies $(t^{\sigma_0})^k = (t^k)^\sigma$;*

(c) *$F_{0,kM}(t) = F_M(t^k)$ for all $M \geq 1$.*

Consequently, for every m', s and e: (32.1) for F_0 with $m = km'$ holds modulo p^e if and only if (32.1) for F with m' holds modulo p^e.

Proof. Let $L(u,\theta_u)$ be an operator annihilating F. Then $L(t^k,\theta_t/k)$ annihilates F_0. Under $u=t^k$ the Frobenius solution $\log(u)F(u)+G(u)$ becomes

$$\log(t^k)F(t^k)+G(t^k)=k\Big(\log(t)F_0(t)+\frac{G(t^k)}{k}\Big).$$

So the normalised logarithmic solution of F_0 is $\log(t)F_0+G(t^k)/k$, and

$$q_0(t)=t\exp\Big(\frac{G(t^k)}{kF(t^k)}\Big)=t\left(\frac{q_F(u)}{u}\right)^{1/k}\Big|_{u=t^k}.$$

This proves (a); integrality holds because $p\nmid k$. Thus $q_0(t)^k=q_F(t^k)$, and inverting gives $t_0(Q)^k=t_F(Q^k)$. Hence $(t^{\sigma_0})^k=t_F(q_0(t)^{pk})=t_F(q_F(t^k)^p)=(t^k)^\sigma$, which is (b). (c) holds because F_0 only contains powers t^{kj}. For the consequence, substitute $u=t^k$:

$$\frac{F_0(t)}{F_0(t^{\sigma_0})}=\frac{F(u)}{F(u^\sigma)},\qquad F_{0,km'p^s}(t)=F_{m'p^s}(u),\qquad F_{0,km'p^{s-1}}(t^{\sigma_0})=F_{m'p^{s-1}}(u^\sigma).$$

A power series in u is divisible by p^e if and only if it is so after the substitution $u=t^k$. □

Proof of Corollary 32.4. For $n=2$ the hypercubic and hyperoctahedral Laurent polynomials have the same constant terms, $\mathrm{CT}((x_1+x_1^{-1})^j(x_2+x_2^{-1})^j)=\mathrm{CT}((x_1+x_1^{-1}+x_2+x_2^{-1})^j)$, equal to $\binom{2k}{k}^2$ for $j=2k$ and 0 for j odd. So $F_0(t)=\sum_k\binom{2k}{k}^2t^{2k}=F(t^2)$ with F the Legendre family of Table 1. For $n=3$, hypercubic, $F_0(t)=\sum_k\binom{2k}{k}^3t^{2k}=F(t^2)$ with F the family $\binom{2n}{n}^3$. In all three cases the vertex coefficient is $\gamma=1$, so the excellent lift of [7, Def. 7.14] is $q_0\mapsto q_0^p$ in the canonical coordinate of F_0. Since the modular $t(q)$ of Table 1 is the mirror map of F (for Legendre see [8, §1] and the discussion in [7, §8], where $t_0(q)^2=\lambda(q^2)/16$; for $\binom{2n}{n}^3$ see [8, App. C, 9.1]), Lemma 36.1 with $k=2$ reduces (32.1) for F_0 and even $m=2m'$ to Theorem 32.1 for F and m', with $w=1$, respectively $w=2$. The primes allowed in [7, Thm. 7.3] are odd for $n=2$ and ≥ 5 for $n=3$ ($\#G=48$), and they do not divide $N=4$. Check [6] of the script recomputes the canonical coordinate of F_0 directly from its Picard–Fuchs equation and verifies (32.1) in the normalisation of [7].

For the simplicial family with n variables, $\mathrm{CT}(g^k)=0$ unless $(n+1)\mid k$, and $\mathrm{CT}(g^{(n+1)j})=((n+1)j)!/j!^{n+1}$. So $F_0(t)=F(t^{n+1})$ with $F=F_{27}$ for $n=2$ and $F=F_{256}$ for $n=3$. The vertex coefficients of g are 1, so the excellent lift is again $q_0\mapsto q_0^p$. The modular $t(q)$ of Table 2 is the mirror map of F; equivalently, by Lemma 36.1(a), the mirror map t_0 of F_0 satisfies $t_0(Q)^{n+1}=t(Q^{n+1})$, which check [8g] verifies with t_0 computed from the Picard–Fuchs equation of F_0. Now Lemma 36.1 with $k=n+1$ (note $p\nmid n+1$ for $p\geq 5$) reduces (32.1) for F_0 and $m=(n+1)m'$ to Theorem 32.2 for F and m'. The symmetry group G of g contains the $(n+1)!$ permutations of its monomials, so the primes allowed in [7, Thm. 7.3] are ≥ 5. Check [8g] also verifies (32.1) for F_0 directly in the normalisation of [7]. □

Proof of Corollary 32.5. Immediate from Theorem 32.1, Lemma 33.3 and §33.5. Among the families of [8, App. B, C] satisfying Assumption (I) or (II), those with Θ an η-product are exactly the families of Table 1. For an η-product, (F4) holds automatically because an η-quotient has no zeros in $\mathbb{H}$. □

37 Families with an elliptic zero

In this section we prove Theorem 32.2. The trace of Q_P is no longer a polynomial, so the argument of §35 has to be replaced by a p-adic estimate.

37.1 The three families

Let $a(q) = \sum_{x,y\in\mathbb{Z}} q^{x^2+xy+y^2}$ be the theta series of the hexagonal lattice, $E_2 = 1 - 24\sum_{n\geq 1}\sigma(n)q^n$, $h_2 = \eta_2^{24}/\eta_1^{24}$ and $h_3 = \eta_3^{12}/\eta_1^{12}$. The data of the three families are in Table 2; $W_N = \left(\begin{smallmatrix} 0 & -1 \\ N & 0 \end{smallmatrix}\right)$ denotes the Fricke matrix.

F	w	Γ	χ	Θ	t	e	k_0	o
$F_{27} = \sum \frac{(3n)!}{n!^3}t^n$	1	$\Gamma_0(3)$	χ_{-3}	a	$\eta_3^9/(\eta_1^3a^3) = \frac{h_3}{1+27h_3}$	$\frac{3+\sqrt{-3}}{6}$	3	1
$F_{256} = \sum \frac{(4n)!}{n!^4}t^n$	2	$\Gamma_0(2)+2$	1	$2E_2(2\tau) - E_2(\tau)$	$h_2/(1+64h_2)^2$	$\frac{1+i}{2}$	4	1
$F_{108} = \sum \frac{(2n)!(3n)!}{n!^5}t^n$	2	$\Gamma_0(3)+3$	1	a^2	$h_3/(1+27h_3)^2$	$\frac{3+\sqrt{-3}}{6}$	6	2

Table 2: Families with an elliptic zero. e is the zero of Θ, k_0 the order of the elliptic point e of Γ, o the order of vanishing of Θ at e in the variable $\tau - e$, and $r = o/k_0 \in \{\frac{1}{3}, \frac{1}{4}, \frac{1}{3}\}$.

We also use a root ϑ of Θ: $\vartheta = \Theta$, $\chi_\vartheta = \chi$ in the first two cases and $\vartheta = a$, $\chi_\vartheta = \chi_{-3}$ in the third case, so that $\Theta = \vartheta^o$ in all cases.

Lemma 37.1. *For each family of Table 2:*

(a) *$F(t(q)) = \Theta(q)$; Θ is a modular form of weight w for $\Gamma_0(N)$ with character χ, and $\Theta|_w W_N = -\Theta$ in the second and third case; $t \in q + q^2\mathbb{Z}[[q]]$ is a Hauptmodul for Γ. The form ϑ is modular for $\Gamma_0(N)$ with character χ_ϑ, and in the third case $a|_1 W_3 = -ia$.*

(b) *e is an elliptic point of Γ whose stabiliser in Γ, modulo scalar matrices, has order k_0.*

(c) *The zeros of Θ in $\mathbb{H}\cup\mathbb{P}^1(\mathbb{Q})$ form the orbit Γe; Θ vanishes at e to order o and ϑ to order 1, in the variable $\tau - e$. The function t is holomorphic on $\mathbb{H}\setminus\Gamma e$ and at all cusps, and has a pole of order k_0 at e in the variable $\tau - e$.*

Proof. (a) The identities $F(t(q)) = \Theta(q)$ are classical (Ramanujan's theories of elliptic functions to alternative bases, see [4, 11] and [8, App. B, C]); check [8a] verifies them to $O(q^{60})$. The series $a(q)$ is a modular form of weight one for $\Gamma_0(3)$ with character χ_{-3}, and Poisson summation gives $a(-1/(3\tau)) = -i\sqrt{3}\,\tau a(\tau)$, that is $a|_1 W_3 = -ia$; hence $a^2|_2 W_3 = -a^2$. Further $2E_2(2\tau) - E_2(\tau) \in M_2(\Gamma_0(2))$ is anti-invariant under W_2, and $(3E_2(3\tau) - E_2(\tau))/2 = a^2$ (check [8a]). The function h_N is a Hauptmodul for $\Gamma_0(N)$ with $h_N \circ W_N = N^{-12/(N-1)}/h_N$, so $h_N/(1+N^{6/(N-1)}h_N)^2$ (with $N^{6/(N-1)} = 64, 27$) is W_N-invariant of degree two in h_N, i.e. a Hauptmodul for $\Gamma_0(N)+N$. In the first case $\eta_3^9/(\eta_1^3a^3) = h_3/(1+27h_3)$ is the identity $a^3 = \eta_1^9/\eta_3^3 + 27\eta_3^9/\eta_1^3$ of [4] (check [8a]).

(b) Put $\gamma_e = \left(\begin{smallmatrix} 1 & -1 \\ 3 & -2 \end{smallmatrix}\right)$ in the first and third case and $\gamma_e = \left(\begin{smallmatrix} 1 & -1 \\ 2 & -1 \end{smallmatrix}\right)$ in the second. Then $\gamma_e \in \Gamma_0(N)$ fixes e, and γ_e has order 3, respectively 2, in $\mathrm{SL}_2(\mathbb{Z})/\{\pm 1\}$. Moreover $W_3 e = e - 1$ and $W_2 e = e - 1$, so TW_N also fixes e. Modulo scalars, the stabiliser of e in Γ is therefore generated by γ_e (first case), respectively by γ_e and TW_N, and has order 3, 4, 6.

(c) We have $j(\gamma_e, e) = 3e - 2 = e^{2\pi i/3}$, respectively $2e - 1 = i$, and $\chi(\gamma_e) = \chi_{-3}(-2) = 1$. Evaluating $\Theta(\gamma_e\tau) = \chi(\gamma_e)j(\gamma_e,\tau)^w\Theta(\tau)$ at $\tau = e$ gives $\Theta(e) = e^{2\pi i/3}\Theta(e)$, respectively $\Theta(e) = i^2\Theta(e)$, so $\Theta(e) = 0$; likewise $a(e) = 0$. Check [8a] verifies the identities

$$a^3 t = \frac{\eta_3^9}{\eta_1^3}, \qquad \Theta^4 t = \eta_1^8\eta_2^8, \qquad \Theta^3 t = \eta_1^6\eta_3^6,$$

that is $\Theta^{1/r}t =$ an η-product, which has neither zeros nor poles in $\mathbb{H}$. As $\Theta(e) = 0$, t has a pole at e. Since t is a Hauptmodul, this is its only pole on $X(\Gamma)$, and it is simple on $X(\Gamma)$; as the local parameter of $X(\Gamma)$ at e is $((\tau - e)/(\tau - \bar{e}))^{k_0}$, the pole has order k_0 in $\tau - e$. The η-identity now shows that $\Theta^{1/r}$ vanishes to order exactly k_0 at e, i.e. Θ to order $o = rk_0$, and that Θ has no zeros in $\mathbb{H}\setminus\Gamma e$. Finally, by the valence formula the zeros of a modular form of weight w on $X_0(N)$, each weighted by the inverse of the order of its stabiliser in $\Gamma_0(N)/\{\pm 1\}$, have total mass

$w[\mathrm{SL}_2(\mathbb{Z}):\Gamma_0(N)]/12$, which is $\frac{1}{3},\frac{1}{2},\frac{2}{3}$. In $\Gamma_0(N)$ the point e has order $3,2,3$, and $o/3, o/2, o/3$ are the same numbers. So Θ has no zeros at the cusps. The statements for $\vartheta = a$ follow from $\Theta = a^2$. □

Symbolic verification of the modular identities and coordinate. The identity

$$a^2 = \frac{3E_2(3\tau) - E_2(\tau)}{2}$$

also follows directly from the valence formula: both sides are holomorphic modular forms of weight two on $\Gamma_0(3)$ with constant term 1, whereas a nonzero difference cannot have order at least one at infinity because the total valence is $2 \cdot 4/12 = 2/3$. For the level-two form $\Theta = 2E_2(2\tau) - E_2(\tau)$, the identity

$$\Theta^2 = \frac{\eta_1^{16}}{\eta_2^8} + 64\frac{\eta_2^{16}}{\eta_1^8}$$

is proved in the same way. Both sides are holomorphic forms of weight four on $\Gamma_0(2)$, and their constant and q coefficients are 1 and 48. A nonzero difference of order at least two would contradict the total valence $4 \cdot 3/12 = 1$. For the two eta quotients, the exponent pairs at levels $1, 2$ are $(16,-8)$ and $(-8,16)$. Their characters are trivial, and their orders at $(\infty, 0)$ on $X_0(2)$ are respectively $(0,1)$ and $(1,0)$, as follows from the eta-quotient criterion and cusp-order formula. Thus both are holomorphic, as required for the valence argument. Substitution of the formulas for t now gives

$$a^3 t = \frac{\eta_3^9}{\eta_1^3}, \qquad \Theta^4 t = \eta_1^8\eta_2^8 \quad (F_{256}), \qquad \Theta^3 t = \eta_1^6\eta_3^6 \quad (F_{108}).$$

The first identity is the definition of t in the $27t$ case; the last follows algebraically from $a^3 = (\eta_1^9/\eta_3^3)(1+27h_3)$ and $\Theta = a^2$.

Write $D = q\,d/dq$. Logarithmic differentiation of the eta products gives $Dh_3/h_3 = a^2$ and $Dh_2/h_2 = 2E_2(2\tau) - E_2(\tau)$. Hence the three modular parametrisations satisfy, respectively,

$$\frac{Dt}{t} = (1-27t)F_{27}(t)^2, \qquad \frac{Dt}{t} = \sqrt{1-256t}\,F_{256}(t), \qquad \frac{Dt}{t} = \sqrt{1-108t}\,F_{108}(t),$$

where each square root has constant term 1. For the second and third identities use

$$\sqrt{1-256t} = \frac{1-64h_2}{1+64h_2}, \qquad \sqrt{1-108t} = \frac{1-27h_3}{1+27h_3}.$$

The Wronskian equation for the Gauss hypergeometric equation gives the first differential identity for its normalized logarithmic coordinate. Clausen's identity expresses the two ${}_3F_2$ periods as the squares of ${}_2F_1(1/8,3/8;1;256t)$ and ${}_2F_1(1/6,1/3;1;108t)$, respectively, and the same Wronskian calculation gives the other two identities. The normalized condition $q = t + O(t^2)$ determines the formal coordinate uniquely. Thus these identities identify the modular coordinate with the Frobenius logarithmic coordinate coefficientwise; finite check [8g] is supplementary.

37.2 Hecke translates of the elliptic point

Let $p \geq 5$ be a prime. For $\beta \in M(N,p)$ put

$$W_\beta = t \circ \beta, \qquad x_\beta = p^{w/2}\chi(\beta)\frac{\Theta}{\Theta|_w\beta},$$

so that $R_\beta = x_\beta\, P(W_\beta)$ in the notation of §34. The proof of Lemma 34.1 applies verbatim, because $\Theta|_w W_N = \pm\Theta$ by Lemma 37.1(a). So for $\gamma \in \Gamma$ the families $(x_\beta \circ \gamma, W_\beta \circ \gamma)$ and (x_β, W_β), indexed by $\Gamma_0(N)\backslash M(N,p)$, agree up to a permutation of the index set. Hence every polynomial

expression in the $p+1$ pairs (x_β, W_β) that is symmetric under permutations of the pairs is a Γ-invariant function. For $\tau_0 \in \mathbb{H}$ let

$$m(\tau_0) = \#\{[\beta] \in \Gamma_0(N)\backslash M(N,p) : \ \beta\tau_0 \in \Gamma e\}.$$

Lemma 37.2. *Let $\tau_0 \in \mathbb{H}$ and $\beta \in M(N,p)$.*

(a) *If $\beta\tau_0 \in \Gamma e$, then in the variable $\tau - \tau_0$ the function W_β has a pole of order k_0, and $\Theta|_w\beta$, $\vartheta|\beta$ vanish to order o and 1. Consequently x_β has a pole of order o at τ_0 if $\tau_0 \notin \Gamma e$, and x_β is holomorphic and nonzero at τ_0 if $\tau_0 \in \Gamma e$.*

(b) *If $\beta\tau_0 \notin \Gamma e$, then W_β and x_β are holomorphic at τ_0, and if moreover $\tau_0 \in \Gamma e$, then x_β vanishes to order o at τ_0.*

(c) *W_β and x_β are holomorphic at every cusp.*

Proof. β is a Möbius transformation which is biholomorphic near τ_0, and $(\Theta|_w\beta)(\tau)$ is $\Theta(\beta\tau)$ times a nowhere vanishing factor. By modularity the orders of t, Θ and ϑ at a point of Γe are those at e. Now (a) and (b) follow from Lemma 37.1(c). For (c), β maps cusps to cusps, where t is holomorphic and Θ does not vanish. □

37.3 Beukers's polynomial

Let $N_p(\vartheta)(q) = \prod_{k=0}^{p-1} \vartheta(\zeta_p^k q^{1/p})$, a series in q, and

$$D = \chi_\vartheta(p)\, \frac{\vartheta(q^p)\, N_p(\vartheta)(q)}{\vartheta(q)^{p+1}}.$$

For $\vartheta = \Theta$ this is Beukers's polynomial $T_{p,p+1}$ [8, §§4,5].

Lemma 37.3. *D is a polynomial in t with coefficients in $\mathbb{Z}$, $D \equiv \chi_\vartheta(p) \pmod p$, and $\deg D \le (p+1)/k_0$. For $\tau_0 \in \mathbb{H} \smallsetminus \Gamma e$, D vanishes at τ_0 to order $m(\tau_0)$ in the variable $\tau - \tau_0$.*

Proof. As in Lemma 34.2, $D = p^{w_\vartheta(p-1)/2} \prod_{[\beta]} \chi_\vartheta(\beta)(\vartheta|\beta)/\vartheta$, where w_ϑ is the weight of ϑ. The argument of Lemma 34.1(a),(b) shows that D is invariant under $\Gamma_0(N)$. In the second case D is W_2-invariant by Lemma 34.1(c). In the third case let $g = W_3$ and $\beta = \left(\begin{smallmatrix} \alpha & b \\ 3c & \delta \end{smallmatrix}\right)$. Then $\beta' = g^{-1}\beta g = \left(\begin{smallmatrix} \delta & -c \\ -3b & \alpha \end{smallmatrix}\right) \in M(3,p)$ and $\chi_{-3}(\alpha) = \chi_{-3}(p)\chi_{-3}(\delta)$. From $a|_1 g = -ia$ we get $((a|\beta)/a)(g\tau) = ((a|\beta')/a)(\tau)$, so the factor of D indexed by β at $g\tau$ equals $\chi_{-3}(p)$ times the factor indexed by β' at τ. Since $\beta \mapsto \beta'$ permutes the classes (Lemma 33.1(H4)) and $\chi_{-3}(p)^{p+1} = 1$, D is W_3-invariant. Thus D is Γ-invariant. By Lemma 37.2, D is holomorphic on $\mathbb{H} \smallsetminus \Gamma e$ and at the cusps and vanishes at $\tau_0 \notin \Gamma e$ to order $m(\tau_0)$; at e the denominator ϑ^{p+1} vanishes to order $p+1$ and the numerator to order $m(e)$, so D has a pole of order $p+1-m(e)$ in $\tau - e$, i.e. of order $(p+1-m(e))/k_0$ in t. Hence D is a polynomial in t of degree at most $(p+1)/k_0$. Its q-expansion lies in $\mathbb{Z}[[q]]$, so $D \in \mathbb{Z}[t]$. Finally, $\prod_k \vartheta(\zeta_p^k z) \equiv \vartheta(z)^p \equiv \vartheta(z^p)$ modulo the ideal generated by $1-\zeta_p$ in $\mathbb{Z}[\zeta_p][[z]]$. Both sides lie in $\mathbb{Z}[[z^p]]$, and $(1-\zeta_p)\mathbb{Z}[\zeta_p] \cap \mathbb{Z} = p\mathbb{Z}$, so $N_p(\vartheta)(q) \equiv \vartheta(q) \pmod p$. Together with $\vartheta(q^p) \equiv \vartheta(q)^p$ this gives $D \equiv \chi_\vartheta(p) \pmod p$. □

37.4 The decay estimate

Consider the Γ-invariant functions

$$e_k = e_k(W_\beta)\ (0 \le k \le p+1), \qquad n_l = \sum_{[\beta]} x_\beta\, e_l(W_\gamma : [\gamma] \ne [\beta])\ (0 \le l \le p),$$

$$S_i = \sum_{[\beta]} x_\beta W_\beta^i\ (i \ge 0),$$

where e_k denotes the k-th elementary symmetric polynomial. Thus $S_i = \mathrm{Tr}_p(t^i)$, and by Lemma 34.2

$$S_i = Q_{t^i} + p^{w+1}\chi(p)\, U_p G_{t^i}. \tag{37.1}$$

Lemma 37.4. (a) *The q-expansions of e_k, n_l, S_i lie in $\mathbb{Z}[[q]]$, and $e_1 \equiv t^p \pmod p$.*

(b) $(-1)^l n_l = \sum_{k=0}^{l}(-1)^k e_k S_{l-k}$ *for $0 \le l \le p$, and* $\sum_{k=0}^{p+1}(-1)^k e_k S_{l-k} = 0$ *for $l \ge p+1$.*

Proof. (a) The pair for β_∞ is $(\Theta(q)/\Theta(q^p), t(q^p))$, and the pairs for β_k are $(\phi(\zeta_p^k q^{1/p}), t(\zeta_p^k q^{1/p}))$ with $\phi(z) = p^w\chi(p)\Theta(z^p)/\Theta(z) \in \mathbb{Z}[[z]]$ (proof of Lemma 34.2). A symmetric polynomial expression in the latter p pairs is a power series in $q^{1/p}$ with coefficients in $\mathbb{Z}[\zeta_p]$, invariant under $q^{1/p} \mapsto \zeta_p q^{1/p}$ and under $\mathrm{Gal}(\mathbb{Q}(\zeta_p)/\mathbb{Q})$, hence lies in $\mathbb{Z}[[q]]$. Moreover $e_1 = t(q^p) + pU_p t \equiv t^p \pmod p$. (b) Compare coefficients in $(\sum_k (-1)^k e_k Y^{p+1-k}) \sum_{i\ge 0} S_i Y^{-i-1} = \prod_\beta (Y - W_\beta) \sum_\beta \frac{x_\beta}{Y - W_\beta} = \sum_\beta x_\beta \prod_{\gamma\ne\beta}(Y - W_\gamma)$. □

Proposition 37.5 (Decay of the Hecke trace)**.** *For all $i, n \ge 0$,*

$$v_p([t^n]S_i) \;\ge\; \frac{k_0}{p+1}\,(n - pi - \lfloor pr \rfloor).$$

Proof. Step 1: three families of polynomials. Put $E_k = e_k D^{k_0}$, $M_l = n_l D^{k_0}$ and $A_0 = S_0 D^o$. We claim that $E_k, M_l, A_0 \in \mathbb{Z}[t]$ with

$$\deg E_k \le p+1, \quad [t^{p+1}]E_1 \equiv 0 \pmod p, \quad \deg M_l \le p, \quad \deg A_0 \le \lfloor pr \rfloor.$$

These functions are Γ-invariant and holomorphic at the cusps (Lemma 37.2(c)). Let $\tau_0 \in \mathbb{H} \smallsetminus \Gamma e$ and $m = m(\tau_0)$. By Lemma 37.2, exactly m of the W_β have a pole at τ_0, of order k_0, and the corresponding x_β have poles of order $o \le k_0$; all other W_β, x_β are holomorphic. Hence e_k and n_l have poles of order at most $k_0 m$ and S_0 of order at most o, while D^{k_0} and D^o vanish to orders $k_0 m$ and $om \ge o$ (Lemma 37.3). So E_k, M_l, A_0 are holomorphic on $\mathbb{H} \smallsetminus \Gamma e$, i.e. polynomials in t; their q-expansions are integral, so they lie in $\mathbb{Z}[t]$. At e let $m_e = m(e)$. By Lemma 37.2, m_e of the W_β have a pole of order k_0 in $\tau - e$, i.e. of order 1 in t, and the corresponding x_β are bounded; the other W_β are bounded and the corresponding x_β vanish to order o in $\tau - e$, i.e. to order r in $1/t$. The pole order of D^{k_0} at e is $p+1-m_e$ in t, and that of D^o is $r(p+1-m_e)$. Therefore, measuring pole orders in t: e_k has a pole of order $\le \min(k, m_e)$, so $\deg E_k \le \min(k, m_e) + p + 1 - m_e \le p+1$. Each term of n_l has a pole of order $\le \min(l, m_e - 1)$ (if β is one of the m_e branches) or $\le \min(l, m_e) - r$ (otherwise), so $\deg M_l \le p + 1 - m_e + \max(\min(l, m_e - 1), \min(l, m_e) - r) \le p + 1 - r$, hence $\le p$. Finally S_0 is bounded at e, and vanishes to order r if $m_e = 0$, so $\deg A_0 \le r(p + 1 - m_e) - r[m_e = 0] \le pr$. Lastly, $E_1 \equiv \chi_\vartheta(p)^{k_0} e_1 \equiv \pm t^p \pmod p$ by Lemmas 37.3 and 37.4(a), so $[t^{p+1}]E_1 \equiv 0 \pmod p$.

Step 2: Gauss norms. For $\rho \ge 1$ and $f = \sum c_n t^n \in \mathbb{Q}_p[[t]]$ put $|f|_\rho = \sup_n |c_n|_p \rho^n$. This is multiplicative and satisfies the ultrametric inequality. Let $R = p^{k_0/(p+1)}$ and $1 \le \rho < R$. By Lemma 37.3, $D = \pm(1 + pC)$ with $C \in \mathbb{Z}[t]$ of degree $\le (p+1)/k_0$, so $|pC|_\rho \le p^{-1}\rho^{(p+1)/k_0} < 1$; hence $D^{-1} = \pm\sum_j (-pC)^j$ and $|D|_\rho = |D^{-1}|_\rho = 1$. Consequently

$$|e_1|_\rho = |E_1|_\rho \le \rho^p, \qquad |e_k|_\rho \le \rho^{p+1}, \qquad |n_l|_\rho \le \rho^p, \qquad |S_0|_\rho \le \rho^{c_0}, \qquad c_0 := \lfloor pr \rfloor;$$

for the first bound note $|[t^{p+1}]E_1|_p \rho^{p+1} \le p^{-1}\rho^{p+1} \le \rho^p$ because $\rho < R \le p$.

Step 3: induction. We show $|S_l|_\rho \le \rho^{pl + c_0}$ for all $l \ge 0$. For $l = 0$ this is Step 2. For $l \ge 1$, Lemma 37.4(b) gives $S_l = (-1)^l n_l - \sum_{k=1}^{\min(l, p+1)} (-1)^k e_k S_{l-k}$ with $n_l := 0$ for $l > p$. Here $|n_l|_\rho \le \rho^p \le \rho^{pl + c_0}$, $|e_1 S_{l-1}|_\rho \le \rho^{p + p(l-1) + c_0}$ and, for $k \ge 2$, $|e_k S_{l-k}|_\rho \le \rho^{p+1+p(l-k)+c_0} \le \rho^{pl + c_0}$.

Step 4. Hence $|[t^n]S_i|_p \le \rho^{pi + c_0 - n}$ for every $\rho \in [1, R)$. If $n \ge pi + c_0$ we let $\rho \to R$ and get $|[t^n]S_i|_p \le p^{-k_0(n - pi - c_0)/(p+1)}$; if $n < pi + c_0$ the assertion is trivial since $S_i \in \mathbb{Z}[[t]]$. □

Remark 37.6. The bound is sharp in the sense that check [8d] finds equality for many (i,n). The rate $k_0/(p+1)$ is the reciprocal of the number of Hecke translates of e on $X(\Gamma)$ for p inert in the CM field of e. These translates are CM points of conductor p; for p inert they lie on the circle $|t|_p = R$ (when C has a unit leading coefficient), which is where the trace has its poles. The proof does not need this description.

37.5 The truncation coefficients

Lemma 37.7. *Let $p \geq 5$ and $c = w+1$ ($c = 2,3,3$ for F_{27}, F_{256}, F_{108}). If n has the digit $p-1$ in exactly d positions of its base-p expansion, then $v_p(a_n) \geq cd$.*

Proof. By Legendre's formula $v_p((kn)!/n!^k) = \sum_{j\geq 1} \lfloor k\{n/p^j\} \rfloor$, where $\{x\}$ is the fractional part; all terms are ≥ 0. If the digit of n at position $j-1$ is $p-1$, then $\{n/p^j\} \geq (p-1)/p$ and the j-th term is $\geq \lfloor k(p-1)/p \rfloor = k-1$ for $p \geq k$. This gives $2d$ for $(3n)!/n!^3$ and $3d$ for $(4n)!/n!^4$. For $(2n)!(3n)!/n!^5$ the j-th term is $\lfloor 2\{n/p^j\} \rfloor + \lfloor 3\{n/p^j\} \rfloor \geq 1+2$. □

Lemma 37.8. *Let $p \geq 5$, $(k_0, o, c) \in \{(3,1,2), (4,1,3), (6,2,3)\}$ and $r = o/k_0$. For $j \geq 0$ let $\kappa = 0$ if $j = 0$ and $\kappa = 1 + \lfloor \log_p j \rfloor$ (the number of base-p digits of j) if $j \geq 1$. Then*

$$\left\lceil \frac{k_0}{p+1}\bigl(p(j+1) - \lfloor pr \rfloor\bigr) \right\rceil \;\geq\; c(\kappa+1).$$

Proof. The left side is at least $\lceil x \rceil$ with $x = \frac{p}{p+1}(k_0(j+1) - o) \geq \frac{5}{6}(k_0(j+1) - o)$, and it suffices that $x > c(\kappa+1) - 1$. For $j = 0$: $\frac{5}{6}(k_0 - o) = \frac{5}{3}, \frac{5}{2}, \frac{10}{3}$, which exceed $c - 1 = 1, 2, 2$. For $j \geq 1$ we have $j + 1 \geq p^{\kappa-1} + 1 \geq 5^{\kappa-1} + 1$, and $\frac{5}{6}(k_0(5^{\kappa-1}+1) - o) > c(\kappa+1) - 1$ holds for $\kappa = 1$ ($\frac{25}{6} > 3$, $\frac{35}{6} > 5$, $\frac{50}{6} > 5$). When κ increases by one, the left side increases by $\frac{5}{6}k_0(5^\kappa - 5^{\kappa-1}) \geq \frac{10}{3}k_0 \geq 10$ and the right side by $c \leq 3$, so the inequality holds for all $\kappa \geq 1$. □

37.6 Proof of Theorem 32.2

Fix $p \geq 5$ and $m \geq 1$, put $K_j = mp^j$, $F_{K_{-1}} = 0$ and $c = w+1$. We prove

$$(\mathrm{H}_s) \qquad \Theta(q)\, F_{K_{s-1}}(t(q^p)) \equiv \Theta(q^p)\, F_{K_s}(t(q)) \pmod{p^{cs}\mathbb{Z}[[q]]}$$

by induction on $s \geq 1$; as in §35 it is equivalent to the theorem. Let $P = F_{K_{s-1}} = \sum_{i<K_{s-1}} a_i t^i$ and $Q = Q_P$. By linearity and Lemma 34.2,

$$Q = \sum_{i<K_{s-1}} a_i S_i - p^{w+1}\chi(p)\, U_p G_P.$$

Let $n \geq K_s$. As in the proof of the claim in §35, (H_{s-1}) gives $p^{w+1}U_pG_P \equiv p^{w+1}F_{K_{s-2}}(t) \pmod{p^{w+1+c(s-1)}}$, so $[t^n]\, p^{w+1}U_pG_P \equiv 0 \pmod{p^{cs}}$ (for $s = 1$ trivially). Now let $i < K_{s-1}$ and $j = K_{s-1} - 1 - i$, with κ as in Lemma 37.8. Since $n - pi \geq K_s - pi = p(j+1)$, Proposition 37.5 and Lemma 37.8 give $v_p([t^n]S_i) \geq c(\kappa+1)$. If $\kappa \leq s-1$, then $j < p^\kappa$ and

$$i = (m-1)p^{s-1} + (p^{s-1} - p^\kappa) + (p^\kappa - 1 - j), \qquad 0 \leq p^\kappa - 1 - j < p^\kappa,$$

so the digits of i in the positions $\kappa, \ldots, s-2$ are $p-1$, and $v_p(a_i) \geq c(s-1-\kappa)$ by Lemma 37.7. In all cases $v_p(a_i) + v_p([t^n]S_i) \geq cs$. Therefore $[t^n]Q \equiv 0 \pmod{p^{cs}}$ for all $n \geq K_s$. On the other hand $[t^n]Q = a_n$ for $n < K_s$ by Lemma 33.4(c). Hence $Q \equiv F_{K_s}(t) \pmod{p^{cs}}$, which is (H_s). □

Remark 37.9. The proof shows where the exponent comes from. The top monomials t^i, i close to mp^{s-1}, contribute through $[t^n]S_i$ with $n - pi \approx p$, which is only divisible by $p^{\lceil (k_0-o)p/(p+1) \rceil}$. This loss is exactly compensated by the c factors of p per digit $p-1$ of i. For the families of Theorem 32.1 the same argument works trivially, because S_i is then a polynomial of degree $< pi + p$.

38 Zagier's case D

In this section we prove Theorem 32.3. Let $a_n = \sum_k \binom{n}{k}^2 \binom{n+k}{k}$ and $F = \sum a_n t^n$, and put

$$t(\tau) = q \prod_{n\geq 1} (1-q^n)^{5(\frac{n}{5})}, \qquad \Theta(\tau) = \prod_{n\equiv \pm 1\,(5)} (1-q^n)^{-3} \prod_{n\not\equiv \pm 1\,(5)} (1-q^n)^2,$$

with $(\frac{n}{5})$ the Legendre symbol. For $d \in (\mathbb{Z}/5\mathbb{Z})^\times$ let $\sigma_d \in \Gamma_0(5)$ be a matrix with lower right entry $\equiv d \pmod 5$; the action of σ_d on functions for $\Gamma_1(5)$ (the diamond operator) depends only on d.

38.1 The modular data

Lemma 38.1. (a) $F(t(q)) = \Theta(q)$, $\Theta^2 t = \eta(5\tau)^5/\eta(\tau)$ *and* $1/t - 11 - t = \eta(\tau)^6/\eta(5\tau)^6$.

(b) t *is a Hauptmodul for* $\Gamma_1(5)$. *It is holomorphic and nonzero on* $\mathbb{H}$, *has a simple zero at* ∞, *a simple pole at the cusp* $\frac{2}{5}$, *and the values* $\frac{-11\pm 5\sqrt{5}}{2}$ *at the cusps* $0, \frac{1}{2}$. *Moreover* $t\circ\sigma_{\pm 2} = -1/t$ *and* $t\circ\sigma_{-1} = t$.

(c) Θ *is a modular form of weight one for* $\Gamma_1(5)$, *without zeros in* $\mathbb{H}$, *with a simple zero at* $\frac{2}{5}$ *and no other zero at the cusps. There is* $c \in \{\pm 1\}$ *with*

$$\Theta|_1\sigma_2 = c\,t\Theta, \qquad \Theta|_1\sigma_3 = -c\,t\Theta, \qquad \Theta|_1\sigma_{-1} = -\Theta.$$

Proof. (a) The first identity is Beukers's modular parametrisation of the Apéry numbers for $\zeta(2)$ [10], see also [23, case D]; the third is Ramanujan's identity for the Rogers–Ramanujan continued fraction $r(\tau)$, since $t = r^5$ [14, 11]. All three are checked to $O(q^{79})$ by check [9a].

(b) That $t = r^5$ is a Hauptmodul for $\Gamma_1(5)$ is classical [14]. It is holomorphic and nonzero on $\mathbb{H}$ by its product expansion, and $t = q + O(q^2)$. The right side of the third identity in (a) is a Hauptmodul for $\Gamma_0(5)$, with its only pole at the cusp ∞ of $X_0(5)$ and its only zero at the cusp 0. Since $X_1(5) \to X_0(5)$ has degree two and t has degree one, t is not $\Gamma_0(5)$-invariant; as $1/t - t$ is, we get $t\circ\sigma_{\pm 2} = -1/t$ and $t\circ\sigma_{\pm 1} = t$. The fibre of $X_1(5) \to X_0(5)$ over ∞ consists of the cusps ∞ and $\sigma_2\infty \sim \frac{2}{5}$, both of width one, and over 0 of 0 and $\sigma_2 0 = \frac{1}{2}$, both of width five. Hence t has its pole at $\frac{2}{5}$, and at 0, $\frac{1}{2}$ it takes the two roots of $t^2 + 11t - 1 = 0$.

(c) $\eta(5\tau)^5/\eta(\tau)$ is a modular form of weight two for $\Gamma_0(5)$ with character $(\frac{\cdot}{5})$ [20, Thm. 1.64], without zeros in $\mathbb{H}$; by Ligozat's formula [20, Thm. 1.65] it vanishes to order one (in $e^{2\pi i\tau}$) at the cusps $\infty, \frac{2}{5}$ and does not vanish at $0, \frac{1}{2}$. By (a) and (b), $\Theta^2 = \eta(5\tau)^5/(\eta(\tau)t)$ is therefore a meromorphic modular form of weight two for $\Gamma_1(5)$, holomorphic and nonzero on $\mathbb{H}$, of order $0, 2, 0, 0$ at $\infty, \frac{2}{5}, 0, \frac{1}{2}$. The series $\Theta \in 1 + q\mathbb{Z}[[q]]$ is a holomorphic square root of it on $\mathbb{H}$, so $\Theta|_1\gamma = \epsilon(\gamma)\Theta$ for $\gamma \in \Gamma_1(5)$, with a character $\epsilon : \Gamma_1(5) \to \{\pm 1\}$. The group $\Gamma_1(5)$ is torsion free, does not contain -1, and $X_1(5)$ has genus zero with four cusps, all regular; so $\Gamma_1(5)$ is the free group generated by parabolic elements AT^hA^{-1} fixing three of the cusps. Since the orders of Θ^2 at the cusps are even, $\Theta|_1 A$ is a power series in $e^{2\pi i\tau/h}$ at each cusp $A\infty$ of width h, and ϵ is trivial on these generators. Thus $\epsilon = 1$, and Θ has the stated zeros.

For $d = \pm 2$ we get from (a), (b) and $(\frac{\pm 2}{5}) = -1$ that $(\Theta|_1\sigma_d)^2 = \Theta^2|_2\sigma_d = -\eta(5\tau)^5/(\eta(\tau)\cdot(-1/t)) = t^2\Theta^2$, so $\Theta|_1\sigma_d = \pm t\Theta$ with a constant sign. If $\Theta|_1\sigma_2 = ct\Theta$, then, as σ_3 acts as σ_2^{-1}, $\Theta = (ct\Theta)|_1\sigma_3 = c\,(-1/t)\,\Theta|_1\sigma_3$, i.e. $\Theta|_1\sigma_3 = -ct\Theta$. Finally $\sigma_{-1} \in -\Gamma_1(5)$ and $\Theta|_1(-1) = -\Theta$. (Numerically $c = 1$, check [9a], but we do not need this.) □

Symbolic identification of the Zagier D coordinate. For the Zagier D parametrisation one similarly has the exact identity

$$\frac{Dt}{t} = (1 - 11t - t^2)\Theta^2.$$

Indeed, the diamond relation $t\circ\sigma_2 = -1/t$ shows that Dt/t is a holomorphic weight-two form on $\Gamma_0(5)$ with quadratic character $(\frac{\cdot}{5})$. The expression on the right has the same transformation

law: under $t \mapsto -1/t$ and $\Theta^2 \mapsto t^2\Theta^2$ it changes sign. The cusp orders in Lemma 38.1 show that it is holomorphic at every cusp. Both sides have expansion $1 - 5q + O(q^2)$. The total valence is $2 \cdot 6/12 = 1$, so their difference, of order at least two at infinity, vanishes identically. The Wronskian equation of

$$(\theta^2 - t(11\theta^2 + 11\theta + 3) - t^2(\theta+1)^2)F = 0$$

gives $d\log q_F/dt = 1/[t(1 - 11t - t^2)F(t)^2]$ for the normalized Frobenius coordinate. Since $F(t(q)) = \Theta(q)$ and $t = q + O(q^2)$, the preceding modular identity proves $q_F(t(q)) = q$. Thus the mirror-map assertion does not rely on finite check [9a].

38.2 Hecke operators on $\Gamma_1(5)$

Let $p \neq 5$ be a prime and $\Delta_p = \{\beta \in M_2(\mathbb{Z}) : \det\beta = p,\ \beta \equiv \left(\begin{smallmatrix}1 & *\\ 0 & p\end{smallmatrix}\right) \pmod 5\}$. Then $\Delta_p = \Gamma_1(5)\left(\begin{smallmatrix}1 & 0\\ 0 & p\end{smallmatrix}\right)\Gamma_1(5)$, and $\Gamma_1(5)\backslash\Delta_p$ has the $p+1$ representatives $\beta_k = \left(\begin{smallmatrix}1 & k\\ 0 & p\end{smallmatrix}\right)$, $0 \le k < p$, and $\beta_\infty = \sigma_p\left(\begin{smallmatrix}p & 0\\ 0 & 1\end{smallmatrix}\right)$ [13, Prop. 5.2.1]. Clearly $\gamma\Delta_p = \Delta_p\gamma = \Delta_p$ for $\gamma \in \Gamma_1(5)$, and $\sigma\Delta_p\sigma^{-1} = \Delta_p$ for $\sigma \in \Gamma_0(5)$, because the diagonal entries modulo 5 are unchanged by conjugation with a matrix that is upper triangular modulo 5.

Put $\check\Theta = t\Theta$ and $\check t = -1/t$. For $P \in \mathbb{Z}[T]$ of degree D and $\beta \in \Delta_p$ let

$$R_\beta = p^{1/2}\,\frac{\Theta\cdot P(t\circ\beta)}{\Theta|_1\beta}, \qquad \check R_\beta = p^{1/2}\,\frac{\check\Theta\cdot P(\check t\circ\beta)}{\check\Theta|_1\beta}, \qquad \Lambda_P = \sum_{[\beta]} R_\beta, \qquad \check\Lambda_P = \sum_{[\beta]}\check R_\beta,$$

the sums over $\Gamma_1(5)\backslash\Delta_p$. Besides Q_P and G_P (with this Θ and t) we use

$$\check Q_P(q) = \frac{\Theta(q)\,P(-1/t(q^p))}{t(q^p)\,\Theta(q^p)}, \qquad \check G_P(z) = \frac{\Theta(z^p)\,P(-1/t(z))}{t(z)\,\Theta(z)},$$

which are Laurent series with integer coefficients.

Lemma 38.2. (a) *Λ_P and $\check\Lambda_P$ are well defined and $\Gamma_1(5)$-invariant, and $\check\Lambda_P = \Lambda_P\circ\sigma_2$.*

(b) *If $p \equiv \pm1 \pmod 5$, then $\Lambda_P = \eta'_p Q_P + p^2U_pG_P$ with $\eta'_p = \pm1$ for $p \equiv \pm1$, and Λ_P is a polynomial in t with integer coefficients of degree at most $p(D+1)-1$.*

(c) *If $p \equiv \pm2 \pmod 5$, let $\eta_p = c$ for $p \equiv 2$ and $\eta_p = -c$ for $p \equiv 3 \pmod 5$. Then*

$$\Lambda_P = \eta_p\check Q_P + p^2U_pG_P, \qquad \check\Lambda_P = -\eta_p\,t\,Q_P + p^2\,t\,U_p\check G_P,$$

Λ_P and $\check\Lambda_P$ are Laurent polynomials in t with integer coefficients, $\check\Lambda_P$ has degree at most $p(D+1)$ in t, and $\check\Lambda_P(t) = \Lambda_P(-1/t)$.

Proof. (a) Since $\Theta|_1\gamma = \Theta$ and $t\circ\gamma = t$ for $\gamma \in \Gamma_1(5)$, the proof of Lemma 34.1(a),(b) (with $\chi = 1$) shows $R_{\gamma\beta} = R_\beta$ and $R_\beta\circ\gamma = R_{\beta\gamma}$, and likewise for $\check R_\beta$. For the last claim let $\beta \in \Delta_p$ and $\beta' = \sigma_2^{-1}\beta\sigma_2 \in \Delta_p$. Then $t(\beta\sigma_2\tau) = t(\sigma_2\beta'\tau) = \check t(\beta'\tau)$, and with $\Theta|_1\sigma_2 = c\check\Theta$,

$$\frac{\Theta(\sigma_2\tau)}{(\Theta|_1\beta)(\sigma_2\tau)} = \frac{(\Theta|_1\sigma_2)(\tau)}{(\Theta|_1\beta\sigma_2)(\tau)} = \frac{c\check\Theta(\tau)}{(\Theta|_1\sigma_2\beta')(\tau)} = \frac{\check\Theta(\tau)}{(\check\Theta|_1\beta')(\tau)}.$$

Hence $R_\beta\circ\sigma_2 = \check R_{\beta'}$, and $\beta \mapsto \beta'$ permutes $\Gamma_1(5)\backslash\Delta_p$.

(b), (c) As in Lemma 34.2, the classes β_k contribute $p^2U_pG_P$ to Λ_P, and to $\check\Lambda_P$ they contribute

$$p^2U_p\Big(\frac{\check\Theta(z^p)\,P(\check t(z))}{\check\Theta(z)}\Big) = p^2\,t\,U_p\check G_P.$$

For β_∞ we have $(\Theta|_1\beta_\infty)(\tau) = p^{1/2}(\Theta|_1\sigma_p)(p\tau)$ and $t\circ\beta_\infty = (t\circ\sigma_p)(p\tau)$. If $p\equiv\pm1$, Lemma 38.1 gives $R_{\beta_\infty} = \eta'_p Q_P$. If $p\equiv\pm2$, it gives $R_{\beta_\infty} = \Theta(\tau)P(-1/t(p\tau))/(\eta_p t(p\tau)\Theta(p\tau)) = \eta_p\check{Q}_P$, and, as $\check{\Theta}|_1\sigma_p = (t\circ\sigma_p)\,\Theta|_1\sigma_p = -\eta_p\Theta$ and $\check{t}\circ\sigma_p = t$, $\check{R}_{\beta_\infty} = t\Theta P(t(p\tau))/(-\eta_p\Theta(p\tau)) = -\eta_p t Q_P$.

It remains to locate the poles. All R_β, $\check{R}_\beta$ are holomorphic on $\mathbb{H}$, because Θ has no zeros and t no poles or zeros there. Let $c' = A\infty$ be a cusp, $A\in\mathrm{SL}_2(\mathbb{Z})$, and for $\beta\in\Delta_p$ write $\beta A = A_\beta\left(\begin{smallmatrix}a' & b'\\ 0 & d'\end{smallmatrix}\right)$ with $A_\beta\in\mathrm{SL}_2(\mathbb{Z})$, $a'd' = p$, as in Lemma 34.3; then $\beta c' = A_\beta\infty$, and exactly one class has $(a',d') = (p,1)$. For a $\Gamma_1(5)$-invariant φ we have $\nu_A(\varphi\circ\beta) = \frac{a'}{d'}\nu_{A_\beta}(\varphi)$, and $\nu_A(f|_1\beta) = \frac{a'}{d'}\nu_{A_\beta}(f)$ for a form f of weight one; here ν is the order in $e^{2\pi i\tau}$, not normalised by the width (proof of Lemma 34.3). If $c'\in\{0,\frac12\}$, the cusp $\beta c'$ again has denominator prime to 5, so lies in the class of 0 or $\frac12$, where $t,\check{t},\Theta,\check{\Theta}$ are finite and nonzero; all branches are holomorphic at c'. Let $c' = x/(5y)$ with $\gcd(x,5y)=1$ be in the class of ∞ ($x\equiv\pm1$) or of $\frac25$ ($x\equiv\pm2$). Writing $\beta = \left(\begin{smallmatrix}\alpha & b\\ 5\gamma & \delta\end{smallmatrix}\right)$ with $\alpha\equiv1 \pmod 5$, we have $\beta(x,5y)^{\mathsf T} = a'(x',5y')^{\mathsf T}$ with $\gcd(x',5y')=1$ and $x'\equiv x/a' \pmod 5$. So the branches with $a'=1$ preserve the classes, and the branch with $a'=p$ multiplies the class by p^{-1}: it preserves the classes if $p\equiv\pm1$ and exchanges the classes of ∞ and $\frac25$ if $p\equiv\pm2$.

At $\frac25$ we have $\nu(t)=-1$, $\nu(\Theta)=1$, $\nu(\check{t})=1$, $\nu(\check{\Theta})=0$, and at ∞ the opposite: $\nu(t)=1$, $\nu(\Theta)=0$, $\nu(\check{t})=-1$, $\nu(\check{\Theta})=1$. If $p\equiv\pm1$, all branches are holomorphic at ∞, and at $\frac25$ the branch with $a'=p$ has $\nu(R_\beta) = 1-p-pD$, the others $\nu(R_\beta) = 1-\frac1p-\frac{D}{p}$. So Λ_P is a polynomial in t of degree at most $p(D+1)-1$. If $p\equiv\pm2$, then at $\frac25$ the branch with $a'=p$ has $\nu(\check{R}_\beta) = 0-p\cdot1+pD\cdot(-1) = -p(D+1)$ and the others $\nu(\check{R}_\beta)\ge0$, while at ∞ poles may occur. So $\check{\Lambda}_P$ is a Laurent polynomial in t of degree at most $p(D+1)$. The $\Gamma_1(5)$-invariant function Λ_P is a rational function of t, and by (a) and $t\circ\sigma_2 = -1/t$ we have $\check{\Lambda}_P(t) = \Lambda_P(-1/t)$; hence $\Lambda_P(t) = \check{\Lambda}_P(-1/t)$ is a Laurent polynomial as well. Integrality of the coefficients follows from the q-expansions, as $t\in q+q^2\mathbb{Z}[[q]]$. □

38.3 Proof of Theorem 32.3

Fix $p\neq5$ and $m\ge1$, and put $K_j = mp^j$, $F_{K_{-1}} = 0$. Note that $\Theta\in1+q\mathbb{Z}[[q]]$ and $t\in q+q^2\mathbb{Z}[[q]]$, so Lemma 33.4 applies.

Case $p\equiv\pm1 \pmod 5$. By Lemma 38.2(b), $Q_P = \eta'_p\Lambda_P - \eta'_p p^2 U_p G_P$ with Λ_P a polynomial in t of degree $\le p\deg P + p - 1$. This is (35.1) with $w=1$ and $\chi(p)$ replaced by η'_p, and the proof of §35 applies word for word.

Case $p\equiv\pm2 \pmod 5$. We prove by simultaneous induction on $s\ge1$ the congruences

$$\begin{aligned}&(\mathrm{H}_s) && \Theta(q)\,F_{K_{s-1}}(t(q^p)) \equiv \Theta(q^p)\,F_{K_s}(t(q)),\\ &(\check{\mathrm{H}}_s) && t(q)\Theta(q)\,F_{K_{s-1}}(-1/t(q^p)) \equiv t(q^p)\Theta(q^p)\,F_{K_s}(-1/t(q)),\end{aligned}$$

modulo p^{2s}, as Laurent series in q. With $P = F_{K_{s-1}}$ they read $Q_P\equiv F_{K_s}(t)$ and $t\check{Q}_P\equiv F_{K_s}(-1/t)$. Let $s\ge1$, assume (H_{s-1}) and $(\check{\mathrm{H}}_{s-1})$ if $s\ge2$, and let $P = F_{K_{s-1}}$, $D = K_{s-1}-1$.

Claim: $p^2U_pG_P \equiv p^2F_{K_{s-2}}(t)$ and $p^2U_p\check{G}_P \equiv p^2F_{K_{s-2}}(-1/t)/t$ modulo p^{2s}. For $s=1$ all four terms are $\equiv0 \pmod{p^2}$. For $s\ge2$, the first congruence follows from (H_{s-1}) as in §35. For the second, $(\check{\mathrm{H}}_{s-1})$ in the variable z says $F_{K_{s-1}}(-1/t(z)) \equiv t(z)\Theta(z)F_{K_{s-2}}(-1/t(z^p))/(t(z^p)\Theta(z^p))$ modulo p^{2s-2}, hence $\check{G}_P(z)\equiv F_{K_{s-2}}(-1/t(z^p))/t(z^p)$, and we apply U_p.

Proof of (H_s). By Lemma 38.2(c) and the claim,

$$Q_P = -\eta_p\frac{\check{\Lambda}_P(t)}{t} + \eta_p p^2 U_p\check{G}_P \equiv L(t) := -\eta_p\frac{\check{\Lambda}_P(t)}{t} + \eta_p p^2\,\frac{F_{K_{s-2}}(-1/t)}{t} \pmod{p^{2s}},$$

where L is a Laurent polynomial whose terms of degree ≥0 have degree $\le p(D+1)-1 = K_s-1$, and whose second summand contains only negative powers of t. Since $Q_P\in F(t)+t^{K_s}\mathbb{Z}[[t]]$

(Lemma 33.4(c)), comparing coefficients gives $[t^n]Q_P \equiv 0$ for $n \geq K_s$, which is (H_s), and moreover the identity of Laurent polynomials

$$\check{\Lambda}_P(t) \equiv -\eta_p\, t\, F_{K_s}(t) + p^2 F_{K_{s-2}}(-1/t) \pmod{p^{2s}}. \tag{38.1}$$

Proof of $(\check{\mathrm{H}}_s)$. By Lemma 38.2(c), (38.1) with t replaced by $-1/t$, and the claim,

$$\begin{aligned}\eta_p \check{Q}_P &= \Lambda_P - p^2 U_p G_P = \check{\Lambda}_P(-1/t) - p^2 U_p G_P \\ &\equiv \frac{\eta_p}{t} F_{K_s}(-1/t) + p^2 F_{K_{s-2}}(t) - p^2 F_{K_{s-2}}(t) = \frac{\eta_p}{t} F_{K_s}(-1/t)\end{aligned}$$

modulo p^{2s}. Multiplying by $\eta_p t$ gives $(\check{\mathrm{H}}_s)$. This completes the induction. □

Remark 38.3. The two traces are the Hecke operators T_p applied to the weight -1 forms $P(t)/\Theta$ and $P(-1/t)/(t\Theta)$, which are exchanged by $\langle 2\rangle$; for $p \equiv \pm 2$ the operator T_p involves $\langle p\rangle$ and therefore mixes the family with its dual. The terms $p^2 F_{K_{s-2}}$ cancel in the last display; this is why no condition on the signs is needed.

39 Limits of the modular method and an elliptic extension

39.1 The hyperoctahedral family in dimension three

Let $c_k = \binom{2k}{k} \sum_j \binom{k}{j}^2 \binom{2j}{j}$, the number of closed walks of length $2k$ on $\mathbb{Z}^3$, and $F(u) = \sum c_k u^k$. The hyperoctahedral family with $n = 3$ is $F(t^2)$, and by Lemma 36.1 Conjecture BV for it is (32.1) for F with $m = 1$, modulo p^{2s}. The family is modular (check [7b]): with the Hauptmodul $v = (\eta_1\eta_6/\eta_2\eta_3)^{12}$ of $\Gamma_0(6) + 6$ of the Apéry family of Table 1,

$$u = \frac{v}{(1+v)^2}, \qquad F(u(q)) = (1+v)\,\frac{\eta_2^7\eta_3^7}{\eta_1^5\eta_6^5},$$

and $u(q)$ is the mirror map of F. The function u is a Hauptmodul for the group $\Gamma_0(6)+$ generated by $\Gamma_0(6)$ and all Atkin–Lehner involutions. Its pole is the fixed point $v = -1$ of the involution $v \mapsto 1/v$, an elliptic point of order $k_0 = 2$ of $\Gamma_0(6)+$, and $\Theta = (1+v)\eta_2^7\eta_3^7/\eta_1^5\eta_6^5$ has a simple zero there. So this is a family of elliptic type with $w = 2$, $o = 1$, $r = \frac{1}{2}$. (Numerically, the degree of the polynomial D of Lemma 37.3 is $(p \mp 1)/2$ according as $p \equiv \pm 1 \pmod 3$ (check [7d]), which indicates that the elliptic point has CM by $\mathbb{Q}(\sqrt{-3})$.)

The proof of Proposition 37.5 applies and gives $v_p([u^n]S_i) \geq \frac{2}{p+1}(n - pi - \lfloor p/2 \rfloor)$ (check [7e] confirms it for $p = 5$). At the top monomials of the truncation this yields only one factor p, and the coefficients c_k gain only one factor p per digit $p-1$, from $\binom{2k}{k}$; the sum $\sum_j \binom{k}{j}^2\binom{2j}{j}$ is a p-adic unit for $k = p-1$ and $k = p^2-1$ (check [7c], $p \leq 13$). Hence, for the monomials at the top of the truncation, the lower bound for $v_p(c_i) + v_p([u^n]S_i)$ that the argument of §37 provides is only about s, not $2s$. In fact, for $p = 5$ and $s = 3$ the termwise bound $\min_i(v_p(c_i) + v_p([u^n]S_i))$ over $125 \leq n < 150$ equals $5 < 6$, while the tail of the trace has valuation ≥ 6 (check [7e]). So the congruence modulo p^{2s}, which holds in all our tests (check [7a]), requires cancellation between the contributions of different monomials; this termwise estimate does not prove the congruence. The full bound is proved by Theorem 2.1. It is consistent with [8, Remark 1.14] that only the exponent $2s$, not $3s$, occurs: here $1/r = 2 < w + 1$.

39.2 Dimension four and higher

For dimensions $n \geq 4$, the modular arguments in this part do not provide a uniform proof for the three standard families. The positive-vertex and matrix arguments of Part I prove the required coefficientwise congruence for every such dimension in the stated full-symmetry prime ranges, without assuming a modular parametrisation. The mirror-quintic simplicial period is included in that theorem.

39.3 A conjecture

Theorems 32.1 and 32.2 and the data for the hyperoctahedral family suggest the following.

Conjecture 39.1. *Let $(N, \Gamma, w, \chi, \Theta, t)$ satisfy* (F1)–(F3) *and the conditions of Lemma 37.1*(c) *for an elliptic point e of order k_0, with $r = o/k_0$. Then for every prime $p \nmid N$ with $p \geq 5$ and all $m, s \geq 1$, congruence* (32.1) *with the excellent lift holds modulo $p^{\min(w+1, \lfloor 1/r \rfloor)s}$.*

For the families of Table 2, this is Theorem 32.2. Its three-dimensional hyperoctahedral specialization follows from Theorem 2.1. The assertion for arbitrary other elliptic-zero families is not claimed in this volume.

40 Computational verification

The script `verify_excellent_lift.py` (Appendix B; standard library only, about two and a half minutes) works with exact integer and rational arithmetic; the three floating point checks are marked as such. It performs the following checks.

[1] $F(t(q)) = \Theta(q)$ to $O(q^{60})$ for the 13 families of Table 1, with F given by the closed formulas of §33.5.

[2] Ligozat orders of Θ and t at all cusps: zeros of Θ = poles of t = the listed cusps; the value of r_Θ; condition (C_p) via Lemma 33.3; the character of Θ; the Atkin–Lehner symmetry of Θ and t.

[3] Proposition 34.4: for $P = F_{mp^{s-1}}$ and for random integer polynomials P, the series $Q_P + p^{w+1}\chi(p)U_pG_P$, expanded in t, has all coefficients of degree $> p \deg P + \lfloor r_\Theta(p-1) \rfloor$ equal to 0, for $p \in \{2, 3, 5, 7, 11\}$, $p \nmid N$.

[4] Theorem 32.1: the valuation of the difference in (H_s), for all primes $p \leq 13$ with $p \nmid N$ and all pairs (s, m) in $\{(1,1), (1,2), (2,1), (2,2), (3,1)\}$ with $mp^s \leq 130$.

[5] Corollary 32.6 for $mp^s \leq 250$.

[6] Conjecture BV in its own normalisation for the two- and three-dimensional hypercubic families. The canonical coordinate is computed from the Picard–Fuchs recurrence by the Frobenius method, the excellent lift is $q \mapsto q^p$, and (32.1) is checked with $m = 2$.

[7] The hyperoctahedral family with $n = 3$ (§39.1): [7a] the congruence modulo p^{2s} for small p, s, with the canonical coordinate computed from the Picard–Fuchs recurrence; [7b] the modular parametrisation and the mirror map; [7c] $v_p(c_{p^k-1}) = k$; [7d] the degree of D; [7e] Proposition 37.5 and the failure of the termwise bound for $p = 5$, $s = 3$.

[8] Theorem 32.2: [8a] the identities of Lemma 37.1 and, in floating point, $\Theta(e) = 0$; [8b] Lemma 37.3 for $p \leq 13$ (and $B = D^2$ for F_{108}); [8c] the polynomials E_k, M_l, A_0 of Proposition 37.5 and their degrees for $p \in \{5, 7\}$; [8d] Proposition 37.5 for $p \leq 13$; [8e] the congruence (H_s) modulo $p^{(w+1)s}$ for $p \leq 13$ and $mp^s \leq 130$; [8f] Lemma 37.8 for $p < 200$, $\kappa \leq 12$, and Lemma 37.7 for sample n; [8g] Conjecture BV for the simplicial families with $n = 2, 3$ in the normalisation of [7], and the mirror map identity; [8h] the trace of Q_1 for F_{27} and $p = 5$ is not a polynomial in t.

[9] Theorem 32.3: [9a] Lemma 38.1(a), the orders of $\eta(5\tau)^5/\eta(\tau)$ at the cusps, the mirror map, and in floating point the action of the diamond operators; [9b] Lemma 38.2 for $p \in \{2, 3, 7, 11, 13, 17, 19\}$ and several P; [9c] (H_s) and ($\check{H}_s$) modulo p^{2s} for $p \in \{2, 3, 7, 11, 13, 17\}$ and $mp^s \leq 125$.

The supplied execution record reports 114 passing checks and is reproduced in Appendix C. These bounded checks, including the explicitly identified floating-point evaluations, supplement the symbolic proofs; they do not establish infinite identities by extrapolation.

Part VI
Alternative elliptic and cusp proofs; exact sharpness

Norm-polynomial trace bounds, explicit Zagier D cusp analysis, and all retained coefficient obstructions.

41 Statements, normalization, and scope

Let

$$F(t)=\sum_{k\ge 0}a_k t^k\in 1+t\mathbb{Z}[[t]],\qquad F_M(t)=\sum_{0\le k<M}a_k t^k.$$

In a modular parametrization $F(t(q))=\Theta(q)$, with $t(q)\in q+q^2\mathbb{Z}[[q]]$, define

$$\sigma_p(t(q))=t(q^p).$$

The inverse series $q(t)$ is integral, so $\sigma_p(t)\in t^p+t^{p+1}\mathbb{Z}[[t]]$ and $\sigma_p(t)\equiv t^p \pmod p$. Every congruence in this note is an identity of complete formal power series, coefficient by coefficient. In particular, it is stronger than checking one coefficient or evaluating at one fixed parameter.

For later use define the cleared defect

$$E_{p,s,m}(t)=F(t)F_{mp^{s-1}}(\sigma_p(t))-F(\sigma_p(t))F_{mp^s}(t). \tag{41.1}$$

Since all denominator series have constant term one, the two statements

$$\frac{F(t)}{F(\sigma_p(t))}\equiv\frac{F_{mp^s}(t)}{F_{mp^{s-1}}(\sigma_p(t))}\pmod{p^{\rho s}\mathbb{Z}_p[[t]]},\qquad E_{p,s,m}\in p^{\rho s}\mathbb{Z}_p[[t]] \tag{41.2}$$

are equivalent.

41.1 The cases proved in this note

Theorem 41.1 (Main conclusions)**.** *For every $p\ge 5$ and all $m,s\ge 1$, (41.2) holds for the following three series, with the indicated exponent:*

$F(t)$	a_k	ρ
${}_2F_1(1/3,2/3;1;27t)$	$(3k)!/(k!)^3$	2
${}_3F_2(1/4,1/2,3/4;1,1;256t)$	$(4k)!/(k!)^4$	3
${}_3F_2(1/3,1/2,2/3;1,1;108t)$	$(3k)!(2k)!/(k!)^5$	3

For Zagier D,

$$a_k=\sum_{j=0}^{k}\binom{k}{j}^2\binom{k+j}{j},$$

the same assertion holds with $\rho=2$ for every prime $p\equiv\pm1 \pmod 5$ and all $m,s\ge 1$. For every prime $p\ne 5$, its first level $s=1$ holds for all $m\ge 1$.

The elliptic cases are proved in Section 42; the proof for Zagier D is in Section 43. These assertions are supported by the proofs, not by extrapolation from the accompanying calculations. Part V gives the modular framework and an alternative elliptic trace argument. The norm-polynomial estimate below is retained as a separate proof.

41.2 Relation to the original simplicial conjecture

For the simplicial Laurent polynomial in d variables,

$$g(x) = x_1 + \cdots + x_d + (x_1 \cdots x_d)^{-1}, \qquad r = d + 1,$$

the original period is

$$\mathcal{F}(z) = \sum_{k \geq 0} \frac{(rk)!}{(k!)^r} z^{rk} = F(z^r).$$

Beukers–Vlasenko's canonical coordinate is $q_0(z) = z \exp(G_0(z)/\mathcal{F}(z))$, where

$$G_0(z) = \sum_{k \geq 1} \frac{(rk)!}{(k!)^r} (H_{rk} - H_k) z^{rk}.$$

The normalized coordinate therefore satisfies

$$q(z^r) = q_0(z)^r.$$

Consequently the lifts are compatible: $(z^\sigma)^r = \sigma_p(z^r)$, and $\mathcal{F}_{rM}(z) = F_M(z^r)$. Thus the normalized case $m = 1$ gives exactly the original case $m = d + 1$, not a differently truncated assertion. For $d = 2, 3$ the symmetry and support-lattice hypotheses of [7, Theorem 7.3] exclude precisely the primes $2, 3$; the conclusions above cover every remaining prime.

41.3 Relation to the unified results

Target	Role of this part and full conclusion
Simplicial $d = 2, 3$	Full conjectured congruence for all admissible primes and all levels; the normalized $d = 3$ result has exponent $3s$.
The $108t$ family	Full formal-series congruence modulo p^{3s} for $p \geq 5$ and all m, s.
Zagier D	Full p^{2s} congruence for $p \equiv \pm 1 \pmod 5$; first level for every $p \neq 5$. All classes and levels are treated in Theorem 48.1.
Hyperoctahedral $d = 3$	An exact coefficient has valuation $2s$ at $p = 5$ for every s. The matching full lower bound is Theorem 2.1.
Dimensions $d \geq 4$	All-level sharpness obstructions for hypercubes, eventual sharpness for simplices, and a dimension-four octahedral obstruction. The full p^{2s} statement is Theorem 2.1.

The last two rows give upper bounds on a possible uniform exponent. The matching lower bound is supplied by Theorem 2.1; together they give optimality in the explicitly stated ranges. No assertion about priority in the literature is needed for the mathematical arguments below.

42 Elliptic zeros: a rational trace estimate

We prove the three elliptic-zero cases of Theorem 41.1. All congruences of series are coefficientwise. Put $F_M(t) = \sum_{0 \leq k < M} a_k t^k$ when $F(t) = \sum_{k \geq 0} a_k t^k$.

42.1 Modular input and notation

Let $p \geq 5$ be prime. In the three cases below the prime is automatically prime to the modular level. We use the following modular data:

$F(t)$	w	e	Γ	a_k
${}_2F_1(1/3, 2/3; 1; 27t)$	1	3	$\Gamma_0(3)$	$(3k)!/(k!)^3$
${}_3F_2(1/4, 1/2, 3/4; 1, 1; 256t)$	2	4	$\Gamma_0(2) + 2$	$(4k)!/(k!)^4$
${}_3F_2(1/3, 1/2, 2/3; 1, 1; 108t)$	2	3	$\Gamma_0(3) + 3$	$(3k)!(2k)!/(k!)^5$

In each case $t(q) \in q + q^2\mathbb{Z}[[q]]$ is a Hauptmodul, $\Theta(q) = F(t(q)) \in 1 + q\mathbb{Z}[[q]]$ is a modular form of weight w, and Θ has just one zero on the compactified modular curve, at the unique pole of t. The ratio of these orders is $1/e$. Thus $C = t\Theta^e$ is a holomorphic modular form. More explicitly, writing $h_N = (\eta_N/\eta_1)^{24/(N-1)}$ for $N = 2, 3$, the three pairs are

	t	Θ	$C = t\Theta^e$
$27t$	$h_3/(1+27h_3)$	θ_H	η_3^9/η_1^3
$256t$	$h_2/(1+64h_2)^2$	$2E_2(2\tau) - E_2(\tau)$	$(\eta_1\eta_2)^8$
$108t$	$h_3/(1+27h_3)^2$	$(3E_2(3\tau) - E_2(\tau))/2$	$(\eta_1\eta_3)^6$.

These are the modular parametrizations in [8, Appendices B and C]. The form θ_H has quadratic character χ_{-3}; in the two weight-two cases the character is trivial on $\Gamma_0(N)$ and may have a sign on the Atkin–Lehner extension. Set $\epsilon = \chi_{-3}(p)$ in weight one and $\epsilon = 1$ in weight two.

Use the $p+1$ Hecke branches

$$t_\infty = t(p\tau), \quad t_b = t((\tau+b)/p), \qquad \Theta_\infty = \Theta(p\tau), \quad \Theta_b = \Theta((\tau+b)/p) \quad (0 \le b < p),$$

and similarly $C_i = C(\beta_i\tau)$, so $t_i = C_i/\Theta_i^e$. The associated modular-form ratios are

$$x_\infty = \frac{\Theta}{\Theta_\infty}, \qquad x_b = \epsilon p^w \frac{\Theta}{\Theta_b}.$$

The Hecke action permutes the pairs (x_i, t_i), including under the Atkin–Lehner extensions. This follows directly from the weight-w slash operator, as in [8, Lemmas 3.6 and 3.7].

Define

$$D(t) = \frac{\Theta_\infty \prod_{b=0}^{p-1} \Theta_b}{\Theta^{p+1}}, \qquad a = \left\lfloor \frac{p}{e} \right\rfloor, \qquad \delta = \left\lfloor \frac{p+1}{e} \right\rfloor.$$

Here $D(0) = 1$.

Lemma 42.1 (Norm-polynomial facts)**.** *The following assertions hold.*

1. *$D \in \mathbb{Z}[t]$, $\deg D \le \delta$, and $D \equiv 1 \pmod p$.*
2. *The polynomial*

$$T(X) = \left(1 - \frac{\Theta_\infty}{\Theta}X\right) \prod_{b=0}^{p-1} \left(p^w - \epsilon\frac{\Theta_b}{\Theta}X\right) = \epsilon D \prod_i (X - x_i)$$

   has coefficients $T_j(t) \in \mathbb{Z}[t]$ satisfying $\deg T_j \le \lfloor j/e \rfloor$.

3. *Writing*

$$D^e \prod_i (X - t_i) = D^e X^{p+1} + A_1(t)X^p + \cdots + A_{p+1}(t),$$

   we have $A_j \in \mathbb{Z}[t]$, $\deg A_j \le p+1$, and $A_1(t) \equiv -t^p \pmod p$.

4. *For every $k \ge 0$ the rational trace*

$$R_k(t) = \sum_i x_i t_i^k$$

   belongs to $D^{-ek-1}\mathbb{Z}[t]$. Its numerator after multiplication by D^{ek+1} has degree at most $(p+1)k + a$.

Proof. Since $\prod_i x_i = \epsilon p^{wp}/D$, the Hecke permutation of the x_i first implies the invariance of D. It then makes the coefficients of $T = \epsilon D \prod_i (X - x_i)$ invariant under Γ. Each coefficient of X^j is holomorphic away from the zero of Θ, and its pole order relative to t is at most j/e. There are no poles at the cusps, since Θ has no cusp zeros. Consequently it is a polynomial in the Hauptmodul

t of the stated degree. Its q-expansion is integral: products of the branches with arguments $\zeta_p^b q^{1/p}$ are invariant under the cyclic rotation and under the Galois group of $\mathbb{Q}(\zeta_p)$, so their coefficients are rational algebraic integers and their exponents are integers. Division by $\Theta \in 1 + q\mathbb{Z}[[q]]$ preserves integrality. Since the inverse series $q(t)$ has integral coefficients, a polynomial in t with integral q-expansion has integral coefficients. The leading coefficient of T is ϵD because p is odd. This proves the polynomiality, degree and integrality assertions for D and T.

The elementary norm identity

$$\prod_{b=0}^{p-1} \Theta(\zeta_p^b q^{1/p}) \equiv \Theta(q) \pmod{p}$$

holds for every integral series Θ: reduce first modulo $1-\zeta_p$, use the freshman's dream, and then use $(1-\zeta_p)\mathbb{Z}[\zeta_p] \cap \mathbb{Z} = p\mathbb{Z}$. Together with $\Theta(q^p) \equiv \Theta(q)^p \pmod{p}$ this gives $D \equiv 1 \pmod{p}$.

For the third assertion use the exact identity

$$D^e \prod_i (X - t_i) = \frac{\prod_i (X\Theta_i^e - C_i)}{\Theta^{e(p+1)}}.$$

Its coefficients are invariant because D and the elementary symmetric functions in the t_i are invariant. Every numerator is holomorphic (all branches of Θ and C are holomorphic modular forms for a common finite-index subgroup). At each cusp this means holomorphy after the appropriate weight slash transformation; the weights of the numerator and denominator agree, so their automorphy factors cancel. The denominator has its only zero at the pole of t. Its pole order there, measured on $X(\Gamma)$, is at most $e(p+1)/e = p+1$. Thus the coefficients are polynomials of degree at most $p+1$. They have integral q-expansions because the elementary symmetric functions in the t_i do, and D is integral; hence they lie in $\mathbb{Z}[t]$. Finally,

$$\sum_i t_i = t(q^p) + p\, U_p t(q) \equiv t(q)^p \pmod{p},$$

so $A_1 = -D^e \sum_i t_i \equiv -t^p \pmod{p}$.

For the last assertion multiply each summand of R_k by D^{ek+1}. Up to the constant factor 1 or ϵp^w, the resulting summand is

$$\frac{C_i^k \prod_{j \neq i} \Theta_j^{ek+1}}{\Theta^{(p+1)(ek+1)-1}}.$$

The same holomorphy and invariance argument bounds the degree of the sum by

$$\left\lfloor \frac{(p+1)(ek+1)-1}{e} \right\rfloor = (p+1)k + a.$$

Its coefficients are integral, since R_k has the integral q-expansion displayed in (42.5) below and D is integral. □

Lemma 42.2 (Elliptic trace filtration)**.** *If $R_k(t) = \sum_{n \geq 0} r_{k,n} t^n$, then*

$$v_p(r_{k,n}) \geq \left\lceil \frac{n - pk - a}{\delta} \right\rceil.$$

In particular, modulo p^u the trace R_k is a polynomial of degree at most $pk + a + \delta(u-1)$ for every integer $u \geq 1$.

Proof. Equip $\mathbb{Q}_p(t)$ with a Gauss norm for which $\|t\| = R$, where $1 \leq R < p^{1/\delta}$. It suffices to take $\log_p R$ rational. Complete this field in its Gauss norm and extend the norm to a common algebraic splitting field containing all paired branches (x_i, t_i). Because $D = 1 + pB$, $\deg B \leq \delta$, and $D(0) = 1$, one has $\|D\| = \|D^{-1}\| = 1$. In the monic polynomial $T/(\epsilon D)$ for the x_i, the coefficient

of X^{p+1-j} has norm at most $R^a \le R^{aj}$, since its numerator has degree $\le \lfloor (p+1-j)/e \rfloor \le a$. The nonarchimedean root bound therefore gives $|x_i| \le R^a$ for every i in any valued splitting field.

The monic polynomial for the t_i has coefficients A_j/D^e. For $j \ge 2$, their norms are at most $R^{p+1} \le R^{pj}$. For $j = 1$ the possible term of degree $p+1$ in A_1 is divisible by p, while all other terms have degree at most p. Since $R < p^{1/\delta} \le p$, this gives $\|A_1/D^e\| \le R^p$ as well. Thus the root bound gives $|t_i| \le R^p$. Consequently

$$\|R_k\| = \left| \sum_i x_i t_i^k \right| \le R^{pk+a}.$$

By Lemma 42.1, R_k has only a power of D as denominator, so its Taylor series at zero converges in this Gauss norm. Hence $|r_{k,n}|_p R^n \le R^{pk+a}$. Letting $R \uparrow p^{1/\delta}$ and using the integrality of $v_p(r_{k,n})$ proves the bound. □

42.2 Carries in the factorial coefficients

Let $K = mp^{s-1}$, $0 \le k < K$, $j = K - k \ge 1$. For $e = 3$ or 4 put $h = \lfloor \log_p(ej-1) \rfloor$. Legendre's formula gives

$$v_p\left(\frac{(ek)!}{(k!)^e}\right) \ge (e-1)\max(0, s-1-h). \tag{42.1}$$

Indeed, for $h < i \le s-1$ we have $p^i \mid K$ and $ej \le p^i$; the residue of k modulo p^i is $p^i - j$, and the corresponding summand in Legendre's formula is exactly $e-1$. All other summands are nonnegative.

For $e = 3, 4$ let $r = e - 1$. The elementary inequalities

$$h \le j-1, \qquad a \le \delta, \qquad r\delta \le p-1$$

hold for every prime $p \ge 5$. The first follows from $p^j \ge pj > ej - 1$; the last follows immediately by considering p modulo e. Using (42.1), the degree bound from Lemma 42.2 for $a_k R_k$ modulo p^{rs} is at most

$$pk + a + \delta\{r(h+1) - 1\}.$$

When a_k is already zero modulo p^{rs} the summand is discarded. The excess of this bound over pK is strictly negative, because

$$a - pj + \delta\{r(h+1)-1\} = r\delta(h+1) + (a-\delta) - pj \le (p-1)j - pj < 0.$$

It follows that

$$\sum_{k<K} a_k R_k(t) \pmod{p^{rs}} \quad \text{is a polynomial of degree strictly less than } pK \tag{42.2}$$

for the $27t$ case with $r = 2$ and the $256t$ case with $r = 3$.

For the $108t$ case put $h = \lfloor \log_p(3j-1) \rfloor$. Each index $h < i \le s-1$ contributes $2+1 = 3$ to the valuation of $a_k = (3k)!(2k)!/(k!)^5$. Thus

$$v_p(a_k) \ge 3\max(0, s-1-h).$$

The excess degree modulo p^{3s} is consequently bounded by

$$E = a - pj + \delta(3h+2). \tag{42.3}$$

If $p \equiv 1 \pmod 3$, then $a = \delta = (p-1)/3$ and $E = (p-1)(h+1) - pj < 0$. If $p \equiv 2 \pmod 3$, write $p = 3a+2$, so $\delta = a+1$ and

$$E = (p+1)h - p(j-1).$$

For $h = 0$ this is nonpositive, with equality only when $j = 1$. For $h = 1$, the inequality $j \geq \lceil (p+1)/3 \rceil$ makes it strictly negative for $p \geq 11$. When $p = 5$, it is $11 - 5j$; the only exception to strict negativity is $j = 2$. For $h \geq 2$, the inequality $j \geq (p^h + 1)/3$ and

$$p(p^h - 2) > 3(p+1)h \qquad (p \geq 5,\ h \geq 2)$$

give strict negativity. The displayed inequality holds at $p = 5, h = 2$ and is preserved as either p or h increases.

It remains to repair $p = 5, j = 2$. If $s = 1$, use $v_p(a_k) \geq 0$ directly: the excess degree is at most $1 - 10 + 2(3-1) = -5$. If $s \geq 2$, then $k = K - 2 \equiv 3 \pmod 5$; the $i = 1$ summand in Legendre's formula is $\lfloor 9/5 \rfloor + \lfloor 6/5 \rfloor = 2$, and all indices $2 \leq i \leq s-1$ contribute 3. Therefore $v_5(a_k) \geq 3(s-1) - 1$, and the excess degree is at most $1 - 10 + 2 \cdot 3 = -3$. We have proved that

$$\sum_{k<K} a_k R_k(t) \pmod{p^{3s}} \quad \text{is a polynomial of degree at most } pK. \tag{42.4}$$

42.3 The induction and the boundary coefficient

The exact trace identity for a polynomial P is

$$\sum_i x_i P(t_i) = Q_P + \epsilon p^{w+1} U_p G_P, \qquad Q_P = \frac{\Theta(q)P(t(q^p))}{\Theta(q^p)}, \quad G_P(q) = \frac{\Theta(q^p)P(t(q))}{\Theta(q)}. \tag{42.5}$$

It follows by summing the p branches $q \mapsto \zeta_p^b q^{1/p}$.

Theorem 42.3 (The three elliptic-zero families)**.** *Let $\sigma(t(q)) = t(q^p)$ be the excellent Frobenius lift in one of the three displayed families. For all primes $p \geq 5$ and all integers $m, s \geq 1$,*

$$\frac{F(t)}{F(\sigma(t))} \equiv \frac{F_{mp^s}(t)}{F_{mp^{s-1}}(\sigma(t))} \pmod{p^{\rho s}\mathbb{Z}_p[[t]]},$$

where $\rho = 2$ for $27t$ and $\rho = 3$ for $256t$ and $108t$.

Proof. Fix m, set $K = mp^{s-1}$, and take $P = F_K$ in (42.5). For $s = 1$, the second term on its right is zero modulo p^ρ, since $w + 1 = \rho$. For $s \geq 2$, the induction hypothesis is equivalent to

$$G_{F_K}(q) \equiv F_{K/p}(t(q^p)) \pmod{p^{\rho(s-1)}}.$$

Hence

$$\epsilon p^{w+1} U_p G_{F_K} \equiv \epsilon p^\rho F_{K/p}(t(q)) \pmod{p^{\rho s}},$$

which is a polynomial in t of degree less than pK. Equations (42.2) and (42.4) therefore show that Q_{F_K} is congruent modulo $p^{\rho s}$ to a polynomial of degree $< pK$ in the first two cases and of degree $\leq pK$ in the $108t$ case.

Since $\sigma(t) = t^p + O(t^{p+1})$,

$$Q_{F_K}(t) = F(t)\frac{F_K(\sigma(t))}{F(\sigma(t))} = F(t) - a_K t^{pK} + O(t^{pK+1}).$$

Thus $Q_{F_K} \equiv F_{pK}$ modulo $p^{\rho s}$ immediately in the $27t$ and $256t$ cases.

In the $108t$ case the only possible remaining coefficient is the one at degree pK, equal to $a_{pK} - a_K$. The factorization

$$a_K = \binom{3K}{K}\binom{2K}{K}^2$$

and Jacobsthal's binomial congruence give

$$a_{mp^s} \equiv a_{mp^{s-1}} \pmod{p^{3s}} \qquad (p \geq 5).$$

For example, apply [21, Lemma 2.1] with equal exponents $r = s$ to each binomial factor. Thus the final coefficient also vanishes. This completes the induction. Finally divide by the unit $F_K(\sigma(t))$ to obtain the asserted quotient congruence. □

Scope. This proves the normalized $27t$ and $256t$ simplicial cases and the requested $108t$ family. Passing from the original simplicial period $F(t^{n+1})$ to the normalized variable gives Conjecture 7.5 for simplicial dimensions $n = 2, 3$ at all primes allowed by the original symmetry/lattice hypothesis (here $p \geq 5$). This particular rational-trace proof concerns the three displayed families. The other standard cases are proved in Part I, and full Zagier D is proved in Parts V and VII.

43 Zagier D at primes congruent to ± 1 modulo five

Let

$$A_n = \sum_{k=0}^{n} \binom{n}{k}^2 \binom{n+k}{k}, \qquad F(t) = \sum_{n\geq 0} A_n t^n, \qquad F_M(t) = \sum_{0\leq n<M} A_n t^n.$$

The modular parametrization in [8, Appendix B, §8.8] is

$$t(q) = q \prod_{n\geq 1} (1-q^n)^{5(\frac{n}{5})}, \qquad F(t(q)) = \Theta(q), \qquad t\Theta^2 = \frac{\eta(5\tau)^5}{\eta(\tau)}. \tag{43.1}$$

Here $q = e^{2\pi i\tau}$, $t \in q + q^2\mathbb{Z}[[q]]$, and $\Theta \in 1 + q\mathbb{Z}[[q]]$. Define the excellent lift by $\sigma_p(t(q)) = t(q^p)$.

Theorem 43.1 (Zagier D for two residue classes)**.** *For every prime $p \equiv 1$ or $-1 \pmod 5$, and all integers $m, s \geq 1$, one has*

$$\frac{F(t)}{F(\sigma_p(t))} \equiv \frac{F_{mp^s}(t)}{F_{mp^{s-1}}(\sigma_p(t))} \pmod{p^{2s}\mathbb{Z}_p[[t]]}.$$

This theorem covers all truncation lengths and all levels s in the indicated prime classes. The final subsection also proves the first level for $p \equiv \pm 2 \pmod 5$ by an independent Laurent-support argument. Every higher level in these classes follows from Theorem 48.1.

43.1 The zero cusp and its divisor

Put

$$H = \pm\Gamma_1(5) = \left\{ \begin{pmatrix} a & b \\ c & d \end{pmatrix} \in \mathrm{SL}_2(\mathbb{Z}) : c \equiv 0, \quad a \equiv d \equiv \pm 1 \pmod 5 \right\}.$$

For $\gamma \in H$, let $\epsilon(\gamma) \in \{\pm 1\}$ be the common residue of its diagonal entries modulo 5. Then

$$t(\gamma\tau) = t(\tau), \qquad \Theta|_1\gamma = \epsilon(\gamma)\Theta.$$

Throughout this section the slash operator is determinant normalized:

$$(f|_w\beta)(\tau) = \det(\beta)^{w/2}(c\tau + d)^{-w} f(\beta\tau).$$

Let

$$\delta = \begin{pmatrix} -2 & -1 \\ 5 & 2 \end{pmatrix}, \qquad c_* = \delta\infty = -2/5.$$

The four cusps of H are represented by $\infty, c_*, 0, \delta 0 = -1/2$, with widths $1, 1, 5, 5$, respectively.

Lemma 43.2 (Divisor data)**.** *The form Θ has a simple zero at c_* and no other zeros, including in the upper half plane. Moreover,*

$$\mathrm{div}(t) = [\infty] - [c_*].$$

Consequently t is a Hauptmodul for H.

Proof. Let χ be the quartic character modulo 5 with $\chi(2) = i$. The standard holomorphic Eisenstein series of weight one

$$E_\chi(q) = \frac{3+i}{10} + \sum_{n\geq 1}\left(\sum_{d|n}\chi(d)\right)q^n, \qquad E_{\bar\chi}(q) = \frac{3-i}{10} + \sum_{n\geq 1}\left(\sum_{d|n}\bar\chi(d)\right)q^n$$

have characters $\chi, \bar\chi$ on $\Gamma_0(5)$. Their constant terms are $L(0,\chi)/2 = (3+i)/10$ and its conjugate. The form in (43.1) is

$$\Theta = \frac{3-i}{2}E_\chi + \frac{3+i}{2}E_{\bar\chi}.$$

Since the lower-right entry of δ is 2, its expansion at c_* is

$$\Theta|_1\delta = i\frac{3-i}{2}E_\chi - i\frac{3+i}{2}E_{\bar\chi} = q + O(q^2).$$

Indeed, the constant term is $i/2 - i/2 = 0$, and the coefficient of q is 1.

The effective index of H is 12: modulo 5, the group $\mathrm{SL}_2(\mathbb{F}_5)$ has order 120 and the image of H has order 10. There are no elliptic points, since the trace of an element of H is congruent to 2 or -2 modulo 5, whereas a nontrivial elliptic element would have trace $0, 1$, or -1. The weight-one valence formula therefore gives total zero order $12/12 = 1$. The zero just found at c_* accounts for this entire divisor.

Write $\phi = \eta(5\tau)^5/\eta(\tau)$. This is a weight-two form on $\Gamma_0(5)$ of character $(5/\cdot)$. It has order 1 at ∞, hence also at c_*, since $\phi|_2\delta = -\phi$. At 0 its order is zero: with $S = \left(\begin{smallmatrix}0 & -1\\ 1 & 0\end{smallmatrix}\right)$,

$$\phi|_2 S = -5^{-5/2}\frac{\eta(\tau/5)^5}{\eta(\tau)},$$

whose leading term is a nonzero constant. Its order is also zero at $\delta 0$, by modularity on $\Gamma_0(5)$. It has no zeros in the upper half plane. The identity $t = \phi/\Theta^2$ now gives the asserted divisor. A nonconstant meromorphic function with exactly one simple pole has degree one as a map to $\mathbb{P}^1$, so it is a Hauptmodul. □

43.2 The Hecke correspondence with its sign

Fix $p \equiv \varepsilon_p \pmod 5$, where $\varepsilon_p \in \{\pm 1\}$. Let $\mathcal{M}_p$ consist of the integral matrices $\beta = \left(\begin{smallmatrix}a & b\\ c & d\end{smallmatrix}\right)$ satisfying

$$\begin{aligned} ad - bc &= p, \qquad c \equiv 0 \pmod 5,\\ (a,d) &\equiv (p,1) \text{ or } (-p,-1) \pmod 5. \end{aligned}$$

For $\beta \in \mathcal{M}_p$, let $\epsilon(\beta) \in \{\pm 1\}$ be its lower-right entry modulo 5. Left and right multiplication by H preserve $\mathcal{M}_p$, and

$$\epsilon(\gamma\beta) = \epsilon(\gamma)\epsilon(\beta) = \epsilon(\beta\gamma) \qquad (\gamma \in H).$$

Lemma 43.3 (Cosets and the distinguished cusp)**.** *The left H-cosets in $\mathcal{M}_p$ have the representatives*

$$\beta_\infty = \begin{pmatrix} p & 0\\ 0 & 1\end{pmatrix}, \qquad \beta_b = \begin{pmatrix} 1 & b\\ 0 & p\end{pmatrix} \quad (0 \leq b < p).$$

Their signs are $\epsilon(\beta_\infty) = 1$ and $\epsilon(\beta_b) = \varepsilon_p$. Also $\mathrm{adj}(\beta) \in \mathcal{M}_p$ *for every $\beta \in \mathcal{M}_p$, and every such β preserves the cusp class c_*.*

Proof. Apply integral Hermite normal form to write $\beta = \gamma\beta_j$, with $\gamma \in \mathrm{SL}_2(\mathbb{Z})$ and β_j one of the displayed matrices. If $j = \infty$, the diagonal entries of $\gamma = \beta\beta_\infty^{-1}$ reduce to the same ± 1 modulo 5, and its lower-left entry is divisible by 5. If $j = b$, then

$$\gamma = \begin{pmatrix} a & (b' - ab)/p\\ c & (d - cb)/p\end{pmatrix} \quad \text{when } \beta = \begin{pmatrix} a & b'\\ c & d\end{pmatrix}.$$

Its diagonal entries reduce to the same $\pm p = \pm 1$ modulo 5, since $p^2 \equiv 1 \pmod 5$, and its lower-left entry is again divisible by 5. Thus $\gamma \in H$ in both cases. Distinct representatives are already distinct modulo $\mathrm{SL}_2(\mathbb{Z})$. The sign assertions follow directly from their lower-right entries. The congruence $p^2 \equiv 1 \pmod 5$ likewise shows that taking adjugates preserves $\mathcal{M}_p$.

A primitive cusp column whose lower component is divisible by 5 has H-class determined by its upper component modulo 5 up to sign. The class c_* has upper component ± 2. Applying a matrix in $\mathcal{M}_p$ multiplies this residue by $\pm p = \pm 1$. The resulting column has common divisor either 1 or p, because its image under the adjugate is p times the original primitive column. Reduction to a primitive column therefore again changes its upper residue only by a sign. This proves cusp stability; all representatives of that cusp have width 1. □

43.3 Polynomiality and the exact trace

For an integer polynomial P of degree d, put

$$R_\beta(\tau) = p^{1/2}\epsilon(\beta)\frac{\Theta(\tau)}{(\Theta|_1\beta)(\tau)}P(t(\beta\tau)), \qquad \mathcal{T}_p(P) = \sum_{H\backslash\mathcal{M}_p} R_\beta.$$

Lemma 43.4 (Zagier D trace polynomial)**.** *There is a polynomial $\Pi_P \in \mathbb{Z}[t]$ of degree at most $pd + p - 1$ such that*

$$\Pi_P(t(q)) = \frac{\Theta(q)P(t(q^p))}{\Theta(q^p)} + \varepsilon_p p^2 U_p\left(\frac{\Theta(q^p)P(t(q))}{\Theta(q)}\right). \tag{43.2}$$

Here $U_p(\sum_n b_n q^n) = \sum_n b_{pn} q^n$.

Proof. The character signs show that $R_{\gamma\beta} = R_\beta$ and $R_\beta(\gamma\tau) = R_{\beta\gamma}(\tau)$ for $\gamma \in H$. Thus $\mathcal{T}_p(P)$ is H-invariant.

No summand has a pole in the upper half plane, by Lemma 43.2. If a cusp c' lies outside the class c_*, then so does $\beta c'$: otherwise applying the adjugate and using Lemma 43.3 would put c' in that class. At these cusps the forms in the numerator and denominator have nonzero leading constants, and t stays finite. There are therefore no poles there.

At c_*, write $\beta\delta = \delta_\beta\beta'$ in integral Hermite normal form, where $\beta' = \left(\begin{smallmatrix} a' & b' \\ 0 & d' \end{smallmatrix}\right)$ and $a'd' = p$. As both c_* and βc_* have width 1 and are H-equivalent, the orders of $\Theta|_1\beta$ and $t \circ \beta$ are respectively a'/d' and $-a'/d'$. Consequently

$$\operatorname{ord}_{c_*} R_\beta = 1 - \frac{a'}{d'}(d+1) \geq -pd - p + 1.$$

It follows that the trace has its only possible pole at c_*, of order at most $pd + p - 1$. Since t is a Hauptmodul with a simple pole there, $\mathcal{T}_p(P) = \Pi_P(t)$ with this degree bound.

The infinity branch equals the first term on the right of (43.2). Each remaining branch is

$$\varepsilon_p p\,\frac{\Theta(q)P(t(\zeta_p^b q^{1/p}))}{\Theta(\zeta_p^b q^{1/p})}.$$

Summing over b gives the second term in (43.2). Both terms have integral q-expansions, since $\Theta(q)$ is an integral unit. As $t = q + O(q^2)$ integrally, coefficient comparison then gives $\Pi_P \in \mathbb{Z}[t]$. □

Proof of Theorem 43.1. Fix $m \geq 1$, put $K_j = mp^j$ for $j \geq 0$, and set $F_{K_{-1}} = 0$. Take $P = F_{K_{s-1}}$ in Lemma 43.4. The degree of its trace polynomial is at most

$$p(K_{s-1} - 1) + p - 1 = K_s - 1.$$

For $s \geq 2$, the previous induction step, after cross multiplication and division by the unit $\Theta(q)$, reads

$$\frac{\Theta(q^p)F_{K_{s-1}}(t(q))}{\Theta(q)} \equiv F_{K_{s-2}}(t(q^p)) \pmod{p^{2(s-1)}}.$$

Applying U_p and multiplying by p^2 gives

$$p^2 U_p\left(\frac{\Theta(q^p)F_{K_{s-1}}(t(q))}{\Theta(q)}\right) \equiv p^2 F_{K_{s-2}}(t(q)) \pmod{p^{2s}}.$$

For $s = 1$, this last assertion is automatic under the convention $F_{K_{-1}} = 0$.

Thus the exact trace identity makes

$$Q(t) = F(t)\frac{F_{K_{s-1}}(\sigma_p(t))}{F(\sigma_p(t))}$$

congruent modulo p^{2s} to an integer polynomial of degree less than K_s. Since $\sigma_p(t) = t^p + O(t^{p+1})$,

$$Q(t) = F(t) + O(t^{K_s}).$$

Coefficient comparison forces $Q(t) \equiv F_{K_s}(t) \pmod{p^{2s}}$. This completes the induction for all m, s. Dividing by the unit $F_{K_{s-1}}(\sigma_p(t))$ gives the claimed quotient congruence. □

43.4 Diamond classes and the Laurent correction

The same divisor calculation also determines the diamond action:

$$\Theta|_1\delta = t\Theta, \qquad t(\delta\tau) = -1/t(\tau). \tag{43.3}$$

For the first identity, the quotient $(\Theta|_1\delta)/\Theta$ has divisor $[\infty] - [c_*]$, so it is a constant multiple of t; its first q-coefficient is 1. The second identity follows from $\phi|_2\delta = -\phi$ and $t = \phi/\Theta^2$.

When $p \equiv \pm 2 \pmod 5$, the diamond action in the Hecke correspondence is therefore non-scalar. In weight -1, it sends

$$\frac{P(t)}{\Theta} \longmapsto \left(\frac{P(t)}{\Theta}\right)\Big|_{-1}\delta = \frac{P(-1/t)}{t\Theta}.$$

This has a pole at the expansion cusp $t = 0$. The corresponding trace can have a negative Laurent part, so Lemma 43.4 does not extend by the same argument. Controlling that negative Laurent part and its regular remainder requires the paired descent proved in Section 48; the present one-cusp argument alone does not provide that control.

43.5 The first level for every prime different from five

The Laurent terms just described prevent the preceding higher-level induction, but do not prevent the initial step.

Theorem 43.5 (Zagier D at the first level)**.** *For every prime $p \neq 5$ and every integer $m \geq 1$,*

$$\frac{F(t)}{F(\sigma_p(t))} \equiv \frac{F_{mp}(t)}{F_m(\sigma_p(t))} \pmod{p^2\mathbb{Z}_p[[t]]}.$$

Proof. The cases $p \equiv \pm 1 \pmod 5$ already follow from Theorem 43.1. Assume $p \equiv 2$ or $3 \pmod 5$. Retain the matrix-defined set $\mathcal{M}_p$ and the sign $\epsilon(\beta) = d \bmod 5$; these definitions still make sense, since its lower-right entries are ± 1 modulo 5. Put

$$\eta_p = \begin{cases} \delta^{-1}, & p \equiv 2 \pmod 5, \\ \delta, & p \equiv 3 \pmod 5, \end{cases} \qquad c_p = \begin{cases} -1, & p \equiv 2 \pmod 5, \\ 1, & p \equiv 3 \pmod 5. \end{cases}$$

The coset argument and the trace below require only $p \nmid 5$; they do not require p to be odd. In particular, $p = 2, 3$ are included. Integral Hermite normal form now gives the left H-cosets

$$\beta_\infty, \qquad \eta_p\beta_b \quad (0 \le b < p).$$

Indeed, in a finite Hermite branch the intervening integral matrix has diagonal residues $\pm(p, p^{-1})$ modulo 5; after multiplication by η_p^{-1} it lies in H. The signs of all displayed representatives are 1. By (43.3),

$$\left(\frac{P(t)}{\Theta}\right)\Big|_{-1}\eta_p = c_p\frac{P(-1/t)}{t\Theta}.$$

The same weight-zero trace construction therefore gives the exact identity

$$\mathcal{T}_p(P) = Q_P + c_p p^2\Theta(q)\,U_p\left(\frac{P(-1/t(q))}{t(q)\Theta(q)}\right), \qquad Q_P = \frac{\Theta(q)P(t(q^p))}{\Theta(q^p)}. \tag{43.4}$$

Here U_p acts on Laurent series as well as power series.

We claim that, for $d = \deg P$,

$$\mathcal{T}_p(P) \in \mathbb{Z}[t, t^{-1}], \qquad \operatorname{supp}_t\mathcal{T}_p(P) \subseteq \left[-\left\lfloor\frac{d+1}{p}\right\rfloor, p(d+1)-1\right]. \tag{43.5}$$

The absence of upper-half-plane poles follows as before. A cusp with denominator prime to 5 cannot map to either ∞ or c_* under $\mathcal{M}_p$, so the trace has no poles at the two other cusps.

To analyze the remaining cusps, apply a determinant-p matrix to a primitive cusp column. Its new common divisor is 1 or p. Among the $p+1$ branches exactly one has common divisor p, and hence ramification p; the others have ramification $1/p$. This follows from Hermite normal form after multiplication by a matrix representing the cusp. At c_*, whose upper residue is ± 2 modulo 5, a branch with common divisor 1 changes that residue to $\pm 2p = \pm 1$, and therefore maps to ∞. The unique branch with common divisor p maps back to c_*. Its pole order in the trace is at most $p(d+1)-1$, since Θ has order 1 at c_*. All other branches are holomorphic there.

At ∞, the unique ramification-p branch maps back to ∞; the other p branches map to c_*. Each of these can have pole order at most $(d+1)/p$. Since the trace is H-invariant and the cusp width is 1, its pole order is an integer, at most $\lfloor(d+1)/p\rfloor$. Thus the trace is a Laurent polynomial with the claimed support. Its Laurent q-expansion is integral by (43.4), because t/q and Θ are integral units. Substitution of the integral inverse series $q(t)$ then proves integrality of its Laurent coefficients, establishing (43.5). Explicitly, $q(t) = tV(t)$ with $V(t) \in 1 + t\mathbb{Z}[[t]]$; its inverse powers remain integral, so substitution sends $\mathbb{Z}((q))$ into $\mathbb{Z}((t))$.

Now take $P = F_m$, so $d = m-1$. The correction in (43.4) is divisible by p^2 in the integral Laurent-series ring. Hence Q_P is congruent modulo p^2 to a Laurent polynomial whose positive degree is less than mp. Since Q_P itself has no negative powers, all negative coefficients of that Laurent polynomial vanish modulo p^2. It is therefore congruent to an ordinary polynomial of degree less than mp. Finally,

$$Q_P(t) = F(t)\frac{F_m(\sigma_p(t))}{F(\sigma_p(t))} = F(t) + O(t^{mp}).$$

Coefficient comparison gives $Q_P \equiv F_{mp} \pmod{p^2}$. Dividing by $F_m(\sigma_p(t))$ completes the proof. □

This Laurent-support argument supplies the first level in the diamond classes. The paired induction of Section 48 supplies every level $s \ge 2$ as well.

44 Exact coefficient obstructions to larger exponents

The results of this section give upper bounds for the modulus that can hold in the full excellent-lift congruence. They complement the full lower bound modulo p^{2s} proved in Theorem 2.1. Each obstruction is an exact calculation of one coefficient, valid for infinitely many levels by proof rather than by extrapolation from computation.

44.1 Normalization and the first two coefficients

Write

$$F(u)=\sum_{k\ge 0}a_k u^k\in 1+u\mathbb{Z}_p[[u]],\qquad q(u)=u\exp(g(u)),\qquad g(u)=\alpha u+O(u^2).$$

Let T be the compositional inverse of q, and put $\sigma(u)=T(q(u)^p)$. For $K=p^s$, define the cleared defect

$$E_s(u)=F(u)F_{K/p}(\sigma(u))-F(\sigma(u))F_K(u),\qquad F_M(u)=\sum_{0\le k<M}a_k u^k.$$

Its divisibility by p^b is equivalent to the quotient congruence modulo p^b, because all denominator series have constant term one.

Lemma 44.1 (The first two possible defect coefficients)**.** *For $p\ge 2$ and $s\ge 1$,*

$$[u^K]E_s=a_K-a_{K/p},\tag{44.1}$$

$$[u^{K+1}]E_s=a_{K+1}-(a_1+K\alpha)a_{K/p}.\tag{44.2}$$

Proof. Since $T(z)=z+O(z^2)$, $\sigma(u)=u^p(1+p\alpha u+O(u^2))$. Rewrite the defect as

$$E_s=(F-F_K)F_{K/p}(\sigma)-F_K(F-F_{K/p})(\sigma).$$

The first summand has coefficients a_K,a_{K+1} at degrees $K,K+1$, since $F_{K/p}(\sigma)=1+O(u^p)$. Up to degree $K+1$, the second summand is

$$a_{K/p}F_K(u)\sigma(u)^{K/p}=a_{K/p}u^K(1+(a_1+K\alpha)u+O(u^2)).$$

This proves both identities. □

We record the normalization explicitly, using the expressions for $F_0(t)$ and $G_0(t)$ in [7, §7, Proposition 9.1]. Here t is the parameter of the original Laurent-polynomial family; u is the normalized parameter used in this section. Let $H_k=\sum_{j=1}^{k}1/j$ and $H_0=0$.

For the hypercubic family in dimension d,

$$u=t^2,\qquad a_k=\binom{2k}{k}^d,\qquad g(u)=\frac{2d\sum_{k\ge1}a_k(H_{2k}-H_k)u^k}{F(u)}.$$

Thus $a_1=2^d$ and $\alpha=d2^d$. For the hyperoctahedral family in dimension d,

$$u=t^2,\qquad a_k=\sum_{k_1+\cdots+k_d=k}\frac{(2k)!}{(k_1!\cdots k_d!)^2},$$

and

$$g(u)=\frac{2}{F(u)}\sum_{k\ge1}u^k\sum_{k_1+\cdots+k_d=k}\frac{(2k)!}{(k_1!\cdots k_d!)^2}(H_{2k}-H_{k_1}).$$

At $k=1$ the inner sum is $1+3(d-1)$; hence $a_1=2d$ and $\alpha=6d-4$. For the simplicial family in dimension d, put $r=d+1$. Then

$$u=t^r, \qquad a_k=\frac{(rk)!}{(k!)^r}, \qquad g(u)=\frac{r\sum_{k\ge1}a_k(H_{rk}-H_k)u^k}{F(u)}.$$

In particular, $a_1=r!$ and $\alpha=r!r(H_r-1)$.

In each case, with $h=2$ or $h=r$, the original coordinate $q_0(t)=t\exp(G_0(t)/F_0(t))$ satisfies $q_0(t)^h=q(t^h)$. The vertex coefficient is one, so $q_0\mapsto q_0^p$ implies $(t^{\sigma_0})^h=\sigma(t^h)$. Furthermore, $F_{0,hM}(t)=F_M(t^h)$. The normalized defects above therefore give exactly the original Beukers–Vlasenko defects with $m=2$ or $m=d+1$ after the injective substitution $u=t^h$. The admissible primes do not divide h. No identification with an unverified Picard–Fuchs logarithmic solution is needed: these formulas use the coordinate in [7, Definition 7.14] itself.

44.2 A binomial estimate

We use Jacobsthal's congruence in the form stated in [21, Lemma 2.1, equation (7)]: for $p\ge5$ and $r,t\ge1$,

$$\binom{p^rn}{p^tk}\Big/\binom{p^{r-1}n}{p^{t-1}k}\equiv1 \pmod{p^{r+t+\min(r,t)}}. \tag{44.3}$$

Endpoint ratios with equal top and bottom are identically one. In particular, for $K=p^s$,

$$\binom{2K}{K}\equiv\binom{2K/p}{K/p} \pmod{p^{3s}}. \tag{44.4}$$

44.3 Hyperoctahedral dimension three

Let

$$b_N=\sum_{k=0}^{N}\binom{N}{k}^2\binom{2k}{k}, \qquad a_N=\binom{2N}{N}b_N.$$

These are the normalized three-dimensional hyperoctahedral coefficients. For $p\ge5$, define

$$S_p=\sum_{k=1}^{(p-1)/2}\frac{\binom{2k}{k}}{k^2}\in\mathbb{F}_p, \qquad \chi_p=\left(\frac{-3}{p}\right).$$

Theorem 44.2 (All-level obstruction in dimension three)**.** *For every $p\ge5$ and $s\ge1$,*

$$a_{p^s}-a_{p^{s-1}}\equiv2\chi_p^{s-1}S_p\,p^{2s} \pmod{p^{2s+1}}. \tag{44.5}$$

In particular,

$$[u^{5^s}]E_s\equiv2(-1)^{s-1}5^{2s} \pmod{5^{2s+1}} \qquad (s\ge1),$$

so the displayed coefficient has valuation exactly $2s$ at every level.

Proof. Put $N=p^s$, $M=p^{s-1}$, and $T_N(k)=\binom{N}{k}^2\binom{2k}{k}$. Compare the terms with $k=pj$ to $T_M(j)$. For $0<j<M$, write $r=v_p(j)\le s-2$. Lucas's theorem, or the identity $\binom{M}{j}=(M/j)\binom{M-1}{j-1}$, gives $v_p\binom{M}{j}=s-1-r$. By (44.3),

$$\binom{N}{pj}\Big/\binom{M}{j}\equiv1 \pmod{p^{s+2r+2}}, \qquad \binom{2pj}{pj}\Big/\binom{2j}{j}\equiv1 \pmod{p^{3r+3}}.$$

Multiplication by the squared binomial coefficient adds valuation at least $2s-2r-2$. The resulting error bounds are $3s$ and $2s+r+1$, both at least $2s+1$. Thus $T_N(pj) \equiv T_M(j) \pmod{p^{2s+1}}$. The endpoint $j=0$ is exact, and $j=M$ follows from (44.4). Consequently,

$$b_N - b_M \equiv \sum_{\substack{0<k<N \\ p\nmid k}} T_N(k) \pmod{p^{2s+1}}.$$

For $p \nmid k$, Lucas gives $\binom{N-1}{k-1} \equiv (-1)^{k-1} \pmod{p}$; hence

$$\frac{b_N - b_M}{p^{2s}} \equiv \sum_{\substack{0<k<p^s \\ p\nmid k}} \frac{\binom{2k}{k}}{k^2} \pmod{p}.$$

If a base-p digit of k exceeds $(p-1)/2$, doubling creates a carry and $\binom{2k}{k} \equiv 0 \pmod{p}$. Otherwise Lucas factors the central binomial coefficient over all digits. The denominator depends only on the nonzero units digit, so the last sum equals

$$S_p \left(\sum_{j=0}^{(p-1)/2} \binom{2j}{j} \right)^{s-1}.$$

Writing $h=(p-1)/2$, we have $\binom{2j}{j} \equiv (-4)^j \binom{h}{j} \pmod{p}$. The parenthesized sum is therefore $(-3)^h = \chi_p$ in $\mathbb{F}_p$. The central-binomial factors at N and M differ by a multiple of p^{3s}, and each is 2 modulo p. This proves (44.5). Finally $S_5 = 2 + 6/4 = 1$ in $\mathbb{F}_5$ and $\chi_5 = -1$; apply (44.1). □

This rules out a congruence modulo 5^{2s+1}, and therefore modulo 5^{3s}, for the full defect at every level. It does not by itself show that all coefficients of the defect are divisible by 5^{2s}; Theorem 2.1 supplies that lower bound. Thus the uniform exponent is exactly $2s$ at $p=5$.

44.4 All higher-dimensional hypercubic families

Theorem 44.3 (Hypercubic obstruction)**.** *Let $d \geq 4$, let $p > d$ be prime, and let $s \geq 1$. For $K = p^s$,*

$$[u^{K+1}]E_s \equiv 2^{2d-1} d(d-3) p^{2s} \pmod{p^{2s+1}}.$$

In particular this coefficient has valuation exactly $2s$ for every such d, p, s.

Proof. Here $a_k = \binom{2k}{k}^d$, $a_1 = 2^d$, and $\alpha = d2^d$. Equation (44.4) gives $a_K \equiv a_{K/p} \pmod{p^{3s}}$. The exact adjacent ratio is

$$\frac{a_{K+1}}{a_K} = 2^d \left(\frac{1+2K}{1+K} \right)^d = 2^d \left(1 + dK + \frac{d(d-3)}{2} K^2 + O(K^3) \right),$$

with p-integral coefficients. Since $3s \geq 2s+1$, substituting into (44.2) yields

$$[u^{K+1}]E_s \equiv 2^{d-1} d(d-3) K^2 a_{K/p} \pmod{p^{2s+1}}.$$

Lucas gives $a_{K/p} \equiv 2^d \pmod{p}$. As $p > d \geq 4$, the resulting constant $2^{2d-1}d(d-3)$ is nonzero modulo p. □

For the full hypercubic symmetry group, $p > d$ is precisely the admissible-prime restriction in these dimensions. Thus the exponent $2s$ cannot be increased even by one throughout the conjectured range. The theorem gives the upper bound, and Theorem 2.1 gives the lower bound. Hence $2s$ is the exact full-series exponent throughout this hypercubic range.

44.5 All higher-dimensional simplicial families

Put $r = d + 1$, and let $H_r^{(2)} = \sum_{j=1}^{r} 1/j^2$. Define

$$A_r = r(H_r - 1), \qquad B_r = \frac{A_r^2 - r^2 H_r^{(2)} + r}{2}, \qquad C_r = (r!)^2 B_r.$$

Theorem 44.4 (Simplicial obstruction and exceptional primes)**.** *Let $r \geq 5$ and let $p > r$ be prime. For every $s \geq 1$,*

$$[u^{p^s+1}]E_s \equiv C_r p^{2s} \pmod{p^{2s+1}}. \tag{44.6}$$

The number C_r is a positive integer. Thus this coefficient has valuation exactly $2s$ whenever $p \nmid C_r$. More generally, setting $c = v_p(C_r)$, one has

$$v_p([u^{p^s+1}]E_s) = 2s + c \qquad \text{for every } s > c. \tag{44.7}$$

Consequently every admissible prime fails to support a p^{3s} strengthening at all sufficiently large levels.

Proof. The coefficients factor as

$$a_K = \frac{(rK)!}{(K!)^r} = \prod_{j=2}^{r} \binom{jK}{K}.$$

Apply (44.3) factor by factor to obtain $a_K \equiv a_{K/p} \pmod{p^{3s}}$. Lucas gives $a_{K/p} \equiv r! \pmod{p}$, a unit since $p > r$. The adjacent ratio expands as

$$R_r(K) := \frac{a_{K+1}}{a_K} = \frac{\prod_{j=1}^{r}(rK + j)}{(1 + K)^r} = r!(1 + A_r K + B_r K^2 + O(K^3)).$$

Indeed, the first two coefficients of its logarithm are A_r and $-(r^2 H_r^{(2)} - r)/2$. Since $a_1 = r!$ and $\alpha = r!A_r$, (44.2) gives the stronger identity

$$[u^{K+1}]E_s = r!B_r a_{K/p} K^2 + O(p^{3s}). \tag{44.8}$$

Reducing $a_{K/p}$ modulo p proves (44.6).

The series $R_r(X)$ lies in $\mathbb{Z}[[X]]$: its numerator is an integral polynomial and $(1 + X)^{-r}$ has integral coefficients. Thus $r!B_r = [X^2]R_r(X)$ is integral, and so is C_r. For positivity, write

$$D_r := \frac{2B_r}{r^2} = 2 \sum_{2 \leq i < j \leq r} \frac{1}{ij} - 1 + \frac{1}{r}.$$

One has $D_4 = 0$ and

$$D_{r+1} - D_r = \frac{2(H_r - 1) - 1/r}{r + 1} > 0 \qquad (r \geq 4).$$

Hence $C_r > 0$ for all $r \geq 5$. Finally, in (44.8), the displayed leading term has valuation $2s + c$, because $r!$ and $a_{K/p}$ are units. For $s > c$ this is smaller than $3s$, proving (44.7). □

For example,

$$C_4 = 0, \qquad C_5 = 69000, \qquad C_6 = 7257600, \qquad C_7 = 727050240.$$

Thus dimension four at $p = 7$ has $[u^{7^s+1}]E_s \equiv 7^{2s} \pmod{7^{2s+1}}$ for every s. In dimension five, the exceptional prime $p = 7$ has $v_7(C_6) = 1$, so the same coefficient has valuation $2s + 1$ for every $s \geq 2$. The vanishing $C_4 = 0$ is a cancellation in this particular necessary obstruction; it is not an independent proof of the stronger three-dimensional congruence.

44.6 Hyperoctahedral dimension four

Let

$$D_k = \sum_{j=0}^{k} \binom{k}{j}^2 \binom{2j}{j} \binom{2k-2j}{k-j}$$

be the Domb numbers. Grouping the four composition indices into two pairs and applying Vandermonde shows that the four-dimensional hyperoctahedral coefficient is $a_k = \binom{2k}{k} D_k$. The established Domb congruence [18, Theorem 1.1, with $A = 2$, $B = C = 1$] and (44.4) give

$$a_{p^s} \equiv a_{p^{s-1}} \pmod{p^{3s}} \qquad (p \geq 5).$$

Theorem 44.5 (Four-dimensional hyperoctahedral obstruction)**.** *For every prime $p \geq 5$ and every $s \geq 1$,*

$$[u^{p^s+1}]E_s \equiv 32p^{2s} \pmod{p^{2s+1}}.$$

This coefficient has valuation exactly $2s$ at every admissible prime and level.

Proof. The Domb recurrence, equivalently its differential equation in [8, Appendix C, §9.7] after removing the indicated sign twist, is

$$k^3 D_k = 2(2k-1)(5k^2-5k+2)D_{k-1} - 64(k-1)^3 D_{k-2}.$$

Multiplication by the central binomial coefficient gives

$$k^4 a_k = 4(2k-1)^2(5k^2-5k+2)a_{k-1} - 256(k-1)^2(2k-1)(2k-3)a_{k-2}.$$

Set $k = K+1$ and expand modulo K^3. Since $K+1$ is a p-adic unit,

$$a_{K+1} \equiv (8 + 20K + 4K^2)a_K + 256K^2 a_{K-1} \pmod{p^{3s}}.$$

Here $a_1 = 8$ and $\alpha = 20$. Use (44.2) and the preceding coefficient supercongruence to obtain

$$[u^{K+1}]E_s \equiv K^2(4a_{K/p} + 256a_{K-1}) \pmod{p^{2s+1}}.$$

Lucas gives $a_{K/p} \equiv 8 \pmod{p}$. The factor $\binom{2K-2}{K-1}$ in a_{K-1} is zero modulo p, since doubling $K-1 = p^s - 1$ creates a carry. The stated congruence follows. □

Interpretation. Theorems 44.2, 44.3, 44.4, and 44.5 prove exact upper obstructions for the possible full-series modulus. The lower bound modulo p^{2s} is Theorem 1.1. Their combination proves sharpness in the parameter ranges explicitly stated; it does not assert exactness at every exceptional simplicial prime or every higher octahedral dimension.

45 Relation to the cited literature

Dwork crystals III supplies the exact formulation of Conjecture 7.5, the canonical coordinates, and the excellent Cartier relation [7, Section 7 and Proposition 9.1]. Its higher formal-derivative statement already gives strong congruences for appropriate Laurent coefficients, but does not by itself identify the finite truncations in (41.2). Section 42 supplies this identification for the two low-dimensional simplicial families by bounding the entire rational trace.

Beukers's modular-form paper supplies the parametrizations and the Hecke permutation of the modular-form ratios [8, Lemmas 3.6–3.7, Appendices B–C]. The additional ingredient here is a bound uniform in the monomial degree k and in the modulus p^u. The crucial estimate is

$$\deg(R_k \bmod p^u) \leq pk + \lfloor p/e \rfloor + \lfloor (p+1)/e \rfloor (u-1).$$

It permits induction through every level, even though the trace has actual complex poles and infinitely many nonzero Taylor coefficients.

Vlasenko's exposition explains the cohomological framework and the distinction between period truncations and Cartier coefficient congruences [22]. Fu [16] gives a cohomological criterion involving a Frobenius-stable rank-two factor containing the holomorphic class, and studies its relation to excellent-lift fixed points. That criterion concerns additional hypotheses at selected parameters. None of those hypotheses is imported into the formal-series theorems in this volume.

Straub's paper proves coefficient supercongruences and Gessel–Lucas relations for sporadic sequences. We use its explicit Jacobsthal lemma [21, Lemma 2.1] to remove the boundary coefficient in the $108t$ argument and to establish the sharpness formulas. Alinquant–Osburn finish the remaining original sporadic two-term coefficient congruence [2]. Such coefficient results are useful inputs, but do not alone imply (41.2). Only the optional dimension-four octahedral calculation needs the established Domb-number result [18] as an additional input.

46 Exact verification and proof dependencies

The complete verification programs for this part are embedded in Appendix D, together with the other programs of the collected work. All arithmetic in these particular programs is integer or rational arithmetic, with explicit prime-power reduction. Their declared finite checks supplement the proofs; no finite initial-segment check is used as an infinite-series argument.

The elliptic checks use the Lambert-series formula

$$\theta_H(q) = 1 + 6\sum_{n\geq 1} \chi_{-3}(n)\frac{q^n}{1-q^n}$$

for the cubic theta function. This avoids a possible truncation error from summing the lattice form $x^2 + xy + y^2$ over a square of insufficient radius. The theoretical proof does not depend on a lattice cutoff. The tests also inspect coefficients beyond the truncation degree, so they are not vacuous initial-segment checks.

For Zagier D, the diamond classes satisfy

$$t(\delta\tau) = -1/t(\tau), \qquad \Theta|_1\delta = t\Theta, \qquad \delta = \begin{pmatrix} -2 & -1 \\ 5 & 2 \end{pmatrix}.$$

The corresponding trace has a negative Laurent part at $t = 0$. Its support is finite and explicitly bounded, giving the independent first-level proof above. The one-cusp polynomial induction does not control its higher-level negative part; the simultaneous Laurent induction of Section 38 and the paired canonical-basis descent of Section 48 provide that control for all $p \neq 5$ and all m, s.

For the three-dimensional hyperoctahedral family and every higher-dimensional standard family, the full lower bound $E_{p,s,m} \in p^{2s}\mathbb{Z}_p[[t]]$ is Theorem 2.1, within its displayed prime conditions. The sharpness theorems above explain why a uniform $3s$ exponent is unavailable in their stated ranges. No higher-level or higher-coefficient case within that standard theorem is left to these calculations.

Part VII
Two-cusp descent and complementary Cartier arguments

All-prime Zagier D, the independent geometric first level, negative-power bases, and octahedral differential refinements.

47 Scope and exact statements

This part retains the complete two-cusp proof for Zagier D and independent auxiliary arguments for the standard families, using the modular data and excellent Cartier theorem cited below. The all-level ordinary-truncation theorem at every odd prime is Theorem 1.1. The earlier elliptic-zero arguments and coefficient upper bounds are not needed in either proof.

For a period $F(t) = \sum_{k\geq 0} a_k t^k$ with $a_0 = 1$, write

$$F_M(t) = \sum_{0\leq k<M} a_k t^k, \qquad \sigma_p(t(q)) = t(q^p).$$

The excellent-lift congruence at level s and multiplier m is

$$\frac{F(t)}{F(\sigma_p(t))} \equiv \frac{F_{mp^s}(t)}{F_{mp^{s-1}}(\sigma_p(t))} \pmod{p^{2s}\mathbb{Z}_p[[t]]}. \tag{47.1}$$

Equivalently, the complete cleared defect

$$E_{p,s,m}(t) = F(t)F_{mp^{s-1}}(\sigma_p(t)) - F(\sigma_p(t))F_{mp^s}(t)$$

belongs to $p^{2s}\mathbb{Z}_p[[t]]$. All congruences are coefficientwise identities of infinite formal series. A congruence for an isolated coefficient is not substituted for this assertion.

Target	Conclusion
Zagier D, $p \equiv \pm 2 \pmod 5$	Full congruence (47.1), for all $m, s \geq 1$, including $p = 2, 3$.
Zagier D, all residue classes	A single proof covers every $p \neq 5$ and all m, s.
Hyperoctahedral dimension three	The full first level and an all-level weighted differential bound are proved. The full U_p descent follows from Theorem 1.1.
All three families in dimensions $n \geq 4$	The full first level is proved with the original prescribed truncation lengths. An auxiliary negative-power basis has an all-level congruence; ordinary truncations at all levels satisfy Theorem 1.1.

47.1 The distinct conclusions of the auxiliary arguments

The Zagier D theorem below covers all s, m and every coefficient. The independent geometric theorem covers the entire formal series at the first level. The negative-power canonical-basis theorem is an all-level theorem for different functions, while the octahedral differential theorem is a weighted all-coefficient statement for $K = p^s$. These additional conclusions and their proofs are retained even though the ordinary positive-power truncation statement is already proved in Part I. The volume makes no assertion of external peer review or historical priority.

47.2 The original families and their normalization

In n variables the three Laurent polynomials are

$$g_{\text{simplex}} = x_1 + \cdots + x_n + (x_1 \cdots x_n)^{-1},$$
$$g_{\text{cube}} = \prod_{i=1}^{n} (x_i + x_i^{-1}),$$
$$g_{\text{octa}} = \sum_{i=1}^{n} (x_i + x_i^{-1}).$$

Their original periods have the form

$$\mathcal{F}(z) = \mathrm{CT}\,\frac{1}{1 - zg} = F(z^h), \qquad h = n+1 \text{ for the simplex}, \quad h = 2 \text{ otherwise}.$$

Conjecture 7.5 prescribes multiplier h in this original variable [7, Conjecture 7.5]. The normalized coordinate satisfies $q(z^h) = q_0(z)^h$, so the excellent lifts commute with this substitution and $\mathcal{F}_{hM}(z) = F_M(z^h)$. Consequently the normalized first-level statement $F/F^\sigma \equiv F_p$ is exactly the prescribed original statement with multiplier h. The original auxiliary arguments in this part use the standard full-symmetry prime ranges of [7, Theorem 7.3]: $p > n$ for cubes and octahedra, and $p > n+1$ for simplices. The global formal statement at every odd prime is Theorem 1.1.

48 The complete Zagier D congruence

Write

$$A_n = \sum_{j=0}^{n} \binom{n}{j}^2 \binom{n+j}{j}, \qquad F(t) = \sum_{n \geq 0} A_n t^n, \qquad F_K(t) = \sum_{0 \leq n < K} A_n t^n.$$

We use the normalized modular parametrization

$$t(q) = q \prod_{n \geq 1} (1 - q^n)^{5(\frac{n}{5})}, \qquad \Theta(q) = F(t(q)), \qquad \sigma_p(t(q)) = t(q^p).$$

The product and the Lambert series for Θ imply $t \in q + q^2\mathbb{Z}[[q]]$ and $\Theta \in 1 + q\mathbb{Z}[[q]]$.

Theorem 48.1 (Zagier D at every prime different from five)**.** *For every prime $p \neq 5$, and all integers $m, s \geq 1$, one has*

$$\frac{F(t)}{F(\sigma_p(t))} \equiv \frac{F_{mp^s}(t)}{F_{mp^{s-1}}(\sigma_p(t))} \pmod{p^{2s}\mathbb{Z}_p[[t]]}.$$

The primes 2 and 3 are included, as are every truncation multiplier m and every level s.

The proof works with the expansions at both distinguished cusps at once. No higher-level congruence is assumed in the construction.

48.1 The two cusps and an integral canonical basis

Let $H = \pm\Gamma_1(5)$, put

$$\delta = \begin{pmatrix} -2 & -1 \\ 5 & 2 \end{pmatrix}, \qquad c_* = \delta\infty = -2/5,$$

and use determinant-normalized slash operators

$$(f|_k\beta)(\tau) = \det(\beta)^{k/2}(c\tau + d)^{-k} f(\beta\tau).$$

For $\gamma \in H$, write $\epsilon(\gamma) \in \{1, -1\}$ for its diagonal residue modulo 5. The weight-one form satisfies $\Theta|_1\gamma = \epsilon(\gamma)\Theta$.

We recall the necessary divisor information, including its justification. The four cusps are $\infty, c_*, 0, -1/2$, of widths $1, 1, 5, 5$. For the quartic character $\chi(2) = i$ modulo 5, let

$$E_\chi = \frac{3+i}{10} + \sum_{n\geq 1}\left(\sum_{d|n}\chi(d)\right)q^n.$$

The modular parametrization of Zagier D gives

$$\Theta = \frac{3-i}{2}E_\chi + \frac{3+i}{2}E_{\bar\chi}, \qquad t\Theta^2 = \frac{\eta(5\tau)^5}{\eta(\tau)}.$$

These are the data in [8, Appendix B, Section 8.8]. Since the lower-right entry of δ is 2, $\Theta|_1\delta = q + O(q^2)$. The group H has effective index 12 and no elliptic points. The weight-one valence formula therefore shows that this simple zero at c_* is the entire zero divisor of Θ. The eta quotient $t\Theta^2$ has order one at each of ∞, c_* and order zero at the other two cusps. Consequently

$$\operatorname{div}(t) = [\infty] - [c_*].$$

In particular t is a Hauptmodul. The leading coefficients and these divisors, together with $(t\Theta^2)|_2\delta = -t\Theta^2$, give

$$\Theta|_1\delta = t\Theta, \qquad t(\delta\tau) = -1/t(\tau). \tag{48.1}$$

On weight -1 forms set

$$\mathcal{D}f = f|_{-1}\delta, \qquad JP(t) = \frac{P(-1/t)}{t}.$$

Thus

$$\mathcal{D}\left(\frac{P(t)}{\Theta}\right) = \frac{JP(t)}{\Theta}, \qquad \mathcal{D}^2 = -1,$$

because $\delta^2 = -I$. Both $P \mapsto JP$ and the modular expansions preserve integral Laurent coefficients.

For every $n \geq 1$ there is a unique form

$$g_n \in \frac{t^{-1}\mathbb{Z}[t^{-1}]}{\Theta}, \qquad g_n(q) = q^{-n} + O(1), \tag{48.2}$$

whose only negative Fourier term is q^{-n}. Indeed the forms t^{-j}/Θ, $j = 1, \ldots, n$, have leading terms q^{-j} with coefficient one. Successively canceling the other negative terms is an integral triangular procedure. The form g_n is holomorphic away from ∞: at c_*, t^{-j}/Θ has order $j - 1 \geq 0$, and at the other cusps both t and Θ have nonzero leading terms. There are no upper-half-plane poles. Conversely every weight -1 form with the same character and poles only at ∞ is determined by its principal part. The difference of two such forms with identical principal part would be a holomorphic form of negative weight, and hence zero.

The diamond transform has the complementary shape

$$\mathcal{D}g_n \in \frac{\mathbb{Z}[t]_{<n}}{\Theta}. \tag{48.3}$$

It is holomorphic away from c_*, and in particular has an integral power-series expansion at ∞. Here $\mathbb{Z}[t]_{<n}$ denotes polynomials of degree less than n. Set $g_{n/p} = 0$ whenever $p \nmid n$.

48.2 Hecke relations for both members of the pair

For Laurent series define

$$U_p\left(\sum_{j\in\mathbb{Z}} b_j q^j\right) = \sum_{j\in\mathbb{Z}} b_{pj} q^j, \qquad V_p f(q) = f(q^p).$$

Only finitely many negative terms occur throughout. Fix a prime $p \neq 5$, and put

$$\eta_p = \begin{cases} I, & p \equiv 1 \pmod 5, \\ \delta^{-1}, & p \equiv 2 \pmod 5, \\ \delta, & p \equiv 3 \pmod 5, \\ -I, & p \equiv 4 \pmod 5. \end{cases}$$

Consider the determinant-p matrices

$$\mathcal{M}_p = \left\{ \begin{pmatrix} a & b \\ c & d \end{pmatrix} : \begin{array}{c} a,b,c,d \in \mathbb{Z}, \quad ad - bc = p, \quad c \equiv 0 \pmod 5, \\ (a,d) \equiv (p,1) \text{ or } (-p,-1) \pmod 5 \end{array} \right\}.$$

Let $\epsilon(\beta)$ be the lower-right residue of β modulo 5, interpreted as 1 or -1, and define

$$\mathcal{H}_p f = p^{1/2} \sum_{H\backslash\mathcal{M}_p} \epsilon(\beta)\, f|_{-1}\beta.$$

The sign is multiplicative under left and right multiplication by H, so this is again a weight -1 form with character ϵ.

Integral Hermite normal form gives the representatives

$$\beta_\infty = \begin{pmatrix} p & 0 \\ 0 & 1 \end{pmatrix}, \qquad \eta_p\beta_b, \quad \beta_b = \begin{pmatrix} 1 & b \\ 0 & p \end{pmatrix} \quad (0 \le b < p),$$

For completeness, in a finite Hermite branch the intervening unimodular matrix has diagonal residues $\pm(p, p^{-1})$ modulo 5; multiplying it by η_p^{-1} places it in H. All displayed representatives have sign one. Write $\mathcal{E}_p f = f|_{-1}\eta_p$. Thus $\mathcal{E}_p$ is $1, -\mathcal{D}, \mathcal{D}, -1$, respectively, in the four residue classes $1, 2, 3, 4$. The Fourier expansions of the representatives give

$$\mathcal{H}_p = V_p + p^2 U_p \mathcal{E}_p. \tag{48.4}$$

The matrix δ normalizes H, and conjugation by δ preserves $\mathcal{M}_p$ and its sign: modulo 5 all the matrices involved are upper triangular and the two diagonal residues are unchanged by conjugation. Therefore

$$\mathcal{H}_p\mathcal{D} = \mathcal{D}\mathcal{H}_p. \tag{48.5}$$

Lemma 48.2 (Exact canonical Hecke relation)**.** *Set*

$$\mathbf{G}_n = \begin{pmatrix} g_n \\ \mathcal{D}g_n \end{pmatrix}, \qquad M_p = \begin{cases} I_2, & p \equiv 1 \pmod 5, \\ \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}, & p \equiv 2 \pmod 5, \\ \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}, & p \equiv 3 \pmod 5, \\ -I_2, & p \equiv 4 \pmod 5. \end{cases}$$

For every $n \ge 1$, with the Hecke operator acting componentwise,

$$V_p\mathbf{G}_n + p^2 M_p U_p \mathbf{G}_n = \mathcal{H}_p\mathbf{G}_n = \mathbf{G}_{pn} + p^2 M_p \mathbf{G}_{n/p}. \tag{48.6}$$

Proof. The first equality follows from (48.4) and $\mathcal{E}_p\mathbf{G}_n = M_p\mathbf{G}_n$. It remains to identify the principal parts of $\mathcal{H}_p g_n$.

First suppose $p \equiv 2$ or $3 \pmod 5$, and write $c_p = -1$ or 1, respectively. At ∞ the only negative term is q^{-pn}, since $\mathcal{D}g_n$ is holomorphic there. The expansion at c_*, obtained using (48.5), is

$$\mathcal{H}_p\mathcal{D}g_n = V_p\mathcal{D}g_n - c_p p^2 U_p g_n,$$

whose principal part is $-c_p p^2 q^{-n/p}$ if $p \mid n$, and zero otherwise. These are exactly the principal parts of $g_{pn} + c_p p^2 \mathcal{D}g_{n/p}$.

Next suppose $p \equiv \varepsilon_p = \pm 1 \pmod 5$. Then $\mathcal{E}_p = \varepsilon_p$, so the principal part of $\mathcal{H}_p g_n = V_p g_n + \varepsilon_p p^2 U_p g_n$ at ∞ is $q^{-pn} + \varepsilon_p p^2 q^{-n/p}$, the latter term being omitted when $p \nmid n$. At c_* the expansion is $V_p\mathcal{D}g_n + \varepsilon_p p^2 U_p \mathcal{D}g_n$, which is holomorphic. Thus its principal parts coincide with those of $g_{pn} + \varepsilon_p p^2 g_{n/p}$.

There are no poles at the other cusps. Indeed a primitive cusp column whose lower entry is not divisible by 5 still has lower entry nonzero modulo 5 after applying a matrix in $\mathcal{M}_p$; division by its possible common factor p does not change that property. Such a cusp cannot map to ∞ or c_*. All summands are holomorphic in the upper half plane as well. In either case the difference is a holomorphic weight -1 form, and must vanish. Applying $\mathcal{D}$ gives the second component of (48.6). □

Lemma 48.3 (Paired Cartier divisibility)**.** *For every $n \geq 1$,*

$$U_p g_n - g_{n/p} \in p^{2v_p(n)}\mathbb{Z}[[q]], \qquad U_p\mathcal{D}g_n - \mathcal{D}g_{n/p} \in p^{2v_p(n)}\mathbb{Z}[[q]].$$

Proof. Both differences are integral. In the first one the principal parts cancel; the second one is holomorphic at ∞. Write their column vector as Δ_n. Apply U_p to (48.6), using $U_pV_p = 1$. The resulting exact vector relation is

$$\Delta_{pn} = p^2 M_p U_p \Delta_n.$$

Iterating once for each factor p in n proves the assertion, since U_p and M_p preserve integral coefficients. □

48.3 The differential residual supplies the weights

Set

$$\theta = t\frac{d}{dt}, \qquad A(t) = 1 - 11t - t^2, \qquad L = \theta^2 - t(11\theta^2 + 11\theta + 3) - t^2(\theta+1)^2.$$

Thus $L = A\theta^2 + (\theta A)\theta - 3t - t^2$ and $LF = 0$. Let $\partial = q\,d/dq$. The canonical-coordinate Wronskian gives

$$\partial = A(t)\Theta^2\theta.$$

One can see the normalization directly: the second solution is $F(t)\log q(t)$, so the Wronskian equation yields $AF^2\,\theta\log q = 1$, its constant being fixed at $t = 0$. It follows by differentiating twice and using $LF = 0$ that, for every Laurent polynomial P,

$$\partial^2\left(\frac{P(t)}{\Theta}\right) = A(t)\Theta^3\,LP(t). \tag{48.7}$$

Write $f_K = F_K(t(q))/\Theta(q)$. The exact truncation residual is

$$LF_K = -K^2 t^K (A_K + A_{K-1}t). \tag{48.8}$$

Indeed all coefficients below t^K cancel by the Apéry recurrence; the only remaining terms have degrees K and $K+1$, with the displayed coefficients. Direct calculation on monomials also gives

$$L(JP)(t) = -t\,(LP)(-1/t).$$

Consequently

$$L(JF_K) = (-1)^K K^2 t^{-K}(A_K t - A_{K-1}). \tag{48.9}$$

Every factor outside K^2 on the right sides of (48.8) and (48.9), after multiplication by $A\Theta^3$, has an integral Laurent q-expansion. Therefore

$$\partial^2 f_K \in K^2\mathbb{Z}[[q]], \qquad \partial^2 \mathcal{D} f_K \in K^2\mathbb{Z}((q)). \tag{48.10}$$

48.4 Descent of the truncation and conclusion

Proof of Theorem 48.1. Let $p \mid K$, and set $r = v_p(K)$. Since $\mathcal{D}f_K$ has poles only at ∞, of order at most K, its canonical expansion is finite:

$$\mathcal{D}f_K = \sum_{n=1}^{K} c_n g_n, \qquad c_n = [q^{-n}]\mathcal{D}f_K \in \mathbb{Z}.$$

The second assertion of (48.10) gives

$$v_p(c_n) + 2v_p(n) \geq 2r. \tag{48.11}$$

Define the smaller-pole form

$$h = \sum_{\substack{1 \leq n \leq K \\ p \mid n}} c_n g_{n/p}.$$

Multiply the two congruences in Lemma 48.3 by c_n, and use (48.11). Term by term the product of the two divisibility factors is at least p^{2r}. It follows that

$$U_p \mathcal{D} f_K \equiv h, \qquad -U_p f_K \equiv \mathcal{D}h \pmod{p^{2r}\mathbb{Z}((q))}. \tag{48.12}$$

By (48.3) there is an integral polynomial P, of degree strictly less than K/p, such that

$$-\mathcal{D}h = \frac{P(t)}{\Theta}.$$

On the other hand $f_K = 1 + O(q^K)$, and hence $U_p f_K = 1 + O(q^{K/p})$. The second congruence in (48.12) now gives

$$P(t(q)) \equiv \Theta(q) \pmod{p^{2r}, q^{K/p}}.$$

The integral inverse $q(t) = t + O(t^2)$, followed by comparison of coefficients of degree $< K/p$, yields

$$P(t) \equiv F_{K/p}(t) \pmod{p^{2r}}.$$

Thus both members of the desired paired descent hold:

$$U_p f_K \equiv f_{K/p}, \qquad U_p \mathcal{D} f_K \equiv \mathcal{D} f_{K/p} \pmod{p^{2r}}. \tag{48.13}$$

For the second one, apply $\mathcal{D}$ to the polynomial congruence $P \equiv F_{K/p}$, which preserves integral Laurent expansions, and use the first part of (48.12).

Finally the first assertion of (48.10) shows that each coefficient of f_K at an exponent not divisible by p is divisible by p^{2r}. These are exactly the coefficients retained by $1 - V_p U_p$. Therefore

$$f_K \equiv V_p U_p f_K \equiv V_p f_{K/p} \pmod{p^{2r}}.$$

Take $K = mp^s$. Then $r \geq s$, so this says

$$\frac{F_{mp^s}(t)}{F(t)} \equiv \frac{F_{mp^{s-1}}(\sigma_p(t))}{F(\sigma_p(t))} \pmod{p^{2s}}.$$

All four denominator series have constant term one. Cross multiplication and division by units gives Theorem 48.1. □

Every step used only $p \nmid 5$. In particular no Jacobsthal congruence, odd-prime assumption, ordinary-point assumption, or higher-level induction hypothesis is required.

49 The first level in every dimension

The following argument proves the full formal-series congruence at $s = 1$ for every dimension of the three families in Conjecture 7.5. The higher-level ordinary-truncation proof is given independently in Part I.

Let g be the simplicial, hypercubic, or hyperoctahedral Laurent polynomial in dimension n, with vertex coefficient one, and let $f = 1 - zg$. Write

$$c_j = \mathrm{CT}(g^j), \qquad \mathcal{F}(z) = \sum_{j\geq 0} c_j z^j, \qquad H_N(z) = \mathrm{CT}(f^{N-1}).$$

Let $\tau = \sigma_p(z)$ be the excellent lift and $\lambda = \mathcal{F}(z)/\mathcal{F}(\tau)$. Assume the prime satisfies the symmetry and support-lattice exclusions of [7, Theorem 7.3]. In particular, $p > n$ for the cubic and octahedral families and $p > n + 1$ for the simplicial family, apart from irrelevant low-dimensional simplifications.

Lemma 49.1 (Second-order geometric approximation)**.** *For the above excellent lift,*

$$\lambda \equiv 2H_p - H_{2p} \pmod{p^2\mathbb{Z}_p[[z]]}.$$

More precisely, write

$$\mathscr{C}_p(f^{2p-1}) = H_{2p}(z) + B_p(z)g(\mathbf{x}).$$

Then

$$\tau \equiv -\frac{B_p(z)}{H_{2p}(z)} \pmod{p^2\mathbb{Z}_p[[z]]}.$$

Proof. All the calculations below may be made in the ring of Laurent series whose coefficients are z-adically convergent integral power series. Put $h(\mathbf{x}) = f^\sigma(\mathbf{x}^p)$. Since $h - f^p$ is divisible by p, the exact identity

$$\frac{1}{f} - \frac{2hf^{p-1} - f^{2p-1}}{h^2} = \frac{(h - f^p)^2}{fh^2}$$

has an integral remainder divisible by p^2. Applying the Cartier operator and using $\mathscr{C}_p(A(\mathbf{x})/h(\mathbf{x})^2) = \mathscr{C}_p(A)/f^\sigma(\mathbf{x})^2$, we obtain

$$\mathscr{C}_p(1/f) \equiv \frac{2f^\sigma \mathscr{C}_p(f^{p-1}) - \mathscr{C}_p(f^{2p-1})}{(f^\sigma)^2} \pmod{p^2}.$$

We justify the two numerator supports explicitly. Let Γ be the lattice spanned by the support of g. Since its index is prime to p, $p\mathbf{u} \in \Gamma$ implies $\mathbf{u} \in \Gamma$. Reflexivity gives integral facet functions L_j with $\Delta = \{L_j \leq 1\}$. A lattice point in $(1 - 1/p)\Delta$ has $L_j \leq 0$ for every j, and hence is zero. A lattice point in $(2 - 1/p)\Delta$ has $L_j \leq 1$, and hence belongs to Δ. The only points of $\Delta \cap \Gamma$ are zero and the vertices. Complete symmetry therefore yields the exact identities

$$\mathscr{C}_p(f^{p-1}) = H_p, \qquad \mathscr{C}_p(f^{2p-1}) = H_{2p} + B_p g.$$

For the hypercube, the restriction to Γ is essential; its points have coordinates all even or all odd.

The excellent Cartier relation $\mathscr{C}_p(1/f) \equiv \lambda/f^\sigma \pmod{p^2\mathscr{F}_2^\sigma}$ is in particular a coefficientwise congruence modulo p^2. Multiplying by $(f^\sigma)^2$ gives

$$\lambda(1 - \tau g) \equiv 2H_p(1 - \tau g) - H_{2p} - B_p g \pmod{p^2}.$$

Since $\mathrm{CT}(g) = 0$, comparison of the coefficients of 1 and g first gives $\lambda \equiv 2H_p - H_{2p}$, and then $\tau H_{2p} \equiv -B_p$. The latter can be divided by the unit H_{2p}, whose constant term is one. □

Theorem 49.2 (Conjecture 7.5 at the first level, in every dimension)**.** *Set $h = 2$ for the hypercubic and hyperoctahedral families and $h = n + 1$ for the simplicial family. At every admissible prime,*

$$\frac{\mathcal{F}(z)}{\mathcal{F}(\sigma_p(z))} \equiv \mathcal{F}_{hp}(z) \pmod{p^2\mathbb{Z}_p[[z]]}.$$

Equivalently, in the normalized variable $u = z^h$,

$$\frac{F(u)}{F(\sigma_p(u))} \equiv F_p(u) \pmod{p^2\mathbb{Z}_p[[u]]}.$$

These are the $s = 1$ cases of the specified original truncation lengths; $\mathcal{F}_h = 1$.

Proof. The coefficient of z^j in $2H_p - H_{2p}$ is

$$c_j w_j, \qquad w_j = (-1)^j \left(2\binom{p-1}{j} - \binom{2p-1}{j}\right),$$

with binomial coefficients zero past the top index. If $0 \le j < p$, the product expansion

$$(-1)^j\binom{ap-1}{j} = \prod_{i=1}^{j}\left(1 - \frac{ap}{i}\right) \equiv 1 - apH_j \pmod{p^2}$$

shows that $w_j \equiv 1 \pmod{p^2}$. For $p \le j < 2p$, Lucas's congruence gives $w_j \equiv 1 \pmod{p}$.

The remaining required divisibilities follow directly from the factorial formulas. In the cubic case, $c_{2k} = \binom{2k}{k}^n$, and for $k < p$ this is divisible by p whenever $2k \ge p$. In the octahedral case,

$$c_{2k} = \sum_{k_1+\cdots+k_n=k} \frac{(2k)!}{(k_1!\cdots k_n!)^2},$$

and every denominator is prime to p when $k < p$; hence the same divisibility holds term by term. In the simplicial case, putting $r = n + 1 < p$,

$$c_{rk} = \frac{(rk)!}{(k!)^r}, \qquad 0 \le k < p, \qquad v_p(c_{rk}) \ge \left\lfloor \frac{rk}{p} \right\rfloor.$$

Thus in all three cases c_j is divisible by p for $p \le j < \min(2p, hp)$, and in the simplicial case it is divisible by p^2 for $2p \le j < hp$. The other two families have $h = 2$, so there is no latter range. It follows coefficientwise that

$$2H_p - H_{2p} \equiv \mathcal{F}_{hp} \pmod{p^2}.$$

Apply Lemma 49.1 and then the normalized coordinate relation $q_0(z)^h = q(z^h)$. □

Limitation. Multiplication of the excellent Cartier relation by $(f^\sigma)^{2p^{s-1}}$ does not automatically preserve the p^{2s} coefficient divisibility of the error. Its support is not restricted to exponent vectors divisible by p^{s-1}. Accordingly, this proof supplies a complete base case in every dimension. The separate higher-level induction is the positive-vertex descent of Part I.

50 A canonical negative-power basis in all three families

The following all-level congruence concerns an auxiliary canonical basis. It is not the ordinary positive-power truncation congruence. It isolates which half of the two-cusp argument survives without a modular parametrization.

Work at an admissible prime p for one of the three completely symmetric families. Put $h = 2$ for the cubic and octahedral families, $h = n + 1$ for the simplicial family, and normalize $u = z^h$.

Write $F(u) = \mathcal{F}(z)$, $q(u) = q_0(z)^h$, and $\Theta(q) = F(u(q))$. Both $q(u)$ and its inverse are integral at p; here integrality always means over $\mathbb{Z}_p$.

There is a unique canonical basis

$$g_N \in \frac{u^{-1}\mathbb{Z}_p[u^{-1}]}{F(u)}, \qquad g_N(q) = q^{-N} + O(1), \qquad N \geq 1.$$

Existence and uniqueness follow from the integral triangular leading terms of u^{-j}/F. Equivalently,

$$g_N(u) = \frac{[F(u)q(u)^{-N}]_{u<0}}{F(u)},$$

where the numerator means the finite negative-power part in u.

Theorem 50.1 (Canonical-basis Frobenius congruence)**.** *For every integer $N \geq 1$,*

$$g_{pN}(q) \equiv g_N(q^p) \pmod{p^{2(1+v_p(N))}\mathbb{Z}_p((q))}.$$

Consequently

$$U_p g_N - g_{N/p} \in p^{2v_p(N)}\mathbb{Z}_p[[q]],$$

where $g_{N/p} = 0$ if $p \nmid N$.

Proof. We first construct an integral triangular family of vertex coefficients. For a positive integer M, take the formal expansion of $1/f$ at the vertex with monomial $(x_1 \cdots x_n)^{-1}$ in the simplex and cube, or x_1^{-1} in the octahedron. We allow z^{-1} in the coefficient ring, as in [7, Proposition 7.17]. Coefficient extraction is performed after localization at z and p-adic completion. The particular series and differences used below have bounded negative support, so their divisibility statements lie in the ordinary subring $\mathbb{Z}_p((z))$; no such assertion is made for every element of the completed coefficient ring. Let $\mathcal{P}_M$ be minus the coefficient at $(x_1 \cdots x_n)^M$, or at x_1^M with the other exponents zero, respectively. Direct geometric expansion gives

$$\mathcal{P}_{hN}(z) = u^{-N}A_N(u), \qquad A_N \in 1 + u\mathbb{Z}[u], \quad \deg A_N < N.$$

For clarity, the three finite formulas are as follows. With $M = hN$, the cubic coefficients are

$$\mathcal{P}_M(z) = z^{-M} \sum_{0 \leq j < M/2} (-1)^{nj} \binom{M-j-1}{j}^n z^{2j}.$$

For the simplex, $r = n + 1$ and

$$\mathcal{P}_M(z) = z^{-M} \sum_{0 \leq j < M/r} (-1)^{(r-1)j} \frac{(M-j-1)!}{(M-rj-1)!(j!)^{r-1}} z^{rj}.$$

For the octahedron,

$$\mathcal{P}_M(z) = z^{-M} \sum_{0 \leq j < M/2} z^{2j} \sum_{k_1+\cdots+k_n=j} (-1)^{k_1} \frac{(M-k_1-1)!}{(M-2j-1)!k_1!(k_2!\cdots k_n!)^2}.$$

Each formula is a coefficient of an integral geometric series, so its integrality does not depend on a factorial-denominator estimate.

Set $R_N = \mathcal{P}_{hN}/\mathcal{F} = A_N/(u^N F)$. The excellent Cartier relation, read at the exponent vector of length hpN, gives

$$R_{pN}(q) \equiv R_N(q^p) \pmod{p^{2(1+v_p(N))}}. \tag{50.1}$$

This is the same coefficient-reading argument as [7, Proposition 7.17], and it applies to even indices as well; the displayed finite formulas verify the required expansions directly. The condition $p \nmid h$ ensures the stated valuation.

There is a second useful consequence of the rank-two connection. Let $\theta_z = z\,d/dz$, and let $L_2 = \theta_z^2 - B\theta_z - A$ be the operator of [7, Proposition 7.6], so $L_2(1/f) \in \mathscr{F}_2$. Reading the same vertex coefficient gives $L_2\mathcal{P}_{hN} \in (hN)^2\mathbb{Z}_p((z))$. The solutions $\mathcal{F}$ and $\mathcal{F}\log q_0$ gauge this operator to a scalar multiple of ∂_0^2, where $\partial_0 = q_0\,d/dq_0$. Explicitly, $\partial_0^2(\mathcal{P}_{hN}/\mathcal{F}) = \mathcal{F}^3W^{-2}L_2\mathcal{P}_{hN}$. The scalar and its inverse are integral units: this follows from $\mathcal{F}(0) = 1$ and the integral unit Wronskian in [7, Proposition 7.9]. Since $\partial = q\,d/dq = h^{-1}\partial_0$, we obtain

$$\partial^2 R_N \in N^2\mathbb{Z}_p((q)). \tag{50.2}$$

Expand $R_N = \sum_{j=1}^{N} c_{N,j}g_j$. The coefficients are its negative Fourier coefficients, so they are integral and $c_{N,N} = 1$. By (50.2),

$$v_p(c_{N,j}) + 2v_p(j) \geq 2v_p(N).$$

Comparing principal parts in (50.1) and using the integral canonical basis yields

$$R_{pN} \equiv \sum_{j=1}^{N} c_{N,j}g_{pj} \pmod{p^{2(1+v_p(N))}}.$$

Subtracting (50.1) now gives

$$\sum_{j=1}^{N} c_{N,j}(g_{pj} - V_p g_j) \equiv 0 \pmod{p^{2(1+v_p(N))}}.$$

Induct on the ordinary positive integer N. For $j < N$, the induction hypothesis contributes $2(1+v_p(j))$ to the valuation, and the displayed bound on $c_{N,j}$ raises the total to at least $2(1+v_p(N))$. The final coefficient is $c_{N,N} = 1$, so the desired congruence for $g_{pN} - V_p g_N$ follows. Applying U_p proves the second assertion when $p \mid N$. For $p \nmid N$, it simply asserts integrality and absence of negative terms, which follow from the canonical principal part. □

Relation to ordinary truncations. The basis consists of negative-power polynomials divided by F. The target truncation F_{p^s}/F belongs to the positive-power space. The integral diamond operation in the Zagier D argument interchanges these two spaces and supplies the missing relation there. No corresponding operation has been established for the general higher-dimensional families, so the theorem above is not a proof of their ordinary truncation congruence. Part I proves that congruence using a different, positive-vertex construction.

51 The three-dimensional hyperoctahedral family: a weighted Cartier reduction

This section proves an all-coefficient differential divisibility statement for the three-dimensional hyperoctahedral family. It is an additional differential refinement; the full ordinary truncation congruence follows from Theorem 2.1.

51.1 The period and its actual Beukers–Vlasenko coordinate

Use the normalized variable $u = t^2$, and put

$$b_n = \sum_{j=0}^{n} \binom{n}{j}^2 \binom{2j}{j}, \qquad a_n = \binom{2n}{n} b_n, \qquad F(u) = \sum_{n\geq 0} a_n u^n.$$

Equivalently,

$$b_n = \sum_{k_1+k_2+k_3=n} \left(\frac{n!}{k_1!k_2!k_3!}\right)^2, \qquad a_n = \sum_{k_1+k_2+k_3=n} \frac{(2n)!}{(k_1!k_2!k_3!)^2}.$$

The normalized logarithmic companion prescribed by [7, §7 and Proposition 9.1] is

$$G(u) = 2\sum_{n\geq 1} u^n \sum_{k_1+k_2+k_3=n} \frac{(2n)!}{(k_1!k_2!k_3!)^2}(H_{2n} - H_{k_1}). \tag{51.1}$$

Thus $q(u) = u\exp(G(u)/F(u))$ is exactly the square of the original BV coordinate, not merely a coordinate with the same first term. It and its inverse $u(q)$ are p-integral for every $p \geq 5$.

Write $\theta = u\,d/du$ and

$$L = \theta^3 - 2u(2\theta+1)(10\theta^2+10\theta+3) + 36u^2(\theta+1)(2\theta+1)(2\theta+3). \tag{51.2}$$

Lemma 51.1 (Logarithmic companion with a telescoping certificate)**.** *Both F and $F\log u + G$, with G as in (51.1), are solutions of $Ly = 0$.*

Proof. For a formal parameter ρ, define

$$b_n(\rho) = \sum_{j=0}^{n} \binom{n+\rho}{j}^2 \binom{2j}{j}, \qquad c_n(\rho) = \frac{(1+2\rho)_{2n}}{(1+\rho)_n^2}, \qquad a_n(\rho) = c_n(\rho)b_n(\rho).$$

Here $(x)_n$ is the rising factorial. We work modulo ρ^2. Let

$$T(z,j) = \binom{z}{j}^2 \binom{2j}{j}, \qquad R(z,j) = \frac{j^3(3j-4z-4)}{(z+1-j)^2}.$$

Direct cancellation of rational functions gives the certificate

$$\begin{aligned}(z+1)^2T(z+1,j) - (10z^2+10z+3)T(z,j) + 9z^2T(z-1,j)\\ = R(z,j+1)T(z,j+1) - R(z,j)T(z,j).\end{aligned}$$

Substitute $z = n-1+\rho$ and sum over $0 \leq j \leq n$. The lower boundary is zero and the upper boundary is $O(\rho^2)$. Although $R(z,n)$ has a displayed denominator ρ^2, its product with $T(z,n)$ is regular: the zero of the binomial square cancels it. Terms beyond the finite cutoffs in the other two sums are also $O(\rho^2)$; when $n = 1$, the factor $9\rho^2$ handles the last sum. Consequently, with $b_{-1}(\rho) = 0$,

$$(n+\rho)^2 b_n(\rho) - \{10(n-1+\rho)^2 + 10(n-1+\rho) + 3\}b_{n-1}(\rho) + 9(n-1+\rho)^2 b_{n-2}(\rho) = O(\rho^2).$$

The ratio

$$\frac{c_n(\rho)}{c_{n-1}(\rho)} = \frac{2(2n+2\rho-1)}{n+\rho}$$

shows that the shifted recurrence for L applied to $a_n(\rho)$ is $c_n(\rho)(n+\rho)$ times the preceding recurrence. Its constant-index residual is ρ^3. Therefore

$$L\left(u^\rho \sum_{n\geq 0} a_n(\rho)u^n\right) = O(\rho^2).$$

Taking its constant and linear terms proves the claim, because

$$c_n'(0) = 2\binom{2n}{n}(H_{2n} - H_n), \qquad b_n'(0) = 2\sum_{j=0}^{n} T(n,j)(H_n - H_{n-j}).$$

Grouping the other two multinomial indices identifies $a_n'(0)$ with the coefficient in (51.1). □

51.2 Symmetric square, Bol identity, and an exact residual

Put

$$A(u) = (1-4u)(1-36u) = 1 - 40u + 144u^2.$$

The monic version of L is the symmetric square of

$$M = \theta^2 + \frac{4u(36u-5)}{A(u)}\theta + \frac{3u(9u-1)}{A(u)}.$$

This follows by direct substitution in the symmetric-square formula

$$(\theta^2 + B\theta + C)^{\mathrm{Sym}^2} = \theta^3 + 3B\theta^2 + (\theta B + 4C + 2B^2)\theta + (2\theta C + 4BC).$$

If $\varphi_0 = \sqrt{F}$ and $\varphi_1 = \varphi_0 \log u + \cdots$ are its normalized solutions, Lemma 51.1 gives $\log q = \varphi_1/\varphi_0$. Their θ-Wronskian is $A^{-1/2}$, normalized to constant term 1, and hence

$$\theta \log q = \frac{1}{F\sqrt{A}}, \qquad \partial := q\frac{d}{dq} = F\sqrt{A}\,\theta. \tag{51.3}$$

It follows, either by direct differentiation or by comparing the leading terms and the three-dimensional kernels, that

$$\partial^3\left(\frac{P(u(q))}{F(u(q))}\right) = F(u(q))^2\sqrt{A(u(q))}\,(LP)(u(q)) \tag{51.4}$$

for every formal series P.

For $K \geq 1$ let $F_K(u) = \sum_{0\leq n<K} a_n u^n$. The recurrence obtained from (51.2) gives the exact identity

$$LF_K = -K^3 a_K u^K + 36K(2K-1)(2K+1)a_{K-1}u^{K+1}. \tag{51.5}$$

In particular, if $K = mp^s$ with $m, s \geq 1$, then

$$LF_K \in p^{2s}\mathbb{Z}_p[u].$$

Indeed, the first coefficient is divisible by p^{3s}, and $\binom{2K-2}{K-1}$ has at least s carries when $K-1$ is doubled in base p. Thus $p^s \mid a_{K-1}$.

51.3 An integral primitive improves the weighted bound

For $j \geq 0$ define

$$I_j(u) = \int_0^u z^j F(z)\,dz = \sum_{n\geq 0} \frac{a_n}{n+j+1}u^{n+j+1}.$$

Lemma 51.2 (Two shifted Catalan divisibilities)**.** *Let $p \geq 5$, $s \geq 1$, and $K = p^s$. Then*

$$I_K(u) \in \mathbb{Z}_p[[u]], \qquad KI_{K-1}(u) \in \mathbb{Z}_p[[u]].$$

Proof. Only the central binomial factor in a_n is needed. If $r = v_p(n+K+1)$, then for $1 \leq i \leq \min(r,s)$, the residue of n modulo p^i is $p^i - 1$. For $s < i \leq r$ it is $p^i - p^s - 1$. Every displayed residue is at least $p^i/2$ when $p \geq 5$. Each corresponding summand in Legendre's formula

$$v_p\binom{2n}{n} = \sum_{i\geq 1}\left(\left\lfloor\frac{2n}{p^i}\right\rfloor - 2\left\lfloor\frac{n}{p^i}\right\rfloor\right)$$

is therefore 1, so $v_p(a_n) \geq r$.

For the second assertion put $r = v_p(n+K)$. If $r \leq s$, the factor K already cancels the denominator. If $r > s$, then for every $s < i \leq r$ the residue of n modulo p^i is $p^i - p^s \geq p^i/2$. Thus $v_p(a_n) \geq r - s$, which again proves integrality. □

Theorem 51.3 (All-coefficient weighted divisibility)**.** *For every prime $p \geq 5$ and every $s \geq 1$, put $K = p^s$ and*

$$H_K(q) = \frac{F_K(u(q))}{F(u(q))} = 1 + \sum_{n \geq 1} h_{K,n} q^n.$$

Then

$$\partial^2 H_K \in p^{2s}\mathbb{Z}_p[[q]]. \tag{51.6}$$

Equivalently, for every $n \geq 1$,

$$v_p(h_{K,n}) \geq \max\{0,\, 2s - 2v_p(n)\}. \tag{51.7}$$

In particular every coefficient with $p \nmid n$ is divisible by p^{2s}.

Proof. From (51.3),

$$\partial I_j(u(q)) = u(q)^{j+1} F(u(q))^2 \sqrt{A(u(q))}.$$

Integrate (51.4) with (51.5) once with respect to $\log q$. All terms have constant term zero, so no integration constant appears. The resulting exact identity is

$$\partial^2 H_K = -K^3 a_K I_{K-1}(u(q)) + 36K(2K-1)(2K+1) a_{K-1} I_K(u(q)). \tag{51.8}$$

The first term is divisible by K^2 by Lemma 51.2; so is the second, because $p^s \mid a_{K-1}$. The integral inverse coordinate $u(q)$ preserves divisibility. Finally $[q^n]\partial^2 H_K = n^2 h_{K,n}$, proving the coefficient bound. □

51.4 A Cartier reformulation of the ordinary truncation congruence

Write $U_p(\sum h_n q^n) = \sum h_{pn} q^n$ and $V_p H(q) = H(q^p)$. The theorem gives

$$H_{p^s} - V_p U_p H_{p^s} \in p^{2s}\mathbb{Z}_p[[q]].$$

Consequently the full desired excellent-lift congruence in the original conjecture is now equivalent to

$$U_p H_{p^s} \equiv H_{p^{s-1}} \pmod{p^{2s}\mathbb{Z}_p[[q]]}. \tag{51.9}$$

Indeed its normalized cleared defect divided by the period units is $V_p H_{p^{s-1}} - H_{p^s}$, and V_p is coefficientwise injective. Equation (51.9) follows, at every level, from Theorem 2.1 and the equivalence just proved. The differential estimate alone controls the nonresonant coefficients and quantifies the loss at the other indices; its role is distinct from the positive-vertex proof of the full Cartier congruence.

51.5 A rigorously compatible Domb transformation

There is also an exact compatible modular transformation. Let

$$d_n = (-1)^n \sum_{j=0}^{n} \binom{n}{j}^2 \binom{2j}{j} \binom{2n-2j}{n-j}, \qquad F_D(v) = \sum_{n \geq 0} d_n v^n,$$

and put $D(v) = 1 + 20v + 64v^2 = (1+4v)(1+16v)$. The twisted Domb operator from [8, Appendix C, §9.7] is

$$L_D = \vartheta^3 + 2v(2\vartheta+1)(5\vartheta^2+5\vartheta+2) + 64v^2(\vartheta+1)^3, \qquad \vartheta = v\frac{d}{dv}.$$

For $u = v/D(v)$, direct substitution gives the operator identity

$$(L[\sqrt{D}\, y])(u(v)) = \frac{\sqrt{D(v)}}{1-64v^2}(L_D y)(v).$$

Here the left side means that $\sqrt{D(v)}y(v)$ is viewed as a function of u via the inverse substitution. Uniqueness of the normalized holomorphic and logarithmic solutions, together with Lemma 51.1, proves

$$F(u(v)) = \sqrt{D(v)}F_D(v), \qquad q_{\mathrm{BV}}(u(v)) = q_D(v).$$

In particular the lift is genuinely transported by this transformation. The Domb mirror parameter is

$$v(q) = \left(\frac{\eta(2\tau)\eta(6\tau)}{\eta(\tau)\eta(3\tau)}\right)^6, \qquad q = e^{2\pi i\tau},$$

and the octahedral form is

$$F(u(q)) = q\frac{d}{dq}\log v(q) = \frac{-E_2(q) + 2E_2(q^2) - 3E_2(q^3) + 6E_2(q^6)}{4}.$$

Transport of the infinite period and coordinate does not by itself transport the ordinary finite truncations. In particular, no full octahedral congruence is inferred solely from the established Domb congruences. Lagrange inversion gives the exact coefficient kernel

$$a_n = \sum_{j=0}^{n} \frac{2j+1}{2n+1}\, d_j[v^{n-j}]D(v)^{n+1/2},$$

which specifies a possible additional truncation-compatibility problem.

52 Exact verification and the role of the auxiliary arguments

The accompanying Python programs use exact integer and rational arithmetic and explicit reduction modulo prime powers. They are self-contained and need only the Python standard library. Their recorded outputs declare all finite coefficient bounds.

For Zagier D the checks include integral canonical principal parts, both components of the exact Hecke identities, the paired Cartier recurrence, Laurent-series descent including its negative powers, and the original full cleared defect beyond its truncation boundary. In particular the computations include $p = 2, 3$, nonsplit primes $7, 13$, split primes $11, 19$, higher levels, and $m = 2$. The largest recorded check reaches q^{1380} at $p = 7, s = 3, m = 2$. The all-dimensional checks verify the geometric numerator, the explicit excellent lift modulo p^2, and the full first-level quotient beyond the cutoff in dimensions three through six. The auxiliary canonical-basis checks cover ten cases in dimensions three through five, including indices with a nontrivial prime-to-p factor. The octahedral checks verify the telescoping certificate, the BV coordinate against the modular inverse through degree 39, and six weighted differential bounds. They also sample the full octahedral congruence of Theorem 2.1 at $(p, s) = (5, 1), (5, 2), (5, 3), (7, 1), (7, 2), (11, 2)$, through the respective degrees $48, 128, 528, 56, 224, 512$; all observed minima are $2s$. These calculations verify the implementation and sample the identities; the infinite statements depend on the written proofs.

52.1 Why the auxiliary arguments require a separate descent

The excellent Cartier theorem gives

$$\mathscr{C}_p(1/f) - \frac{\mathcal{F}(z)}{\mathcal{F}(\sigma_p(z))}\frac{1}{f^\sigma} \in p^2\mathscr{F}_2^\sigma.$$

Reading a spatial coefficient with exponent vector divisible by p^{s-1} supplies a factor p^{2s}. The operation defining the ordinary period truncation is a different linear functional. In the notation of [22, Section 4.3], it is obtained from period maps involving powers of $zg(\mathbf{x})$. Their generic

divisibility does not automatically double merely because the Cartier error is a second formal derivative.

The proof of Section 49 avoids that difficulty at the first level using the exact two-term geometric numerator. For levels $s \geq 2$, multiplying a Cartier error by a larger polynomial can mix spatial exponent vectors with smaller p-adic valuation. A valid induction must control this mixing, or prove a separate descent identity for the actual truncations. The auxiliary theorem in Section 50 controls negative-power canonical functions at all levels, but supplies no identification with the ordinary positive-power truncations. The required ordinary-tail identification is instead supplied by the positive-vertex basis in Part I.

The new Zagier D proof resolves its analogous difficulty by the paired canonical-basis recurrence. Its basis forms have poles in a finite set of cusps preserved by the Hecke correspondence. In the three-dimensional hyperoctahedral modular parametrization, the relevant zero of the weight-two form is elliptic, so that same finite-cusp argument is not directly available. Section 51 instead proves the second-derivative estimate at all levels and isolates the exact additional relation (51.9). Theorem 2.1 supplies that relation and the higher-dimensional ordinary-truncation bounds by positive interpolation and matrix block descent.

A An independent coefficientwise certificate in dimension four

This appendix gives a second, independent exact certificate for one case of Theorem 2.1. Its finite polynomial calculation proves every coefficient at the specified prime and level.

Theorem A.1 (Dimension four, $p = 5$, $s = 2$)**.** *Let*

$$a_j = \binom{2j}{j} \sum_{k=0}^{j} \binom{j}{k}^2 \binom{2k}{k} \binom{2j-2k}{j-k}, \qquad F(u) = \sum_{j \geq 0} a_j u^j.$$

Let $\sigma(u)$ be the actual normalized Beukers–Vlasenko excellent lift for the four-dimensional hyperoctahedral family. Then

$$F(u)F_5(\sigma(u)) - F(\sigma(u))F_{25}(u) \in 625\mathbb{Z}_5[[u]].$$

This is the entire formal-series congruence in Conjecture 7.5 at $p = 5, s = 2$, with original truncation lengths 50 and 10.

The proof is an exact finite polynomial certificate. Crucially, the rational lift used in the certificate is constructed from vertex polynomials, independently of the desired ordinary truncation identity.

A.1 Vertex polynomials and a derivative bound

Put

$$b_j = \sum_{k=0}^{j} \binom{j}{k}^2 \binom{2k}{k}, \qquad A_K(u) = \sum_{j=0}^{K-1} u^j \sum_{\ell=0}^{j} (-1)^\ell \frac{(2K-\ell-1)!\, b_{j-\ell}}{(2K-2j-1)!\, \ell!\, (j-\ell)!^2}.$$

Each summand is an integer: the factorial ratio is a multinomial coefficient. Also $A_K(0) = 1$. Expanding $1/(1 - z\sum_{i=1}^{4}(x_i + x_i^{-1}))$ in the vertex chamber where x_1 is dominant, the negative of the coefficient at x_1^{-2K}, with all other exponents zero, is

$$P_{2K}(u) = u^{-K} A_K(u), \qquad u = z^2.$$

Consequently the excellent Cartier relation [7, Theorem 7.3] gives, with $\lambda = F(u)/F(\sigma(u))$,

$$P_{2pK}(u) - \lambda P_{2K}(\sigma(u)) \in p^{2+2v_p(K)}\mathbb{Z}_p((u)). \tag{A.1}$$

These displayed series have bounded negative support; no identification of the whole completed coefficient ring with $\mathbb{Z}_p((u))$ is needed.

We shall also use

$$\theta C_K \in p^{v_p(K)}\mathbb{Z}_p[[u]], \qquad C_K = \frac{A_{2K}}{A_K}, \qquad \theta = u\frac{d}{du}. \tag{A.2}$$

Here is a direct justification from the second-order quotient. Set $R_K = P_{2K}/F$, and use the actual canonical coordinate q, with $\partial = q\, d/dq$. Reading the coefficient at the same vertex in the relation of [7, Proposition 7.6], then dividing by the period and changing to the canonical coordinate, gives

$$\partial^2 R_K \in p^{2v_p(K)}\mathbb{Z}_p((q)).$$

The change-of-coordinate and gauge factors are integral units by [7, Proposition 7.9 and Corollary 7.11]; the factor 2 from $u = z^2$ is a unit here. If $r = v_p(K)$, then for every nonzero Laurent index j, integrality of R_K and $j^2[q^j]R_K \in p^{2r}\mathbb{Z}_p$ imply $j[q^j]R_K \in p^r\mathbb{Z}_p$. Thus $\partial R_K \in p^r\mathbb{Z}_p((q))$, and the same holds for θR_K. Since $C_K = u^K R_{2K}/R_K$, equation (A.2) follows. In particular, for $K = 25$, if $C_K = \sum c_j u^j$, then $jc_j \in 25\mathbb{Z}_5$.

A.2 Constructing the lift modulo 625

From now on set $p = 5$, $N = 25$, $X = u^5$, and write $\sigma(u) = XS(u)$, where $S \in 1 + 5\mathbb{Z}_5[[u]]$. Apply (A.1) at $K = N, 2N$ and eliminate λ. Because all A_K have constant term one, this gives

$$S^N C_{5N}(u) - C_N(XS) \in 5^6 \mathbb{Z}_5[[u]]. \tag{A.3}$$

Let $h = S - 1$. Divide the left-hand side by $N = 25$ and expand at $h = 0$. Its coefficients through degree three will be denoted a, b, c, d, respectively. Then

$$a \in 5\mathbb{Z}_5[[u]], \qquad b(0) = 1, \qquad b, c, d \in \mathbb{Z}_5[[u]].$$

For completeness, the higher terms may be discarded modulo 625 on the domain $h \in 5\mathbb{Z}_5[[u]]$. Indeed, $jc_j \in N\mathbb{Z}_5$ implies

$$\binom{j}{r} c_j = \frac{jc_j}{r} \binom{j-1}{r-1} \in \frac{N}{r} \mathbb{Z}_5 \quad (r \geq 1),$$

and the same estimate holds for $\binom{N}{r}$. After division by N, the term of degree r in h therefore has valuation at least $r - v_5(r)$. This is at least four whenever $r \geq 4$. The assertion $a \in 5\mathbb{Z}_5[[u]]$ also follows from (A.3) and these estimates. Thus

$$a + bh + ch^2 + dh^3 \equiv 0 \pmod{625}.$$

As b is a unit, the unique root with $h \equiv 0 \pmod 5$ is

$$h \equiv -\frac{a}{b} - \frac{ca^2}{b^3} + \left(\frac{d}{b^4} - \frac{2c^2}{b^5} \right) a^3 \pmod{625}. \tag{A.4}$$

This assertion can also be checked by substitution; uniqueness follows by factoring the difference of two roots, whose quotient has unit constant term modulo five.

We now describe the exact polynomial calculation of a, b, c, d. All input polynomials are computed modulo $5^6 = 15625$. For $0 \leq r \leq 3$, put

$$a_r = [h^r] A_N(X(1+h)), \qquad b_r = [h^r] A_{2N}(X(1+h)).$$

Define

$$T_0 = b_0, \qquad T_r = b_r a_0^r - \sum_{i=1}^{r} a_i T_{r-i} a_0^{i-1} \quad (1 \leq r \leq 3).$$

Then $[h^r] C_N(X(1+h)) = T_r / a_0^{r+1}$. Let $\mathcal{A}_r \in (\mathbb{Z}/625\mathbb{Z})[u]$ be the coefficientwise quotient

$$\mathcal{A}_r = \frac{\binom{N}{r} A_{10N} a_0^4 - A_{5N} T_r a_0^{3-r}}{N} \pmod{625}. \tag{A.5}$$

The numerators are divisible by N, as follows from the preceding argument and is also checked coefficientwise in the certificate. All four coefficients a, b, c, d have the common unit denominator $A_{5N} a_0^4$; their respective numerators are

$$A = \mathcal{A}_0, \quad B = \mathcal{A}_1, \quad C = \mathcal{A}_2, \quad D = \mathcal{A}_3.$$

Thus (A.4) becomes

$$h \equiv -\frac{A}{B} - \frac{CA^2}{B^3} + \left(\frac{D}{B^4} - \frac{2C^2}{B^5} \right) A^3 \pmod{625}.$$

The calculation checks $A \in 5\mathbb{Z}[u]/625$, $A(0) = 0$, and $B(0) = 1$. The four polynomial degrees after reduction are $489, 537, 537, 527$, respectively.

A.3 The ordinary truncation certificate

The first relation (A.1) gives

$$\lambda \equiv \frac{S^{25}A_{125}(u)}{A_{25}(XS)} \pmod{625}.$$

Because $h \in 5\mathbb{Z}_5[[u]]$, $S^{25} \equiv 1 + 25h \pmod{625}$. Define the four explicit integral polynomials

$$L_r = A_{125}(u)([h^r]F_5(X(1+h)) + 25[h^{r-1}]F_5(X(1+h))) - F_{25}(u)[h^r]A_{25}(X(1+h)),$$

where a coefficient with index -1 means zero. The target congruence is therefore equivalent to $L_0 + L_1h + L_2h^2 + L_3h^3 \equiv 0 \pmod{625}$. Substituting the independently constructed root and clearing the unit denominator B^5 reduces it to the finite identity

$$\begin{aligned} 0 \equiv\ & L_0B^5 - L_1AB^4 + (L_2B^3 - L_1CB^2)A^2 \\ & + (L_1DB - 2L_1C^2 + 2L_2CB - L_3B^2)A^3 \pmod{625}. \end{aligned} \tag{A.6}$$

The accompanying standalone program `certificate_octa4_p5_s2.py` computes the four vertex polynomials directly from their factorial sums, constructs (A.5), and verifies that every coefficient of (A.6) is zero modulo 625. The degree bound of this cleared identity is 2819. All operations are integer polynomial addition, multiplication, and exact division by 25; there is no power-series cutoff, numerical approximation, or reconstruction of the lift from the target truncations. The machine-readable coefficient certificate is `certificate_octa4_p5_s2.json`. This finite identity proves $\lambda F_5(\sigma) - F_{25} \in 625\mathbb{Z}_5[[u]]$. Multiplication by the period unit $F(\sigma)$ proves Theorem A.1.

B The original modular verification program

The following is the complete supplied verification program. Its comments and output labels retain the original manuscript's local section numbers and the former "open case" label; current theorem status is given in Part I. The operations have not been altered in consolidation. The finite and floating-point checks have the limited roles described in Section 40.

```
#!/usr/bin/env python3
"""
Exact-arithmetic checks for the paper
  "Hecke traces and the Beukers--Vlasenko supercongruences for excellent Frobenius lifts".
Standard library only.  Run:  python3 verify_excellent_lift.py
Every check prints PASS/FAIL; the script exits with status 1 if any check fails.

Notation: q-series are integer lists [c_0, c_1, ...] truncated at a length M.
For a family (Theta, t, F) with F(t(q)) = Theta(q) and a prime p:
   Q_P(q)   = Theta(q) P(t(q^p)) / Theta(q^p)
   G_P(z)   = Theta(z^p) P(t(z)) / Theta(z)
   Tr(Q_P)  = Q_P + p^(w+1) chi(p) U_p(G_P)          (Lemma 3.3)
"""
import sys, random, cmath
from math import comb, factorial, gcd, floor, ceil
from fractions import Fraction as Fr

FAILS = []
def report(name, ok, detail=""):
    print(("PASS " if ok else "FAIL ") + name + ((" :: " + detail) if detail else ""), flush=True)
    if not ok: FAILS.append(name)

# ---------------------------------------------------------------- series utilities
def mul(a, b, M):
    c = [0]*M
    for i in range(min(M, len(a))):
        x = a[i]
        if x == 0: continue
        for j in range(min(M-i, len(b))):
            y = b[j]
            if y: c[i+j] += x*y
    return c
def inv(a, M):
    assert a[0] == 1
    b = [0]*M; b[0] = 1
    for n in range(1, M):
        s = 0
        for k in range(1, min(n, len(a)-1)+1):
            if a[k]: s += a[k]*b[n-k]
        b[n] = -s
    return b
def subqp(a, p, M):                       # a(q) -> a(q^p)
    c = [0]*M
    for i, x in enumerate(a):
        if i*p >= M: break
        c[i*p] = x
    return c
def polyt(coeffs, t, M):                  # sum coeffs[k] t^k  (Horner)
    res = [0]*M
    for cf in reversed(coeffs):
        res = mul(res, t, M); res[0] += cf
    return res
def t_expand(f, t, M):                    # f(q) = sum_k c_k t(q)^k ; t = q + O(q^2)
    rem = list(f[:M]); out = []; tp = [1] + [0]*(M-1)
    for k in range(M):
        c = rem[k]; out.append(c)
        if c:
            for i in range(M): rem[i] -= c*tp[i]
        tp = mul(tp, t, M)
    return out
def vp(x, p):
    if x == 0: return 10**9
    v = 0
    while x % p == 0: x //= p; v += 1
    return v
```

```
def vp_series(a, p): return min(vp(x, p) for x in a)

def etaseries(exps, M):
    """prod_m eta(m tau)^{e_m} as an integer q-series (integral q-shift required)."""
    sh = Fr(sum(m*e for m, e in exps.items()), 24)
    assert sh.denominator == 1 and sh >= 0
    P = [0]*M; P[0] = 1
    for m, e in exps.items():
        for n in range(1, M):
            k = m*n
            if k >= M: break
            for _ in range(abs(e)):
                if e > 0:
                    for i in range(M-1, k-1, -1): P[i] -= P[i-k]
                else:
                    for i in range(k, M): P[i] += P[i-k]
    sh = int(sh)
    return [0]*sh + P[:M-sh]
def cubic_theta(M):
    a = [0]*M; R = int((4*M/3)**0.5) + 3          # x^2+xy+y^2 < M forces |x|,|y| < sqrt(4M/3)
    for x in range(-R, R+1):
        for y in range(-R, R+1):
            v = x*x + x*y + y*y
            if v < M: a[v] += 1
    return a
def sigma(n): return sum(d for d in range(1, n+1) if n % d == 0)

# ------------------------------------------------------------------ characters
def chi_m4(p): return 0 if p % 2 == 0 else (1 if p % 4 == 1 else -1)
def chi_m3(p): return 0 if p % 3 == 0 else (1 if p % 3 == 1 else -1)
def chi_5(p):  return 0 if p % 5 == 0 else (1 if p % 5 in (1, 4) else -1)
def chi_1(p):  return 1

def C(n, k): return comb(n, k) if 0 <= k <= n else 0

# ------------------------------------------------------------------ the families of cusp type
# (name, weight w, level N, group, character chi, Theta eta-exponents, t eta-exponents,
#  zero cusp denominators (d|N) of Theta, r_Theta, closed formula for a_n)
CUSP = [
 ("Legendre 2F1(1/2,1/2;1|16t)",1,4,"G0(4)",chi_m4,{2:10,1:-4,4:-4},{1:8,4:16,2:-24},[2],Fr(1,2),
   lambda n: C(2*n,n)**2),
 ("Zagier A (Franel)",1,6,"G0(6)",chi_m3,{2:1,3:6,1:-2,6:-3},{1:3,6:9,2:-3,3:-9},[3],Fr(1),
   lambda n: sum(C(n,k)**3 for k in range(n+1))),
 ("Zagier B",1,9,"G0(9)",chi_m3,{1:3,3:-1},{9:3,1:-3},[1],Fr(1),
   lambda n: sum((-3)**(n-3*k)*factorial(n)//(factorial(n-3*k)*factorial(k)**3) for k in range(n//3+1))),
 ("Zagier C",1,6,"G0(6)",chi_m3,{3:1,2:6,1:-3,6:-2},{6:8,1:4,2:-8,3:-4},[2],Fr(1),
   lambda n: sum(C(n,k)**2*C(2*k,k) for k in range(n+1))),
 ("Zagier E",1,8,"G0(8)",chi_m4,{2:10,1:-4,4:-4},{1:4,4:2,8:4,2:-10},[2],Fr(1),
   lambda n: sum(4**(n-2*k)*C(n,2*k)*C(2*k,k)**2 for k in range(n//2+1))),
 ("Zagier F",1,6,"G0(6)",chi_m3,{1:6,6:1,2:-3,3:-2},{6:5,2:1,1:-5,3:-1},[1],Fr(1),
   lambda n: sum((-8)**(n-k)*C(n,k)*sum(C(k,j)**3 for j in range(k+1)) for k in range(n+1))),
 ("3F2(1/2,1/2,1/2;1,1|64t)",2,4,"G0(4)+4",chi_1,{2:20,1:-8,4:-8},{1:24,4:24,2:-48},[2],Fr(1,2),
   lambda n: C(2*n,n)**3),
 ("Apery (gamma)",2,6,"G0(6)+6",chi_1,{2:7,3:7,1:-5,6:-5},{1:12,6:12,2:-12,3:-12},[2,3],Fr(1),
   lambda n: sum(C(n,k)**2*C(n+k,k)**2 for k in range(n+1))),
 ("Almkvist (eta)",2,5,"G0(5)",chi_5,{1:5,5:-1},{5:6,1:-6},[1],Fr(1),
   lambda n: (-1)**n*sum((-1)**k*C(n,k)**3*(C(4*n-5*k-1,3*n)+C(4*n-5*k,3*n)) for k in range(n+1))),
 ("Domb (alpha)",2,6,"G0(6)+3",chi_1,{1:4,3:4,2:-2,6:-2},{2:6,6:6,1:-6,3:-6},[1,3],Fr(1),
   lambda n: (-1)**n*sum(C(n,k)**2*C(2*k,k)*C(2*n-2*k,n-k) for k in range(n+1))),
 ("Almkvist-Zudilin (delta)",2,6,"G0(6)+2",chi_1,{1:3,2:3,3:-1,6:-1},{3:4,6:4,1:-4,2:-4},[1,2],Fr(1),
   lambda n: sum((-3)**(n-3*k)*factorial(n+k)//(factorial(k)**4*factorial(n-3*k)) for k in range(n//3+1))),
 ("Almkvist (epsilon)",2,8,"G0(8)+8",chi_1,{2:6,4:6,1:-4,8:-4},{1:8,8:8,2:-8,4:-8},[2,4],Fr(1),
   lambda n: sum(C(n,k)**2*C(2*k,n)**2 for k in range(n+1))),
 ("Almkvist (zeta)",2,9,"G0(9)+9",chi_1,{3:10,1:-3,9:-3},{1:6,9:6,3:-12},[3],Fr(1),
   lambda n: sum(C(n,k)**2*C(n,l)*C(k,l)*C(k+l,n) for k in range(n+1) for l in range(n+1))),
]

def fam_series(f, M):
    name,w,N,grp,chi,thx,tx,zd,rth,an = f
    return etaseries(thx, M), etaseries(tx, M)

# ------------------------------------------------------------------ Check 1: parametrisations
def check_param():
    M = 60
```

```
    for f in CUSP:
        th, t = fam_series(f, M)
        A = [f[9](n) for n in range(M)]
        ok = (polyt(A, t, M) == th) and t[0] == 0 and t[1] == 1 and th[0] == 1
        report(f"[1] F(t(q)) = Theta(q) to O(q^{M}) for {f[0]}", ok)

# ------------------------------------------------------------------ Check 2: Ligozat orders at cusps
def ligozat(exps, N, d):
    # order of vanishing of prod eta(delta tau)^{r_delta} at a cusp a/d (d|N) of Gamma_0(N),
    # in the local uniformiser of X_0(N) at that cusp
    return Fr(N, 24)*sum(Fr(gcd(d, de)**2*r, gcd(d, N//d)*d*de) for de, r in exps.items())
def check_cusps():
    for f in CUSP:
        name,w,N,grp,chi,thx,tx,zd,rth,an = f
        divs = [d for d in range(1, N+1) if N % d == 0]
        oth = {d: ligozat(thx, N, d) for d in divs}
        ot  = {d: ligozat(tx, N, d) for d in divs}
        zeros = [d for d in divs if oth[d] > 0]
        poles = [d for d in divs if ot[d] < 0]
        ratios = {d: oth[d]/(-ot[d]) for d in zeros}
        weight_ok = Fr(sum(thx.values()), 2) == w and sum(tx.values()) == 0
        ok = weight_ok and zeros == zd and poles == zd and all(oth[d] >= 0 for d in divs) \
             and all(r == rth for r in ratios.values())
        # cusp-class condition (C): gcd(e, N/e) in {1,2}, or (N=9,e=3: the two cusps 1/3, 2/3 are
        # interchanged by w_9 and both have width 1)
        cond = all(gcd(e, N//e) in (1, 2) for e in zd) or (N == 9 and zd == [3])
        # character of the eta-quotient (Gordon-Hughes-Newman): chi(d) = ((-1)^w s / d), s = prod delta^
    r_delta
        s = Fr(1)
        for de, r in thx.items(): s *= Fr(de)**r
        D = (-1)**w * s.numerator * s.denominator          # same square class as (-1)^w s
        def legendre(a, p):
            a %= p
            return 0 if a == 0 else (1 if pow(a, (p-1)//2, p) == 1 else -1)
        char_ok = all(legendre(D, p) == chi(p) for p in [5, 7, 11, 13, 17, 19, 23, 29, 31, 37] if N % p and
    D % p)
        tchar = Fr(1)
        for de, r in tx.items(): tchar *= Fr(de)**r
        tchar_ok = all(legendre(tchar.numerator*tchar.denominator, p) == 1
                       for p in [5, 7, 11, 13, 17, 19, 23, 29, 31, 37] if N % p)
        # Atkin-Lehner symmetry: W_e maps eta_delta to eta_{e delta / gcd(e,delta)^2}
        al_ok = True
        if "+" in grp:
            e = int(grp.split("+")[1])
            for ex in (thx, tx):
                img = {}
                for de, r in ex.items():
                    d2 = e*de//gcd(e, de)**2
                    img[d2] = img.get(d2, 0) + r
                al_ok &= (img == ex)
        report(f"[2] zeros of Theta = poles of t = cusps with denominators {zd}, r_Theta = {rth}, (C) holds,
     "
                f"character and Atkin-Lehner symmetry ({grp}) ok: {name}",
                ok and cond and char_ok and tchar_ok and al_ok, f"ord Theta {dict(oth)}, ord t {dict(ot)}")

# ------------------------------------------------------------------ the trace
def trace_poly(f, p, P, M):
    """t-expansion (length M) of Tr(Q_P) = Q_P + p^{w+1} chi(p) U_p(G_P)."""
    name,w,N,grp,chi,thx,tx,zd,rth,an = f
    MZ = p*M
    th, t = fam_series(f, MZ)
    Pz = polyt(P, t, MZ)
    G = mul(mul(subqp(th[:M], p, MZ), inv(th, MZ), MZ), Pz, MZ)
    UpG = [G[p*n] for n in range(M)]
    thq, tq = th[:M], t[:M]
    Q = mul(mul(thq, inv(subqp(thq, p, M), M), M), subqp(Pz[:M], p, M), M)
    Tr = [Q[n] + p**(w+1)*chi(p)*UpG[n] for n in range(M)]
    return t_expand(Tr, tq, M)

def check_trace_lemma():
    rnd = random.Random(20261107)
    for f in CUSP:
        name,w,N,grp,chi,thx,tx,zd,rth,an = f
        A = [an(n) for n in range(200)]
```

```
        allok = True; info = []
        for p in [2, 3, 5, 7, 11]:
            if N % p == 0: continue
            cases = []
            for (s, m) in [(1, 1), (1, 2), (2, 1)]:
                if m*p**s <= 60: cases.append(("F_{%d}" % (m*p**(s-1)), A[:m*p**(s-1)]))
            for L in [3, 6]:
                cases.append(("random deg %d" % (L-1), [1] + [rnd.randint(-9, 9) for _ in range(L-1)]))
            for lab, P in cases:
                L = len(P)
                D = p*(L-1) + int(rth*(p-1))
                M = D + 20
                T = trace_poly(f, p, P, M)
                exact = all(x == 0 for x in T[D+1:])
                allok &= exact
                info.append(f"p={p},{lab}:deg<={D}:{'ok' if exact else 'NO'}")
        report(f"[3] Tr(Q_P) is a polynomial in t of degree <= p(L-1)+floor(r(p-1)) (exact, to t^(D+19)): {
    name}",
               allok, "; ".join(info[:6]) + (" ..." if len(info) > 6 else ""))

# ------------------------------------------------------------------ Check 4: main theorem
def cong_val(f, p, s, m, M):
    name,w,N,grp,chi,thx,tx,zd,rth,an = f
    th, t = fam_series(f, M)
    K = m*p**s
    A = [an(n) for n in range(K)]
    lhs = mul(th, polyt(A[:m*p**(s-1)], subqp(t, p, M), M), M)        # Theta(q) F_{K_{s-1}}(t(q^p))
    rhs = mul(subqp(th, p, M), polyt(A, t, M), M)                     # Theta(q^p) F_{K_s}(t(q))
    return vp_series([x-y for x, y in zip(lhs, rhs)], p)

def check_main():
    for f in CUSP:
        name,w,N,grp,chi,thx,tx,zd,rth,an = f
        allok = True; vals = []
        for p in [2, 3, 5, 7, 11, 13]:
            if N % p == 0: continue
            for (s, m) in [(1, 1), (1, 2), (2, 1), (2, 2), (3, 1)]:
                K = m*p**s
                if K > 130: continue
                M = min(2*K + 10, 200)
                v = cong_val(f, p, s, m, M)
                allok &= (v >= (w+1)*s)
                vals.append(f"p={p},s={s},m={m}:v={v}")
        report(f"[4] Theta(q)F_(mp^(s-1))(t(q^p)) = Theta(q^p)F_(mp^s)(t(q)) mod p^((w+1)s), w={w}: {name}",
               allok, " ".join(vals))

# ------------------------------------------------------------------ Check 5: coefficient corollary
def check_coeff():
    for f in CUSP:
        name,w,N,grp,chi,thx,tx,zd,rth,an = f
        ok = True; cnt = 0
        for p in [2, 3, 5, 7, 11, 13]:
            if N % p == 0: continue
            for s in [1, 2, 3]:
                for m in range(1, 6):
                    if m*p**s > 250: continue
                    cnt += 1
                    ok &= (an(m*p**s) - an(m*p**(s-1))) % p**((w+1)*s) == 0
        report(f"[5] a(mp^s) = a(mp^(s-1)) mod p^((w+1)s) ({cnt} cases): {name}", ok)

# ------------------------------------------------------------------ Check 6: BV3 Conjecture 7.5 in its own
    normalisation
def frob_canonical(rec, order_gap, M):
    """F_0(t) and canonical coordinate q(t) = t exp(F_1/F_0) from a recurrence
       n^k c_n = sum_j R_j(n) c_{n-j}, via the Frobenius method n -> n+eps (dual numbers)."""
    # rec(nn) returns (lead(nn), [(j, coeff(nn))...]) for nn a dual number (x, dx)
    def dm(x, y): return (x[0]*y[0], x[0]*y[1] + x[1]*y[0])
    cs = [(Fr(1), Fr(0))]
    for n in range(1, M):
        lead, terms = rec((Fr(n), Fr(1)))
        s = (Fr(0), Fr(0))
        for j, cf in terms:
            if n-j >= 0:
                pr = dm(cf, cs[n-j]); s = (s[0] + pr[0], s[1] + pr[1])
```

```
        if lead[0] == 0:
            raise ValueError
        cs.append((s[0]/lead[0], (s[1]*lead[0] - s[0]*lead[1])/(lead[0]**2)))
    F0 = [x[0] for x in cs]; F1 = [x[1] for x in cs]
    # g = F1/F0, E = exp(g)
    inv0 = [Fr(0)]*M; inv0[0] = Fr(1)
    for n in range(1, M): inv0[n] = -sum(F0[k]*inv0[n-k] for k in range(1, n+1))
    g = [sum(F1[i]*inv0[n-i] for i in range(n+1)) for n in range(M)]
    E = [Fr(0)]*M; E[0] = Fr(1)
    for n in range(1, M): E[n] = sum(k*g[k]*E[n-k] for k in range(1, n+1))/n
    q = [Fr(0)] + E[:M-1]
    return F0, q
def series_inverse_fn(q, M):
    """compositional inverse t(Q) of q(t) = t + ..."""
    T = [Fr(0)]*M; T[1] = Fr(1)
    for n in range(2, M):
        comp = [Fr(0)]*(n+1); Tp = [Fr(1)] + [Fr(0)]*n
        for k in range(1, n+1):
            Tp = [sum(Tp[i]*T[j-i] for i in range(j+1)) for j in range(n+1)]
            if q[k]:
                for j in range(n+1): comp[j] += q[k]*Tp[j]
        T[n] = -comp[n]
    return T
def fmul(a, b, M):
    c = [Fr(0)]*M
    for i in range(min(M, len(a))):
        if a[i] == 0: continue
        for j in range(min(M-i, len(b))):
            if b[j]: c[i+j] += a[i]*b[j]
    return c
def fcompose(a, b, M):          # a(b(t)), b(0)=0
    res = [Fr(0)]*M
    for cf in reversed(a[:M]):
        res = fmul(res, b, M); res[0] += cf
    return res
def finv(a, M):
    b = [Fr(0)]*M; b[0] = 1/a[0]
    for n in range(1, M): b[n] = -sum(a[k]*b[n-k] for k in range(1, n+1))/a[0]
    return b
def vpF(x, p):
    if x == 0: return 10**9
    return vp(x.numerator, p) - vp(x.denominator, p)
def bv_check(label, rec, gap, mBV, plist, slist, M, expect, tag="[6]"):
    F0, q = frob_canonical(rec, gap, M)
    T = series_inverse_fn(q, M)                         # mirror map t(Q)
    for p in plist:
        qp = [Fr(0)]*M
        qpow = [Fr(1)] + [Fr(0)]*(M-1)
        for _ in range(p): qpow = fmul(qpow, q, M)        # q(t)^p
        tsig = fcompose(T, qpow, M)                       # t^sigma(t) = t(q(t)^p)
        for s in slist:
            K = mBV*p**s
            if K >= M: continue
            Ft = F0; Fs = fcompose(F0, tsig, M)
            A = fcompose(F0[:mBV*p**(s-1)], tsig, M)
            lhs = fmul(Ft, A, M); rhs = fmul(Fs, F0[:K] + [Fr(0)]*(M-K), M)
            v = min(vpF(x-y, p) for x, y in zip(lhs, rhs))
            ok = v >= expect(s)
            report(f"{tag} BV3 (19) with m={mBV}, excellent lift q->q^p: {label}, p={p}, s={s}", ok,
                   f"v_p(F(t)F_(mp^(s-1))(t^s) - F(t^s)F_(mp^s)(t)) = {v} (required >= {expect(s)})")
def check_bv3():
    lin = lambda a0, a1, n: (a0 + a1*n[0], a1*n[1])
    def dmul(*xs):
        r = (Fr(1), Fr(0))
        for x in xs: r = (r[0]*x[0], r[0]*x[1] + r[1]*x[0])
        return r
    sc = lambda c, x: (c*x[0], c*x[1])
    # hypercubic = hyperoctahedral in dimension 2:  n^2 c_n = 16 (n-1)^2 c_{n-2}
    rec2 = lambda n: (dmul(n, n), [(2, sc(16, dmul(lin(-1, 1, n), lin(-1, 1, n))))])
    bv_check("hypercubic/hyperoctahedral n=2 (c_{2k}=C(2k,k)^2)", rec2, 2, 2, [3, 5, 7], [1, 2], 110, lambda
      s: 2*s)
    # hypercubic n=3:  n^3 c_n = 64 (n-1)^3 c_{n-2}
    rec3 = lambda n: (dmul(n, n, n), [(2, sc(64, dmul(lin(-1, 1, n), lin(-1, 1, n), lin(-1, 1, n))))])
    bv_check("hypercubic n=3 (c_{2k}=C(2k,k)^3)", rec3, 2, 2, [5, 7], [1, 2], 110, lambda s: 3*s)
```

```
# ------------------------------------------------------------------ Check 7: families outside the theorem
def elliptic_families(M):
    out = []
    a = cubic_theta(M)
    t27 = mul(etaseries({3:9, 1:-3}, M), inv(mul(mul(a, a, M), a, M), M), M)
    out.append(("2F1(1/3,2/3;1|27t)  [zero of Theta at elliptic point of G0(3)]", 1, 3, chi_m3, a, t27))
    h2 = etaseries({2:24, 1:-24}, M); d = [1] + [0]*(M-1)
    d = [d[i] + 64*h2[i] for i in range(M)]
    t256 = mul(h2, inv(mul(d, d, M), M), M)
    E22 = [1] + [24*(sigma(n) - (2*sigma(n//2) if n % 2 == 0 else 0)) for n in range(1, M)]
    out.append(("3F2(1/4,1/2,3/4;1,1|256t)  [elliptic, G0(2)+2]", 2, 2, chi_1, E22, t256))
    h3 = etaseries({3:12, 1:-12}, M); d = [1] + [0]*(M-1)
    d = [d[i] + 27*h3[i] for i in range(M)]
    t108 = mul(h3, inv(mul(d, d, M), M), M)
    E23 = [1] + [12*(sigma(n) - (3*sigma(n//3) if n % 3 == 0 else 0)) for n in range(1, M)]
    out.append(("3F2(1/3,1/2,2/3;1,1|108t)  [elliptic, G0(3)+3]", 2, 3, chi_1, E23, t108))
    return out
def check_ell_nonpoly():
    name, w, N, chi, th0, t0 = elliptic_families(400)[0]
    p, P = 5, [1]
    M = 60; MZ = p*M
    a = cubic_theta(MZ)
    t = mul(etaseries({3:9, 1:-3}, MZ), inv(mul(mul(a, a, MZ), a, MZ), MZ), MZ)
    G = mul(subqp(a[:M], p, MZ), inv(a, MZ), MZ)
    Q = mul(a[:M], inv(subqp(a[:M], p, M), M), M)
    Tr = [Q[n] + p**2*chi(p)*G[p*n] for n in range(M)]
    T = t_expand(Tr, t[:M], M)
    hi = [(n, vp(T[n], p)) for n in range(5, 40, 6)]
    report("[8h] for 2F1(1/3,2/3;1|27t), p=5: Tr(Q_1) is not a polynomial in t; "
           "its high t-coefficients are nonzero but p-adically small", any(T[n] != 0 for n in range(10, M-5)
    ),
           "(n, v_5 of coefficient): " + str(hi))


def check_hyperoct3():
    # [7c] hyperoctahedral family in dimension 3 (BV3 Conj. 7.5, outside Thm 1.1):
    #      c_{2k} = C(2k,k) sum_j C(k,j)^2 C(2j,j); in u = t^2 the recurrence
    #      k^3 a_k = 2(2k-1)(10k^2-10k+3) a_{k-1} - 36(k-1)(2k-1)(2k-3) a_{k-2}
    lin = lambda a0, a1, n: (a0 + a1*n[0], a1*n[1])
    def dmul(*xs):
        r = (Fr(1), Fr(0))
        for x in xs: r = (r[0]*x[0], r[0]*x[1] + r[1]*x[0])
        return r
    sc = lambda c, x: (c*x[0], c*x[1])
    quad = lambda n: (10*n[0]**2 - 10*n[0] + 3, (20*n[0] - 10)*n[1])
    rec = lambda n: (dmul(n, n, n), [(1, sc(2, dmul(lin(-1, 2, n), quad(n)))),
                                     (2, sc(-36, dmul(lin(-1, 1, n), lin(-1, 2, n), lin(-3, 2, n))))])
    F0, q = frob_canonical(rec, 1, 70)
    ok_seq = all(F0[k] == C(2*k, k)*sum(C(k, j)**2*C(2*j, j) for j in range(k+1)) for k in range(70))
    T = series_inverse_fn(q, 70)
    integral = all(x.denominator == 1 for x in T)
    M = 60
    uq = [int(x) for x in T[:M]]
    A = [int(x) for x in F0[:M]]
    th = polyt(A, uq, M)
    vals = []
    ok = ok_seq and integral
    for (p, s) in [(5, 1), (7, 1), (11, 1), (5, 2)]:
        K = p**s
        lhs = mul(th, polyt(A[:p**(s-1)], subqp(uq, p, M), M), M)
        rhs = mul(subqp(th, p, M), polyt(A[:K], uq, M), M)
        v = vp_series([x-y for x, y in zip(lhs, rhs)], p)
        ok &= (v >= 2*s)
        vals.append(f"p={p},s={s}:v={v}")
    report("[7a] (numerical, open case) hyperoctahedral n=3 of BV3 Conj. 7.5: congruence mod p^(2s) "
           "(the valuation is exactly 2s, not 3s)", ok, " ".join(vals))
    # [7b] modular structure: u = v/(1+v)^2, Theta = (1+v) Theta_Apery, v = (eta1 eta6/eta2 eta3)^12
    M = 60
    v = etaseries({1: 12, 6: 12, 2: -12, 3: -12}, M)
    opv = [(1 if i == 0 else 0) + v[i] for i in range(M)]
    u = mul(v, inv(mul(opv, opv, M), M), M)
    thh = mul(opv, etaseries({2: 7, 3: 7, 1: -5, 6: -5}, M), M)
    ok7b = (u == uq[:M]) and (t_expand(thh, u, M) == A[:M])
```

```
    report("[7b] hyperoctahedral n=3: mirror map u = v/(1+v)^2 and F(u(q)) = (1+v) eta2^7 eta3^7/(eta1^5
     eta6^5), "
           "v = (eta1 eta6/(eta2 eta3))^12", ok7b)
    # [7c] the truncation coefficients have only one carry per digit p-1: v_p(c_(p^k-1)) = k
    bz = lambda n: sum(C(n, j)**2*C(2*j, j) for j in range(n+1))
    ok7c = all(vp(bz(p**k - 1), p) == 0 and vp(C(2*(p**k-1), p**k-1)*bz(p**k-1), p) == k
               for p in [5, 7, 11, 13] for k in [1, 2])
    report("[7c] hyperoctahedral n=3: v_p(c_(p^k-1)) = k for p in {5,7,11,13}, k in {1,2} "
           "(only one factor p per digit p-1, so the carry bound of Section 6 fails)", ok7c)


# ------------------------------------------------------------------ Check 8: families with an elliptic zero

def normseries(f, p, M):
    """prod_{k mod p} f(zeta^k z) as a series in q = z^p (f in 1 + zZ[[z]] given to precision p*M)."""
    MZ = p*M
    F = [Fr(x) for x in f[:MZ]]
    finv = [Fr(0)]*MZ; finv[0] = Fr(1)
    for n in range(1, MZ):
        finv[n] = -sum(F[k]*finv[n-k] for k in range(1, n+1) if F[k])
    zl = [sum(i*F[i]*finv[n-i] for i in range(1, n+1) if F[i]) for n in range(MZ)]   # z f'/f
    g = [Fr(0)] + [p*zl[p*n]/(p*n) for n in range(1, M)]                             # log N
    E = [Fr(0)]*M; E[0] = Fr(1)
    for n in range(1, M): E[n] = sum(k*g[k]*E[n-k] for k in range(1, n+1))/n
    assert all(x.denominator == 1 for x in E)
    return [int(x) for x in E]

def ell_data(name, M):
    """(w, chi, Theta, t, vartheta, k0, ord, lam, coefficient formula) for the three families."""
    if name == "27t":
        a = cubic_theta(M)
        t = mul(etaseries({3: 9, 1: -3}, M), inv(mul(mul(a, a, M), a, M), M), M)
        return dict(w=1, chi=chi_m3, th=a, t=t, vth=a, k0=3, ordz=1, lam=1,
                    an=lambda n: factorial(3*n)//factorial(n)**3, N=3)
    if name == "256t":
        h2 = etaseries({2: 24, 1: -24}, M); d = [1] + [0]*(M-1)
        d = [d[i] + 64*h2[i] for i in range(M)]
        t = mul(h2, inv(mul(d, d, M), M), M)
        E = [1] + [24*(sigma(n) - (2*sigma(n//2) if n % 2 == 0 else 0)) for n in range(1, M)]
        return dict(w=2, chi=chi_1, th=E, t=t, vth=E, k0=4, ordz=1, lam=1,
                    an=lambda n: factorial(4*n)//factorial(n)**4, N=2)
    if name == "108t":
        h3 = etaseries({3: 12, 1: -12}, M); d = [1] + [0]*(M-1)
        d = [d[i] + 27*h3[i] for i in range(M)]
        t = mul(h3, inv(mul(d, d, M), M), M)
        E = [1] + [12*(sigma(n) - (3*sigma(n//3) if n % 3 == 0 else 0)) for n in range(1, M)]
        a = cubic_theta(M)
        return dict(w=2, chi=chi_1, th=E, t=t, vth=a, k0=6, ordz=2, lam=1, vchi=chi_m3,
                    an=lambda n: C(2*n, n)*factorial(3*n)//factorial(n)**3, N=3)
    raise ValueError

ELL = ["27t", "256t", "108t"]

def check_ell_param():
    M = 60
    for nm in ELL:
        D = ell_data(nm, M)
        F = t_expand(D['th'], D['t'], M)
        ok = all(F[n] == D['an'](n) for n in range(M))
        report(f"[8a] F(t(q)) = Theta(q) to O(q^{M}) for {nm}", ok)
    # eta identities giving the zeros of Theta and the pole of t
    a = cubic_theta(M)
    b3 = etaseries({1: 9, 3: -3}, M); c3 = [27*x for x in etaseries({3: 9, 1: -3}, M)]
    D = ell_data("27t", M)
    ok1 = mul(mul(a, a, M), a, M) == [x + y for x, y in zip(b3, c3)]
    ok2 = mul(mul(mul(a, a, M), a, M), D['t'], M) == etaseries({3: 9, 1: -3}, M)
    report("[8a] 27t: a^3 = eta1^9/eta3^3 + 27 eta3^9/eta1^3 and a^3 t = eta3^9/eta1^3 (no zeros in H)", ok1
      and ok2)
    D = ell_data("256t", M)
    E2 = mul(D['th'], D['th'], M)
    ok1 = E2 == [x + 64*y for x, y in zip(etaseries({1: 16, 2: -8}, M), etaseries({2: 16, 1: -8}, M))]
    ok2 = mul(mul(E2, E2, M), D['t'], M) == etaseries({1: 8, 2: 8}, M)
    report("[8a] 256t: E22^2 = eta1^16/eta2^8 + 64 eta2^16/eta1^8 and E22^4 t = eta1^8 eta2^8", ok1 and ok2)
    D = ell_data("108t", M)
```

```
    ok1 = D['th'] == mul(a, a, M)
    ok2 = mul(mul(mul(D['th'], D['th'], M), D['th'], M), D['t'], M) == etaseries({1: 6, 3: 6}, M)
    report("[8a] 108t: E23 = a^2 and E23^3 t = eta1^6 eta3^6", ok1 and ok2)
    # floating point: Theta vanishes at the elliptic point e
    def qs(c, tau, n=200):
        q = cmath.exp(2j*cmath.pi*tau)
        return sum(c[k]*q**k for k in range(min(n, len(c))))
    e3 = complex(0.5, 1/(2*3**0.5)); e4 = complex(0.5, 0.5)
    vals = []
    for nm, e in [("27t", e3), ("256t", e4), ("108t", e3)]:
        D = ell_data(nm, 200)
        vals.append(abs(qs(D['th'], e)))
    report("[8a] |Theta(e)| at e=(3+sqrt-3)/6, (1+i)/2, (3+sqrt-3)/6 (floating point)", max(vals) < 1e-12,
           " ".join(f"{v:.1e}" for v in vals))

def Dpoly(D, p, M):
    """the polynomial D(t) = chi'(p) vth(q^p) N(vth)(q) / vth(q)^(p+1), expanded in t."""
    vth = D['vth']; MZ = p*M
    Nm = normseries(vth[:MZ] if len(vth) >= MZ else vth, p, M)
    num = mul(subqp(vth[:M], p, M), Nm, M)
    den = [1] + [0]*(M-1)
    for _ in range(p+1): den = mul(den, vth[:M], M)
    c = D.get('vchi', D['chi'])(p)
    return t_expand([c*x for x in mul(num, inv(den, M), M)], D['t'][:M], M)

def deg_tail(f, upto):
    nz = [n for n in range(upto) if f[n] != 0]
    return max(nz) if nz else -1

def check_ell_D():
    for nm in ELL:
        details = []; ok = True
        for p in [5, 7, 11, 13]:
            M = 36
            D = ell_data(nm, p*M)
            Dt = Dpoly(D, p, M)
            dg = deg_tail(Dt, M - 2)
            eps = Dt[0]
            dstar = Fr((p+1)*D['lam'], D['k0'])
            good = eps in (1, -1) and all((Dt[n] - (eps if n == 0 else 0)) % p == 0 for n in range(M)) and
     dg <= dstar
            if nm == "108t":      # also B_Theta = D^2
                Bt = Dpoly(dict(D, vth=D['th'], vchi=D['chi']), p, M)
                good &= Bt[:M-2] == mul(Dt, Dt, M)[:M-2]
            ok &= good
            details.append(f"p={p}:deg={dg}<={dstar},D(0)={eps}")
        report(f"[8b] D(t) in Z[t], D = +-1 mod p, deg D <= (p+1)lam/k0{' and B = D^2' if nm=='108t' else
     ''}: {nm}",
               ok, " ".join(details))

def sym_functions(D, p, M):
    """q-series of e_k (k=0..p+1), n_l (l=0..p), S_0 for the pairs (x_beta, W_beta)."""
    w = D['w']; chi = D['chi'](p); th = D['th']; t = D['t']; MZ = p*M
    x0 = mul(th[:M], inv(subqp(th[:M], p, M), M), M)
    W0 = subqp(t[:M], p, M)
    base = mul(subqp(th[:M], p, MZ), inv(th[:MZ], MZ), MZ)
    tj = [1] + [0]*(MZ-1)
    Pw = [None]; X = []
    for j in range(p+1):
        Gj = mul(base, tj, MZ)
        X.append([p**(w+1)*chi*Gj[p*n] for n in range(M)])
        if j >= 1: Pw.append([p*tj[p*n] for n in range(M)])
        tj = mul(tj, t[:MZ], MZ)
    # elementary symmetric functions of the p non-principal W's (Newton)
    e = [[Fr(1)] + [Fr(0)]*(M-1)]
    for j in range(1, p+1):
        acc = [Fr(0)]*M
        for i in range(1, j+1):
            pr = mul(e[j-i], Pw[i], M)
            sg = 1 if i % 2 == 1 else -1
            acc = [a_ + sg*b_ for a_, b_ in zip(acc, pr)]
        e.append([x/j for x in acc])
    assert all(x.denominator == 1 for row in e for x in row)
    e = [[int(x) for x in row] for row in e] + [[0]*M]
```

```
    eall = [[e[k][n] + (mul(W0, e[k-1], M)[n] if k >= 1 else 0) for n in range(M)] for k in range(p+2)]
    Y = []
    for l in range(p+1):
        acc = [0]*M
        for j in range(l+1):
            pr = mul(e[l-j], X[j], M)
            acc = [a_ + (-1)**j*b_ for a_, b_ in zip(acc, pr)]
        Y.append(acc)
    n = []
    for l in range(p+1):
        v = mul(x0, e[l], M)
        v = [a_ + b_ for a_, b_ in zip(v, Y[l])]
        if l >= 1: v = [a_ + b_ for a_, b_ in zip(v, mul(W0, Y[l-1], M))]
        n.append(v)
    S0 = [a_ + b_ for a_, b_ in zip(x0, X[0])]
    return eall, n, S0

def check_ell_sym():
    for nm in ELL:
        ok = True; det = []
        for p in [5, 7]:
            M = p + 16
            D = ell_data(nm, p*M)
            k0, o, lam = D['k0'], D['ordz'], D['lam']
            mu, e0 = k0//lam, o//lam
            Dt = Dpoly(D, p, M)
            Dq = polyt(Dt[:deg_tail(Dt, M-2)+1], D['t'][:M], M)
            Dmu = [1] + [0]*(M-1)
            for _ in range(mu): Dmu = mul(Dmu, Dq, M)
            De0 = [1] + [0]*(M-1)
            for _ in range(e0): De0 = mul(De0, Dq, M)
            eall, nl, S0 = sym_functions(D, p, M)
            tq = D['t'][:M]
            Ek = [t_expand(mul(ek, Dmu, M), tq, M) for ek in eall]
            Ml = [t_expand(mul(x, Dmu, M), tq, M) for x in nl]
            A0 = t_expand(mul(S0, De0, M), tq, M)
            lim = M - 2
            g1 = all(deg_tail(E, lim) <= p+1 for E in Ek[1:])
            g2 = Ek[1][p+1] % p == 0
            g3 = all(deg_tail(x, lim) <= p for x in Ml)
            g4 = deg_tail(A0, lim) <= floor(Fr(p*o, k0))
            # mod p: E_1 = eps^mu t^p
            eps = Dt[0]
            g5 = all((Ek[1][n] - (eps**mu if n == p else 0)) % p == 0 for n in range(lim))
            ok &= g1 and g2 and g3 and g4 and g5
            det.append(f"p={p}:degE<=p+1:{g1},[t^(p+1)]E1=0modp:{g2},degM<=p:{g3},degA0<=floor(pr):{g4},E1
     =+-t^p mod p:{g5}")
        report(f"[8c] polynomiality and degrees of E_k = e_k D^k0, M_l = n_l D^k0, A_0 = S_0 D^o (Section 6)
     : {nm}", ok, "; ".join(det))

def monotraces(D, p, imax, M):
    w = D['w']; c = D['chi'](p); th = D['th']; t = D['t']; MZ = p*M
    x0 = mul(th[:M], inv(subqp(th[:M], p, M), M), M)
    tp = subqp(t[:M], p, M)
    base = mul(subqp(th[:M], p, MZ), inv(th[:MZ], MZ), MZ)
    out = []; ti = [1] + [0]*(MZ-1); tpi = [1] + [0]*(M-1)
    for i in range(imax+1):
        Q = mul(x0, tpi, M); G = mul(base, ti, MZ)
        out.append(t_expand([Q[n] + p**(w+1)*c*G[p*n] for n in range(M)], t[:M], M))
        ti = mul(ti, t[:MZ], MZ); tpi = mul(tpi, tp, M)
    return out

def check_ell_decay():
    for nm in ELL:
        ok = True; det = []
        for p in [5, 7, 11, 13]:
            imax = 3 if p < 11 else 2
            M = p*imax + 50
            D = ell_data(nm, p*M)
            r = Fr(D['ordz'], D['k0']); dstar = Fr((p+1)*D['lam'], D['k0']); c0 = floor(p*r)
            S = monotraces(D, p, imax, M)
            slack = 10**9
            for i in range(imax+1):
                for n in range(M):
```

```
                    if S[i][n] == 0: continue
                    b = ceil(Fr(n - p*i - c0)/dstar)
                    if vp(S[i][n], p) < b: ok = False
                    if n >= p*i + p: slack = min(slack, vp(S[i][n], p) - b)
            det.append(f"p={p}:min slack={slack}")
        report(f"[8d] decay estimate v_p([t^n]T_p(t^i)) >= (n-pi-floor(pr))/delta*: {nm}", ok, " ".join(det)
    )

def check_ell_cong():
    for nm in ELL:
        ok = True; det = []
        for p in [5, 7, 11, 13]:
            for (s, m) in [(1, 1), (1, 2), (1, 3), (2, 1), (2, 2), (3, 1)]:
                K = m*p**s
                if K > 130: continue
                M = K + 25
                D = ell_data(nm, M)
                w = D['w']; th = D['th'][:M]; t = D['t'][:M]
                A = [D['an'](n) for n in range(K)]
                lhs = mul(th, polyt(A[:m*p**(s-1)], subqp(t, p, M), M), M)
                rhs = mul(subqp(th, p, M), polyt(A, t, M), M)
                v = vp_series([x-y for x, y in zip(lhs, rhs)], p)
                ok &= v >= (w+1)*s
                det.append(f"p={p},s={s},m={m}:v={v}")
        report(f"[8e] Theorem B: Theta(q)F_(mp^(s-1))(t(q^p)) = Theta(q^p)F_(mp^s)(t(q)) mod p^((w+1)s): {nm
    }",
               ok, " ".join(det))

def check_ell_carry():
    # the inequalities used in the proof of Theorem B, for p < 200 and digit counts k <= 12
    ok = True
    for nm, c, L0, e0, r in [("27t", 2, 3, 1, Fr(1, 3)), ("256t", 3, 4, 1, Fr(1, 4)), ("108t", 3, 6, 2, Fr(1,
     3))]:
        for p in [x for x in range(5, 200) if all(x % d for d in range(2, x))]:
            dstar = Fr(p+1, L0)
            for k in range(0, 13):
                j = 0 if k == 0 else p**(k-1)
                dec = ceil(Fr(p*(j+1) - floor(p*r))/dstar)
                ok &= dec >= c*(k+1)
        # carry bound per digit p-1
        an = ell_data(nm, 10)['an']
        for p in [5, 7, 11, 13]:
            for n in [p**3 - 1, 2*p**3 - 1, p**3 - 2, p**2 - 1 + 3*p**3]:
                digits = 0; x = n
                while x:
                    digits += (x % p == p-1); x //= p
                ok &= vp(an(n), p) >= c*digits
    report("[8f] arithmetic of the induction: decay at offset p(j+1) >= c(k+1) (p<200, k<=12) and "
           "v_p(a_n) >= c * #(digits p-1)", ok)

def check_hyperoct3_termwise():
    # [7e] the decay estimate holds for the hyperoctahedral family (k0=2, o=1, r=1/2), but the termwise bound
    #      of Section 6 is too weak for s=3: it gives 5 < 6, while the tail of the trace is divisible by p^6
    M0 = 200
    v = etaseries({1: 12, 6: 12, 2: -12, 3: -12}, 5*M0)
    opv = [(1 if i == 0 else 0) + v[i] for i in range(5*M0)]
    u = mul(v, inv(mul(opv, opv, 5*M0), 5*M0), 5*M0)
    th = mul(opv, etaseries({2: 7, 3: 7, 1: -5, 6: -5}, 5*M0), 5*M0)
    D = dict(w=2, chi=chi_1, th=th, t=u)
    an = lambda n: C(2*n, n)*sum(C(n, j)**2*C(2*j, j) for j in range(n+1))
    p, s, m = 5, 3, 1
    Ks, Ks1 = m*p**s, m*p**(s-1)
    M = Ks + 25
    S = monotraces(D, p, Ks1 - 1, M)
    a = [an(i) for i in range(Ks1)]
    dstar, c0 = Fr(p+1, 2), p//2
    de = all(S[i][n] == 0 or vp(S[i][n], p) >= ceil(Fr(n - p*i - c0)/dstar) for i in range(Ks1) for n in
     range(M))
    tw = min(min(vp(a[i], p) + vp(S[i][n], p) for i in range(Ks1)) for n in range(Ks, M))
    tr = min(vp(sum(a[i]*S[i][n] for i in range(Ks1)), p) for n in range(Ks, M))
    # [7d] degree of the polynomial D: (p-1)/2 for p = 1 mod 3 and (p+1)/2 for p = 2 mod 3 (CM by Q(sqrt(-3))
     )
    degs = []
    for pp in [5, 7, 11, 13]:
```

```
        MM = 30
        Dt = Dpoly(dict(vth=th, chi=chi_1, t=u), pp, MM)
        dg = deg_tail(Dt, MM - 2)
        degs.append((pp, dg, (pp - 1)//2 if pp % 3 == 1 else (pp + 1)//2))
    report("[7d] hyperoctahedral n=3: deg D = (p-1)/2 if p = 1 mod 3 and (p+1)/2 if p = 2 mod 3, p in
     {5,7,11,13}",
           all(x[1] == x[2] for x in degs), str(degs))
    report("[7e] hyperoctahedral n=3, p=5, s=3: decay estimate holds; termwise bound min_i v(a_i)+v([u^n]S_i
     ) "
           "over n in [125,150) is 5 < 6, but the tail of Tr(Q_P) has valuation >= 6", de and tw == 5 and tr
      >= 6,
           f"decay ok={de}, termwise={tw}, true={tr}")

def check_bv_simplicial():
    lin = lambda a0, a1, n: (a0 + a1*n[0], a1*n[1])
    def dmul(*xs):
        r = (Fr(1), Fr(0))
        for x in xs: r = (r[0]*x[0], r[0]*x[1] + r[1]*x[0])
        return r
    sc = lambda c, x: (c*x[0], c*x[1])
    rec2 = lambda n: (dmul(n, n), [(3, sc(27, dmul(lin(-1, 1, n), lin(-2, 1, n))))])
    bv_check("simplicial n=2 (c_{3k}=(3k)!/k!^3)", rec2, 3, 3, [5, 7], [1, 2], 110, lambda s: 2*s, "[8g]")
    rec3 = lambda n: (dmul(n, n, n), [(4, sc(256, dmul(lin(-1, 1, n), lin(-2, 1, n), lin(-3, 1, n))))])
    bv_check("simplicial n=3 (c_{4k}=(4k)!/k!^4)", rec3, 4, 4, [5, 7], [1, 2], 110, lambda s: 3*s, "[8g]")
    # the modular t(q) of 27t / 256t is the mirror map:  t_0(Q)^(n+1) = t(Q^(n+1))
    for nm, rec, k in [("27t", rec2, 3), ("256t", rec3, 4)]:
        M = 61
        F0, q = frob_canonical(rec, k, M)
        T = series_inverse_fn(q, M)
        Tk = [Fr(1)] + [Fr(0)]*(M-1)
        for _ in range(k): Tk = fmul(Tk, T, M)
        D = ell_data(nm, M)
        tk = subqp(D['t'][:M], k, M)
        report(f"[8g] mirror map: t_0(Q)^{k} = t(Q^{k}) for the simplicial family attached to {nm}",
               all(Tk[n] == tk[n] for n in range(M)))


# ------------------------------------------------------------------ Check 9: Zagier's case D


def leg5(n):
    r = n % 5
    return 0 if r == 0 else (1 if r in (1, 4) else -1)
def tD(M):
    """t = q prod (1-q^n)^(5 (n/5))"""
    P = [0]*M; P[0] = 1
    for n in range(1, M):
        e = 5*leg5(n)
        for _ in range(abs(e)):
            if e > 0:
                for i in range(M-1, n-1, -1): P[i] -= P[i-n]
            else:
                for i in range(n, M): P[i] += P[i-n]
    return [0] + P[:M-1]
def thetaD(M):
    """Theta = prod_{n = +-1 mod 5} (1-q^n)^(-3) prod_{other n} (1-q^n)^2"""
    P = [0]*M; P[0] = 1
    for n in range(1, M):
        e = -3 if n % 5 in (1, 4) else 2
        for _ in range(abs(e)):
            if e > 0:
                for i in range(M-1, n-1, -1): P[i] -= P[i-n]
            else:
                for i in range(n, M): P[i] += P[i-n]
    return P
aD = lambda n: sum(C(n, k)**2*C(n+k, k) for k in range(n+1))

# Laurent series: (v, a) = q^v * sum_i a[i] q^i
def lmul(A, B, M): return (A[0] + B[0], mul(A[1], B[1], M))
def ladd(A, B, M):
    v = min(A[0], B[0]); out = [0]*M
    for (w, a) in (A, B):
        for i, x in enumerate(a):
            if 0 <= i + w - v < M: out[i + w - v] += x
```

```
    return (v, out)
def lscale(A, c): return (A[0], [c*x for x in A[1]])
def lUp(A, p, M):
    v, a = A
    v0 = -((-v)//p)                      # ceil(v/p)
    return (v0, [a[p*n - v] if 0 <= p*n - v < len(a) else 0 for n in range(v0, v0 + M)])
def lsubqp(A, p, M):
    v, a = A
    return (p*v, subqp(a, p, M))
def to_t(A, t, M):
    """Laurent q-series -> dict {j: c_j} with A = sum_j c_j t^j (t = q u, u a unit), M relative terms"""
    v, a = A
    u = t[1:] + [0]
    if v < 0:
        uu = [1] + [0]*(M-1)
        for _ in range(-v): uu = mul(uu, u, M)       # A t^{-v} = a u^{-v} (power series)
        f = mul(a, uu, M)
    else:
        f = [0]*v + a[:M-v]
    c = t_expand(f, t[:M], M)
    sh = v if v < 0 else 0
    return {j + sh: c[j] for j in range(M) if c[j]}
def Pof(P, X, M):                                # P(X) for a Laurent series X
    res = (0, [0]*M)
    pw = (0, [1] + [0]*(M-1))
    for cf in P:
        res = ladd(res, lscale(pw, cf), M) if cf else res
        pw = lmul(pw, X, M)
    return res

def zd_series(M):
    t = tD(M); th = thetaD(M)
    u = t[1:] + [0]
    tinv = (-1, inv(u, M))                       # 1/t
    return t, th, tinv

def check_zd_param():
    M = 80
    t, th, tinv = zd_series(M)
    A = [aD(n) for n in range(M)]
    ok1 = polyt(A, t, M) == th
    ok2 = mul(mul(th, th, M), t, M) == etaseries({5: 5, 1: -1}, M)
    # 1/t - 11 - t = eta1^6/eta5^6, i.e. (1 - 11 t - t^2) = t eta1^6/eta5^6
    lhs3 = [(1 if n == 0 else 0) - 11*t[n] - mul(t, t, M)[n] for n in range(M)]
    # t * eta1^6/eta5^6 = (t/q) * prod (1-q^n)^6 (1-q^(5n))^(-6)
    u = t[1:] + [0]
    P6 = [0]*M; P6[0] = 1
    for nn in range(1, M):
        for e, mm in ((6, 1), (-6, 5)):
            k = mm*nn
            if k >= M: continue
            for _ in range(abs(e)):
                if e > 0:
                    for i in range(M-1, k-1, -1): P6[i] -= P6[i-k]
                else:
                    for i in range(k, M): P6[i] += P6[i-k]
    ok3 = lhs3[:M-1] == mul(u, P6, M)[:M-1]          # u = t/q is known to O(q^(M-1))
    # Ligozat: eta5^5/eta1 has order 1 at the cusp infinity of X_0(5) (width 1) and order 0 at the cusp 0
    ok3 = ok3 and ligozat({5: 5, 1: -1}, 5, 5) == 1 and ligozat({5: 5, 1: -1}, 5, 1) == 0
    report("[9a] Zagier D: F(t(q)) = Theta(q), Theta^2 t = eta5^5/eta1 and 1/t - 11 - t = eta1^6/eta5^6 to O
     (q^79); orders of eta5^5/eta1 at the cusps of X_0(5) are 1, 0",
           ok1 and ok2 and ok3)
    # diamond operators (floating point): t o<d> and Theta|<d> for d = 2, 3, 4
    MM = 400
    tt, thh, _ = zd_series(MM)
    def ev(c, tau):
        q = cmath.exp(2j*cmath.pi*tau)
        return sum(c[k]*q**k for k in range(MM))
    res = []
    for (sig, d) in [((3, 1, 5, 2), 2), ((2, 1, 5, 3), 3), ((4, 1, 15, 4), 4)]:
        a, b, c, dd = sig
        tau = complex(-0.4, 0.2) if d != 4 else complex(-4/15, 0.07)
        st = (a*tau + b)/(c*tau + dd)
        tv, tsv = ev(tt, tau), ev(tt, st)
```

```
        thv, thsv = ev(thh, tau), ev(thh, st)
        if d in (2, 3):
            res.append((tv*tsv, thsv/((c*tau + dd)*tv*thv)))
        else:
            res.append((tsv/tv, thsv/((c*tau + dd)*thv)))
    # the modular t is the mirror map of the Apery zeta(2) equation:  n^2 a_n = (11n^2-11n+3) a_(n-1) + (n-1)
     ^2 a_(n-2)
    def dm(x, y): return (x[0]*y[0], x[0]*y[1] + x[1]*y[0])
    def rec(n):
        n2 = dm(n, n)
        quad = (11*n2[0] - 11*n[0] + 3, 11*n2[1] - 11*n[1])
        nm1 = (n[0] - 1, n[1])
        return n2, [(1, quad), (2, dm(nm1, nm1))]
    F0, qc = frob_canonical(rec, 2, 50)
    T = series_inverse_fn(qc, 50)
    okm = all(F0[n] == aD(n) for n in range(50)) and all(T[n] == tD(50)[n] for n in range(50))
    report("[9a] Zagier D: the canonical coordinate of the Apery zeta(2) equation inverts t(q), i.e. t is
     the mirror map", okm)
    c2 = res[0][1].real
    ok = (abs(res[0][0] + 1) < 1e-9 and abs(res[1][0] + 1) < 1e-9 and abs(res[2][0] - 1) < 1e-9
          and abs(abs(c2) - 1) < 1e-9 and abs(res[1][1] + c2) < 1e-9 and abs(res[2][1] + 1) < 1e-9)
    report("[9a] diamond operators on Gamma_1(5): t o<2> = t o<3> = -1/t, Theta|<2> = c t Theta, "
           "Theta|<3> = -c t Theta, t o<4> = t, Theta|<4> = -Theta (floating point)", ok,
           f"c = {c2:+.12f}; t(<2>tau)t(tau) = {res[0][0]:.3e}")
    return round(c2)

def traces_zd(p, P, M, c):
    """exact Laurent q-series of the two Hecke traces for p = +-2 mod 5, or the trace for p = +-1 mod 5"""
    D = len(P) - 1
    MZ = p*(M + D + 3)
    t, th, tinv = zd_series(MZ)
    tq, thq = t[:M], th[:M]
    _, _, tinvq = zd_series(M)
    thinvZ = inv(th, MZ)
    Gz = mul(mul(subqp(th, p, MZ), thinvZ, MZ), polyt(P, t, MZ), MZ)
    UG = lUp((0, Gz), p, M)
    Q = (0, mul(mul(thq, inv(subqp(thq, p, M), M), M), polyt(P, subqp(tq, p, M), M), M))
    if p % 5 in (1, 4):
        eta1 = 1 if p % 5 == 1 else -1
        return ladd(lscale(Q, eta1), lscale(UG, p*p), M)
    eta = c if p % 5 == 2 else -c
    # Qcheck = Theta P(-1/t(q^p)) / (t(q^p) Theta(q^p))
    tinv_p = lsubqp(tinvq, p, M)
    Qc = lmul(lmul((0, thq), Pof(P, lscale(tinv_p, -1), M), M),
              lmul(tinv_p, (0, inv(subqp(thq, p, M), M)), M), M)
    Lam = ladd(lscale(Qc, eta), lscale(UG, p*p), M)
    # Gcheck(z) = Theta(z^p) P(-1/t(z)) / (t(z) Theta(z))
    tinvZ = (-1, inv(t[1:] + [0], MZ))
    Gc = lmul(lmul((0, subqp(th, p, MZ)), Pof(P, lscale(tinvZ, -1), MZ), MZ),
              lmul(tinvZ, (0, thinvZ), MZ), MZ)
    UGc = lUp(Gc, p, M)
    Lamc = ladd(lscale(lmul((0, tq), Q, M), -eta), lscale(lmul((0, tq), UGc, M), p*p), M)
    return Lam, Lamc

def check_zd_traces(c):
    random.seed(5)
    ok = True; det = []
    for p in [2, 3, 7, 13, 17, 11, 19]:
        for P in [[1], [1, 3, 19], [random.randint(-9, 9) for _ in range(4)]]:
            if P[-1] == 0: P[-1] = 1
            D = len(P) - 1; K = p*(D + 1)
            M = K + 30
            out = traces_zd(p, P, M, c)
            if p % 5 in (1, 4):
                Ld = to_t(out, zd_series(M)[0], M - 5)
                good = all(j >= 0 for j in Ld) and max(Ld) <= K - 1
                det.append(f"p={p},D={D}:poly deg {max(Ld)}<={K-1}" if good else f"p={p}:FAIL")
            else:
                Lam, Lamc = out
                t = zd_series(M)[0]
                L = to_t(Lam, t, M - 5)
                Lc = to_t(Lamc, t, M - 5)
                topc = max(Lc); bot = min(L)
                # Laurent polynomials: L supported in [-K, (D+1)//p - 1], Lc in [1-(D+1)//p, K]
```

```
                good = topc <= K and bot >= -K
                good &= max(L) <= (D + 1)//p - 1 + 0 and min(Lc) >= 1 - (D + 1)//p
                # Lc(t) = L(-1/t): coefficient of t^j in Lc equals (-1)^j * coefficient of t^(-j) in L
                good &= all(Lc.get(j, 0) == (-1)**j*L.get(-j, 0) for j in range(-K, K + 1))
                det.append(f"p={p},D={D}:top(Lc)={topc}<={K}")
            ok &= good
    report("[9b] Hecke traces on Gamma_1(5): p=+-1: eta'Q+p^2U_pG polynomial of degree<p(D+1); "
           "p=+-2: Lambda = eta Qcheck + p^2 U_pG and Lambda' = -eta t Q + p^2 t U_p Gcheck are Laurent "
           "polynomials, deg Lambda' <= p(D+1), Lambda'(t) = Lambda(-1/t)", ok, " ".join(det[:12]) + " ...")

def check_zd_cong():
    ok = True; det = []
    for p in [2, 3, 7, 11, 13, 17]:
        for (s, m) in [(1, 1), (1, 2), (2, 1), (2, 2), (3, 1)]:
            K = m*p**s
            if K > 125: continue
            Kp = m*p**(s-1)
            M = K + 20
            t, th, tinv = zd_series(M)
            A = [aD(n) for n in range(K)]
            lhs = mul(th, polyt(A[:Kp], subqp(t, p, M), M), M)
            rhs = mul(subqp(th, p, M), polyt(A, t, M), M)
            vH = vp_series([x - y for x, y in zip(lhs, rhs)], p)
            # dual congruence: t Theta F_(K')(-1/t(q^p)) / (t(q^p) Theta(q^p))  =  F_K(-1/t(q))
            tinv_p = lsubqp(tinv, p, M)
            L1 = lmul(lmul((0, mul(t, th, M)), Pof(A[:Kp], lscale(tinv_p, -1), M), M),
                      lmul(tinv_p, (0, inv(subqp(th, p, M), M)), M), M)
            R1 = Pof(A, lscale(tinv, -1), M)
            Dif = ladd(L1, lscale(R1, -1), M)
            vD = min(vp(x, p) for x in Dif[1][:M - 2*K//p - 5]) if any(Dif[1]) else 10**9
            ok &= vH >= 2*s and vD >= 2*s
            det.append(f"p={p},s={s},m={m}:v(H)={vH},v(D)={vD}")
    report("[9c] Theorem C: (H_s) and the dual congruence (D_s) modulo p^(2s) for Zagier D", ok, " ".join(
     det))


if __name__ == "__main__":
    check_param()
    check_cusps()
    check_trace_lemma()
    check_main()
    check_coeff()
    check_bv3()
    check_hyperoct3()
    check_hyperoct3_termwise()
    check_ell_param()
    check_ell_D()
    check_ell_sym()
    check_ell_decay()
    check_ell_cong()
    check_ell_carry()
    check_bv_simplicial()
    check_ell_nonpoly()
    c = check_zd_param()
    check_zd_traces(c)
    check_zd_cong()
    print()
    print("ALL CHECKS PASSED" if not FAILS else f"{len(FAILS)} CHECK(S) FAILED: {FAILS}")
    sys.exit(1 if FAILS else 0)
```

C Recorded output of the modular verification program

This is the execution record supplied with the source manuscript, not a newly claimed run. Its historical "open case" wording is superseded by Theorem 2.1.

```
PASS [1] F(t(q)) = Theta(q) to O(q^60) for Legendre 2F1(1/2,1/2;1|16t)
PASS [1] F(t(q)) = Theta(q) to O(q^60) for Zagier A (Franel)
PASS [1] F(t(q)) = Theta(q) to O(q^60) for Zagier B
PASS [1] F(t(q)) = Theta(q) to O(q^60) for Zagier C
PASS [1] F(t(q)) = Theta(q) to O(q^60) for Zagier E
PASS [1] F(t(q)) = Theta(q) to O(q^60) for Zagier F
PASS [1] F(t(q)) = Theta(q) to O(q^60) for 3F2(1/2,1/2,1/2;1,1|64t)
PASS [1] F(t(q)) = Theta(q) to O(q^60) for Apery (gamma)
PASS [1] F(t(q)) = Theta(q) to O(q^60) for Almkvist (eta)
PASS [1] F(t(q)) = Theta(q) to O(q^60) for Domb (alpha)
PASS [1] F(t(q)) = Theta(q) to O(q^60) for Almkvist-Zudilin (delta)
PASS [1] F(t(q)) = Theta(q) to O(q^60) for Almkvist (epsilon)
PASS [1] F(t(q)) = Theta(q) to O(q^60) for Almkvist (zeta)
PASS [2] zeros of Theta = poles of t = cusps with denominators [2], r_Theta = 1/2, (C) holds, character and
     Atkin-Lehner symmetry (G0(4)) ok: Legendre 2F1(1/2,1/2;1|16t) :: ord Theta {1: Fraction(0, 1), 2:
     Fraction(1, 2), 4: Fraction(0, 1)}, ord t {1: Fraction(0, 1), 2: Fraction(-1, 1), 4: Fraction(1, 1)}
PASS [2] zeros of Theta = poles of t = cusps with denominators [3], r_Theta = 1, (C) holds, character and
     Atkin-Lehner symmetry (G0(6)) ok: Zagier A (Franel) :: ord Theta {1: Fraction(0, 1), 2: Fraction(0, 1),
      3: Fraction(1, 1), 6: Fraction(0, 1)}, ord t {1: Fraction(0, 1), 2: Fraction(0, 1), 3: Fraction(-1, 1),
      6: Fraction(1, 1)}
PASS [2] zeros of Theta = poles of t = cusps with denominators [1], r_Theta = 1, (C) holds, character and
     Atkin-Lehner symmetry (G0(9)) ok: Zagier B :: ord Theta {1: Fraction(1, 1), 3: Fraction(0, 1), 9:
     Fraction(0, 1)}, ord t {1: Fraction(-1, 1), 3: Fraction(0, 1), 9: Fraction(1, 1)}
PASS [2] zeros of Theta = poles of t = cusps with denominators [2], r_Theta = 1, (C) holds, character and
     Atkin-Lehner symmetry (G0(6)) ok: Zagier C :: ord Theta {1: Fraction(0, 1), 2: Fraction(1, 1), 3:
     Fraction(0, 1), 6: Fraction(0, 1)}, ord t {1: Fraction(0, 1), 2: Fraction(-1, 1), 3: Fraction(0, 1), 6:
      Fraction(1, 1)}
PASS [2] zeros of Theta = poles of t = cusps with denominators [2], r_Theta = 1, (C) holds, character and
     Atkin-Lehner symmetry (G0(8)) ok: Zagier E :: ord Theta {1: Fraction(0, 1), 2: Fraction(1, 1), 4:
     Fraction(0, 1), 8: Fraction(0, 1)}, ord t {1: Fraction(0, 1), 2: Fraction(-1, 1), 4: Fraction(0, 1), 8:
      Fraction(1, 1)}
PASS [2] zeros of Theta = poles of t = cusps with denominators [1], r_Theta = 1, (C) holds, character and
     Atkin-Lehner symmetry (G0(6)) ok: Zagier F :: ord Theta {1: Fraction(1, 1), 2: Fraction(0, 1), 3:
     Fraction(0, 1), 6: Fraction(0, 1)}, ord t {1: Fraction(-1, 1), 2: Fraction(0, 1), 3: Fraction(0, 1), 6:
      Fraction(1, 1)}
PASS [2] zeros of Theta = poles of t = cusps with denominators [2], r_Theta = 1/2, (C) holds, character and
     Atkin-Lehner symmetry (G0(4)+4) ok: 3F2(1/2,1/2,1/2;1,1|64t) :: ord Theta {1: Fraction(0, 1), 2:
     Fraction(1, 1), 4: Fraction(0, 1)}, ord t {1: Fraction(1, 1), 2: Fraction(-2, 1), 4: Fraction(1, 1)}
PASS [2] zeros of Theta = poles of t = cusps with denominators [2, 3], r_Theta = 1, (C) holds, character and
      Atkin-Lehner symmetry (G0(6)+6) ok: Apery (gamma) :: ord Theta {1: Fraction(0, 1), 2: Fraction(1, 1),
     3: Fraction(1, 1), 6: Fraction(0, 1)}, ord t {1: Fraction(1, 1), 2: Fraction(-1, 1), 3: Fraction(-1, 1),
      6: Fraction(1, 1)}
PASS [2] zeros of Theta = poles of t = cusps with denominators [1], r_Theta = 1, (C) holds, character and
     Atkin-Lehner symmetry (G0(5)) ok: Almkvist (eta) :: ord Theta {1: Fraction(1, 1), 5: Fraction(0, 1)},
     ord t {1: Fraction(-1, 1), 5: Fraction(1, 1)}
PASS [2] zeros of Theta = poles of t = cusps with denominators [1, 3], r_Theta = 1, (C) holds, character and
      Atkin-Lehner symmetry (G0(6)+3) ok: Domb (alpha) :: ord Theta {1: Fraction(1, 1), 2: Fraction(0, 1), 3:
      Fraction(1, 1), 6: Fraction(0, 1)}, ord t {1: Fraction(-1, 1), 2: Fraction(1, 1), 3: Fraction(-1, 1),
     6: Fraction(1, 1)}
PASS [2] zeros of Theta = poles of t = cusps with denominators [1, 2], r_Theta = 1, (C) holds, character and
      Atkin-Lehner symmetry (G0(6)+2) ok: Almkvist-Zudilin (delta) :: ord Theta {1: Fraction(1, 1), 2:
     Fraction(1, 1), 3: Fraction(0, 1), 6: Fraction(0, 1)}, ord t {1: Fraction(-1, 1), 2: Fraction(-1, 1), 3:
      Fraction(1, 1), 6: Fraction(1, 1)}
PASS [2] zeros of Theta = poles of t = cusps with denominators [2, 4], r_Theta = 1, (C) holds, character and
      Atkin-Lehner symmetry (G0(8)+8) ok: Almkvist (epsilon) :: ord Theta {1: Fraction(0, 1), 2: Fraction(1,
      1), 4: Fraction(1, 1), 8: Fraction(0, 1)}, ord t {1: Fraction(1, 1), 2: Fraction(-1, 1), 4: Fraction
     (-1, 1), 8: Fraction(1, 1)}
PASS [2] zeros of Theta = poles of t = cusps with denominators [3], r_Theta = 1, (C) holds, character and
     Atkin-Lehner symmetry (G0(9)+9) ok: Almkvist (zeta) :: ord Theta {1: Fraction(0, 1), 3: Fraction(1, 1),
      9: Fraction(0, 1)}, ord t {1: Fraction(1, 1), 3: Fraction(-1, 1), 9: Fraction(1, 1)}
PASS [3] Tr(Q_P) is a polynomial in t of degree <= p(L-1)+floor(r(p-1)) (exact, to t^(D+19)): Legendre 2F1
     (1/2,1/2;1|16t) :: p=3,F_{1}:deg<=1:ok; p=3,F_{2}:deg<=4:ok; p=3,F_{3}:deg<=7:ok; p=3,random deg 2:deg
     <=7:ok; p=3,random deg 5:deg<=16:ok; p=5,F_{1}:deg<=2:ok ...
PASS [3] Tr(Q_P) is a polynomial in t of degree <= p(L-1)+floor(r(p-1)) (exact, to t^(D+19)): Zagier A (
     Franel) :: p=5,F_{1}:deg<=4:ok; p=5,F_{2}:deg<=9:ok; p=5,F_{5}:deg<=24:ok; p=5,random deg 2:deg<=14:ok;
      p=5,random deg 5:deg<=29:ok; p=7,F_{1}:deg<=6:ok ...
PASS [3] Tr(Q_P) is a polynomial in t of degree <= p(L-1)+floor(r(p-1)) (exact, to t^(D+19)): Zagier B :: p
     =2,F_{1}:deg<=1:ok; p=2,F_{2}:deg<=3:ok; p=2,F_{2}:deg<=3:ok; p=2,random deg 2:deg<=5:ok; p=2,random
```

```
    deg 5:deg<=11:ok; p=5,F_{1}:deg<=4:ok ...
PASS [3] Tr(Q_P) is a polynomial in t of degree <= p(L-1)+floor(r(p-1)) (exact, to t^(D+19)): Zagier C :: p
    =5,F_{1}:deg<=4:ok; p=5,F_{2}:deg<=9:ok; p=5,F_{5}:deg<=24:ok; p=5,random deg 2:deg<=14:ok; p=5,random
    deg 5:deg<=29:ok; p=7,F_{1}:deg<=6:ok ...
PASS [3] Tr(Q_P) is a polynomial in t of degree <= p(L-1)+floor(r(p-1)) (exact, to t^(D+19)): Zagier E :: p
    =3,F_{1}:deg<=2:ok; p=3,F_{2}:deg<=5:ok; p=3,F_{3}:deg<=8:ok; p=3,random deg 2:deg<=8:ok; p=3,random
    deg 5:deg<=17:ok; p=5,F_{1}:deg<=4:ok ...
PASS [3] Tr(Q_P) is a polynomial in t of degree <= p(L-1)+floor(r(p-1)) (exact, to t^(D+19)): Zagier F :: p
    =5,F_{1}:deg<=4:ok; p=5,F_{2}:deg<=9:ok; p=5,F_{5}:deg<=24:ok; p=5,random deg 2:deg<=14:ok; p=5,random
    deg 5:deg<=29:ok; p=7,F_{1}:deg<=6:ok ...
PASS [3] Tr(Q_P) is a polynomial in t of degree <= p(L-1)+floor(r(p-1)) (exact, to t^(D+19)): 3F2
    (1/2,1/2,1/2;1,1|64t) :: p=3,F_{1}:deg<=1:ok; p=3,F_{2}:deg<=4:ok; p=3,F_{3}:deg<=7:ok; p=3,random deg
    2:deg<=7:ok; p=3,random deg 5:deg<=16:ok; p=5,F_{1}:deg<=2:ok ...
PASS [3] Tr(Q_P) is a polynomial in t of degree <= p(L-1)+floor(r(p-1)) (exact, to t^(D+19)): Apery (gamma)
    :: p=5,F_{1}:deg<=4:ok; p=5,F_{2}:deg<=9:ok; p=5,F_{5}:deg<=24:ok; p=5,random deg 2:deg<=14:ok; p=5,
    random deg 5:deg<=29:ok; p=7,F_{1}:deg<=6:ok ...
PASS [3] Tr(Q_P) is a polynomial in t of degree <= p(L-1)+floor(r(p-1)) (exact, to t^(D+19)): Almkvist (eta)
     :: p=2,F_{1}:deg<=1:ok; p=2,F_{2}:deg<=3:ok; p=2,F_{2}:deg<=3:ok; p=2,random deg 2:deg<=5:ok; p=2,
    random deg 5:deg<=11:ok; p=3,F_{1}:deg<=2:ok ...
PASS [3] Tr(Q_P) is a polynomial in t of degree <= p(L-1)+floor(r(p-1)) (exact, to t^(D+19)): Domb (alpha)
    :: p=5,F_{1}:deg<=4:ok; p=5,F_{2}:deg<=9:ok; p=5,F_{5}:deg<=24:ok; p=5,random deg 2:deg<=14:ok; p=5,
    random deg 5:deg<=29:ok; p=7,F_{1}:deg<=6:ok ...
PASS [3] Tr(Q_P) is a polynomial in t of degree <= p(L-1)+floor(r(p-1)) (exact, to t^(D+19)): Almkvist-
    Zudilin (delta) :: p=5,F_{1}:deg<=4:ok; p=5,F_{2}:deg<=9:ok; p=5,F_{5}:deg<=24:ok; p=5,random deg 2:deg
    <=14:ok; p=5,random deg 5:deg<=29:ok; p=7,F_{1}:deg<=6:ok ...
PASS [3] Tr(Q_P) is a polynomial in t of degree <= p(L-1)+floor(r(p-1)) (exact, to t^(D+19)): Almkvist (
    epsilon) :: p=3,F_{1}:deg<=2:ok; p=3,F_{2}:deg<=5:ok; p=3,F_{3}:deg<=8:ok; p=3,random deg 2:deg<=8:ok;
    p=3,random deg 5:deg<=17:ok; p=5,F_{1}:deg<=4:ok ...
PASS [3] Tr(Q_P) is a polynomial in t of degree <= p(L-1)+floor(r(p-1)) (exact, to t^(D+19)): Almkvist (zeta
    ) :: p=2,F_{1}:deg<=1:ok; p=2,F_{2}:deg<=3:ok; p=2,F_{2}:deg<=3:ok; p=2,random deg 2:deg<=5:ok; p=2,
    random deg 5:deg<=11:ok; p=5,F_{1}:deg<=4:ok ...
PASS [4] Theta(q)F_(mp^(s-1))(t(q^p)) = Theta(q^p)F_(mp^s)(t(q)) mod p^((w+1)s), w=1: Legendre 2F1
    (1/2,1/2;1|16t) :: p=3,s=1,m=1:v=2 p=3,s=1,m=2:v=4 p=3,s=2,m=1:v=4 p=3,s=2,m=2:v=6 p=3,s=3,m=1:v=6 p=5,
    s=1,m=1:v=2 p=5,s=1,m=2:v=2 p=5,s=2,m=1:v=4 p=5,s=2,m=2:v=4 p=5,s=3,m=1:v=6 p=7,s=1,m=1:v=2 p=7,s=1,m=2:
    v=2 p=7,s=2,m=1:v=4 p=7,s=2,m=2:v=4 p=11,s=1,m=1:v=2 p=11,s=1,m=2:v=2 p=11,s=2,m=1:v=4 p=13,s=1,m=1:v=2
     p=13,s=1,m=2:v=2
PASS [4] Theta(q)F_(mp^(s-1))(t(q^p)) = Theta(q^p)F_(mp^s)(t(q)) mod p^((w+1)s), w=1: Zagier A (Franel) :: p
    =5,s=1,m=1:v=2 p=5,s=1,m=2:v=2 p=5,s=2,m=1:v=4 p=5,s=2,m=2:v=4 p=5,s=3,m=1:v=6 p=7,s=1,m=1:v=2 p=7,s=1,
    m=2:v=2 p=7,s=2,m=1:v=4 p=7,s=2,m=2:v=4 p=11,s=1,m=1:v=2 p=11,s=1,m=2:v=2 p=11,s=2,m=1:v=4 p=13,s=1,m=1:
    v=2 p=13,s=1,m=2:v=2
PASS [4] Theta(q)F_(mp^(s-1))(t(q^p)) = Theta(q^p)F_(mp^s)(t(q)) mod p^((w+1)s), w=1: Zagier B :: p=2,s=1,m
    =1:v=2 p=2,s=1,m=2:v=4 p=2,s=2,m=1:v=4 p=2,s=2,m=2:v=6 p=2,s=3,m=1:v=6 p=5,s=1,m=1:v=2 p=5,s=1,m=2:v=2
    p=5,s=2,m=1:v=4 p=5,s=2,m=2:v=4 p=5,s=3,m=1:v=6 p=7,s=1,m=1:v=2 p=7,s=1,m=2:v=2 p=7,s=2,m=1:v=4 p=7,s=2,
    m=2:v=4 p=11,s=1,m=1:v=2 p=11,s=1,m=2:v=2 p=11,s=2,m=1:v=4 p=13,s=1,m=1:v=2 p=13,s=1,m=2:v=2
PASS [4] Theta(q)F_(mp^(s-1))(t(q^p)) = Theta(q^p)F_(mp^s)(t(q)) mod p^((w+1)s), w=1: Zagier C :: p=5,s=1,m
    =1:v=2 p=5,s=1,m=2:v=2 p=5,s=2,m=1:v=4 p=5,s=2,m=2:v=4 p=5,s=3,m=1:v=6 p=7,s=1,m=1:v=2 p=7,s=1,m=2:v=2
    p=7,s=2,m=1:v=4 p=7,s=2,m=2:v=4 p=11,s=1,m=1:v=2 p=11,s=1,m=2:v=2 p=11,s=2,m=1:v=4 p=13,s=1,m=1:v=2 p
    =13,s=1,m=2:v=2
PASS [4] Theta(q)F_(mp^(s-1))(t(q^p)) = Theta(q^p)F_(mp^s)(t(q)) mod p^((w+1)s), w=1: Zagier E :: p=3,s=1,m
    =1:v=2 p=3,s=1,m=2:v=2 p=3,s=2,m=1:v=4 p=3,s=2,m=2:v=4 p=3,s=3,m=1:v=6 p=5,s=1,m=1:v=2 p=5,s=1,m=2:v=2
    p=5,s=2,m=1:v=4 p=5,s=2,m=2:v=4 p=5,s=3,m=1:v=6 p=7,s=1,m=1:v=2 p=7,s=1,m=2:v=2 p=7,s=2,m=1:v=4 p=7,s=2,
    m=2:v=4 p=11,s=1,m=1:v=2 p=11,s=1,m=2:v=2 p=11,s=2,m=1:v=4 p=13,s=1,m=1:v=2 p=13,s=1,m=2:v=2
PASS [4] Theta(q)F_(mp^(s-1))(t(q^p)) = Theta(q^p)F_(mp^s)(t(q)) mod p^((w+1)s), w=1: Zagier F :: p=5,s=1,m
    =1:v=2 p=5,s=1,m=2:v=2 p=5,s=2,m=1:v=4 p=5,s=2,m=2:v=4 p=5,s=3,m=1:v=6 p=7,s=1,m=1:v=2 p=7,s=1,m=2:v=2
    p=7,s=2,m=1:v=4 p=7,s=2,m=2:v=4 p=11,s=1,m=1:v=2 p=11,s=1,m=2:v=2 p=11,s=2,m=1:v=4 p=13,s=1,m=1:v=2 p
    =13,s=1,m=2:v=2
PASS [4] Theta(q)F_(mp^(s-1))(t(q^p)) = Theta(q^p)F_(mp^s)(t(q)) mod p^((w+1)s), w=2: 3F2(1/2,1/2,1/2;1,1|64
    t) :: p=3,s=1,m=1:v=3 p=3,s=1,m=2:v=6 p=3,s=2,m=1:v=6 p=3,s=2,m=2:v=9 p=3,s=3,m=1:v=9 p=5,s=1,m=1:v=3 p
    =5,s=1,m=2:v=3 p=5,s=2,m=1:v=6 p=5,s=2,m=2:v=6 p=5,s=3,m=1:v=9 p=7,s=1,m=1:v=3 p=7,s=1,m=2:v=3 p=7,s=2,
    m=1:v=6 p=7,s=2,m=2:v=6 p=11,s=1,m=1:v=3 p=11,s=1,m=2:v=3 p=11,s=2,m=1:v=6 p=13,s=1,m=1:v=3 p=13,s=1,m
    =2:v=3
PASS [4] Theta(q)F_(mp^(s-1))(t(q^p)) = Theta(q^p)F_(mp^s)(t(q)) mod p^((w+1)s), w=2: Apery (gamma) :: p=5,s
    =1,m=1:v=3 p=5,s=1,m=2:v=3 p=5,s=2,m=1:v=6 p=5,s=2,m=2:v=6 p=5,s=3,m=1:v=9 p=7,s=1,m=1:v=3 p=7,s=1,m=2:
    v=3 p=7,s=2,m=1:v=6 p=7,s=2,m=2:v=6 p=11,s=1,m=1:v=3 p=11,s=1,m=2:v=3 p=11,s=2,m=1:v=6 p=13,s=1,m=1:v=3
     p=13,s=1,m=2:v=3
PASS [4] Theta(q)F_(mp^(s-1))(t(q^p)) = Theta(q^p)F_(mp^s)(t(q)) mod p^((w+1)s), w=2: Almkvist (eta) :: p=2,
    s=1,m=1:v=3 p=2,s=1,m=2:v=6 p=2,s=2,m=1:v=6 p=2,s=2,m=2:v=9 p=2,s=3,m=1:v=9 p=3,s=1,m=1:v=3 p=3,s=1,m=2:
    v=3 p=3,s=2,m=1:v=6 p=3,s=2,m=2:v=6 p=3,s=3,m=1:v=9 p=7,s=1,m=1:v=3 p=7,s=1,m=2:v=3 p=7,s=2,m=1:v=6 p=7,
    s=2,m=2:v=6 p=11,s=1,m=1:v=3 p=11,s=1,m=2:v=3 p=11,s=2,m=1:v=6 p=13,s=1,m=1:v=3 p=13,s=1,m=2:v=3
PASS [4] Theta(q)F_(mp^(s-1))(t(q^p)) = Theta(q^p)F_(mp^s)(t(q)) mod p^((w+1)s), w=2: Domb (alpha) :: p=5,s
    =1,m=1:v=3 p=5,s=1,m=2:v=3 p=5,s=2,m=1:v=6 p=5,s=2,m=2:v=6 p=5,s=3,m=1:v=9 p=7,s=1,m=1:v=3 p=7,s=1,m=2:
    v=3 p=7,s=2,m=1:v=6 p=7,s=2,m=2:v=6 p=11,s=1,m=1:v=3 p=11,s=1,m=2:v=3 p=11,s=2,m=1:v=6 p=13,s=1,m=1:v=3
     p=13,s=1,m=2:v=3
PASS [4] Theta(q)F_(mp^(s-1))(t(q^p)) = Theta(q^p)F_(mp^s)(t(q)) mod p^((w+1)s), w=2: Almkvist-Zudilin (
```

```
    delta) :: p=5,s=1,m=1:v=3 p=5,s=1,m=2:v=3 p=5,s=2,m=1:v=6 p=5,s=2,m=2:v=6 p=5,s=3,m=1:v=9 p=7,s=1,m=1:v
    =3 p=7,s=1,m=2:v=3 p=7,s=2,m=1:v=6 p=7,s=2,m=2:v=6 p=11,s=1,m=1:v=3 p=11,s=1,m=2:v=3 p=11,s=2,m=1:v=6 p
    =13,s=1,m=1:v=3 p=13,s=1,m=2:v=3
PASS [4] Theta(q)F_(mp^(s-1))(t(q^p)) = Theta(q^p)F_(mp^s)(t(q)) mod p^((w+1)s), w=2: Almkvist (epsilon) ::
    p=3,s=1,m=1:v=3 p=3,s=1,m=2:v=3 p=3,s=2,m=1:v=6 p=3,s=2,m=2:v=6 p=3,s=3,m=1:v=9 p=5,s=1,m=1:v=3 p=5,s=1,
    m=2:v=3 p=5,s=2,m=1:v=6 p=5,s=2,m=2:v=6 p=5,s=3,m=1:v=9 p=7,s=1,m=1:v=3 p=7,s=1,m=2:v=3 p=7,s=2,m=1:v=6
     p=7,s=2,m=2:v=6 p=11,s=1,m=1:v=3 p=11,s=1,m=2:v=3 p=11,s=2,m=1:v=6 p=13,s=1,m=1:v=3 p=13,s=1,m=2:v=3
PASS [4] Theta(q)F_(mp^(s-1))(t(q^p)) = Theta(q^p)F_(mp^s)(t(q)) mod p^((w+1)s), w=2: Almkvist (zeta) :: p
    =2,s=1,m=1:v=3 p=2,s=1,m=2:v=6 p=2,s=2,m=1:v=6 p=2,s=2,m=2:v=9 p=2,s=3,m=1:v=9 p=5,s=1,m=1:v=3 p=5,s=1,
    m=2:v=3 p=5,s=2,m=1:v=6 p=5,s=2,m=2:v=6 p=5,s=3,m=1:v=9 p=7,s=1,m=1:v=3 p=7,s=1,m=2:v=3 p=7,s=2,m=1:v=6
     p=7,s=2,m=2:v=6 p=11,s=1,m=1:v=3 p=11,s=1,m=2:v=3 p=11,s=2,m=1:v=6 p=13,s=1,m=1:v=3 p=13,s=1,m=2:v=3
PASS [5] a(mp^s) = a(mp^(s-1)) mod p^((w+1)s) (50 cases): Legendre 2F1(1/2,1/2;1|16t)
PASS [5] a(mp^s) = a(mp^(s-1)) mod p^((w+1)s) (35 cases): Zagier A (Franel)
PASS [5] a(mp^s) = a(mp^(s-1)) mod p^((w+1)s) (50 cases): Zagier B
PASS [5] a(mp^s) = a(mp^(s-1)) mod p^((w+1)s) (35 cases): Zagier C
PASS [5] a(mp^s) = a(mp^(s-1)) mod p^((w+1)s) (50 cases): Zagier E
PASS [5] a(mp^s) = a(mp^(s-1)) mod p^((w+1)s) (35 cases): Zagier F
PASS [5] a(mp^s) = a(mp^(s-1)) mod p^((w+1)s) (50 cases): 3F2(1/2,1/2,1/2;1,1|64t)
PASS [5] a(mp^s) = a(mp^(s-1)) mod p^((w+1)s) (35 cases): Apery (gamma)
PASS [5] a(mp^s) = a(mp^(s-1)) mod p^((w+1)s) (53 cases): Almkvist (eta)
PASS [5] a(mp^s) = a(mp^(s-1)) mod p^((w+1)s) (35 cases): Domb (alpha)
PASS [5] a(mp^s) = a(mp^(s-1)) mod p^((w+1)s) (35 cases): Almkvist-Zudilin (delta)
PASS [5] a(mp^s) = a(mp^(s-1)) mod p^((w+1)s) (50 cases): Almkvist (epsilon)
PASS [5] a(mp^s) = a(mp^(s-1)) mod p^((w+1)s) (50 cases): Almkvist (zeta)
PASS [6] BV3 (19) with m=2, excellent lift q->q^p: hypercubic/hyperoctahedral n=2 (c_{2k}=C(2k,k)^2), p=3, s
    =1 :: v_p(F(t)F_(mp^(s-1))(t^s) - F(t^s)F_(mp^s)(t)) = 2 (required >= 2)
PASS [6] BV3 (19) with m=2, excellent lift q->q^p: hypercubic/hyperoctahedral n=2 (c_{2k}=C(2k,k)^2), p=3, s
    =2 :: v_p(F(t)F_(mp^(s-1))(t^s) - F(t^s)F_(mp^s)(t)) = 4 (required >= 4)
PASS [6] BV3 (19) with m=2, excellent lift q->q^p: hypercubic/hyperoctahedral n=2 (c_{2k}=C(2k,k)^2), p=5, s
    =1 :: v_p(F(t)F_(mp^(s-1))(t^s) - F(t^s)F_(mp^s)(t)) = 2 (required >= 2)
PASS [6] BV3 (19) with m=2, excellent lift q->q^p: hypercubic/hyperoctahedral n=2 (c_{2k}=C(2k,k)^2), p=5, s
    =2 :: v_p(F(t)F_(mp^(s-1))(t^s) - F(t^s)F_(mp^s)(t)) = 4 (required >= 4)
PASS [6] BV3 (19) with m=2, excellent lift q->q^p: hypercubic/hyperoctahedral n=2 (c_{2k}=C(2k,k)^2), p=7, s
    =1 :: v_p(F(t)F_(mp^(s-1))(t^s) - F(t^s)F_(mp^s)(t)) = 2 (required >= 2)
PASS [6] BV3 (19) with m=2, excellent lift q->q^p: hypercubic/hyperoctahedral n=2 (c_{2k}=C(2k,k)^2), p=7, s
    =2 :: v_p(F(t)F_(mp^(s-1))(t^s) - F(t^s)F_(mp^s)(t)) = 4 (required >= 4)
PASS [6] BV3 (19) with m=2, excellent lift q->q^p: hypercubic n=3 (c_{2k}=C(2k,k)^3), p=5, s=1 :: v_p(F(t)F_
    (mp^(s-1))(t^s) - F(t^s)F_(mp^s)(t)) = 3 (required >= 3)
PASS [6] BV3 (19) with m=2, excellent lift q->q^p: hypercubic n=3 (c_{2k}=C(2k,k)^3), p=5, s=2 :: v_p(F(t)F_
    (mp^(s-1))(t^s) - F(t^s)F_(mp^s)(t)) = 6 (required >= 6)
PASS [6] BV3 (19) with m=2, excellent lift q->q^p: hypercubic n=3 (c_{2k}=C(2k,k)^3), p=7, s=1 :: v_p(F(t)F_
    (mp^(s-1))(t^s) - F(t^s)F_(mp^s)(t)) = 3 (required >= 3)
PASS [6] BV3 (19) with m=2, excellent lift q->q^p: hypercubic n=3 (c_{2k}=C(2k,k)^3), p=7, s=2 :: v_p(F(t)F_
    (mp^(s-1))(t^s) - F(t^s)F_(mp^s)(t)) = 6 (required >= 6)
PASS [7a] (numerical, open case) hyperoctahedral n=3 of BV3 Conj. 7.5: congruence mod p^(2s) (the valuation
    is exactly 2s, not 3s) :: p=5,s=1:v=2 p=7,s=1:v=2 p=11,s=1:v=2 p=5,s=2:v=4
PASS [7b] hyperoctahedral n=3: mirror map u = v/(1+v)^2 and F(u(q)) = (1+v) eta2^7 eta3^7/(eta1^5 eta6^5), v
     = (eta1 eta6/(eta2 eta3))^12
PASS [7c] hyperoctahedral n=3: v_p(c_(p^k-1)) = k for p in {5,7,11,13}, k in {1,2} (only one factor p per
    digit p-1, so the carry bound of Section 6 fails)
PASS [7d] hyperoctahedral n=3: deg D = (p-1)/2 if p = 1 mod 3 and (p+1)/2 if p = 2 mod 3, p in {5,7,11,13}
    :: [(5, 3, 3), (7, 3, 3), (11, 6, 6), (13, 6, 6)]
PASS [7e] hyperoctahedral n=3, p=5, s=3: decay estimate holds; termwise bound min_i v(a_i)+v([u^n]S_i) over
    n in [125,150) is 5 < 6, but the tail of Tr(Q_P) has valuation >= 6 :: decay ok=True, termwise=5, true
    =6
PASS [8a] F(t(q)) = Theta(q) to O(q^60) for 27t
PASS [8a] F(t(q)) = Theta(q) to O(q^60) for 256t
PASS [8a] F(t(q)) = Theta(q) to O(q^60) for 108t
PASS [8a] 27t: a^3 = eta1^9/eta3^3 + 27 eta3^9/eta1^3 and a^3 t = eta3^9/eta1^3 (no zeros in H)
PASS [8a] 256t: E22^2 = eta1^16/eta2^8 + 64 eta2^16/eta1^8 and E22^4 t = eta1^8 eta2^8
PASS [8a] 108t: E23 = a^2 and E23^3 t = eta1^6 eta3^6
PASS [8a] |Theta(e)| at e=(3+sqrt-3)/6, (1+i)/2, (3+sqrt-3)/6 (floating point) :: 3.5e-16 3.2e-16 1.8e-16
PASS [8b] D(t) in Z[t], D = +-1 mod p, deg D <= (p+1)lam/k0: 27t :: p=5:deg=2<=2,D(0)=-1 p=7:deg=2<=8/3,D(0)
    =1 p=11:deg=4<=4,D(0)=-1 p=13:deg=4<=14/3,D(0)=1
PASS [8b] D(t) in Z[t], D = +-1 mod p, deg D <= (p+1)lam/k0: 256t :: p=5:deg=1<=3/2,D(0)=1 p=7:deg=2<=2,D(0)
    =1 p=11:deg=3<=3,D(0)=1 p=13:deg=3<=7/2,D(0)=1
PASS [8b] D(t) in Z[t], D = +-1 mod p, deg D <= (p+1)lam/k0 and B = D^2: 108t :: p=5:deg=1<=1,D(0)=-1 p=7:
    deg=1<=4/3,D(0)=1 p=11:deg=2<=2,D(0)=-1 p=13:deg=2<=7/3,D(0)=1
PASS [8c] polynomiality and degrees of E_k = e_k D^k0, M_l = n_l D^k0, A_0 = S_0 D^o (Section 6): 27t :: p
    =5:degE<=p+1:True,[t^(p+1)]E1=0modp:True,degM<=p:True,degA0<=floor(pr):True,E1=+-t^p mod p:True; p=7:
    degE<=p+1:True,[t^(p+1)]E1=0modp:True,degM<=p:True,degA0<=floor(pr):True,E1=+-t^p mod p:True
PASS [8c] polynomiality and degrees of E_k = e_k D^k0, M_l = n_l D^k0, A_0 = S_0 D^o (Section 6): 256t :: p
    =5:degE<=p+1:True,[t^(p+1)]E1=0modp:True,degM<=p:True,degA0<=floor(pr):True,E1=+-t^p mod p:True; p=7:
    degE<=p+1:True,[t^(p+1)]E1=0modp:True,degM<=p:True,degA0<=floor(pr):True,E1=+-t^p mod p:True
```

```
PASS [8c] polynomiality and degrees of E_k = e_k D^k0, M_l = n_l D^k0, A_0 = S_0 D^o (Section 6): 108t :: p
    =5:degE<=p+1:True,[t^(p+1)]E1=0modp:True,degM<=p:True,degA0<=floor(pr):True,E1=+-t^p mod p:True; p=7:
    degE<=p+1:True,[t^(p+1)]E1=0modp:True,degM<=p:True,degA0<=floor(pr):True,E1=+-t^p mod p:True
PASS [8d] decay estimate v_p([t^n]T_p(t^i)) >= (n-pi-floor(pr))/delta*: 27t :: p=5:min slack=0 p=7:min slack
    =0 p=11:min slack=0 p=13:min slack=0
PASS [8d] decay estimate v_p([t^n]T_p(t^i)) >= (n-pi-floor(pr))/delta*: 256t :: p=5:min slack=1 p=7:min
    slack=0 p=11:min slack=0 p=13:min slack=0
PASS [8d] decay estimate v_p([t^n]T_p(t^i)) >= (n-pi-floor(pr))/delta*: 108t :: p=5:min slack=0 p=7:min
    slack=1 p=11:min slack=0 p=13:min slack=0
PASS [8e] Theorem B: Theta(q)F_(mp^(s-1))(t(q^p)) = Theta(q^p)F_(mp^s)(t(q)) mod p^((w+1)s): 27t :: p=5,s=1,
    m=1:v=2 p=5,s=1,m=2:v=3 p=5,s=1,m=3:v=3 p=5,s=2,m=1:v=4 p=5,s=2,m=2:v=5 p=5,s=3,m=1:v=6 p=7,s=1,m=1:v=2
     p=7,s=1,m=2:v=2 p=7,s=1,m=3:v=3 p=7,s=2,m=1:v=4 p=7,s=2,m=2:v=4 p=11,s=1,m=1:v=2 p=11,s=1,m=2:v=2 p=11,
    s=1,m=3:v=2 p=11,s=2,m=1:v=4 p=13,s=1,m=1:v=2 p=13,s=1,m=2:v=2 p=13,s=1,m=3:v=2
PASS [8e] Theorem B: Theta(q)F_(mp^(s-1))(t(q^p)) = Theta(q^p)F_(mp^s)(t(q)) mod p^((w+1)s): 256t :: p=5,s
    =1,m=1:v=4 p=5,s=1,m=2:v=5 p=5,s=1,m=3:v=6 p=5,s=2,m=1:v=7 p=5,s=2,m=2:v=8 p=5,s=3,m=1:v=10 p=7,s=1,m=1:
    v=3 p=7,s=1,m=2:v=4 p=7,s=1,m=3:v=4 p=7,s=2,m=1:v=6 p=7,s=2,m=2:v=7 p=11,s=1,m=1:v=3 p=11,s=1,m=2:v=3 p
    =11,s=1,m=3:v=4 p=11,s=2,m=1:v=6 p=13,s=1,m=1:v=3 p=13,s=1,m=2:v=3 p=13,s=1,m=3:v=3
PASS [8e] Theorem B: Theta(q)F_(mp^(s-1))(t(q^p)) = Theta(q^p)F_(mp^s)(t(q)) mod p^((w+1)s): 108t :: p=5,s
    =1,m=1:v=4 p=5,s=1,m=2:v=5 p=5,s=1,m=3:v=6 p=5,s=2,m=1:v=7 p=5,s=2,m=2:v=8 p=5,s=3,m=1:v=10 p=7,s=1,m=1:
    v=3 p=7,s=1,m=2:v=3 p=7,s=1,m=3:v=4 p=7,s=2,m=1:v=6 p=7,s=2,m=2:v=6 p=11,s=1,m=1:v=3 p=11,s=1,m=2:v=3 p
    =11,s=1,m=3:v=3 p=11,s=2,m=1:v=6 p=13,s=1,m=1:v=3 p=13,s=1,m=2:v=3 p=13,s=1,m=3:v=3
PASS [8f] arithmetic of the induction: decay at offset p(j+1) >= c(k+1) (p<200, k<=12) and v_p(a_n) >= c *
    #(digits p-1)
PASS [8g] BV3 (19) with m=3, excellent lift q->q^p: simplicial n=2 (c_{3k}=(3k)!/k!^3), p=5, s=1 :: v_p(F(t)
    F_(mp^(s-1))(t^s) - F(t^s)F_(mp^s)(t)) = 2 (required >= 2)
PASS [8g] BV3 (19) with m=3, excellent lift q->q^p: simplicial n=2 (c_{3k}=(3k)!/k!^3), p=5, s=2 :: v_p(F(t)
    F_(mp^(s-1))(t^s) - F(t^s)F_(mp^s)(t)) = 4 (required >= 4)
PASS [8g] BV3 (19) with m=3, excellent lift q->q^p: simplicial n=2 (c_{3k}=(3k)!/k!^3), p=7, s=1 :: v_p(F(t)
    F_(mp^(s-1))(t^s) - F(t^s)F_(mp^s)(t)) = 2 (required >= 2)
PASS [8g] BV3 (19) with m=4, excellent lift q->q^p: simplicial n=3 (c_{4k}=(4k)!/k!^4), p=5, s=1 :: v_p(F(t)
    F_(mp^(s-1))(t^s) - F(t^s)F_(mp^s)(t)) = 4 (required >= 3)
PASS [8g] BV3 (19) with m=4, excellent lift q->q^p: simplicial n=3 (c_{4k}=(4k)!/k!^4), p=5, s=2 :: v_p(F(t)
    F_(mp^(s-1))(t^s) - F(t^s)F_(mp^s)(t)) = 7 (required >= 6)
PASS [8g] BV3 (19) with m=4, excellent lift q->q^p: simplicial n=3 (c_{4k}=(4k)!/k!^4), p=7, s=1 :: v_p(F(t)
    F_(mp^(s-1))(t^s) - F(t^s)F_(mp^s)(t)) = 3 (required >= 3)
PASS [8g] mirror map: t_0(Q)^3 = t(Q^3) for the simplicial family attached to 27t
PASS [8g] mirror map: t_0(Q)^4 = t(Q^4) for the simplicial family attached to 256t
PASS [8h] for 2F1(1/3,2/3;1|27t), p=5: Tr(Q_1) is not a polynomial in t; its high t-coefficients are nonzero
     but p-adically small :: (n, v_5 of coefficient): [(5, 2), (11, 5), (17, 8), (23, 11), (29, 14), (35,
    17)]
PASS [9a] Zagier D: F(t(q)) = Theta(q), Theta^2 t = eta5^5/eta1 and 1/t - 11 - t = eta1^6/eta5^6 to O(q^79);
     orders of eta5^5/eta1 at the cusps of X_0(5) are 1, 0
PASS [9a] Zagier D: the canonical coordinate of the Apery zeta(2) equation inverts t(q), i.e. t is the
    mirror map
PASS [9a] diamond operators on Gamma_1(5): t o<2> = t o<3> = -1/t, Theta|<2> = c t Theta, Theta|<3> = -c t
    Theta, t o<4> = t, Theta|<4> = -Theta (floating point) :: c = +1.000000000000; t(<2>tau)t(tau) = -1.000
    e+00-1.491e-16j
PASS [9b] Hecke traces on Gamma_1(5): p=+-1: eta'Q+p^2U_pG polynomial of degree<p(D+1); p=+-2: Lambda = eta
    Qcheck + p^2 U_pG and Lambda' = -eta t Q + p^2 t U_p Gcheck are Laurent polynomials, deg Lambda' <= p(D
    +1), Lambda'(t) = Lambda(-1/t) :: p=2,D=0:top(Lc)=2<=2 p=2,D=2:top(Lc)=6<=6 p=2,D=3:top(Lc)=8<=8 p=3,D
    =0:top(Lc)=3<=3 p=3,D=2:top(Lc)=9<=9 p=3,D=3:top(Lc)=12<=12 p=7,D=0:top(Lc)=7<=7 p=7,D=2:top(Lc)=21<=21
     p=7,D=3:top(Lc)=28<=28 p=13,D=0:top(Lc)=13<=13 p=13,D=2:top(Lc)=39<=39 p=13,D=3:top(Lc)=52<=52 ...
PASS [9c] Theorem C: (H_s) and the dual congruence (D_s) modulo p^(2s) for Zagier D :: p=2,s=1,m=1:v(H)=2,v(
    D)=2 p=2,s=1,m=2:v(H)=4,v(D)=4 p=2,s=2,m=1:v(H)=4,v(D)=4 p=2,s=2,m=2:v(H)=6,v(D)=6 p=2,s=3,m=1:v(H)=6,v
    (D)=6 p=3,s=1,m=1:v(H)=2,v(D)=2 p=3,s=1,m=2:v(H)=2,v(D)=2 p=3,s=2,m=1:v(H)=4,v(D)=4 p=3,s=2,m=2:v(H)=4,
    v(D)=4 p=3,s=3,m=1:v(H)=6,v(D)=6 p=7,s=1,m=1:v(H)=2,v(D)=2 p=7,s=1,m=2:v(H)=2,v(D)=2 p=7,s=2,m=1:v(H)=4,
    v(D)=4 p=7,s=2,m=2:v(H)=4,v(D)=4 p=11,s=1,m=1:v(H)=2,v(D)=2 p=11,s=1,m=2:v(H)=2,v(D)=2 p=11,s=2,m=1:v(H
    )=4,v(D)=4 p=13,s=1,m=1:v(H)=2,v(D)=2 p=13,s=1,m=2:v(H)=2,v(D)=2 p=17,s=1,m=1:v(H)=2,v(D)=2 p=17,s=1,m
    =2:v(H)=2,v(D)=2

ALL CHECKS PASSED
```

D Complete verification programs

This appendix embeds the complete programs recovered from the three verification archives, in addition to the base program in Appendix B. Each is a finite, reproducible computation with its scope stated in the program. The independent certificate appears first. The other programs perform bounded validation of the corresponding general proofs. Recorded JSON arrays can be regenerated from these listings; they are not hidden assumptions of the symbolic theorems.

D.1 certificate_octa4_p5_s2.py

Original source: `certificate_octa4_p5_s2.py`; 156 lines.

```
#!/usr/bin/env python3
"""Exact polynomial certificate for hyperoctahedral n=4, p=5, s=2.

The rational excellent lift is constructed only from BV vertex-polynomial
congruences.  The last polynomial identity then proves the ordinary
truncation congruence at EVERY formal-series coefficient.
Standard library only; no floating point and no power-series cutoff.
"""
from math import comb, factorial
from pathlib import Path
import json
import time
import hashlib

P, R, PREC = 5, 2, 4
N = P**R
MOD = P**PREC
WORKMOD = N*MOD

def trim(a):
    a = list(a)
    while len(a)>1 and not a[-1]:
        a.pop()
    return a

def add(a,b,mod=MOD):
    c=[0]*max(len(a),len(b))
    for i,x in enumerate(a): c[i]+=x
    for i,x in enumerate(b): c[i]+=x
    return trim([x%mod for x in c])

def scale(a,k,mod=MOD): return trim([(k*x)%mod for x in a])

def mul(a,b,mod=MOD):
    a=trim([x%mod for x in a]); b=trim([x%mod for x in b])
    if a==[0] or b==[0]: return [0]
    count=min(len(a),len(b))
    width=(count.bit_length()+2*mod.bit_length()+7)//8
    aa=int.from_bytes(b''.join(x.to_bytes(width,'little') for x in a),'little')
    bb=int.from_bytes(b''.join(x.to_bytes(width,'little') for x in b),'little')
    length=len(a)+len(b)-1
    raw=(aa*bb).to_bytes(length*width,'little')
    return trim([int.from_bytes(raw[i*width:(i+1)*width],'little')%mod for i in range(length)])

def power(a,k,mod=MOD):
    out=[1]
    while k:
        if k&1: out=mul(out,a,mod)
        k//=2
        if k: a=mul(a,a,mod)
    return out

def subst_power(a,p):
    out=[0]*(p*(len(a)-1)+1)
    for j,x in enumerate(a): out[p*j]=x
    return trim(out)

def taylor_scale(a,r,p=P,mod=MOD):
```

```
    # [h^r] a(u^p(1+h)).
    return subst_power([(x*comb(j,r))%mod if j>=r else 0 for j,x in enumerate(a)],p)

def b3_sequence(length):
    # Sum over three composition indices of multinomial(n; k)^2.
    return [sum(comb(j,k)**2*comb(2*k,k) for k in range(j+1)) for j in range(length)]

def vertex(K,b3,mod):
    # P_{2K}(u)=u^{-K} A_K(u), the negative of the dominant-vertex coefficient.
    fac=[factorial(j) for j in range(2*K)]
    ans=[]
    for j in range(K):
        total=0
        for ell in range(j+1):
            den=fac[2*K-2*j-1]*fac[ell]*fac[j-ell]**2
            num=fac[2*K-ell-1]
            assert num%den==0
            term=(num//den)*b3[j-ell]
            total+=(-1 if ell&1 else 1)*term
        ans.append(total%mod)
    assert ans[0]==1
    return trim(ans)

def oct4(length):
    return [comb(2*j,j)*sum(comb(j,k)**2*comb(2*k,k)*comb(2*(j-k),j-k)
                            for k in range(j+1)) for j in range(length)]

def build():
    started=time.time()
    b3=b3_sequence(2*P*N)
    av={k:vertex(k,b3,WORKMOD) for k in (N,2*N,P*N,2*P*N)}
    a=av[N]; b=av[2*N]; ap=av[P*N]; bp=av[2*P*N]
    ar=[taylor_scale(a,r,mod=WORKMOD) for r in range(4)]
    br=[taylor_scale(b,r,mod=WORKMOD) for r in range(4)]
    a0powers=[power(ar[0],r,WORKMOD) for r in range(5)]
    # T_r/ar[0]^(r+1) = [h^r] C_N(u^p(1+h)), C_N=A_2N/A_N.
    T=[]
    for r in range(4):
        tr=mul(br[r],a0powers[r],WORKMOD)
        for i in range(1,r+1):
            tr=add(tr,scale(mul(mul(ar[i],T[r-i],WORKMOD),a0powers[i-1],WORKMOD),-1,WORKMOD),WORKMOD)
        T.append(tr)
    # A,B,C,D share the unit denominator ap*ar[0]^4 after division by N.
    ABCD=[]
    for r in range(4):
        numerator=add(scale(mul(bp,a0powers[4],WORKMOD),comb(N,r),WORKMOD),
                      scale(mul(mul(ap,T[r],WORKMOD),a0powers[3-r],WORKMOD),-1,WORKMOD),WORKMOD)
        assert all(x%N==0 for x in numerator), ('N divisibility',r)
        ABCD.append(trim([(x//N)%MOD for x in numerator]))
    A,B,C,D=ABCD
    assert A[0]==0 and all(x%P==0 for x in A)
    assert B[0]==1
    assert C[0]==12 and D[0]==92

    # Independent target truncations enter ONLY at this point.
    f=oct4(P*P)
    fsmall=f[:P]
    L=[]
    for r in range(4):
        lhs=taylor_scale(fsmall,r)
        if r>=1:
            lhs=add(lhs,scale(taylor_scale(fsmall,r-1),N))
        L.append(add(mul(ap,lhs),scale(mul(f,ar[r]),-1)))

    # h=S-1 = -A/B - C A^2/B^3 + (D/B^4-2 C^2/B^5) A^3 (mod 625).
    # Substitute into L0+L1*h+L2*h^2+L3*h^3 and clear the unit B^5.
    Bpow=[power(B,r) for r in range(6)]
    A2=mul(A,A); A3=mul(A2,A)
    target=add(mul(L[0],Bpow[5]),scale(mul(mul(L[1],A),Bpow[4]),-1))
    second=add(mul(L[2],Bpow[3]),scale(mul(mul(L[1],C),Bpow[2]),-1))
    target=add(target,mul(second,A2))
    third=add(mul(mul(L[1],D),B),scale(mul(L[1],mul(C,C)),-2))
    third=add(third,scale(mul(mul(L[2],C),B),2))
    third=add(third,scale(mul(L[3],Bpow[2]),-1))
```

```
    target=add(target,mul(third,A3))
    assert target==[0], ('target not identically zero',[(i,x) for i,x in enumerate(target) if x][:10])

    payload={'p':P,'s':2,'dimension':4,'modulus':MOD,'vertex_precision':WORKMOD,
             'vertex_indices':list(av),'vertex_polynomials':av,
             'lift_coefficients_common_denominator':ABCD,
             'truncation_target_coefficients':L}
    serialized=json.dumps(payload,sort_keys=True,separators=(',',':'))
    report={'theorem':'Every coefficient of E_2 for hyperoctahedral dimension four is divisible by 5^4.',
            'all_polynomial_coefficients_checked':True,
            'power_series_cutoff':None,
            'lift_constructed_from':'BV vertex polynomials only; no target truncation assumption',
            'p':P,'s':2,'modulus':MOD,'intermediate_modulus':WORKMOD,
            'rational_lift_polynomial_degrees':[len(a)-1 for a in ABCD],
            'cleared_identity_degree_bound':max(len(L[r])-1+5*(len(B)-1) for r in range(4)),
            'cleared_identity_zero':target==[0],
            'certificate_sha256':hashlib.sha256(serialized.encode()).hexdigest(),
            'elapsed_seconds':round(time.time()-started,3)}
    base=Path(__file__).resolve().parent
    (base/'certificate_octa4_p5_s2.json').write_text(json.dumps(payload,indent=2)+'\n')
    (base/'certificate_octa4_p5_s2_report.json').write_text(json.dumps(report,indent=2)+'\n')
    print(json.dumps(report,indent=2))

if __name__=='__main__': build()
```

Recorded certificate result. The archived report is reproduced below. The identity and its degree bound can be regenerated by the preceding program.

```
{
  "theorem": "Every coefficient of E_2 for hyperoctahedral dimension four is divisible by 5^4.",
  "all_polynomial_coefficients_checked": true,
  "power_series_cutoff": null,
  "lift_constructed_from": "BV vertex polynomials only; no target truncation assumption",
  "p": 5,
  "s": 2,
  "modulus": 625,
  "intermediate_modulus": 15625,
  "rational_lift_polynomial_degrees": [
    489,
    537,
    537,
    527
  ],
  "cleared_identity_degree_bound": 2819,
  "cleared_identity_zero": true,
  "certificate_sha256": "d7dbc0fd7427660d97e9d8cacf26577f0af474f6ac447c8ad1cf2f78d69bcd6b",
  "elapsed_seconds": 0.642
}
```

D.2 verify_elliptic.py

Original source: `verify_elliptic.py`; 431 lines.

```
#!/usr/bin/env python3
"""Finite exact-arithmetic checks of the elliptic-trace proof.

This is supporting verification, not a replacement for the all-s proof.
All power-series lengths and arithmetic precisions are recorded.  The
full-congruence checks always include terms strictly beyond the cutoff.
Standard-library Python only.  Run from any directory.
"""
from __future__ import annotations

import json
from math import comb, isqrt
from pathlib import Path
from time import perf_counter
```

```
def trim(a):
    a = list(a)
    while len(a) > 1 and a[-1] == 0:
        a.pop()
    return a


def zmul(a, b, n):
    """Exact integer convolution, truncated at q^n."""
    out = [0] * n
    for i, x in enumerate(a[:n]):
        if x:
            for j, y in enumerate(b[:n-i]):
                if y:
                    out[i+j] += x*y
    return out


def mmul(a, b, n, mod):
    """Exact modular convolution by carry-free Kronecker substitution."""
    if n <= 0:
        return []
    a = trim([x % mod for x in a[:n]])
    b = trim([x % mod for x in b[:n]])
    if not a or not b or a == [0] or b == [0]:
        return [0]*n
    count = min(len(a), len(b))
    width = max(1, (count.bit_length() + 2*mod.bit_length() + 7)//8)
    aa = int.from_bytes(b''.join(x.to_bytes(width, 'little') for x in a), 'little')
    bb = int.from_bytes(b''.join(x.to_bytes(width, 'little') for x in b), 'little')
    # Each convolution coefficient is < count*mod^2 < 256^width.
    size = len(a)+len(b)-1
    raw = (aa*bb).to_bytes(size*width, 'little')
    out = [int.from_bytes(raw[i*width:(i+1)*width], 'little') % mod
           for i in range(min(n, size))]
    return out+[0]*(n-len(out))


def zinv(a, n):
    assert a[0] == 1
    b = [1]+[0]*(n-1)
    for k in range(1, n):
        b[k] = -sum(a[j]*b[k-j] for j in range(1, min(k+1, len(a))))
    return b


def minv(a, n, mod):
    assert a[0] % mod == 1
    b = [1]
    while len(b) < n:
        size = min(2*len(b), n)
        ab = mmul(a[:size], b, size, mod)
        correction = [(-x) % mod for x in ab]
        correction[0] = (correction[0]+2) % mod
        b = mmul(b, correction, size, mod)
    return b


def mpow(a, k, n, mod):
    out = [1]+[0]*(n-1)
    while k:
        if k & 1:
            out = mmul(out, a, n, mod)
        k //= 2
        if k:
            a = mmul(a, a, n, mod)
    return out


def vsub(a, p, n):
    out = [0]*n
    for k, x in enumerate(a):
        if p*k >= n:
```

```
            break
        out[p*k] = x
    return out


def mcompose(a, t, n, mod):
    """Baby-step/giant-step polynomial composition; t(0)=0."""
    assert t[0] == 0
    a = a[:n]
    if not a:
        return [0]*n
    block = isqrt(len(a))+1
    powers = [[1]+[0]*(n-1)]
    for _ in range(block):
        powers.append(mmul(powers[-1], t, n, mod))
    out = [0]*n
    for start in reversed(range(0, len(a), block)):
        out = mmul(out, powers[block], n, mod)
        for r, x in enumerate(a[start:start+block]):
            x %= mod
            if x:
                out = [(v+x*u) % mod for v, u in zip(out, powers[r])]
    return out


def texpand(f, t, n, mod=None):
    """Recover the t-series from its q-series, exactly or modulo mod."""
    assert t[0] == 0 and t[1] == 1
    rem = list(f[:n])+[0]*max(0, n-len(f))
    powers = [1]+[0]*(n-1)
    out = []
    for k in range(n):
        c = rem[k] if mod is None else rem[k] % mod
        out.append(c)
        if c:
            for i in range(k, n):
                rem[i] -= c*powers[i]
                if mod is not None:
                    rem[i] %= mod
        powers = zmul(powers, t, n) if mod is None else mmul(powers, t, n, mod)
    return out


def valuation(x, p, cap):
    if x == 0:
        return cap
    v = 0
    while v < cap and x % p == 0:
        v += 1
        x //= p
    return v


def character3(n):
    return 0 if n % 3 == 0 else (1 if n % 3 == 1 else -1)


def divisor_sums(n):
    out = [0]*n
    for d in range(1, n):
        for j in range(d, n, d):
            out[j] += d
    return out


def eta_h(level, n, sig):
    """Exact q*(eta_N/eta_1 product) via its logarithmic derivative."""
    exponent = 24//(level-1)
    logderiv = [0]+[exponent*(sig[j]-(level*sig[j//level] if j % level == 0 else 0))
                    for j in range(1, n)]
    h = [1]+[0]*(n-1)
    for k in range(1, n):
        value = sum(logderiv[j]*h[k-j] for j in range(1, k+1))
        assert value % k == 0
```

```
        h[k] = value//k
    return [0]+h[:n-1]


def theta_h(n):
    # Exact Lambert series.  No finite lattice-radius approximation.
    out = [1]+[0]*(n-1)
    for d in range(1, n):
        c = 6*character3(d)
        for j in range(d, n, d):
            out[j] += c
    return out


class Family:
    def __init__(self, label, weight, e, level, scale, target):
        self.label, self.w, self.e, self.level, self.scale, self.target = (
            label, weight, e, level, scale, target)

    def epsilon(self, p):
        return character3(p) if self.w == 1 else 1

    def coefficient(self, k):
        if self.label == '27t':
            return comb(3*k, k)*comb(2*k, k)
        if self.label == '256t':
            return comb(4*k, k)*comb(3*k, k)*comb(2*k, k)
        return comb(3*k, k)*comb(2*k, k)**2

    def parameters(self, n, mod=None):
        h = H[self.level][:n]
        den = [1]+[self.scale*x for x in h[1:]]
        if mod is None:
            invden = zinv(den, n)
            factor = invden if self.w == 1 else zmul(invden, invden, n)
            t = zmul(h, factor, n)
        else:
            invden = minv(den, n, mod)
            factor = invden if self.w == 1 else mmul(invden, invden, n, mod)
            t = mmul(h, factor, n, mod)
        if self.label == '27t':
            th = TH[:n]
        elif self.label == '256t':
            th = [1]+[24*(SIG[j]-(2*SIG[j//2] if j % 2 == 0 else 0))
                      for j in range(1, n)]
        else:
            th = [1]+[12*(SIG[j]-(3*SIG[j//3] if j % 3 == 0 else 0))
                      for j in range(1, n)]
        if mod is not None:
            th = [x % mod for x in th]
        return th, t


FAMILIES = [Family('27t', 1, 3, 3, 27, 2),
            Family('256t', 2, 4, 2, 64, 3),
            Family('108t', 2, 3, 3, 27, 3)]


def full_congruence(fam, p, s, m):
    cutoff = m*p**s
    lower = m*p**(s-1)
    n = 2*cutoff+9
    target = fam.target*s
    prec = target+2
    mod = p**prec
    th, t = fam.parameters(n, mod)
    coeff = [fam.coefficient(k) % mod for k in range(cutoff)]
    big = mcompose(coeff, t, n, mod)
    smalln = (n-1)//p+1
    small = mcompose(coeff[:lower], t[:smalln], smalln, mod)
    lhs = mmul(th, vsub(small, p, n), n, mod)
    rhs = mmul(vsub(th, p, n), big, n, mod)
    error = [(x-y) % mod for x, y in zip(lhs, rhs)]
    v = min(valuation(x, p, prec) for x in error)
```

```
    assert n > cutoff and v >= target, (fam.label, p, s, m, v)
    return {'family': fam.label, 'p': p, 's': s, 'm': m,
            'cutoff': cutoff, 'q_coefficients_checked': n,
            'required_valuation': target, 'arithmetic_precision': prec,
            'observed_minimum_valuation': v,
            'valuation_is_lower_bound': v == prec, 'passed': True}


def monomial_trace(fam, p, k, prec=6):
    a, delta = p//fam.e, (p+1)//fam.e
    n = p*k+a+delta*(prec-1)+9
    length = p*n
    mod = p**prec
    th, t = fam.parameters(length, mod)
    tk = mpow(t, k, length, mod)
    G = mmul(mmul(vsub(th, p, length), minv(th, length, mod), length, mod),
             tk, length, mod)
    Q = mmul(mmul(th[:n], minv(vsub(th, p, n), n, mod), n, mod),
             vsub(tk, p, n), n, mod)
    trq = [(Q[j]+fam.epsilon(p)*p**(fam.w+1)*G[p*j]) % mod for j in range(n)]
    trt = texpand(trq, t[:n], n, mod)
    for j, c in enumerate(trt):
        bound = max(0, -(-(j-p*k-a)//delta))
        assert c % p**min(bound, prec) == 0, (fam.label, p, k, j, bound, c)
    degrees = []
    for r in range(1, prec+1):
        observed = max((j for j, c in enumerate(trt) if c % p**r), default=-1)
        expected = p*k+a+delta*(r-1)
        assert observed <= expected
        degrees.append({'precision': r, 'observed_degree_in_checked_range': observed,
                        'proved_bound': expected})
    return {'family': fam.label, 'p': p, 'monomial_k': k,
            't_coefficients_checked': n, 'input_q_coefficients': length,
            'arithmetic_precision': prec, 'degrees': degrees, 'passed': True}


def elementary_from_power_sums(sums, count, n):
    elementary = [[1]+[0]*(n-1)]
    for j in range(1, count+1):
        acc = [0]*n
        for r in range(1, j+1):
            value = zmul(elementary[j-r], sums[r], n)
            sign = 1 if r % 2 else -1
            acc = [x+sign*y for x, y in zip(acc, value)]
        assert all(x % j == 0 for x in acc)
        elementary.append([x//j for x in acc])
    return elementary


def exact_norm_polynomials(fam, p):
    """Exact integer q-series, not modular arithmetic, for norm lemmas."""
    n = p+10
    length = p*n
    th, t = fam.parameters(length)
    thshort, tshort = th[:n], t[:n]
    thinv = zinv(thshort, n)
    invpowers = [[1]+[0]*(n-1)]
    for j in range(p+1):
        invpowers.append(zmul(invpowers[-1], thinv, n))

    thpower = [1]+[0]*(length-1)
    theta_sums = {0: None}
    for r in range(1, p+1):
        thpower = zmul(thpower, th, length)
        theta_sums[r] = [p*thpower[p*j] for j in range(n)]
    elementary_theta = elementary_from_power_sums(theta_sums, p, n)
    Dq = zmul(zmul(vsub(thshort, p, n), elementary_theta[p], n), invpowers[p+1], n)
    Dt = texpand(Dq, tshort, n)
    delta = (p+1)//fam.e
    assert Dt[0] == 1 and all(x == 0 for x in Dt[delta+1:])
    assert all(x % p == 0 for x in Dt[1:])

    inner = []
    for j in range(p+1):
```

```
        c = p**(fam.w*(p-j))*(-fam.epsilon(p))**j
        inner.append([c*x for x in zmul(elementary_theta[j], invpowers[j], n)])
    ratio = zmul(vsub(thshort, p, n), thinv, n)
    Tq = []
    for j in range(p+2):
        left = inner[j] if j <= p else [0]*n
        right = zmul(ratio, inner[j-1], n) if j else [0]*n
        item = [x-y for x, y in zip(left, right)]
        Tq.append(item)
        coefficients = texpand(item, tshort, n)
        assert all(x == 0 for x in coefficients[j//fam.e+1:])
    assert Tq[-1] == [fam.epsilon(p)*x for x in Dq]

    # Verify the actual character factor in R_0=-T_p/(epsilon D).
    ratio_full = zmul(vsub(th, p, length), zinv(th, length), length)
    Q0 = zmul(thshort, zinv(vsub(thshort, p, n), n), n)
    R0 = [Q0[j]+fam.epsilon(p)*p**(fam.w+1)*ratio_full[p*j] for j in range(n)]
    check = zmul(Dq, R0, n)
    assert all(Tq[p][j]+fam.epsilon(p)*check[j] == 0 for j in range(n))

    tpower = [1]+[0]*(length-1)
    t_sums = {0: None}
    for r in range(1, p+2):
        tpower = zmul(tpower, t, length)
        V = vsub(tpower, p, n)
        t_sums[r] = [p*tpower[p*j]+V[j] for j in range(n)]
    elementary_t = elementary_from_power_sums(t_sums, p+1, n)
    Depower = [1]+[0]*(n-1)
    for _ in range(fam.e):
        Depower = zmul(Depower, Dq, n)
    for j in range(p+2):
        Aq = [(-1)**j*x for x in zmul(Depower, elementary_t[j], n)]
        At = texpand(Aq, tshort, n)
        assert all(x == 0 for x in At[p+2:])
        if j == 1:
            assert all((x+(1 if k == p else 0)) % p == 0 for k, x in enumerate(At))
    return {'family': fam.label, 'p': p, 'epsilon': fam.epsilon(p),
            'q_coefficients_checked': n, 'input_q_coefficients': length,
            'arithmetic': 'exact integers', 'norm_D_coefficients': Dt[:delta+1],
            'T_coefficient_degree_checks': p+2,
            'parameter_norm_coefficient_degree_checks': p+2,
            'A1_frobenius_congruence': True, 'trace_character_check': True,
            'passed': True}


def implementation_checks():
    # Independently compare packed multiplication with direct convolution.
    for mod in (25, 7**5, 11**8):
        for n in (5, 31, 73):
            a = [(j*j+13*j+7) % mod for j in range(n)]
            b = [(-j**3+17*j-9) % mod for j in range(n)]
            assert mmul(a, b, n, mod) == [x % mod for x in zmul(a, b, n)]
            a[0] = 1
            assert minv(a, n, mod) == [x % mod for x in zinv(a, n)]
    # Check the composition routine against a short direct Horner computation.
    mod, n = 5**8, 37
    t = [0, 1]+[(j*j+3) % mod for j in range(2, n)]
    a = [j*j+1 for j in range(18)]
    h = [0]*n
    for c in reversed(a):
        h = mmul(h, t, n, mod)
        h[0] = (h[0]+c) % mod
    assert mcompose(a, t, n, mod) == h


def main():
    global SIG, TH, H
    start = perf_counter()
    implementation_checks()
    maximum = 650
    SIG = divisor_sums(maximum)
    TH = theta_h(maximum)
    H = {level: eta_h(level, maximum, SIG) for level in (2, 3)}
    output = {'scope': 'Finite exact checks supporting the written proof; not a proof by testing.',
```

```
            'implementation_checks': True, 'parametrizations': [],
            'norm_polynomials': [], 'monomial_filtration': [], 'full_congruences': []}
    for fam in FAMILIES:
        n, mod = 80, 5**8
        th, t = fam.parameters(n, mod)
        assert mcompose([fam.coefficient(k) for k in range(n)], t, n, mod) == th
        output['parametrizations'].append({'family': fam.label, 'q_coefficients_checked': n,
                                           'modulus': mod, 'passed': True})
        for p in (5, 7):
            result = exact_norm_polynomials(fam, p)
            output['norm_polynomials'].append(result)
            print('PASS norm', fam.label, p, 'D =', result['norm_D_coefficients'], flush=True)
        for p in (5, 7):
            for k in (0, 1, 3):
                output['monomial_filtration'].append(monomial_trace(fam, p, k))
        if fam.label == '108t':
            output['monomial_filtration'].append(monomial_trace(fam, 5, 4, 7))
        for p in (5, 7, 11):
            for s in (1, 2):
                for m in (1, 2):
                    result = full_congruence(fam, p, s, m)
                    output['full_congruences'].append(result)
                    print('PASS congruence', fam.label, 'p', p, 's', s, 'm', m,
                          'valuation', result['observed_minimum_valuation'],
                          'checked q degrees 0..', result['q_coefficients_checked']-1, flush=True)
    output['seconds'] = round(perf_counter()-start, 3)
    output['counts'] = {k: len(output[k]) for k in
                        ('parametrizations', 'norm_polynomials', 'monomial_filtration', 'full_congruences')}
    output['all_passed'] = True
    destination = Path(__file__).resolve().parent
    (destination/'elliptic_validation_results.json').write_text(json.dumps(output, indent=2)+'\n')
    summary = [
        'ELLIPTIC TRACE VALIDATION: ALL FINITE CHECKS PASSED',
        'These checks support the written proof; they do not prove the infinite statement.',
        'Corrected theta_H is computed by its exact Lambert series.',
        'All modular polynomial degree checks use exact integers.',
        'All full-congruence checks include coefficients beyond the truncation cutoff.',
        'Counts: '+json.dumps(output['counts'], sort_keys=True),
        'Elapsed seconds: '+str(output['seconds']),
    ]
    (destination/'elliptic_validation_results.txt').write_text('\n'.join(summary)+'\n')
    print('\n'.join(summary), flush=True)


if __name__ == '__main__':
    main()
```

D.3 verify_sharpness.py

Original source: `verify_sharpness.py`; 171 lines.

```
"""Bounded exact-arithmetic validation of the all-s proofs in Exceptional_Excellent_Lift_Supercongruences.tex,
      Section 4.
This script is not a substitute for those proofs and does not claim a full-series theorem.
Python 3 standard library only; run from any current directory.
"""
from fractions import Fraction as Q
from math import comb, factorial
import json
from pathlib import Path


def vp(n,p):
    if n==0:return None
    v=0
    while n%p==0:n//=p;v+=1
    return v

def central(n):return comb(2*n,n)
def oct3(n):return central(n)*sum(comb(n,k)**2*central(k) for k in range(n+1))
def domb(n):return sum(comb(n,k)**2*central(k)*central(n-k) for k in range(n+1))
```

```
def oct4(n):return central(n)*domb(n)
def cube(d,n):return central(n)**d
def simplex(r,n):return factorial(r*n)//factorial(n)**r

def C_r(r):
    H=sum(Q(1,j) for j in range(1,r+1))
    H2=sum(Q(1,j*j) for j in range(1,r+1))
    C=Q(factorial(r)**2,2)*((r*(H-1))**2-r*r*H2+r)
    assert C.denominator==1
    return int(C)

out=[]
# The dimension-three exact refinement, including s=1,2,3 at p=5.
for p in [5,7,11]:
    h=(p-1)//2
    S=sum(central(k)*pow(k,-2,p) for k in range(1,h+1))%p
    chi=1 if pow(p-3,h,p)==1 else -1
    for s in [1,2,3]:
        K=p**s
        z=oct3(K)-oct3(K//p)
        predicted=2*pow(chi,s-1)*S%p
        assert z%p**(2*s)==0
        actual=z//p**(2*s)%p
        assert actual==predicted
        out.append(dict(family='octahedral3',p=p,s=s,index=K,valuation=vp(z,p),unit_mod_p=actual,predicted=
     predicted))
# Hyperoctahedral dimension four (uses the established Domb congruence in the proof).
for p in [5,7]:
    for s in [1,2,3]:
        K=p**s
        z=oct4(K+1)-(8+20*K)*oct4(K//p)
        assert z%p**(2*s)==0
        assert z//p**(2*s)%p==32%p
        out.append(dict(family='octahedral4',p=p,s=s,index=K+1,valuation=vp(z,p),unit_mod_p=z//p**(2*s)%p,
     predicted=32%p))
# General hypercubic formula in two dimensions of interest and multiple primes.
for d,p in [(4,5),(4,7),(5,7),(5,11),(6,7)]:
    for s in [1,2,3]:
        K=p**s
        z=cube(d,K+1)-(2**d+K*d*2**d)*cube(d,K//p)
        predicted=2**(2*d-1)*d*(d-3)%p
        assert z%p**(2*s)==0
        assert z//p**(2*s)%p==predicted
        assert vp(z,p)==2*s
        out.append(dict(family='cube',dimension=d,p=p,s=s,index=K+1,valuation=vp(z,p),unit_mod_p=z//p**(2*s)
     %p,predicted=predicted))
# Simplex dimensions four and five, including the genuine exceptional p=7 when r=6.
for r,p in [(5,7),(5,11),(6,7),(6,11)]:
    H=sum(Q(1,j) for j in range(1,r+1))
    alpha=factorial(r)*r*(H-1)
    assert alpha.denominator==1
    for s in [1,2,3]:
        K=p**s
        z=int(simplex(r,K+1)-(factorial(r)+K*alpha)*simplex(r,K//p))
        predicted=C_r(r)%p
        assert z%p**(2*s)==0
        assert z//p**(2*s)%p==predicted
        if predicted:assert vp(z,p)==2*s
        out.append(dict(family='simplex',dimension=r-1,p=p,s=s,index=K+1,valuation=vp(z,p),unit_mod_p=z//p
     **(2*s)%p,predicted=predicted))
# Check Domb recurrence used in the optional proof directly from its binomial definition.
for k in range(2,16):
    assert k**3*domb(k)==2*(2*k-1)*(5*k*k-5*k+2)*domb(k-1)-64*(k-1)**3*domb(k-2)
# Compare first-jet formula with full formal-series computation at bounded sizes.
# Independent formal-series implementation (no local imports).
from fractions import Fraction as Q
from math import comb, factorial
import json

def mul(a,b,N):
 c=[Q(0)]*N
 for i,x in enumerate(a[:N]):
  if x:
   for j,y in enumerate(b[:N-i]):
```

```
    if y:c[i+j]+=x*y
 return c

def div(a,b,N):
 c=[Q(0)]*N
 for k in range(N):c[k]=(a[k] if k<len(a) else 0)-sum(b[j]*c[k-j] for j in range(1,min(k+1,len(b))))
 return c

def exp(g,N):
 c=[Q(1)]+[Q(0)]*(N-1)
 for k in range(1,N):c[k]=sum(j*g[j]*c[k-j] for j in range(1,min(k+1,len(g))))/k
 return c

def param(n,M,kind):
 H=[Q(0)]
 for k in range(1,(n+1)*M+1):H.append(H[-1]+Q(1,k))
 if kind=='cube':
  F=[comb(2*k,k)**n for k in range(M)]
  G=[2*n*F[k]*(H[2*k]-H[k]) for k in range(M)]
 elif kind=='simplex':
  F=[factorial((n+1)*k)//factorial(k)**(n+1) for k in range(M)]
  G=[(n+1)*F[k]*(H[(n+1)*k]-H[k]) for k in range(M)]
 elif kind=='octa':
  B=[1]*M
  for d in range(n-2):B=[sum(comb(k,j)**2*B[k-j] for j in range(k+1)) for k in range(M)]
  Bn=[sum(comb(k,j)**2*B[k-j] for j in range(k+1)) for k in range(M)]
  S=[sum(comb(k,j)**2*B[k-j]*H[j] for j in range(k+1)) for k in range(M)]
  F=[comb(2*k,k)*Bn[k] for k in range(M)]
  G=[2*comb(2*k,k)*(H[2*k]*Bn[k]-S[k]) for k in range(M)]
 return F,div(G,F,M)

def sigma(g,p,M):
 # Lagrange inversion T_j=1/j * [u^{j-1}] exp(-j*g(u)).
 J=(M-1)//p
 T=[Q(0)]+[exp([-j*x for x in g[:j]],j)[j-1]/j for j in range(1,J+1)]
 w=[Q(0)]*p+exp([p*x for x in g],M-p)
 c=[Q(0)]*M
 wp=[Q(1)]+[Q(0)]*(M-1)
 for j in range(1,J+1):
  wp=mul(wp,w,M)
  for k in range(M):c[k]+=T[j]*wp[k]
 return c

def compose(a,b,M):
 c=[Q(0)]*M
 for x in reversed(a[:M]):
  c=mul(c,b,M);c[0]+=x
 return c

def val(x,p):
 if x==0:return 999999
 if not isinstance(x,Q):x=Q(x)
 n,d=x.numerator,x.denominator;v=0
 while n%p==0:n//=p;v+=1
 while d%p==0:d//=p;v-=1
 return v

def check(n,p,s,kind,M):
 F,g=param(n,M,kind)
 sig=sigma(g,p,M)
 A=mul(F,compose(F[:p**(s-1)],sig,M),M)
 B=mul(F[:p**s],compose(F[:(M-1)//p+1],sig,M),M)
 E=[a-b for a,b in zip(A,B)]
 v=min(val(x,p) for x in E)
 i=next(i for i,x in enumerate(E) if val(x,p)==v)
 return {'kind':kind,'n':n,'p':p,'s':s,'M':M,'v':v,'first_min_index':i,'coefficient':str(E[i]),'q2':str(g
     [1]),'lift_integral':all(x.denominator==1 for x in sig)}


for kind,d,p,s in [('octa',3,5,1),('octa',3,5,2),('octa',4,5,1),('cube',4,5,1),('simplex',4,7,1)]:
    K=p**s;M=K+3
    F,g=param(d,M,kind);sig=sigma(g,p,M)
    lhs=mul(F,compose(F[:K//p],sig,M),M)
    rhs=mul(F[:K],compose(F[:(M-1)//p+1],sig,M),M)
```

```
    assert lhs[K]-rhs[K]==F[K]-F[K//p]
    assert lhs[K+1]-rhs[K+1]==F[K+1]-(F[1]+K*g[1])*F[K//p]

path=Path(__file__).with_name('sharpness_validation.json')
path.write_text(json.dumps({'description':'Finite exact-arithmetic validation; all-s proofs are in
     Exceptional_Excellent_Lift_Supercongruences.tex, Section 4','checks':out,'count':len(out)},indent=2)+'\
     n')
print('PASS:',len(out),'refinement checks; 14 Domb recurrence checks; 10 formal first-jet identities')
print('Constants:',[(r,C_r(r)) for r in range(2,9)])
for x in out:
    if x.get('family') in ['octahedral3','simplex'] and x['p'] in [5,7]:print(json.dumps(x))
```

D.4 verify_zagier_d.py

Original source: `verify_zagier_d.py`; 130 lines.

```
#!/usr/bin/env python3
"""Independent finite checks for both Zagier D theorems.
All arithmetic is exact. These checks support, but do not replace, the proof.
"""
from math import comb

def mul(a, b, N, modulus=None):
    c = [0]*N
    for i, x in enumerate(a[:N]):
        if x:
            for j, y in enumerate(b[:N-i]):
                if y:
                    c[i+j] += x*y
    if modulus:
        c = [x % modulus for x in c]
    return c

def inv(a, N, modulus=None):
    assert a[0] == 1
    b = [1]+[0]*(N-1)
    for n in range(1,N):
        b[n] = -sum(a[k]*b[n-k] for k in range(1,min(n+1,len(a))))
        if modulus:
            b[n] %= modulus
    return b

def poly(a, t, N, modulus=None):
    b=[0]*N
    for x in a[::-1]:
        b=mul(b,t,N,modulus)
        b[0]=(b[0]+x) if modulus is None else (b[0]+x)%modulus
    return b

def dilation(a,p,N):
    b=[0]*N
    for n,x in enumerate(a):
        if n*p>=N: break
        b[n*p]=x
    return b

def data(N, modulus=None):
    b=[1]+[0]*(N-1)
    for k in range(1,N):
        e=0 if k%5==0 else 5 if k%5 in (1,4) else -5
        for _ in range(abs(e)):
            if e>0:
                for j in range(N-1,k-1,-1):
                    b[j]-=b[j-k]
                    if modulus:b[j]%=modulus
            else:
                for j in range(k,N):
                    b[j]+=b[j-k]
                    if modulus:b[j]%=modulus
    t=[0]+b[:-1]
    theta=[1]+[0]*(N-1)
```

```
    ct=[0,3,1,-1,-3]
    for k in range(1,N):
        for j in range(k,N,k):
            theta[j]+=ct[k%5]
    if modulus:theta=[x%modulus for x in theta]
    return t,theta

def apery(M):
    a=[1]
    if M==1:return a
    a.append(3)
    for n in range(2,M):
        b=(11*n*n-11*n+3)*a[n-1]+(n-1)**2*a[n-2]
        assert b%(n*n)==0
        a.append(b//(n*n))
    return a

def t_expansion(f,t,N):
    b=list(f[:N]);out=[];power=[1]+[0]*(N-1)
    for k in range(N):
        c=b[k];out.append(c)
        for j in range(k,N):b[j]-=c*power[j]
        power=mul(power,t,N)
    return out

def v_p(x,p):
    if not x:return 10**9
    v=0
    while x%p==0:v+=1;x//=p
    return v

def check_trace(p,P):
    eps=1 if p%5==1 else -1
    bound=p*(len(P)-1)+p-1
    L=bound+9
    N=p*(L-1)+1
    t,theta=data(N)
    theta_p=dilation(theta,p,N)
    Q=mul(theta,mul(poly(P,dilation(t,p,N),N),inv(theta_p,N),N),N)
    G=mul(theta_p,mul(poly(P,t,N),inv(theta,N),N),N)
    trace=[Q[n]+eps*p*p*G[p*n] for n in range(L)]
    coefficients=t_expansion(trace,t,L)
    assert all(x==0 for x in coefficients[bound+1:])
    print("TRACE PASS",{"p":p,"P":P,"proved_degree_bound":bound,
                        "finite_verified_through_t_degree":L-1,
                        "constant_coefficient":coefficients[0]},flush=True)

def check_congruence(p,s,m,N):
    K=m*p**s;L=K//p
    assert N>K
    modulus=p**(2*s+1)
    t,theta=data(N,modulus)
    A=apery(K)
    lhs=mul(theta,poly(A[:L],dilation(t,p,N),N,modulus),N,modulus)
    rhs=mul(dilation(theta,p,N),poly(A,t,N,modulus),N,modulus)
    differences=[(x-y)%modulus for x,y in zip(lhs,rhs)]
    minimum=min(v_p(x,p) for x in differences)
    assert minimum>=2*s
    print("CONGRUENCE PASS",{"p":p,"s":s,"m":m,"through_q_degree":N-1,
                             "minimum_valuation_capped_at":2*s+1,
                             "observed_minimum":minimum if minimum<10**9 else ">="+str(2*s+1)},
          flush=True)

if __name__=="__main__":
    t,theta=data(65)
    assert poly(apery(65),t,65)==theta
    print("MODULAR PARAMETRIZATION CHECK PASS through q^64",flush=True)
    for p,P in [(11,[1]),(19,[1]),(11,[2,-3])]:
        check_trace(p,P)
    for values in [(11,1,1,45),(19,1,1,55),(29,1,1,70),(31,1,1,75),
                   (11,1,2,60),(19,1,2,85),(11,2,1,170),(19,2,1,410),
                   (11,2,2,300), (2,1,1,22),(2,1,2,25),
                   (3,1,1,25),(3,1,2,30),(7,1,2,45)]:
        check_congruence(*values)
```

D.5 verify_octahedral_interpolation.py

Original source: `verify_octahedral_interpolation.py`; 421 lines.

```
"""Bounded independent checks for the all-dimensional octahedral proof.

Standard library only.  Besides checking interpolation and direct defects,
this script constructs the proof's row operator D from a truncated restricted
unit-factorial series and the symmetric Bessel operator reduction.  Thus its
row checks do not merely solve the desired identities by interpolation.
The infinite theorem is proved in Section 3 of the accompanying self-contained research note.
"""

from fractions import Fraction as Q
from math import comb, factorial
from pathlib import Path
import json


INF = 10**9


def vp(x, p):
    x = Q(x)
    if not x:
        return INF
    a, b, v = x.numerator, x.denominator, 0
    while a % p == 0:
        a //= p
        v += 1
    while b % p == 0:
        b //= p
        v -= 1
    return v


def residue(x, modulus):
    x = Q(x)
    return x.numerator*pow(x.denominator, -1, modulus) % modulus


def poly_add(A, B):
    C = A.copy()
    for k, x in B.items():
        C[k] = C.get(k, 0)+x
    return {k: x for k, x in C.items() if x}


def poly_scale(A, c):
    return {k: c*x for k, x in A.items() if c*x}


def poly_mul(A, B):
    C = {}
    for i, x in A.items():
        for j, y in B.items():
            C[i+j] = C.get(i+j, 0)+x*y
    return {k: x for k, x in C.items() if x}


def poly_mod(A, modulus):
    return {k: residue(x, modulus) for k, x in A.items()
            if residue(x, modulus)}


def poly_mul_mod(A, B, modulus):
    C = {}
    for i, x in A.items():
        for j, y in B.items():
            C[i+j] = (C.get(i+j, 0)+x*y) % modulus
    return {k: x for k, x in C.items() if x}


def sum_of_powers_polynomials(degree):
```

```
    # S_d(X)=sum_{0<=k<X} k^d, using Stirling numbers and falling factorials.
    stirling = [[1]]
    for d in range(1, degree+1):
        old = stirling[-1]
        row = [0]*(d+1)
        for a in range(1, d+1):
            row[a] = (old[a-1] if a-1 < len(old) else 0)
            if a < len(old):
                row[a] += a*old[a]
        stirling.append(row)
    falling = [{0: Q(1)}]
    for k in range(degree+1):
        falling.append(poly_mul(falling[-1], {1: Q(1), 0: Q(-k)}))
    result = []
    for d in range(degree+1):
        S = {}
        for a in range(d+1):
            S = poly_add(S, poly_scale(falling[a+1], Q(stirling[d][a], a+1)))
        result.append(S)
    return result


def unit_factorial_polynomials(p, precision, check_through):
    """Construct T_r^full=1-2p H_r X+p^2T_r modulo p^precision."""
    modulus = p**precision
    cutoff = precision+2
    sums = sum_of_powers_polynomials(2*cutoff)
    logA = {}
    for m in range(1, cutoff+1):
        coefficient = Q((-1)**(m+1)*p**(2*m), m)*sum(
            (Q(1, (a*(p-a))**m) for a in range(1, (p-1)//2+1)), Q(0))
        S = {}
        for b in range(m+1):
            S = poly_add(S, poly_scale(sums[m+b], comb(m, b)))
        logA = poly_add(logA, poly_scale(S, coefficient))
    logA = {k: x for k, x in logA.items() if vp(x, p) < precision}
    assert all(vp(x, p) >= 2 for x in logA.values())
    exponent = poly_scale(logA, -2)
    expA, power = {0: Q(1)}, {0: Q(1)}
    for k in range(1, precision+1):
        power = poly_mul(power, exponent)
        expA = poly_add(expA, poly_scale(power, Q(1, factorial(k))))
    expA = {k: x for k, x in expA.items() if vp(x, p) < precision}
    short = {0: Q(1)}
    polys = []
    H = Q(0)
    for r in range(p):
        if r:
            H += Q(1, r)
            factor = {d: Q((-1)**d*(d+1)*p**d, r**d)
                      for d in range(precision)}
            short = poly_mul(short, factor)
            short = {k: x for k, x in short.items() if vp(x, p) < precision}
        T = poly_mod(poly_mul(expA, short), modulus)
        assert T.get(0, 0) % (p*p) == 1
        assert (T.get(1, 0)-residue(-2*p*H, modulus)) % (p*p) == 0
        assert all(x % (p*p) == 0 for k, x in T.items() if k >= 2)
        polys.append(T)
    unit = [1]
    for a in range(1, p*check_through+p):
        unit.append(unit[-1]*(a if a % p else 1) % modulus)
    c = factorial(p-1)
    for ell in range(check_through+1):
        for r in range(p):
            exact = pow(c, 2*ell, modulus)*factorial(r)**2*pow(
                unit[p*ell+r], -2, modulus) % modulus
            evaluated = sum(x*pow(ell, k, modulus) for k, x in polys[r].items()) % modulus
            assert evaluated == exact, (p, precision, ell, r)
    return polys


def residue_weight_polynomials(n, p, precision, polys):
    modulus = p**precision
    current = {(0, ()): 1}
```

```
    for coordinate in range(n):
        following = {}
        for (residue_sum, exponents), x in current.items():
            for r in range(p):
                scale = pow(factorial(r), -2, modulus)
                for degree, y in polys[r].items():
                    value = x*scale*y % modulus
                    if not value:
                        continue
                    key = (residue_sum+r, exponents+(degree,))
                    following[key] = (following.get(key, 0)+value) % modulus
        current = {key: x for key, x in following.items() if x}
    result = {}
    for (residue_sum, exponents), x in current.items():
        result.setdefault(residue_sum, {})[exponents] = x
    for P in result.values():
        for exponents, x in P.items():
            if sum(exponents) >= 2:
                assert x % (p*p) == 0
            swapped = exponents[1:2]+exponents[:1]+exponents[2:]
            assert P.get(swapped, 0) == x
        linear = []
        for i in range(n):
            e = tuple(1 if k == i else 0 for k in range(n))
            linear.append(P.get(e, 0))
        assert len(set(x % (p*p) for x in linear)) == 1
    return result


def bessel_operator_basis(n, modulus):
    # O_k gives E_k=O_k(x,theta) Phi modulo N^2, in normal order x^a theta^b.
    operators = [{(0, 0): 1}, {(0, 1): 1}]
    for k in range(1, n):
        out = {}
        for (a, b), x in operators[k].items():
            out[(a, b+1)] = (out.get((a, b+1), 0)+x) % modulus
            out[(a, b)] = (out.get((a, b), 0)+a*x) % modulus
        for (a, b), x in operators[k-1].items():
            out[(a+1, b)] = (out.get((a+1, b), 0)-(n-k+1)*x) % modulus
        inverse = pow(k+1, -1, modulus)
        operators.append({key: x*inverse % modulus for key, x in out.items()
                          if x*inverse % modulus})
    return operators


def reduce_symmetric_polynomial(P, n, modulus, operators):
    maximum = max((max(e) for e in P), default=0)
    normal = [({0: 1}, {})]
    for exponent in range(maximum):
        A, B = normal[-1]
        nextA = {k: k*x % modulus for k, x in A.items() if k*x % modulus}
        nextA = poly_add(nextA, {k+1: x for k, x in B.items()})
        nextB = poly_add(A, {k: k*x % modulus for k, x in B.items() if k*x % modulus})
        normal.append((poly_mod(nextA, modulus), poly_mod(nextB, modulus)))
    out = {}
    for k in range(n+1):
        coefficient = {}
        for exponents, x in P.items():
            term = {0: x}
            for i, e in enumerate(exponents):
                term = poly_mul_mod(term, normal[e][1 if i < k else 0], modulus)
                if not term:
                    break
            for a, y in term.items():
                coefficient[a] = (coefficient.get(a, 0)+y) % modulus
        for a, x in coefficient.items():
            for (b, order), y in operators[k].items():
                key = (a+b, order)
                out[key] = (out.get(key, 0)+x*y) % modulus
    return {key: x for key, x in out.items() if x}


def octahedral_matrix(n, bound, modulus):
    """Use the even exponential convolution B_n=A_(n-1) B_1."""
```

```
    even, B1 = [], []
    for j in range(bound+1):
        row, value = [], 1
        for k in range(2*j+1):
            row.append(value % modulus)
            if k < 2*j:
                value = value*(2*j-k)//(k+1)
        even.append(row[::2])
        B1.append([row[j-N] for N in range(j+1)])
    one = [row[0] for row in B1]
    lower = one
    for dimension in range(2, n):
        lower = [sum(even[j][k]*one[k]*lower[j-k] for k in range(j+1)) % modulus
                 for j in range(bound+1)]
    matrix = []
    for j in range(bound+1):
        row = [0]*(j+1)
        for k in range(j+1):
            multiplier = even[j][k]*lower[j-k] % modulus
            if multiplier:
                for N, value in enumerate(B1[k]):
                    row[N] = (row[N]+multiplier*value) % modulus
        assert row[j] == 1
        matrix.append(row)
    return matrix


def construct_and_check_rows(n, p, precision, bound):
    modulus = p**precision
    polys = unit_factorial_polynomials(p, precision, bound//p+2)
    weights = residue_weight_polynomials(n, p, precision, polys)
    operators = bessel_operator_basis(n, modulus)
    reduced = {s: reduce_symmetric_polynomial(P, n, modulus, operators)
               for s, P in weights.items()}
    assert all(x % (p*p) == 0 for A in reduced.values()
               for (degree, order), x in A.items() if degree > 0)
    B = octahedral_matrix(n, bound, modulus)
    a = [row[0] for row in B]
    unit = [1]
    for j in range(1, 2*bound+1):
        unit.append(unit[-1]*(j if j % p else 1) % modulus)
    c = factorial(p-1)
    tested_columns = tested_boundaries = 0
    for j in range(bound+1):
        L, b = divmod(j, p)
        epsilon = (2*b)//p
        D = []
        for i in range(L+1):
            total = 0
            for d in range(min(n-1, L-i)+1):
                A = reduced.get(b+p*d, {})
                value = sum(A.get((L-d-i, k), 0)*pow(i, k, modulus)
                            for k in range(n+1)) % modulus
                total += p**(2*d)*pow(c, -2*(L-d), modulus)*value
            factorial_ratio = factorial(2*L+epsilon)//factorial(2*i)
            D.append(p**epsilon*unit[2*j]*factorial_ratio*total % modulus)
        assert (a[j]-sum(D[i]*a[i] for i in range(L+1))) % modulus == 0, (
            'central', n, p, j)
        for N in range(1, L+1):
            required = min(precision, 2+2*vp(N, p))
            delta = B[j][p*N]-sum(D[i]*B[i][N] for i in range(N, L+1))
            assert delta % (p**required) == 0, ('shifted', n, p, j, N, required)
            tested_columns += 1
        for i in range(L):
            required = min(precision, 2+2*(vp(factorial(L), p)-vp(factorial(i), p)))
            assert D[i]*a[i] % (p**required) == 0, ('boundary', n, p, j, i, required)
            tested_boundaries += 1
    return dict(dimension=n, p=p, working_precision=precision,
                through_row=bound, unit_factorial_interpolation_passed=True,
                symmetric_Bessel_reduction_constructed=True,
                central_identity_passed=True, shifted_columns_tested=tested_columns,
                boundary_terms_tested=tested_boundaries,
                all_row_operator_checks_passed=True)
```

```
def interpolation_vector(B, K, bound, modulus):
    c = [0]*(bound+1)
    for j in range(K, bound+1):
        c[j] = (B[j][0]-sum(c[N]*B[j][N] for N in range(K, j))) % modulus
    return c


def check_descent(n, p, s, m, bound):
    K = m*p**s
    v = vp(K, p)
    modulus = p**(2*v+3)
    B = octahedral_matrix(n, bound, modulus)
    c = interpolation_vector(B, K, bound, modulus)
    d = interpolation_vector(B, K//p, bound//p, modulus)
    minimum, first = INF, None
    for N in range(1, bound+1):
        assert vp(c[N], p) >= max(0, 2*v-2*vp(N, p))
        expected = d[N//p] if N % p == 0 else 0
        value = vp((c[N]-expected) % modulus, p)
        assert value >= 2*v, (n, p, s, m, N, value)
        if value < minimum:
            minimum, first = value, N
    return dict(dimension=n, p=p, s=s, m=m, K=K, through_column=bound,
                required_valuation=2*v, observed_minimum_valuation=minimum,
                first_minimum_column=first, weighted_bound_passed=True,
                descent_passed=True)


def mul(a, b, length):
    out = [Q(0)]*length
    for i, x in enumerate(a[:length]):
        if x:
            for j, y in enumerate(b[:length-i]):
                if y:
                    out[i+j] += x*y
    return out


def divide(a, b, length):
    assert b[0] == 1
    out = [Q(0)]*length
    for k in range(length):
        out[k] = (a[k] if k < len(a) else 0)-sum(
            b[j]*out[k-j] for j in range(1, min(k+1, len(b))))
    return out


def exponential(g, length):
    out = [Q(1)]+[Q(0)]*(length-1)
    for k in range(1, length):
        out[k] = sum(j*g[j]*out[k-j]
                     for j in range(1, min(k+1, len(g))))/k
    return out


def compose(a, b, length):
    out = [Q(0)]*length
    for coefficient in reversed(a[:length]):
        out = mul(out, b, length)
        out[0] += coefficient
    return out


def canonical_lift(g, p, length):
    J = (length-1)//p
    inverse = [Q(0)]+[
        exponential([-j*x for x in g[:j]], j)[j-1]/j
        for j in range(1, J+1)]
    qp = [Q(0)]*p+exponential([p*x for x in g], length-p)
    return compose(inverse, qp, length)


def exact_period(n, length):
```

```
    b = [1]*length
    for dimension in range(n-1):
        b = [sum(comb(j, k)**2*b[j-k] for k in range(j+1)) for j in range(length)]
    return [comb(2*j, j)*b[j] for j in range(length)]


def check_direct_defect(n, p, s, m, length):
    K = m*p**s
    assert length > K
    H = [Q(0)]
    for j in range(1, 2*length+1):
        H.append(H[-1]+Q(1, j))
    F = exact_period(n, length)
    lower = exact_period(n-1, length)
    G = [2*sum(comb(2*j, 2*k)*comb(2*k, k)*lower[j-k]*(H[2*j]-H[k])
               for k in range(j+1)) for j in range(length)]
    g = divide(G, F, length)
    sig = canonical_lift(g, p, length)
    assert all(vp(x, p) >= 0 for x in sig)
    left = mul(F, compose(F[:K//p], sig, length), length)
    right = mul(F[:K], compose(F, sig, length), length)
    values = [vp(x-y, p) for x, y in zip(left, right)]
    minimum = min(values)
    assert minimum >= 2*vp(K, p)
    return dict(dimension=n, p=p, s=s, m=m, through_u_degree=length-1,
                actual_BV_logarithmic_companion=True, lift_p_integral=True,
                required_valuation=2*vp(K, p), observed_minimum_valuation=minimum,
                first_minimum_degree=values.index(minimum), passed=True)


def main():
    results = dict(scope='Bounded exact checks of the explicit row construction, interpolation, and direct
     defects; the infinite theorem is proved separately.',
                   row_operator_checks=[], interpolation_checks=[], direct_defect_checks=[])
    for case in [(3, 5, 6, 145), (4, 5, 6, 145)]:
        result = construct_and_check_rows(*case)
        results['row_operator_checks'].append(result)
        print('PASS constructed rows', json.dumps(result), flush=True)
    for case in [(3, 5, 3, 1, 300), (4, 5, 3, 1, 300),
                 (5, 7, 2, 1, 160), (6, 7, 2, 1, 160),
                 (3, 5, 2, 2, 150)]:
        result = check_descent(*case)
        results['interpolation_checks'].append(result)
        print('PASS descent', json.dumps(result), flush=True)
    for case in [(3, 5, 3, 1, 137), (4, 5, 3, 1, 137), (5, 7, 2, 1, 75)]:
        result = check_direct_defect(*case)
        results['direct_defect_checks'].append(result)
        print('PASS direct defect', json.dumps(result), flush=True)
    results['all_passed'] = True
    destination = Path(__file__).with_name('octahedral_interpolation_validation.json')
    destination.write_text(json.dumps(results, indent=2)+'\n')
    print('ALL CHECKS PASSED', flush=True)


if __name__ == '__main__':
    main()
```

D.6 verify_positive_interpolation.py

Original source: `verify_positive_interpolation.py`; 218 lines.

```
"""Bounded exact checks for the positive-vertex proof.

Standard library only.  The theorem, including every series coefficient,
is proved in Section 2 of the accompanying self-contained research note.  This script tests three
independent components: the factorial block lemma, the integer triangular
descent, and direct ordinary truncation defects computed from the canonical
coordinate.  Finite arithmetic checks are not used in place of the proof.
"""

from fractions import Fraction as Q
```

```
from functools import lru_cache
from math import comb, factorial
from pathlib import Path
import json


def vp(x, p):
    x = Q(x)
    if not x:
        return 10**9
    a, b, v = x.numerator, x.denominator, 0
    while a % p == 0:
        a //= p
        v += 1
    while b % p == 0:
        b //= p
        v -= 1
    return v


def coefficient_matrix(family, dimension):
    n = dimension

    @lru_cache(None)
    def entry(j, N):
        if N > j or N < 0:
            return 0
        if family == 'cube':
            return comb(2*j, j-N)**n
        r = n+1
        return factorial(r*j)//(factorial(j+(r-1)*N)*factorial(j-N)**(r-1))

    return entry


def check_blocks(family, n, p, bound):
    B = coefficient_matrix(family, n)
    count, slack = 0, 10**9
    for j in range(bound+1):
        l = j//p
        d = Q(B(j, 0), B(l, 0))
        assert vp(d, p) >= 0
        for N in range(1, l+1):
            required = 2+2*vp(N, p)
            value = vp(B(j, p*N)-d*B(l, N), p)
            assert value >= required, (family, n, p, j, N, value, required)
            count += 1
            if value < 10**9:
                slack = min(slack, value-required)
    return dict(family=family, dimension=n, p=p, through_row=bound,
                tested_entries=count, minimum_extra_valuation=slack,
                passed=True)


def interpolation_vector(B, K, bound, modulus):
    c = [0]*(bound+1)
    for j in range(K, bound+1):
        c[j] = (B(j, 0)-sum(c[N]*(B(j, N) % modulus)
                            for N in range(K, j))) % modulus
    return c


def check_descent(family, n, p, s, m, bound):
    K = m*p**s
    v = vp(K, p)
    modulus = p**(2*v+3)
    B = coefficient_matrix(family, n)
    c = interpolation_vector(B, K, bound, modulus)
    d = interpolation_vector(B, K//p, bound//p, modulus)
    minimum, first = 10**9, None
    for N in range(1, bound+1):
        assert vp(c[N], p) >= max(0, 2*v-2*vp(N, p))
        expected = d[N//p] if N % p == 0 else 0
        value = vp((c[N]-expected) % modulus, p)
```

```
        assert value >= 2*v, (family, n, p, s, m, N, value)
        if value < minimum:
            minimum, first = value, N
    return dict(family=family, dimension=n, p=p, s=s, m=m, K=K,
                through_column=bound, working_modulus=modulus,
                required_valuation=2*v, observed_minimum_valuation=minimum,
                first_minimum_column=first, weighted_bound_passed=True,
                descent_passed=True)


def mul(a, b, length):
    out = [Q(0)]*length
    for i, x in enumerate(a[:length]):
        if x:
            for j, y in enumerate(b[:length-i]):
                if y:
                    out[i+j] += x*y
    return out


def divide(a, b, length):
    assert b[0] == 1
    out = [Q(0)]*length
    for k in range(length):
        out[k] = (a[k] if k < len(a) else 0)-sum(
            b[j]*out[k-j] for j in range(1, min(k+1, len(b))))
    return out


def exponential(g, length):
    assert not g or g[0] == 0
    out = [Q(1)]+[Q(0)]*(length-1)
    for k in range(1, length):
        out[k] = sum(j*g[j]*out[k-j]
                     for j in range(1, min(k+1, len(g))))/k
    return out


def compose(a, b, length):
    out = [Q(0)]*length
    for coefficient in reversed(a[:length]):
        out = mul(out, b, length)
        out[0] += coefficient
    return out


def parameters(family, n, length):
    H = [Q(0)]
    for j in range(1, (n+1)*length+1):
        H.append(H[-1]+Q(1, j))
    B = coefficient_matrix(family, n)
    F = [B(j, 0) for j in range(length)]
    if family == 'cube':
        G = [2*n*F[j]*(H[2*j]-H[j]) for j in range(length)]
    else:
        r = n+1
        G = [r*F[j]*(H[r*j]-H[j]) for j in range(length)]
    return F, divide(G, F, length)


def canonical_lift(g, p, length):
    # Lagrange inversion of q=u exp(g(u)), followed by q -> q^p.
    J = (length-1)//p
    inverse = [Q(0)]+[
        exponential([-j*x for x in g[:j]], j)[j-1]/j
        for j in range(1, J+1)]
    qp = [Q(0)]*p+exponential([p*x for x in g], length-p)
    return compose(inverse, qp, length)


def check_full_defect(family, n, p, s, m, length):
    K = m*p**s
    assert length > K
    F, g = parameters(family, n, length)
```

```
    sig = canonical_lift(g, p, length)
    assert all(vp(x, p) >= 0 for x in sig)
    left = mul(F, compose(F[:K//p], sig, length), length)
    right = mul(F[:K], compose(F, sig, length), length)
    difference = [x-y for x, y in zip(left, right)]
    values = [vp(x, p) for x in difference]
    minimum = min(values)
    required = 2*vp(K, p)
    assert minimum >= required, (family, n, p, s, m, minimum)
    return dict(family=family, dimension=n, p=p, s=s, m=m,
                through_u_degree=length-1, required_valuation=required,
                observed_minimum_valuation=minimum,
                first_minimum_degree=values.index(minimum),
                lift_p_integral=True, passed=True)


def main():
    results = dict(
        scope='Bounded exact arithmetic supporting the all-level proof; not an infinite proof.',
        block_checks=[], interpolation_checks=[], direct_defect_checks=[])
    block_cases = [
        ('cube', 1, 3, 120), ('cube', 4, 5, 150),
        ('cube', 6, 7, 150), ('cube', 8, 3, 100),
        ('simplex', 4, 7, 150), ('simplex', 5, 7, 120),
        ('simplex', 6, 3, 100), ('simplex', 9, 5, 100)]
    for case in block_cases:
        result = check_blocks(*case)
        results['block_checks'].append(result)
        print('PASS block', json.dumps(result), flush=True)

    descent_cases = [
        ('cube', 4, 5, 1, 1, 130), ('cube', 4, 5, 2, 1, 180),
        ('cube', 4, 5, 3, 1, 400), ('cube', 4, 5, 2, 2, 160),
        ('cube', 5, 7, 2, 1, 180), ('cube', 6, 7, 2, 1, 180),
        ('simplex', 4, 7, 1, 1, 120), ('simplex', 4, 7, 2, 1, 160),
        ('simplex', 4, 7, 2, 2, 180), ('simplex', 5, 7, 2, 1, 160),
        ('simplex', 6, 11, 2, 1, 200)]
    for case in descent_cases:
        result = check_descent(*case)
        results['interpolation_checks'].append(result)
        print('PASS descent', json.dumps(result), flush=True)

    defect_cases = [
        ('cube', 4, 5, 2, 1, 81), ('cube', 4, 5, 3, 1, 139),
        ('cube', 4, 5, 2, 2, 75), ('cube', 5, 7, 2, 1, 79),
        ('simplex', 4, 7, 2, 1, 82), ('simplex', 5, 7, 2, 1, 77),
        ('simplex', 4, 7, 2, 2, 111)]
    for case in defect_cases:
        result = check_full_defect(*case)
        results['direct_defect_checks'].append(result)
        print('PASS direct defect', json.dumps(result), flush=True)

    results['all_passed'] = True
    destination = Path(__file__).with_name('positive_interpolation_validation.json')
    destination.write_text(json.dumps(results, indent=2)+'\n')
    print('ALL CHECKS PASSED', flush=True)


if __name__ == '__main__':
    main()
```

D.7 verify_canonical_basis.py

Original source: `verify_canonical_basis.py`; 224 lines.

```
"""Finite exact checks for the all-level canonical negative-power basis theorem.

Standard-library only. The all-prime, all-dimension statement is proved in
canonical_negative_basis.tex; the checks below are not an infinite proof.
"""
from fractions import Fraction as Q
```

```
from math import comb, factorial
from pathlib import Path
import json


def mul(a, b, N):
    c = [Q(0)] * N
    for i, x in enumerate(a[:N]):
        if x:
            for j, y in enumerate(b[:N-i]):
                if y:
                    c[i+j] += x*y
    return c


def div(a, b, N):
    assert b[0] == 1
    c = [Q(0)] * N
    for k in range(N):
        c[k] = (a[k] if k < len(a) else 0) - sum(
            b[j]*c[k-j] for j in range(1, min(k+1, len(b))))
    return c


def exp(g, N):
    c = [Q(1)] + [Q(0)]*(N-1)
    for k in range(1, N):
        c[k] = sum(j*g[j]*c[k-j] for j in range(1, min(k+1, len(g))))/k
    return c


def compose(a, b, N):
    c = [Q(0)]*N
    for x in reversed(a[:N]):
        c = mul(c, b, N)
        c[0] += x
    return c


def power(a, k, N):
    c = [Q(1)] + [Q(0)]*(N-1)
    for _ in range(k):
        c = mul(c, a, N)
    return c


def vp(x, p):
    x = Q(x)
    if not x:
        return 1000000
    n, d, v = x.numerator, x.denominator, 0
    while n % p == 0:
        v += 1
        n //= p
    while d % p == 0:
        v -= 1
        d //= p
    return v


def octa_coefficients(n, N):
    b = [1]*N
    for _ in range(n-1):
        b = [sum(comb(k,j)**2*b[k-j] for j in range(k+1)) for k in range(N)]
    return [comb(2*k,k)*b[k] for k in range(N)]


def parameters(kind, n, N):
    harmonic = [Q(0)]
    for k in range(1, (n+1)*N+1):
        harmonic.append(harmonic[-1]+Q(1,k))
    if kind == 'cube':
        f = [comb(2*k,k)**n for k in range(N)]
        g = [2*n*f[k]*(harmonic[2*k]-harmonic[k]) for k in range(N)]
```

```
    elif kind == 'simplex':
        r = n+1
        f = [factorial(r*k)//factorial(k)**r for k in range(N)]
        g = [r*f[k]*(harmonic[r*k]-harmonic[k]) for k in range(N)]
    else:
        b = [1]*N
        for _ in range(n-2):
            b = [sum(comb(k,j)**2*b[k-j] for j in range(k+1)) for k in range(N)]
        bn = [sum(comb(k,j)**2*b[k-j] for j in range(k+1)) for k in range(N)]
        hsum = [sum(comb(k,j)**2*b[k-j]*harmonic[j] for j in range(k+1)) for k in range(N)]
        f = [comb(2*k,k)*bn[k] for k in range(N)]
        g = [2*comb(2*k,k)*(harmonic[2*k]*bn[k]-hsum[k]) for k in range(N)]
    return f, div(g, f, N)


def canonical_lift(g, p, N):
    # Revert q=u exp(g) by Lagrange inversion, then substitute q(u)^p.
    J = (N-1)//p
    reverse = [Q(0)] + [exp([-j*x for x in g[:j]],j)[j-1]/j
                        for j in range(1,J+1)]
    qp = [Q(0)]*p+exp([p*x for x in g],N-p)
    return compose(reverse,qp,N)


def hasse(f, h, N, length):
    result = [Q(0)]*length
    for k in range(min(length,(N-1)//h+1)):
        result[k] = (-1)**(h*k)*comb(N-1,h*k)*f[k]
    return result


def vertex_coefficient(kind,n,p,j):
    if kind == 'cube':
        return comb(p+2*j,j)**n
    if kind == 'simplex':
        r = n+1
        return factorial(p+r*j)//(factorial(p+j)*factorial(j)**(r-1))
    lower = octa_coefficients(n-1,j+1)
    return sum(comb(p+2*j,2*l)*lower[l]*comb(p+2*(j-l),j-l)
               for l in range(j+1))


def run_case(kind,n,p):
    N = 3*p+9
    h = n+1 if kind == 'simplex' else 2
    f,g = parameters(kind,n,N)
    sig = canonical_lift(g,p,N)
    hp = hasse(f,h,p,N)
    h2p = hasse(f,h,2*p,N)
    poly = [2*x-y for x,y in zip(hp,h2p)]
    assert all(vp(poly[k]-(f[k] if k<p else 0),p)>=2 for k in range(N))
    b = [Q(0)]*N
    for j in range((p-1)//h+1):
        b[j] = (-1)**(p+h*j)*comb(2*p-1,p+h*j)*vertex_coefficient(kind,n,p,j)
    ratio = div([-x for x in b],h2p,N)
    approximate_lift = [Q(0)]*p+power(ratio,h,N-p)
    assert all(vp(x-y,p)>=2 for x,y in zip(sig,approximate_lift))
    fp_sig = compose(f,sig,N)
    quotient = div(f,fp_sig,N)
    difference = [x-y for x,y in zip(quotient,poly)]
    assert all(vp(x,p)>=2 for x in difference)
    return dict(family=kind,dimension=n,p=p,through_u_degree=N-1,
                minimum_valuation=min(vp(x,p) for x in difference),
                lift_formula_passed=True,quotient_formula_passed=True,
                polynomial_identification_passed=True)



def vertex_polynomial(kind,n,N):
    h=n+1 if kind=='simplex' else 2
    M=h*N
    if kind=='cube':
        return [(-1)**(n*j)*comb(M-j-1,j)**n for j in range(N)]
    if kind=='simplex':
```

```
        r=n+1
        result=[]
        for j in range(N):
            numerator=factorial(M-j-1)
            denominator=factorial(M-r*j-1)*factorial(j)**(r-1)
            assert numerator % denominator == 0
            result.append((-1)**((r-1)*j)*(numerator//denominator))
        return result
    lower=octa_coefficients(n-1,N)
    return [sum((-1)**(j-l)*comb(M-j+l-1,2*l)*comb(M-j-l-1,j-l)*lower[l]
                for l in range(j+1)) for j in range(N)]


def twice_q_derivative_cleared(a,F,g,K,N):
    # If R=u^-K*a/F, return u^K*(q d/dq)^2 R.
    b=div(a,F,N)
    scale=[1]+[j*g[j] for j in range(1,N)]
    first=div([(j-K)*b[j] for j in range(N)],scale,N)
    return div([(j-K)*first[j] for j in range(N)],scale,N)


def canonical_case(kind,n,p,K):
    # Test n=pK, so the required modulus is p^(2*(1+v_p(K))).
    L=p*K
    N=L+12
    expected=2*(1+vp(K,p))
    F,g=parameters(kind,n,N+p)
    sig=canonical_lift(g,p,N+p)
    Fs=compose(F,sig,N)
    ratio=power(div([1],sig[p:],N),K,N)
    A_L=vertex_polynomial(kind,n,L)
    A_K=vertex_polynomial(kind,n,K)
    assert A_L[0]==A_K[0]==1
    R_error=[x-y for x,y in zip(mul(A_L,Fs,N),
             mul(ratio,mul(compose(A_K,sig,N),F,N),N))]
    R_val=min(vp(x,p) for x in R_error)
    assert R_val>=expected
    D2=twice_q_derivative_cleared(A_L,F,g,L,N)
    derivative_val=min(vp(x,p) for x in D2)
    assert derivative_val>=expected
    # Canonical numerator comes from [F exp(-N*g)]_<N.
    C_L=mul(F,exp([-L*x for x in g],L),L)
    C_K=mul(F,exp([-K*x for x in g],K),K)
    assert all(vp(x,p)>=0 for x in C_L+C_K)
    error=[x-y for x,y in zip(mul(C_L,Fs,N),
           mul(ratio,mul(compose(C_K,sig,N),F,N),N))]
    minimum=min(vp(x,p) for x in error)
    assert minimum>=expected
    result=dict(family=kind,dimension=n,p=p,base_index=K,canonical_index=L,
                required_valuation=expected,cleared_coefficients_checked=N,
                tested_laurent_exponents=[-L,N-1-L],
                canonical_frobenius_minimum=minimum,
                vertex_frobenius_minimum=R_val,
                second_derivative_minimum=derivative_val,
                all_passed=True)
    return result


if __name__=='__main__':
    cases=[('cube',4,5,1),('cube',4,5,5),('cube',4,5,10),('cube',5,7,7),
           ('simplex',4,7,1),('simplex',4,7,7),('simplex',5,7,7),
           ('octa',3,5,1),('octa',3,5,5),('octa',4,5,5)]
    results=[]
    for case in cases:
        result=canonical_case(*case)
        results.append(result)
        print('PASS',json.dumps(result),flush=True)
    output=dict(scope='Finite exact checks for an auxiliary negative-power canonical basis, not the positive
      truncation conjecture.',
                all_passed=True,cases=results)
    Path(__file__).with_name('canonical_basis_validation.json').write_text(json.dumps(output,indent=2)+'\n')
```

D.8 verify_first_level.py

Original source: `verify_first_level.py`; 159 lines.

```
"""Finite exact checks for the all-dimensional first-level theorem.

Standard-library only. The all-prime, all-dimension statement is proved in
first_level_all_dimensions.tex; the checks below are not an infinite proof.
"""
from fractions import Fraction as Q
from math import comb, factorial
from pathlib import Path
import json


def mul(a, b, N):
    c = [Q(0)] * N
    for i, x in enumerate(a[:N]):
        if x:
            for j, y in enumerate(b[:N-i]):
                if y:
                    c[i+j] += x*y
    return c


def div(a, b, N):
    assert b[0] == 1
    c = [Q(0)] * N
    for k in range(N):
        c[k] = (a[k] if k < len(a) else 0) - sum(
            b[j]*c[k-j] for j in range(1, min(k+1, len(b))))
    return c


def exp(g, N):
    c = [Q(1)] + [Q(0)]*(N-1)
    for k in range(1, N):
        c[k] = sum(j*g[j]*c[k-j] for j in range(1, min(k+1, len(g))))/k
    return c


def compose(a, b, N):
    c = [Q(0)]*N
    for x in reversed(a[:N]):
        c = mul(c, b, N)
        c[0] += x
    return c


def power(a, k, N):
    c = [Q(1)] + [Q(0)]*(N-1)
    for _ in range(k):
        c = mul(c, a, N)
    return c


def vp(x, p):
    x = Q(x)
    if not x:
        return 1000000
    n, d, v = x.numerator, x.denominator, 0
    while n % p == 0:
        v += 1
        n //= p
    while d % p == 0:
        v -= 1
        d //= p
    return v


def octa_coefficients(n, N):
    b = [1]*N
    for _ in range(n-1):
        b = [sum(comb(k,j)**2*b[k-j] for j in range(k+1)) for k in range(N)]
```

```
    return [comb(2*k,k)*b[k] for k in range(N)]


def parameters(kind, n, N):
    harmonic = [Q(0)]
    for k in range(1, (n+1)*N+1):
        harmonic.append(harmonic[-1]+Q(1,k))
    if kind == 'cube':
        f = [comb(2*k,k)**n for k in range(N)]
        g = [2*n*f[k]*(harmonic[2*k]-harmonic[k]) for k in range(N)]
    elif kind == 'simplex':
        r = n+1
        f = [factorial(r*k)//factorial(k)**r for k in range(N)]
        g = [r*f[k]*(harmonic[r*k]-harmonic[k]) for k in range(N)]
    else:
        b = [1]*N
        for _ in range(n-2):
            b = [sum(comb(k,j)**2*b[k-j] for j in range(k+1)) for k in range(N)]
        bn = [sum(comb(k,j)**2*b[k-j] for j in range(k+1)) for k in range(N)]
        hsum = [sum(comb(k,j)**2*b[k-j]*harmonic[j] for j in range(k+1)) for k in range(N)]
        f = [comb(2*k,k)*bn[k] for k in range(N)]
        g = [2*comb(2*k,k)*(harmonic[2*k]*bn[k]-hsum[k]) for k in range(N)]
    return f, div(g, f, N)


def canonical_lift(g, p, N):
    # Revert q=u exp(g) by Lagrange inversion, then substitute q(u)^p.
    J = (N-1)//p
    reverse = [Q(0)] + [exp([-j*x for x in g[:j]],j)[j-1]/j
                        for j in range(1,J+1)]
    qp = [Q(0)]*p+exp([p*x for x in g],N-p)
    return compose(reverse,qp,N)


def hasse(f, h, N, length):
    result = [Q(0)]*length
    for k in range(min(length,(N-1)//h+1)):
        result[k] = (-1)**(h*k)*comb(N-1,h*k)*f[k]
    return result


def vertex_coefficient(kind,n,p,j):
    if kind == 'cube':
        return comb(p+2*j,j)**n
    if kind == 'simplex':
        r = n+1
        return factorial(p+r*j)//(factorial(p+j)*factorial(j)**(r-1))
    lower = octa_coefficients(n-1,j+1)
    return sum(comb(p+2*j,2*l)*lower[l]*comb(p+2*(j-l),j-l)
               for l in range(j+1))


def run_case(kind,n,p):
    N = 3*p+9
    h = n+1 if kind == 'simplex' else 2
    f,g = parameters(kind,n,N)
    sig = canonical_lift(g,p,N)
    hp = hasse(f,h,p,N)
    h2p = hasse(f,h,2*p,N)
    poly = [2*x-y for x,y in zip(hp,h2p)]
    assert all(vp(poly[k]-(f[k] if k<p else 0),p)>=2 for k in range(N))
    b = [Q(0)]*N
    for j in range((p-1)//h+1):
        b[j] = (-1)**(p+h*j)*comb(2*p-1,p+h*j)*vertex_coefficient(kind,n,p,j)
    ratio = div([-x for x in b],h2p,N)
    approximate_lift = [Q(0)]*p+power(ratio,h,N-p)
    assert all(vp(x-y,p)>=2 for x,y in zip(sig,approximate_lift))
    fp_sig = compose(f,sig,N)
    quotient = div(f,fp_sig,N)
    difference = [x-y for x,y in zip(quotient,poly)]
    assert all(vp(x,p)>=2 for x in difference)
    return dict(family=kind,dimension=n,p=p,through_u_degree=N-1,
                minimum_valuation=min(vp(x,p) for x in difference),
                lift_formula_passed=True,quotient_formula_passed=True,
```

```
                polynomial_identification_passed=True)


if __name__ == '__main__':
    cases=[('cube',4,5),('cube',5,7),('cube',6,7),
           ('simplex',4,7),('simplex',5,7),('simplex',6,11),
           ('octa',3,5),('octa',4,5),('octa',5,7),('octa',6,7)]
    output=[]
    for case in cases:
        result=run_case(*case)
        output.append(result)
        print('PASS',json.dumps(result),flush=True)
    target=Path(__file__).with_name('first_level_validation.json')
    target.write_text(json.dumps(dict(scope='Finite exact checks supporting the all-dimensional s=1 theorem.
     ',
                                      all_passed=True,cases=output),indent=2)+'\n')
```

D.9 verify_full_zagier.py

Original source: `verify_full_zagier.py`; 386 lines.

```
#!/usr/bin/env python3
"""Bounded exact checks of the complete Zagier D proof.

The infinite theorem is proved in full_zagier_proof.tex; these calculations
test the modular normalization, canonical-basis Hecke identities, Cartier
recurrence, and complete finite portions of the truncation congruences.
Python standard library only. Run from any current directory.
"""
from __future__ import annotations

from math import isqrt
from pathlib import Path
from time import perf_counter
import json


def zmul(a, b, n):
    c = [0]*n
    for i, x in enumerate(a[:n]):
        if x:
            for j, y in enumerate(b[:n-i]):
                if y:
                    c[i+j] += x*y
    return c


def zinv(a, n):
    assert a[0] == 1
    b = [1]+[0]*(n-1)
    for k in range(1, n):
        b[k] = -sum(a[j]*b[k-j] for j in range(1, min(k+1, len(a))))
    return b


def zcompose(a, t, n):
    out = [0]*n
    for c in reversed(a):
        out = zmul(out, t, n)
        out[0] += c
    return out


def trim(a):
    a = list(a)
    while len(a) > 1 and a[-1] == 0:
        a.pop()
    return a


def mmul(a, b, n, mod):
```

```
    if n <= 0:
        return []
    a = trim([x % mod for x in a[:n]])
    b = trim([x % mod for x in b[:n]])
    if not a or not b or a == [0] or b == [0]:
        return [0]*n
    count = min(len(a), len(b))
    width = max(1, (count.bit_length()+2*mod.bit_length()+7)//8)
    aa = int.from_bytes(b''.join(x.to_bytes(width, 'little') for x in a), 'little')
    bb = int.from_bytes(b''.join(x.to_bytes(width, 'little') for x in b), 'little')
    size = len(a)+len(b)-1
    raw = (aa*bb).to_bytes(size*width, 'little')
    out = [int.from_bytes(raw[i*width:(i+1)*width], 'little') % mod
           for i in range(min(n, size))]
    return out+[0]*(n-len(out))


def minv(a, n, mod):
    assert a[0] % mod == 1
    b = [1]
    while len(b) < n:
        size = min(2*len(b), n)
        ab = mmul(a[:size], b, size, mod)
        correction = [(-x) % mod for x in ab]
        correction[0] = (correction[0]+2) % mod
        b = mmul(b, correction, size, mod)
    return b


def mcompose(a, t, n, mod):
    assert t[0] == 0
    a = a[:n]
    if not a:
        return [0]*n
    block = isqrt(len(a))+1
    powers = [[1]+[0]*(n-1)]
    for _ in range(block):
        powers.append(mmul(powers[-1], t, n, mod))
    out = [0]*n
    for start in reversed(range(0, len(a), block)):
        out = mmul(out, powers[block], n, mod)
        for r, x in enumerate(a[start:start+block]):
            x %= mod
            if x:
                out = [(v+x*u) % mod for v, u in zip(out, powers[r])]
    return out


def mpow(a, k, n, mod):
    out = [1]+[0]*(n-1)
    while k:
        if k & 1:
            out = mmul(out, a, n, mod)
        k //= 2
        if k:
            a = mmul(a, a, n, mod)
    return out


def vsub(a, p, n):
    out = [0]*n
    for k, x in enumerate(a):
        if p*k >= n:
            break
        out[p*k] = x
    return out


def vp(x, p, cap=10**6):
    if x == 0:
        return cap
    v = 0
    while v < cap and x % p == 0:
        x //= p
```

```
        v += 1
    return v


def make_data(n):
    b = [1]+[0]*(n-1)
    for k in range(1, n):
        e = 0 if k % 5 == 0 else 5 if k % 5 in (1, 4) else -5
        for _ in range(abs(e)):
            if e > 0:
                for j in range(n-1, k-1, -1):
                    b[j] -= b[j-k]
            else:
                for j in range(k, n):
                    b[j] += b[j-k]
    theta = [1]+[0]*(n-1)
    ct = [0, 3, 1, -1, -3]
    for k in range(1, n):
        for j in range(k, n, k):
            theta[j] += ct[k % 5]
    return [0]+b[:-1], theta


def apery(n):
    out = [1, 3]
    for k in range(2, n):
        value = (11*k*k-11*k+3)*out[k-1]+(k-1)**2*out[k-2]
        assert value % (k*k) == 0
        out.append(value//(k*k))
    return out[:n]


T = []
TH = []
CANONICAL = []


def canonical_polynomials(n):
    """g_j = sum_k CANONICAL[j][k] t^(-k)/Theta."""
    hinv = zinv(T[1:], n+1)
    thinv = zinv(TH, n+1)
    powers = [1]+[0]*n
    polys = [[0]]
    for j in range(1, n+1):
        powers = zmul(powers, hinv, n+1)
        unit = zmul(powers, thinv, n+1)
        poly = [0]*j+[1]
        for k in range(1, j):
            c = unit[j-k]
            for r, x in enumerate(polys[k]):
                poly[r] -= c*x
        polys.append(poly)
    return polys


def g_series(n, positive_length):
    """Dictionary giving g_n from q^(-n) through q^(positive_length-1)."""
    if n == 0:
        return {}
    length = n+positive_length
    hinv = zinv(T[1:], length)
    thinv = zinv(TH, length)
    powers = [1]+[0]*(length-1)
    out = [0]*length
    for j in range(1, n+1):
        powers = zmul(powers, hinv, length)
        c = CANONICAL[n][j]
        if not c:
            continue
        unit = zmul(powers, thinv, length)
        for r in range(length-(n-j)):
            out[n-j+r] += c*unit[r]
    return {j-n: x for j, x in enumerate(out)}
```

```
def dg_series(n, length):
    if n == 0:
        return [0]*length
    poly = [(-1)**j*CANONICAL[n][j] for j in range(1, n+1)]
    return zmul(zcompose(poly, T, length), zinv(TH, length), length)


def check_hecke(p, n, length):
    c = -1 if p % 5 == 2 else 1
    split = p % 5 in (1, 4)
    eps = 1 if p % 5 == 1 else -1
    gn = g_series(n, p*(length-1)+1)
    gpn = g_series(p*n, length)
    gsmall = g_series(n//p, length) if n % p == 0 else {}
    dg = dg_series(n, p*(length-1)+1)
    dgpn = dg_series(p*n, length)
    dgsmall = dg_series(n//p, length) if n % p == 0 else [0]*length
    for j in range(-p*n, length):
        vgn = gn.get(j//p, 0) if j % p == 0 else 0
        udg = dg[p*j] if j >= 0 else 0
        dsmall = dgsmall[j] if j >= 0 else 0
        vdg = dg[j//p] if j >= 0 and j % p == 0 else 0
        ugn = gn.get(p*j, 0)
        if split:
            assert vgn+eps*p*p*ugn == gpn.get(j, 0)+eps*p*p*gsmall.get(j, 0)
            assert vdg+eps*p*p*udg == (dgpn[j] if j >= 0 else 0)+eps*p*p*dsmall
        else:
            assert vgn+c*p*p*udg == gpn.get(j, 0)+c*p*p*dsmall
            assert vdg-c*p*p*ugn == (dgpn[j] if j >= 0 else 0)-c*p*p*gsmall.get(j, 0)
    return dict(p=p, basis_index=n, q_degrees_checked=[-p*n, length-1],
                exact_integer_identities=2, passed=True)


def delta(p, n, length):
    gn = g_series(n, p*(length-1)+1)
    gsmall = g_series(n//p, length) if n % p == 0 else {}
    dg = dg_series(n, p*(length-1)+1)
    dgsmall = dg_series(n//p, length) if n % p == 0 else [0]*length
    # The negative parts cancel identically; inspect all retained nonnegative terms.
    return ([gn.get(p*j, 0)-gsmall.get(j, 0) for j in range(length)],
            [dg[p*j]-dgsmall[j] for j in range(length)])


def check_delta(p, n, length):
    c = -1 if p % 5 == 2 else 1
    lhs = delta(p, p*n, length)
    rhs = delta(p, n, p*(length-1)+1)
    if p % 5 in (1, 4):
        eps = 1 if p % 5 == 1 else -1
        assert lhs[0] == [eps*p*p*rhs[0][p*j] for j in range(length)]
        assert lhs[1] == [eps*p*p*rhs[1][p*j] for j in range(length)]
    else:
        assert lhs[0] == [c*p*p*rhs[1][p*j] for j in range(length)]
        assert lhs[1] == [-c*p*p*rhs[0][p*j] for j in range(length)]
    target = 2*vp(p*n, p)
    assert all(x % p**target == 0 for row in lhs for x in row)
    return dict(p=p, basis_index=p*n, through_q_degree=length-1,
                proved_precision=target, recurrence_passed=True,
                divisibility_passed=True)


def check_full(p, s, m):
    K = m*p**s
    length = 2*K+9
    cap = 2*s+2
    mod = p**cap
    th = [x % mod for x in TH[:length]]
    t = [x % mod for x in T[:length]]
    a = apery(K)
    tp = vsub(t, p, length)
    thp = vsub(th, p, length)
    lhs = mmul(th, mcompose(a[:K//p], tp, length, mod), length, mod)
    rhs = mmul(thp, mcompose(a, t, length, mod), length, mod)
```

```
    defects = [(x-y) % mod for x, y in zip(lhs, rhs)]
    minimum = min(vp(x, p, cap) for x in defects)
    assert minimum >= 2*s
    return dict(p=p, s=s, m=m, through_q_degree=length-1,
                arithmetic_modulus=mod, required_valuation=2*s,
                observed_minimum_valuation=minimum,
                minimum_is_capped=(minimum == cap), passed=True)


def dual_series(K, positive_length, mod):
    length = K+positive_length
    t = [x % mod for x in T[:length+1]]
    th = [x % mod for x in TH[:length]]
    a = apery(K)
    rev = [(-1)**(K-1-j)*a[K-1-j] % mod for j in range(K)]
    unit = mmul(mcompose(rev, t, length, mod),
                mpow(minv(t[1:], length, mod), K, length, mod),
                length, mod)
    return mmul(unit, minv(th, length, mod), length, mod)


def check_dual(p, r, m, positive_length):
    K = m*p**r
    cap = 2*r+2
    mod = p**cap
    lhs = dual_series(K, p*(positive_length-1)+1, mod)
    rhs = dual_series(K//p, positive_length, mod)
    defects = [(lhs[p*j+K]-rhs[j+K//p]) % mod
               for j in range(-K//p, positive_length)]
    minimum = min(vp(x, p, cap) for x in defects)
    assert minimum >= 2*r
    return dict(p=p, r=r, m=m,
                q_degrees_checked=[-K//p, positive_length-1],
                arithmetic_modulus=mod, required_valuation=2*r,
                observed_minimum_valuation=minimum,
                minimum_is_capped=(minimum == cap), passed=True)


def main():
    global T, TH, CANONICAL
    start = perf_counter()
    full_cases = [(p, s, m)
                  for p, levels in [(2, (2, 3, 4)), (3, (2, 3)),
                                    (7, (2, 3)), (13, (2,))]
                  for s in levels for m in (1, 2)]
    full_cases += [(11, 2, 1), (11, 2, 2), (19, 2, 1)]
    maximum = max(2*m*p**s+10 for p, s, m in full_cases)
    T, TH = make_data(maximum)
    # Basic normalization checked with exact integers.
    assert zcompose(apery(50), T, 50) == TH[:50]
    aa = [-11*x-y for x, y in zip(T[:50], zmul(T, T, 50))]
    aa[0] += 1
    assert [j*T[j] for j in range(50)] == zmul(T, zmul(aa, zmul(TH, TH, 50), 50), 50)
    a = apery(50)
    assert all(k*k*a[k] == (11*k*k-11*k+3)*a[k-1]+(k-1)**2*a[k-2]
               for k in range(2, 50))
    mod = 7**7
    test_a = [(-k*k+11*k-3) % mod for k in range(29)]
    test_b = [(2*k*k-7*k+5) % mod for k in range(29)]
    assert mmul(test_a, test_b, 29, mod) == [x % mod for x in zmul(test_a, test_b, 29)]
    test_a[0] = 1
    assert minv(test_a, 29, mod) == [x % mod for x in zinv(test_a, 29)]
    CANONICAL = canonical_polynomials(49)
    assert CANONICAL[1] == [0, 1]
    assert CANONICAL[2] == [0, -7, 1]
    assert CANONICAL[3] == [0, 19, -12, 1]
    for n in range(1, 50):
        gs = g_series(n, 1)
        assert gs[-n] == 1 and all(gs[j] == 0 for j in range(-n+1, 0))
    out = dict(scope="Finite exact checks; the all-prime/all-level statement is proved in the note.",
               implementation_checks=True, modular_parametrization=True,
               canonical_wronskian_identity=True,
               canonical_principal_parts_checked=49,
               hecke_identities=[], paired_cartier=[], dual_truncations=[],
```

```
                full_congruences=[])
    for p in (2, 3, 7):
        for n in (1, 2, p):
            r = check_hecke(p, n, 12)
            out["hecke_identities"].append(r)
            print("PASS exact Hecke", r, flush=True)
        for n in (1, p):
            r = check_delta(p, n, 8)
            out["paired_cartier"].append(r)
            print("PASS exact paired recurrence", r, flush=True)
    for p in (11, 19):
        for n in (1, 2):
            r = check_hecke(p, n, 12)
            out["hecke_identities"].append(r)
            print("PASS exact split Hecke", r, flush=True)
    r = check_delta(11, 1, 8)
    out["paired_cartier"].append(r)
    print("PASS exact split paired recurrence", r, flush=True)
    for values in [(2, 2, 1, 25), (2, 3, 2, 25), (3, 1, 1, 25),
                   (3, 2, 2, 25), (7, 1, 1, 25), (7, 2, 2, 25),
                   (13, 1, 1, 25)]:
        r = check_dual(*values)
        out["dual_truncations"].append(r)
        print("PASS dual truncation", r, flush=True)
    for values in full_cases:
        r = check_full(*values)
        out["full_congruences"].append(r)
        print("PASS full finite congruence", r, flush=True)
    out["counts"] = {k: len(out[k]) for k in
                     ("hecke_identities", "paired_cartier",
                      "dual_truncations", "full_congruences")}
    out["all_passed"] = True
    out["seconds"] = round(perf_counter()-start, 3)
    destination = Path(__file__).resolve().parent
    (destination/"full_zagier_validation.json").write_text(json.dumps(out, indent=2)+"\n")
    print("ALL CHECKS PASSED", out["counts"], "seconds", out["seconds"], flush=True)


if __name__ == "__main__":
    main()
```

D.10 verify_octahedral_progress.py

Original source: `verify_octahedral_progress.py`; 247 lines.

```
#!/usr/bin/env python3
"""Finite exact checks for the octahedral weighted Cartier reduction.

The written proof, not these bounded checks, establishes the weighted theorem.
The full positive-truncation congruence for all levels remains unresolved.
Python 3.10+ standard library only; no workspace imports or network required.
"""
from pathlib import Path
from fractions import Fraction
from math import comb, isqrt
import json
import time

def trim(a):
    a = list(a)
    while len(a) > 1 and a[-1] == 0:
        a.pop()
    return a

def mmul(a, b, n, mod):
    """Exact modular convolution by carry-free Kronecker substitution."""
    if n <= 0:
        return []
    a = trim([x % mod for x in a[:n]])
    b = trim([x % mod for x in b[:n]])
    if not a or not b or a == [0] or b == [0]:
        return [0]*n
```

```
    count = min(len(a), len(b))
    width = max(1, (count.bit_length() + 2*mod.bit_length() + 7)//8)
    aa = int.from_bytes(b''.join(x.to_bytes(width, 'little') for x in a), 'little')
    bb = int.from_bytes(b''.join(x.to_bytes(width, 'little') for x in b), 'little')
    # Each convolution coefficient is < count*mod^2 < 256^width.
    size = len(a)+len(b)-1
    raw = (aa*bb).to_bytes(size*width, 'little')
    out = [int.from_bytes(raw[i*width:(i+1)*width], 'little') % mod
           for i in range(min(n, size))]
    return out+[0]*(n-len(out))

def minv(a, n, mod):
    assert a[0] % mod == 1
    b = [1]
    while len(b) < n:
        size = min(2*len(b), n)
        ab = mmul(a[:size], b, size, mod)
        correction = [(-x) % mod for x in ab]
        correction[0] = (correction[0]+2) % mod
        b = mmul(b, correction, size, mod)
    return b

def mpow(a, k, n, mod):
    out = [1]+[0]*(n-1)
    while k:
        if k & 1:
            out = mmul(out, a, n, mod)
        k //= 2
        if k:
            a = mmul(a, a, n, mod)
    return out

def vsub(a, p, n):
    out = [0]*n
    for k, x in enumerate(a):
        if p*k >= n:
            break
        out[p*k] = x
    return out

def mcompose(a, t, n, mod):
    """Baby-step/giant-step polynomial composition; t(0)=0."""
    assert t[0] == 0
    a = a[:n]
    if not a:
        return [0]*n
    block = isqrt(len(a))+1
    powers = [[1]+[0]*(n-1)]
    for _ in range(block):
        powers.append(mmul(powers[-1], t, n, mod))
    out = [0]*n
    for start in reversed(range(0, len(a), block)):
        out = mmul(out, powers[block], n, mod)
        for r, x in enumerate(a[start:start+block]):
            x %= mod
            if x:
                out = [(v+x*u) % mod for v, u in zip(out, powers[r])]
    return out

def valuation(x, p, cap):
    if x == 0:
        return cap
    v = 0
    while v < cap and x % p == 0:
        v += 1
        x //= p
    return v

def divisor_sums(n):
    out = [0]*n
    for d in range(1, n):
        for j in range(d, n, d):
            out[j] += d
    return out

def eta_h6(n,mod):
```

```
    out=[1]+[0]*(n-1)
    for k in range(1,n):
        e=-5+(k%2==0)-(k%3==0)+5*(k%6==0)
        if e>0:
            for _ in range(e):
                for j in range(n-1,k-1,-1):out[j]=(out[j]-out[j-k])%mod
        else:
            for _ in range(-e):
                for j in range(k,n):out[j]=(out[j]+out[j-k])%mod
    return [0]+out[:-1]

def params(n,mod):
    h=eta_h6(n,mod);h2=mmul(h,h,n,mod)
    h3=mmul(h2,h,n,mod)
    num=[(x+17*y+72*z)%mod for x,y,z in zip(h,h2,h3)]
    Q=[(18*x+72*y)%mod for x,y in zip(h,h2)];Q[0]=1
    qinv=minv(Q,n,mod)
    u=mmul(num,mmul(qinv,qinv,n,mod),n,mod)
    sig=divisor_sums(n)
    th=[1]+[(6*sig[k]-(12*sig[k//2]if k%2==0 else 0)
             +(18*sig[k//3]if k%3==0 else 0)
             -(36*sig[k//6]if k%6==0 else 0))%mod for k in range(1,n)]
    return u,th

def coeffs(n,mod):
    # Exact P-recursion, then reduce; divisions by n^3 need not be units.
    a=[1,6]
    for k in range(2,n):
        num=2*(2*k-1)*(10*k*k-10*k+3)*a[-1]-36*(k-1)*(2*k-1)*(2*k-3)*a[-2]
        assert num%(k**3)==0
        a.append(num//(k**3))
    return [x%mod for x in a[:n]]

def check(p,s,m,n):
    K=m*p**s;L=K//p;cap=2*s+2;mod=p**cap
    start=time.perf_counter();u,th=params(n,mod)
    a=coeffs(K,mod)
    big=mcompose(a,u,n,mod)
    tiny=(n-1)//p+1
    small=mcompose(a[:L],u[:tiny],tiny,mod)
    E=[(x-y)%mod for x,y in zip(mmul(th,vsub(small,p,n),n,mod),mmul(vsub(th,p,n),big,n,mod))]
    vals=[valuation(z,p,cap) for z in E]
    result=dict(p=p,s=s,m=m,through_q_degree=n-1,min_valuation=min(vals),first_min_index=vals.index(min(vals
    )),seconds=round(time.perf_counter()-start,2))
    print(json.dumps(result),flush=True)
    assert min(vals)>=2*s
    return result


def exact_bv_coordinate(n):
    # The BV multinomial formula, independently of eta products and PF recurrences.
    H=[Fraction(0)]
    for k in range(1,2*n+1):H.append(H[-1]+Fraction(1,k))
    F=[];G=[]
    for k in range(n):
        b=sum(comb(k,j)**2*comb(2*j,j) for j in range(k+1))
        weighted=sum(comb(k,j)**2*comb(2*j,j)*H[k-j] for j in range(k+1))
        F.append(comb(2*k,k)*b)
        G.append(2*comb(2*k,k)*(H[2*k]*b-weighted))
    g=[Fraction(0)]*n
    for k in range(n):g[k]=G[k]-sum(F[j]*g[k-j] for j in range(1,k+1))
    ex=[Fraction(1)]+[Fraction(0)]*(n-1)
    for k in range(1,n):ex[k]=sum(j*g[j]*ex[k-j] for j in range(1,k+1))/k
    q=[Fraction(0)]+ex[:-1]
    assert all(c.denominator==1 for c in q)
    return [int(c) for c in q]

def vp(n,p):
    if n==0:return 10**9
    out=0
    while n%p==0:n//=p;out+=1
    return out

def check_weighted(p,s,n):
    K=p**s;cap=2*s+3;mod=p**cap
    u,th=params(n,mod)
```

```
    fk=mcompose(coeffs(K,mod),u,n,mod)
    H=mmul(fk,minv(th,n,mod),n,mod)
    assert all((j*j*H[j])%p**(2*s)==0 for j in range(1,n))
    assert H[0]==1 and all(H[j]==0 for j in range(1,min(K,n)))
    return dict(p=p,s=s,K=K,through_q_degree=n-1,precision=cap,
                all_squared_derivative_coefficients_divisible=True)

def primitive_checks(p,s,limit):
    K=p**s
    for n in range(limit):
        c=comb(2*n,n)
        assert vp(c,p)>=vp(n+K+1,p)
        assert s+vp(c,p)>=vp(n+K,p)
    return dict(p=p,s=s,K=K,central_binomial_indices_checked=limit,
                shifted_integrality_passed=True)

# Symbolic bivariate-polynomial verification of the exact telescoping certificate.
def padd(a,b):
    c=dict(a)
    for k,v in b.items():c[k]=c.get(k,0)+v
    return {k:v for k,v in c.items() if v}
def pscale(a,c):return {k:v*c for k,v in a.items() if v*c}
def pmul(a,b):
    c={}
    for (i,j),x in a.items():
        for (k,l),y in b.items():c[i+k,j+l]=c.get((i+k,j+l),0)+x*y
    return {k:v for k,v in c.items() if v}
def ppow(a,n):
    c={(0,0):1}
    for _ in range(n):c=pmul(c,a)
    return c
def certificate_check():
    one={(0,0):1};j={(1,0):1};z={(0,1):1}
    shift=padd(padd(z,one),pscale(j,-1))
    P=padd(pscale(j,3),pscale(padd(z,one),-4))
    Pnext=padd(P,pscale(one,3))
    lhs=padd(pscale(pmul(pmul(padd(pscale(j,2),one),Pnext),ppow(shift,2)),2),
             pscale(pmul(ppow(j,3),P),-1))
    A=padd(padd(pscale(ppow(z,2),10),pscale(z,10)),pscale(one,3))
    rhs=padd(padd(ppow(padd(z,one),4),pscale(pmul(A,ppow(shift,2)),-1)),
             pscale(pmul(ppow(padd(z,pscale(j,-1)),2),ppow(shift,2)),9))
    assert lhs==rhs
    return True

def main():
    start=time.perf_counter()
    assert certificate_check()
    n=40;mod=5**10
    u,th=params(n,mod)
    assert mcompose(coeffs(n,mod),u,n,mod)==th
    qb=exact_bv_coordinate(n)
    assert mcompose(qb,u,n,mod)==[0,1]+[0]*(n-2)
    result={'scope':'Finite checks supporting the written weighted theorem; no full all-level positive-
     truncation proof is claimed.',
            'symbolic_telescoping_certificate':True,
            'bv_coordinate_check':{'through_degree':n-1,'modulus':mod,'passed':True},
            'weighted':[],'primitives':[],'full_congruence_bounded_probes':[]}
    for p,s in [(5,1),(5,2),(5,3),(7,1),(7,2),(11,2)]:
        n=4*p**s+29
        result['weighted'].append(check_weighted(p,s,n))
        result['primitives'].append(primitive_checks(p,s,3*p**s+20))
        result['full_congruence_bounded_probes'].append(check(p,s,1,n))
    result['seconds']=round(time.perf_counter()-start,3)
    result['all_passed']=True
    target=Path(__file__).resolve().parent
    (target/'octahedral_progress_validation.json').write_text(json.dumps(result,indent=2)+'\n')
    print('PASS: exact symbolic certificate, BV coordinate check, 6 weighted series checks, 6 shifted-
     integrality ranges, 6 bounded full-congruence probes.')
    print('The resonant U_p congruence for every coefficient and every level is still not proved.')

if __name__=='__main__':main()
```

E Complete exact certificate program

The multiplication routine encodes polynomial coefficients as base 2^{8w} digits, with w chosen so that each unreduced convolution coefficient is strictly smaller than that base. Thus its packed integer product is exact coefficientwise. All divisions used to construct the lift equation are checked before reduction; all cleared denominators have constant term one.

```
#!/usr/bin/env python3
"""Finite whole-series certificates for specified small-prime octahedral cases.

This establishes only the levels actually executed. It does not establish an
all-level theorem. All algebra is exact over finite integer polynomial rings.
"""
from math import comb, factorial, isqrt
from pathlib import Path
import json, time, hashlib, argparse

def trim(a):
    a=list(a)
    while len(a)>1 and a[-1]==0:a.pop()
    return a
def add(a,b,mod):
    c=[0]*max(len(a),len(b))
    for i,x in enumerate(a):c[i]+=x
    for i,x in enumerate(b):c[i]+=x
    return trim([x%mod for x in c])
def scale(a,k,mod):return trim([(k*x)%mod for x in a])
def mul(a,b,mod):
    a=trim([x%mod for x in a]);b=trim([x%mod for x in b])
    if a==[0] or b==[0]:return [0]
    width=(min(len(a),len(b)).bit_length()+2*mod.bit_length()+7)//8
    aa=int.from_bytes(b''.join(x.to_bytes(width,'little') for x in a),'little')
    bb=int.from_bytes(b''.join(x.to_bytes(width,'little') for x in b),'little')
    length=len(a)+len(b)-1
    raw=(aa*bb).to_bytes(length*width,'little')
    return trim([int.from_bytes(raw[i*width:(i+1)*width],'little')%mod for i in range(length)])
def power(a,k,mod):
    out=[1]
    while k:
        if k&1:out=mul(out,a,mod)
        k//=2
        if k:a=mul(a,a,mod)
    return out
def taylor(a,r,p,mod):
    out=[0]*(p*(len(a)-1)+1)
    for j,x in enumerate(a):
        if j>=r:out[p*j]=(x*comb(j,r))%mod
    return trim(out)
def vp(n,p):
    if n==0:return 10**9
    v=0
    while n%p==0:n//=p;v+=1
    return v
def composition_squares(dimension,length):
    values=[1]*length
    for _ in range(dimension-1):
        values=[sum(comb(j,k)**2*values[j-k] for k in range(j+1))
                for j in range(length)]
    return values

def vertex(K,b3,mod):
    fac=[factorial(j) for j in range(2*K)]
    out=[]
    for j in range(K):
        total=0
        for ell in range(j+1):
            num=fac[2*K-ell-1]
            den=fac[2*K-2*j-1]*fac[ell]*fac[j-ell]**2
            assert num%den==0
            total+=(-1 if ell%2 else 1)*(num//den)*b3[j-ell]
        out.append(total%mod)
    assert out[0]==1
```

```
    return trim(out)
def octa_period(dimension,length):
    values=composition_squares(dimension,length)
    return [comb(2*j,j)*values[j] for j in range(length)]

def profiles(weights,limit,at=0,counts=()):
    if at==len(weights):
        if sum(counts):yield counts
        return
    for k in range(limit//weights[at]+1):
        yield from profiles(weights,limit-k*weights[at],at+1,counts+(k,))

def build(dimension,p,s):
    started=time.time();prec=2*s;yprec=prec-1
    assert s>=2 and p>=3 and dimension>=2
    assert all(p%q for q in range(2,isqrt(p)+1))
    n=p**(prec-2);mod=p**prec;ymod=p**yprec;workmod=p*n*ymod
    degree=max(k for k in range(1,3*yprec+1) if k-1-vp(k,p)<yprec)
    print(json.dumps({'stage':'setup','s':s,'N':n,'modulus':mod,'lift_polynomial_degree':degree}),flush=True
     )
    b3=composition_squares(dimension-1,2*p*n)
    av={k:vertex(k,b3,workmod) for k in (n,2*n,p*n,2*p*n)}
    a,b,ap,bp=[av[k] for k in (n,2*n,p*n,2*p*n)]
    ar=[taylor(a,r,p,workmod) for r in range(degree+1)]
    br=[taylor(b,r,p,workmod) for r in range(degree+1)]
    a0pow=[power(ar[0],r,workmod) for r in range(degree+2)]
    T=[]
    for r in range(degree+1):
        tr=mul(br[r],a0pow[r],workmod)
        for i in range(1,r+1):
            tr=add(tr,scale(mul(mul(ar[i],T[r-i],workmod),a0pow[i-1],workmod),-1,workmod),workmod)
        T.append(tr)
    coeff=[]
    for r in range(degree+1):
        num=add(scale(mul(bp,a0pow[degree+1],workmod),comb(n,r),workmod),scale(mul(mul(ap,T[r],workmod),
     a0pow[degree-r],workmod),-1,workmod),workmod)
        num=scale(num,p**r,workmod)
        assert all(x%(p*n)==0 for x in num),('divisibility',r)
        coeff.append(trim([(x//(p*n))%ymod for x in num]))
    A,B=coeff[:2]
    assert A[0]==0 and B[0]==1
    weights=[min(vp(v,p) for v in c) for c in coeff[2:]]
    assert all(w>=1 for w in weights)
    # Multivariate Lagrange: y=z+sum_m (R)!/((R-m+1)!prod m_r!)
    #   *(-1)^m prod c_r^m_r * z^(R-m+1), z=-A/B, c_r=Q_r/B.
    # Each tree coefficient is an integer, so profiles of valuation>=yprec vanish.
    ps=list(profiles(weights,yprec-1))
    maxden=max([1]+[1+sum((i+2)*c for i,c in enumerate(pr)) for pr in ps])
    bpows=[power(B,j,ymod) for j in range(maxden+1)]
    apows=[power(A,j,ymod) for j in range(maxden+1)]
    Ynum=scale(mul(A,bpows[maxden-1],ymod),-1,ymod)
    terms_info=[]
    for pr in ps:
        m=sum(pr);Rsum=sum((i+2)*c for i,c in enumerate(pr));zexp=Rsum-m+1
        den=factorial(zexp)
        for c in pr:den*=factorial(c)
        assert factorial(Rsum)%den==0
        tree=factorial(Rsum)//den
        if vp(tree,p)+sum(w*c for w,c in zip(weights,pr))>=yprec:continue
        term=apows[zexp]
        for q,c in zip(coeff[2:],pr):
            if c:term=mul(term,power(q,c,ymod),ymod)
        term=mul(term,bpows[maxden-Rsum-1],ymod)
        term=scale(term,(-1 if (Rsum+1)%2 else 1)*tree,ymod)
        Ynum=add(Ynum,term,ymod)
        terms_info.append({'profile':pr,'tree_coefficient':tree})
    Yden=bpows[maxden]
    root=[0]
    for r,c in enumerate(coeff):
        root=add(root,mul(mul(c,power(Ynum,r,ymod),ymod),power(Yden,degree-r,ymod),ymod),ymod)
    assert root==[0],('root',[(i,v) for i,v in enumerate(root) if v][:10])
    print(json.dumps({'stage':'lift_certified','s':s,'degrees':[len(Ynum)-1,len(Yden)-1],'elapsed_seconds':
     round(time.time()-started,3)}),flush=True)
    Q=scale(Ynum,p,mod);R=Yden
```

```
    assert Q[0]==0 and R[0]==1
    f=octa_period(dimension,p**s);fs=f[:p**(s-1)]
    L=[]
    for r in range(prec):
        lhs=taylor(fs,r,p,mod)
        if r:lhs=add(lhs,scale(taylor(fs,r-1,p,mod),n,mod),mod)
        L.append(add(mul(ap,lhs,mod),scale(mul(f,ar[r],mod),-1,mod),mod))
    target=[0];degree_bound=0
    for r,c in enumerate(L):
        term=mul(mul(c,power(Q,r,mod),mod),power(R,prec-1-r,mod),mod)
        degree_bound=max(degree_bound,len(term)-1)
        target=add(target,term,mod)
    assert target==[0],('target',[(i,v) for i,v in enumerate(target) if v][:10])
    payload={'p':p,'s':s,'m':1,'dimension':dimension,'modulus':mod,'intermediate_modulus':workmod,'
     vertex_indices':list(av),'vertex_polynomials':av,'lift_equation_coefficients':coeff,'lagrange_profiles':
     terms_info,'lift_y_numerator':Ynum,'lift_y_denominator':Yden,'target_L':L}
    packed=json.dumps(payload,sort_keys=True,separators=(',',':'))
    report={'theorem':f'Every coefficient of E_({p},{s},1) for octahedral dimension {dimension} is divisible
      by {p}^{2*s}.','dimension':dimension,'p':p,'s':s,'m':1,'modulus':mod,'intermediate_modulus':workmod,'
     all_polynomial_coefficients_checked':True,'power_series_cutoff':None,'lift_constructed_from':'BV vertex
      polynomials; no target truncation assumption','lift_equation_root_identity_zero':root==[0],'
     cleared_target_identity_zero':target==[0],'lift_equation_degree':degree,'coefficient_polynomial_degrees
     ':[len(z)-1 for z in coeff],'lift_y_fraction_degrees':[len(Ynum)-1,len(Yden)-1],'
     lift_y_denominator_power':maxden,'cleared_identity_degree_bound':degree_bound,'certificate_sha256':
     hashlib.sha256(packed.encode()).hexdigest(),'elapsed_seconds':round(time.time()-started,3)}
    base=Path(__file__).resolve().parent;stem=f'certificate_octa{dimension}_p{p}_s{s}'
    (base/(stem+'.json')).write_text(json.dumps(payload,indent=2)+'\n')
    (base/(stem+'_report.json')).write_text(json.dumps(report,indent=2)+'\n')
    print(json.dumps(report,indent=2))
    return report

if __name__=='__main__':
    parser=argparse.ArgumentParser()
    parser.add_argument('--dimension',type=int,required=True)
    parser.add_argument('--p',type=int,required=True)
    parser.add_argument('--s',type=int,required=True)
    args=parser.parse_args();build(args.dimension,args.p,args.s)
```

F Complete polynomial arithmetic and individual-certificate source

The following source is supplied as `verify_dimension_families.py`. The dimension-uniform verifier in the next appendix imports this file. All convolution uses exact integer arithmetic. For packed multiplication, the byte width is chosen so that the largest possible convolution coefficient is strictly below its base; hence there is no carry between stored coefficients.

```
#!/usr/bin/env python3
"""Finite whole-series certificates for specified small-prime octahedral cases.

This establishes only the levels actually executed. It does not establish an
all-level theorem. All algebra is exact over finite integer polynomial rings.
"""
from math import comb, factorial, isqrt
from pathlib import Path
import json, time, hashlib, argparse

def trim(a):
    a=list(a)
    while len(a)>1 and a[-1]==0:a.pop()
    return a
def add(a,b,mod):
    c=[0]*max(len(a),len(b))
    for i,x in enumerate(a):c[i]+=x
    for i,x in enumerate(b):c[i]+=x
    return trim([x%mod for x in c])
def scale(a,k,mod):return trim([(k*x)%mod for x in a])
def mul(a,b,mod):
    a=trim([x%mod for x in a]);b=trim([x%mod for x in b])
    if a==[0] or b==[0]:return [0]
    width=(min(len(a),len(b)).bit_length()+2*mod.bit_length()+7)//8
    aa=int.from_bytes(b''.join(x.to_bytes(width,'little') for x in a),'little')
    bb=int.from_bytes(b''.join(x.to_bytes(width,'little') for x in b),'little')
    length=len(a)+len(b)-1
    raw=(aa*bb).to_bytes(length*width,'little')
    return trim([int.from_bytes(raw[i*width:(i+1)*width],'little')%mod for i in range(length)])
def power(a,k,mod):
    out=[1]
    while k:
        if k&1:out=mul(out,a,mod)
        k//=2
        if k:a=mul(a,a,mod)
    return out
def taylor(a,r,p,mod):
    out=[0]*(p*(len(a)-1)+1)
    for j,x in enumerate(a):
        if j>=r:out[p*j]=(x*comb(j,r))%mod
    return trim(out)
def vp(n,p):
    if n==0:return 10**9
    v=0
    while n%p==0:n//=p;v+=1
    return v
def composition_squares(dimension,length):
    values=[1]*length
    for _ in range(dimension-1):
        values=[sum(comb(j,k)**2*values[j-k] for k in range(j+1))
                for j in range(length)]
    return values

def vertex(K,b3,mod):
    fac=[factorial(j) for j in range(2*K)]
    out=[]
    for j in range(K):
        total=0
        for ell in range(j+1):
            num=fac[2*K-ell-1]
            den=fac[2*K-2*j-1]*fac[ell]*fac[j-ell]**2
            assert num%den==0
            total+=(-1 if ell%2 else 1)*(num//den)*b3[j-ell]
```

```
        out.append(total%mod)
    assert out[0]==1
    return trim(out)
def octa_period(dimension,length):
    values=composition_squares(dimension,length)
    return [comb(2*j,j)*values[j] for j in range(length)]

def profiles(weights,limit,at=0,counts=()):
    if at==len(weights):
        if sum(counts):yield counts
        return
    for k in range(limit//weights[at]+1):
        yield from profiles(weights,limit-k*weights[at],at+1,counts+(k,))

def build(dimension,p,s,write_files=True,verbose=True,m=1):
    started=time.time();prec=2*s;yprec=prec-1
    assert s>=2 and p>=3 and dimension>=2
    assert all(p%q for q in range(2,isqrt(p)+1))
    n=p**(prec-2);mod=p**prec;ymod=p**yprec;workmod=p*n*ymod
    degree=max(k for k in range(1,3*yprec+1) if k-1-vp(k,p)<yprec)
    if verbose:print(json.dumps({'stage':'setup','s':s,'N':n,'modulus':mod,'lift_polynomial_degree':degree}),
     flush=True)
    b3=composition_squares(dimension-1,2*p*n)
    av={k:vertex(k,b3,workmod) for k in (n,2*n,p*n,2*p*n)}
    a,b,ap,bp=[av[k] for k in (n,2*n,p*n,2*p*n)]
    ar=[taylor(a,r,p,workmod) for r in range(degree+1)]
    br=[taylor(b,r,p,workmod) for r in range(degree+1)]
    a0pow=[power(ar[0],r,workmod) for r in range(degree+2)]
    T=[]
    for r in range(degree+1):
        tr=mul(br[r],a0pow[r],workmod)
        for i in range(1,r+1):
            tr=add(tr,scale(mul(mul(ar[i],T[r-i],workmod),a0pow[i-1],workmod),-1,workmod),workmod)
        T.append(tr)
    coeff=[]
    for r in range(degree+1):
        num=add(scale(mul(bp,a0pow[degree+1],workmod),comb(n,r),workmod),scale(mul(mul(ap,T[r],workmod),
     a0pow[degree-r],workmod),-1,workmod),workmod)
        num=scale(num,p**r,workmod)
        assert all(x%(p*n)==0 for x in num),('divisibility',r)
        coeff.append(trim([(x//(p*n))%ymod for x in num]))
    A,B=coeff[:2]
    assert A[0]==0 and B[0]==1
    weights=[min(vp(v,p) for v in c) for c in coeff[2:]]
    assert all(w>=1 for w in weights)
    # Multivariate Lagrange: y=z+sum_m (R)!/((R-m+1)!prod m_r!)
    #   *(-1)^m prod c_r^m_r * z^(R-m+1), z=-A/B, c_r=Q_r/B.
    # Each tree coefficient is an integer, so profiles of valuation>=yprec vanish.
    ps=list(profiles(weights,yprec-1))
    maxden=max([1]+[1+sum((i+2)*c for i,c in enumerate(pr)) for pr in ps])
    bpows=[power(B,j,ymod) for j in range(maxden+1)]
    apows=[power(A,j,ymod) for j in range(maxden+1)]
    Ynum=scale(mul(A,bpows[maxden-1],ymod),-1,ymod)
    terms_info=[]
    for pr in ps:
        tree_count=sum(pr);Rsum=sum((i+2)*c for i,c in enumerate(pr));zexp=Rsum-tree_count+1
        den=factorial(zexp)
        for c in pr:den*=factorial(c)
        assert factorial(Rsum)%den==0
        tree=factorial(Rsum)//den
        if vp(tree,p)+sum(w*c for w,c in zip(weights,pr))>=yprec:continue
        term=apows[zexp]
        for q,c in zip(coeff[2:],pr):
            if c:term=mul(term,power(q,c,ymod),ymod)
        term=mul(term,bpows[maxden-Rsum-1],ymod)
        term=scale(term,(-1 if (Rsum+1)%2 else 1)*tree,ymod)
        Ynum=add(Ynum,term,ymod)
        terms_info.append({'profile':pr,'tree_coefficient':tree})
    Yden=bpows[maxden]
    root=[0]
    for r,c in enumerate(coeff):
        root=add(root,mul(mul(c,power(Ynum,r,ymod),ymod),power(Yden,degree-r,ymod),ymod),ymod)
    assert root==[0],('root',[(i,v) for i,v in enumerate(root) if v][:10])
    if verbose:print(json.dumps({'stage':'lift_certified','s':s,'degrees':[len(Ynum)-1,len(Yden)-1],'
```

```
     elapsed_seconds':round(time.time()-started,3)}),flush=True)
    Q=scale(Ynum,p,mod);R=Yden
    assert Q[0]==0 and R[0]==1
    f=octa_period(dimension,m*p**s);fs=f[:m*p**(s-1)]
    L=[]
    for r in range(prec):
        lhs=taylor(fs,r,p,mod)
        if r:lhs=add(lhs,scale(taylor(fs,r-1,p,mod),n,mod),mod)
        L.append(add(mul(ap,lhs,mod),scale(mul(f,ar[r],mod),-1,mod),mod))
    target=[0];degree_bound=0
    for r,c in enumerate(L):
        term=mul(mul(c,power(Q,r,mod),mod),power(R,prec-1-r,mod),mod)
        degree_bound=max(degree_bound,len(term)-1)
        target=add(target,term,mod)
    assert target==[0],('target',[(i,v) for i,v in enumerate(target) if v][:10])
    payload={'p':p,'s':s,'m':m,'dimension':dimension,'modulus':mod,'intermediate_modulus':workmod,'
     vertex_indices':list(av),'vertex_polynomials':av,'lift_equation_coefficients':coeff,'lagrange_profiles':
     terms_info,'lift_y_numerator':Ynum,'lift_y_denominator':Yden,'target_L':L}
    packed=json.dumps(payload,sort_keys=True,separators=(',',':'))
    report={'theorem':f'Every coefficient of E_({p},{s},{m}) for octahedral dimension {dimension} is
     divisible by {p}^{2*s}.','dimension':dimension,'p':p,'s':s,'m':m,'modulus':mod,'intermediate_modulus':
     workmod,'all_polynomial_coefficients_checked':True,'power_series_cutoff':None,'lift_constructed_from':'
     BV vertex polynomials; no target truncation assumption','lift_equation_root_identity_zero':root==[0],'
     cleared_target_identity_zero':target==[0],'lift_equation_degree':degree,'coefficient_polynomial_degrees
     ':[len(z)-1 for z in coeff],'lift_y_fraction_degrees':[len(Ynum)-1,len(Yden)-1],'
     lift_y_denominator_power':maxden,'cleared_identity_degree_bound':degree_bound,'certificate_sha256':
     hashlib.sha256(packed.encode()).hexdigest(),'elapsed_seconds':round(time.time()-started,3)}
    base=Path(__file__).resolve().parent;stem=f'certificate_octa{dimension}_p{p}_s{s}_m{m}'
    if write_files:(base/(stem+'.json')).write_text(json.dumps(payload,indent=2)+'\n')
    if write_files:(base/(stem+'_report.json')).write_text(json.dumps(report,indent=2)+'\n')
    if verbose:print(json.dumps(report,indent=2))
    return report

if __name__=='__main__':
    parser=argparse.ArgumentParser()
    parser.add_argument('--dimension',type=int,required=True)
    parser.add_argument('--p',type=int,required=True)
    parser.add_argument('--s',type=int,required=True)
    args=parser.parse_args();build(args.dimension,args.p,args.s)
```

G Complete dimension-uniform verifier

Save this appendix as `certify_all_dimensions_newton.py` beside the previous source. The commands

```
python3 certify_all_dimensions_newton.py --s 2
python3 certify_all_dimensions_newton.py --s 3
```

reproduce the two dimension-uniform proofs and write the exact JSON certificates and summaries. A run at another level must pass every assertion before it establishes any additional result.

```
#!/usr/bin/env python3
"""Uniform dimension certificates through ordinary-polynomial Newton models.

This is an exact proof for the level actually executed, not an induction on s.
"""
from pathlib import Path
from math import comb,factorial
import time,json,hashlib,argparse
import verify_dimension_families as v

def main(s):
    started=time.time();p=3;prec=2*s;k=prec-1;a=prec-1
    N=p**(prec-2);M=p**prec;ymod=p**k;work=p**(4*s-2)
    degree=max(r for r in range(1,3*k+1) if r-1-v.vp(r,p)<k)
    def factorial_threshold(exponent):
        r=0;valuation=0
        while valuation<exponent:
```

```
        r+=1;valuation+=v.vp(r,p)
    return r
ninput=factorial_threshold(a+k)
nmodel=factorial_threshold(k)
ntarget=factorial_threshold(prec)
length=2*p*N
print(json.dumps({'stage':'setup','level':s,'input_dimensions':ninput,'model_terms':nmodel,'
 target_dimensions':ntarget,'length':length}),flush=True)
choose=[[comb(j,i)**2%work for i in range(j+1)] for j in range(length)]
comps=[[1]+[0]*(length-1)]
for n in range(1,2+ninput):
    last=comps[-1]
    comps.append([sum(choose[j][i]*last[j-i] for i in range(j+1))%work for j in range(length)])
matrices={}
for K in [N,2*N,p*N,2*p*N]:
    fac=[factorial(j) for j in range(2*K)]
    rows=[]
    for j in range(K):
        row=[]
        for ell in range(j+1):
            num=fac[2*K-ell-1];den=fac[2*K-2*j-1]*fac[ell]*fac[j-ell]**2
            assert num%den==0
            row.append(((-1 if ell%2 else 1)*(num//den))%work)
        rows.append(row)
    matrices[K]=rows
print(json.dumps({'stage':'integer_matrices_ready','elapsed_seconds':round(time.time()-started,3)}),
 flush=True)
def av_for(n):
    bj=comps[n-1]
    return {K:v.trim([sum(c*bj[j-ell] for ell,c in enumerate(row))%work for j,row in enumerate(rows)])
 for K,rows in matrices.items()}
def coeff_for(av):
    aa,bb,ap,bp=[av[K] for K in [N,2*N,p*N,2*p*N]]
    ar=[v.taylor(aa,r,p,work) for r in range(degree+1)]
    br=[v.taylor(bb,r,p,work) for r in range(degree+1)]
    a0pow=[v.power(ar[0],r,work) for r in range(degree+2)]
    T=[]
    for r in range(degree+1):
        tr=v.mul(br[r],a0pow[r],work)
        for i in range(1,r+1):tr=v.add(tr,v.scale(v.mul(v.mul(ar[i],T[r-i],work),a0pow[i-1],work),-1,
 work),work)
        T.append(tr)
    qs=[]
    for r in range(degree+1):
        num=v.add(v.scale(v.mul(bp,a0pow[degree+1],work),comb(N,r),work),v.scale(v.mul(v.mul(ap,T[r],
 work),a0pow[degree-r],work),-1,work),work)
        num=v.scale(num,p**r,work)
        assert all(c%(p*N)==0 for c in num)
        qs.append(v.trim([(c//(p*N))%ymod for c in num]))
    return qs
avcache={};samples=[]
for n in range(2,2+ninput):
    av=av_for(n);avcache[n]=av;samples.append(coeff_for(av))
width=max(len(q) for s0 in samples for q in s0)
diffs=[[[q[j] if j<len(q) else 0 for j in range(width)] for q in s0] for s0 in samples]
newton=[];ordinary=[];bad=[]
for r in range(ninput):
    dr=diffs[0];vr=v.vp(factorial(r),p)
    need=min(vr,k)
    for q in range(degree+1):
        for j,c in enumerate(dr[q]):
            if c%p**need:bad.append([r,q,j,c,need])
    newton.append([v.trim(q) for q in dr])
    if r<nmodel:
        unit=factorial(r)//p**vr;iv=pow(unit,-1,M)
        ordinary.append([[((c//p**vr)*iv)%M for c in q] for q in dr])
    diffs=[[[ (diffs[i+1][q][j]-diffs[i][q][j])%ymod for j in range(width)] for q in range(degree+1)]
 for i in range(len(diffs)-1)]
if bad:
    print(json.dumps({'stage':'model_integrality_failed','bad':bad[:20]}),flush=True)
    raise AssertionError('ordinary model fails')
print(json.dumps({'stage':'ordinary_model_certified','model_degree':nmodel-1,'elapsed_seconds':round(
 time.time()-started,3)}),flush=True)
def model_at(n):
```

```
        out=[[0]*width for _ in range(degree+1)];fall=1;x=n-2
        for r,dr in enumerate(ordinary):
            if r:fall=(fall*(x-r+1))%M
            for q in range(degree+1):
                for j,c in enumerate(dr[q]):out[q][j]=(out[q][j]+fall*c)%M
        return [v.trim(q) for q in out]
    for n,actual in zip(range(2,2+ninput),samples):
        assert all(v.trim([c%ymod for c in q])==aa for q,aa in zip(model_at(n),actual))
    weights=[r-1-v.vp(r,p) for r in range(2,degree+1)]
    profiles=list(v.profiles(weights,k-1))
    maxden=max([1]+[1+sum((i+2)*c for i,c in enumerate(pr)) for pr in profiles])
    def root_from_model(qs):
        A,B=qs[:2];bpows=[v.power(B,r,M) for r in range(maxden+1)];apows=[v.power(A,r,M) for r in range(
     maxden+1)]
        Y=v.scale(v.mul(A,bpows[maxden-1],M),-1,M)
        for pr in profiles:
            count=sum(pr);Rsum=sum((i+2)*c for i,c in enumerate(pr));zexp=Rsum-count+1
            den=factorial(zexp)
            for c in pr:den*=factorial(c)
            assert factorial(Rsum)%den==0
            tree=factorial(Rsum)//den
            if v.vp(tree,p)+sum(w*c for w,c in zip(weights,pr))>=k:continue
            term=apows[zexp]
            for q,c in zip(qs[2:],pr):
                if c:term=v.mul(term,v.power(q,c,M),M)
            term=v.mul(term,bpows[maxden-Rsum-1],M)
            Y=v.add(Y,v.scale(term,(-1 if (Rsum+1)%2 else 1)*tree,M),M)
        return Y,bpows[maxden]
    reports=[]
    for n in range(2,2+ntarget):
        qs=model_at(n);Y,Z=root_from_model(qs)
        assert Z[0]%p!=0
        root=[0]
        for r,q in enumerate(qs):root=v.add(root,v.mul(v.mul(q,v.power(Y,r,ymod),ymod),v.power(Z,degree-r,
     ymod),ymod),ymod)
        assert root==[0],('root',n)
        av=avcache[n];ap=av[p*N];ar=[v.taylor(av[N],r,p,M) for r in range(prec)]
        f=[comb(2*j,j)*comps[n][j]%M for j in range(p**s)];fs=f[:p**(s-1)]
        L=[]
        for r in range(prec):
            lhs=v.taylor(fs,r,p,M)
            if r:lhs=v.add(lhs,v.scale(v.taylor(fs,r-1,p,M),N,M),M)
            L.append(v.add(v.mul(ap,lhs,M),v.scale(v.mul(f,ar[r],M),-1,M),M))
        Q=v.scale(Y,p,M);target=[0];bound=0
        for r,c in enumerate(L):
            term=v.mul(v.mul(c,v.power(Q,r,M),M),v.power(Z,prec-1-r,M),M)
            bound=max(bound,len(term)-1);target=v.add(target,term,M)
        assert target==[0],('target',n,[(i,c) for i,c in enumerate(target) if c][:10])
        report={'dimension':n,'root_identity_zero':True,'target_identity_zero':True,'target_degree_bound':
     bound,'Y_degree':len(Y)-1,'Z_degree':len(Z)-1}
        reports.append(report)
        print(json.dumps({'stage':'dimension_target_checked',**report,'elapsed_seconds':round(time.time()-
     started,3)}),flush=True)
    payload={'p':p,'s':s,'m':1,'work_modulus':work,'target_modulus':M,'input_dimensions':list(range(2,2+
     ninput)),'input_lift_coefficients':samples,'newton_coefficients':newton,'
     ordinary_model_coefficients_in_falling_basis':ordinary,'lagrange_profiles':profiles,'
     fixed_denominator_exponent':maxden,'target_reports':reports}
    packed=json.dumps(payload,sort_keys=True,separators=(',',':'))
    summary={'theorem':f'For all octahedral dimensions n>=2, every coefficient of E_(3,{s},1) is divisible
     by 3^{2*s}.','prime':p,'level':s,'multiplier':1,'target_modulus':M,'work_modulus':work,'
     input_dimension_count':ninput,'ordinary_model_degree':nmodel-1,'target_dimension_count':ntarget,'
     all_input_difference_integrality_checked':True,'all_polynomial_coefficients_checked':True,'
     power_series_cutoff':None,'certificate_sha256':hashlib.sha256(packed.encode()).hexdigest(),'
     elapsed_seconds':round(time.time()-started,3)}
    base=Path(__file__).resolve().parent
    (base/f'all_dimensions_newton_s{s}_certificate.json').write_text(json.dumps(payload,indent=2)+'\n')
    (base/f'all_dimensions_newton_s{s}_summary.json').write_text(json.dumps(summary,indent=2)+'\n')
    print(json.dumps(summary,indent=2))
if __name__=='__main__':
    parser=argparse.ArgumentParser();parser.add_argument('--s',type=int,required=True);args=parser.
     parse_args();main(args.s)
```

H Exact modular input matrices and model-only mode

The following source is `certify_all_dimensions_newton_fast.py`. It requires NumPy in addition to the previous arithmetic source. For level 4, use `-s 4 -model-only`. The overflow bound is checked before any integer matrix multiplication. The output model checkpoint is consumed by the next appendix.

```
#!/usr/bin/env python3
"""Uniform dimension certificates through ordinary-polynomial Newton models.

This is an exact proof for the level actually executed, not an induction on s.
"""
from pathlib import Path
from math import comb,factorial
import time,json,hashlib,argparse
import numpy as np
import verify_dimension_families as v

def main(s,model_only=False):
    started=time.time();p=3;prec=2*s;k=prec-1;a=prec-1
    N=p**(prec-2);M=p**prec;ymod=p**k;work=p**(4*s-2)
    degree=max(r for r in range(1,3*k+1) if r-1-v.vp(r,p)<k)
    def factorial_threshold(exponent):
        r=0;valuation=0
        while valuation<exponent:
            r+=1;valuation+=v.vp(r,p)
        return r
    ninput=factorial_threshold(a+k)
    nmodel=factorial_threshold(k)
    ntarget=factorial_threshold(prec)
    length=2*p*N
    print(json.dumps({'stage':'setup','level':s,'input_dimensions':ninput,'model_terms':nmodel,'
     target_dimensions':ntarget,'length':length}),flush=True)
    assert work*work*length < 2**63
    exponent=4*s-2
    facunit=[1]*(2*length);facval=[0]*(2*length)
    for j in range(1,2*length):
        w=j;val=0
        while w%p==0:w//=p;val+=1
        facunit[j]=(facunit[j-1]*w)%work;facval[j]=facval[j-1]+val
    invunit=np.array([pow(w,-1,work) for w in facunit],dtype=np.int64)
    facunit=np.array(facunit,dtype=np.int64);facval=np.array(facval,dtype=np.int64)
    ppows=np.array([p**e if e<exponent else 0 for e in range(2*int(facval[-1])+1)],dtype=np.int64)
    choose=np.zeros((length,length),dtype=np.int64)
    for j in range(length):
        ii=np.arange(j+1);vv=2*(facval[j]-facval[ii]-facval[j-ii])
        uu=facunit[j]*invunit[ii]%work;uu=uu*invunit[j-ii]%work
        choose[j,:j+1]=(uu*uu%work)*ppows[vv]%work
    comps=[np.zeros(length,dtype=np.int64)];comps[0][0]=1
    for n in range(1,2+ninput):comps.append(choose.dot(comps[-1])%work)
    del choose
    print(json.dumps({'stage':'dimension_coefficients_ready','elapsed_seconds':round(time.time()-started,3)
     }),flush=True)
    matrices={}
    for K in [N,2*N,p*N,2*p*N]:
        mat=np.zeros((K,K),dtype=np.int64)
        for j in range(K):
            ii=np.arange(j+1);aa=2*K-j+ii-1;bb=2*K-2*j-1;ell=j-ii
            vv=facval[aa]-facval[bb]-facval[ell]-2*facval[ii]
            assert min(vv)>=0
            uu=facunit[aa]*invunit[bb]%work;uu=uu*invunit[ell]%work
            uu=uu*(invunit[ii]*invunit[ii]%work)%work
            row=uu*ppows[vv]%work;mat[j,:j+1]=np.where(ell%2,(-row)%work,row)
        matrices[K]=mat
    print(json.dumps({'stage':'integer_matrices_ready','elapsed_seconds':round(time.time()-started,3)}),
     flush=True)
    def av_for(n):
        bj=comps[n-1]
        return {K:v.trim((mat.dot(bj[:K])%work).tolist()) for K,mat in matrices.items()}
    def coeff_for(av):
        aa,bb,ap,bp=[av[K] for K in [N,2*N,p*N,2*p*N]]
        ar=[v.taylor(aa,r,p,work) for r in range(degree+1)]
```

```
    br=[v.taylor(bb,r,p,work) for r in range(degree+1)]
    a0pow=[v.power(ar[0],r,work) for r in range(degree+2)]
    T=[]
    for r in range(degree+1):
        tr=v.mul(br[r],a0pow[r],work)
        for i in range(1,r+1):tr=v.add(tr,v.scale(v.mul(v.mul(ar[i],T[r-i],work),a0pow[i-1],work),-1,
 work),work)
        T.append(tr)
    qs=[]
    for r in range(degree+1):
        num=v.add(v.scale(v.mul(bp,a0pow[degree+1],work),comb(N,r),work),v.scale(v.mul(v.mul(ap,T[r],
 work),a0pow[degree-r],work),-1,work),work)
        num=v.scale(num,p**r,work)
        assert all(c%(p*N)==0 for c in num)
        qs.append(v.trim([(c//(p*N))%ymod for c in num]))
    return qs
avcache={};samples=[]
for n in range(2,2+ninput):
    av=av_for(n);avcache[n]=av;samples.append(coeff_for(av))
width=max(len(q) for s0 in samples for q in s0)
diffs=[[[q[j] if j<len(q) else 0 for j in range(width)] for q in s0] for s0 in samples]
newton=[];ordinary=[];bad=[]
for r in range(ninput):
    dr=diffs[0];vr=v.vp(factorial(r),p)
    need=min(vr,k)
    for q in range(degree+1):
        for j,c in enumerate(dr[q]):
            if c%p**need:bad.append([r,q,j,c,need])
    newton.append([v.trim(q) for q in dr])
    if r<nmodel:
        unit=factorial(r)//p**vr;iv=pow(unit,-1,M)
        ordinary.append([[((c//p**vr)*iv)%M for c in q] for q in dr])
    diffs=[[[ (diffs[i+1][q][j]-diffs[i][q][j])%ymod for j in range(width)] for q in range(degree+1)]
 for i in range(len(diffs)-1)]
if bad:
    print(json.dumps({'stage':'model_integrality_failed','bad':bad[:20]}),flush=True)
    raise AssertionError('ordinary model fails')
print(json.dumps({'stage':'ordinary_model_certified','model_degree':nmodel-1,'elapsed_seconds':round(
 time.time()-started,3)}),flush=True)
def model_at(n):
    out=[[0]*width for _ in range(degree+1)];fall=1;x=n-2
    for r,dr in enumerate(ordinary):
        if r:fall=(fall*(x-r+1))%M
        for q in range(degree+1):
            for j,c in enumerate(dr[q]):out[q][j]=(out[q][j]+fall*c)%M
    return [v.trim(q) for q in out]
for n,actual in zip(range(2,2+ninput),samples):
    assert all(v.trim([c%ymod for c in q])==aa for q,aa in zip(model_at(n),actual))
weights=[r-1-v.vp(r,p) for r in range(2,degree+1)]
profiles=list(v.profiles(weights,k-1))
maxden=max([1]+[1+sum((i+2)*c for i,c in enumerate(pr)) for pr in profiles])
def root_from_model(qs):
    A,B=qs[:2];bpows=[v.power(B,r,M) for r in range(maxden+1)];apows=[v.power(A,r,M) for r in range(
 maxden+1)]
    Y=v.scale(v.mul(A,bpows[maxden-1],M),-1,M)
    for pr in profiles:
        count=sum(pr);Rsum=sum((i+2)*c for i,c in enumerate(pr));zexp=Rsum-count+1
        den=factorial(zexp)
        for c in pr:den*=factorial(c)
        assert factorial(Rsum)%den==0
        tree=factorial(Rsum)//den
        if v.vp(tree,p)+sum(w*c for w,c in zip(weights,pr))>=k:continue
        term=apows[zexp]
        for q,c in zip(qs[2:],pr):
            if c:term=v.mul(term,v.power(q,c,M),M)
        term=v.mul(term,bpows[maxden-Rsum-1],M)
        Y=v.add(Y,v.scale(term,(-1 if (Rsum+1)%2 else 1)*tree,M),M)
    return Y,bpows[maxden]
reports=[]
partial={'p':p,'s':s,'work_modulus':work,'input_dimension_count':ninput,'model_degree':nmodel-1,'
 input_lift_coefficients':samples,'newton_coefficients':newton,'
 ordinary_model_coefficients_in_falling_basis':ordinary,'lagrange_profiles':profiles,'
 fixed_denominator_exponent':maxden}
Path(__file__).resolve().parent.joinpath(f'newton_s{s}_model_checkpoint.json').write_text(json.dumps(
```

```
    partial,separators=(',',':'))+'\n')
    if model_only:
        print(json.dumps({'stage':'model_only_complete','level':s,'input_dimensions':ninput,'model_degree':
    nmodel-1,'all_input_difference_integrality_checked':True,'checkpoint':str(Path(__file__).resolve().
    parent/f'newton_s{s}_model_checkpoint.json')}),flush=True)
        return
    for n in range(2,2+ntarget):
        qs=model_at(n);Y,Z=root_from_model(qs)
        assert Z[0]%p!=0
        root=[0]
        for r,q in enumerate(qs):root=v.add(root,v.mul(v.mul(q,v.power(Y,r,ymod),ymod),v.power(Z,degree-r,
    ymod),ymod),ymod)
        assert root==[0],('root',n)
        av=avcache[n];ap=av[p*N];ar=[v.taylor(av[N],r,p,M) for r in range(prec)]
        f=[comb(2*j,j)*int(comps[n][j])%M for j in range(p**s)];fs=f[:p**(s-1)]
        L=[]
        for r in range(prec):
            lhs=v.taylor(fs,r,p,M)
            if r:lhs=v.add(lhs,v.scale(v.taylor(fs,r-1,p,M),N,M),M)
            L.append(v.add(v.mul(ap,lhs,M),v.scale(v.mul(f,ar[r],M),-1,M),M))
        Q=v.scale(Y,p,M);target=[0];bound=0
        for r,c in enumerate(L):
            term=v.mul(v.mul(c,v.power(Q,r,M),M),v.power(Z,prec-1-r,M),M)
            bound=max(bound,len(term)-1);target=v.add(target,term,M)
        assert target==[0],('target',n,[(i,c) for i,c in enumerate(target) if c][:10])
        report={'dimension':n,'root_identity_zero':True,'target_identity_zero':True,'target_degree_bound':
    bound,'Y_degree':len(Y)-1,'Z_degree':len(Z)-1}
        reports.append(report)
        Path(__file__).resolve().parent.joinpath(f'newton_s{s}_target_progress.json').write_text(json.dumps(
    reports,indent=2)+'\n')
        print(json.dumps({'stage':'dimension_target_checked',**report,'elapsed_seconds':round(time.time()-
    started,3)}),flush=True)
    payload={'p':p,'s':s,'m':1,'work_modulus':work,'target_modulus':M,'input_dimensions':list(range(2,2+
    ninput)),'input_lift_coefficients':samples,'newton_coefficients':newton,'
    ordinary_model_coefficients_in_falling_basis':ordinary,'lagrange_profiles':profiles,'
    fixed_denominator_exponent':maxden,'target_reports':reports}
    packed=json.dumps(payload,sort_keys=True,separators=(',',':'))
    summary={'theorem':f'For all octahedral dimensions n>=2, every coefficient of E_(3,{s},1) is divisible
    by 3^{2*s}.','prime':p,'level':s,'multiplier':1,'target_modulus':M,'work_modulus':work,'
    input_dimension_count':ninput,'ordinary_model_degree':nmodel-1,'target_dimension_count':ntarget,'
    all_input_difference_integrality_checked':True,'all_polynomial_coefficients_checked':True,'
    power_series_cutoff':None,'certificate_sha256':hashlib.sha256(packed.encode()).hexdigest(),'
    elapsed_seconds':round(time.time()-started,3)}
    base=Path(__file__).resolve().parent
    (base/f'all_dimensions_newton_s{s}_certificate.json').write_text(json.dumps(payload,indent=2)+'\n')
    (base/f'all_dimensions_newton_s{s}_summary.json').write_text(json.dumps(summary,indent=2)+'\n')
    print(json.dumps(summary,indent=2))
if __name__=='__main__':
    parser=argparse.ArgumentParser();parser.add_argument('--s',type=int,required=True);parser.add_argument('
    --model-only',action='store_true');args=parser.parse_args();main(args.s,args.model_only)
```

I Compact Hensel roots and complete target certificates

Save this source as `compact_hensel.py` in the same directory. It checks the final entire root and target polynomial identities, and saves their explicit coefficient arrays. The optional summary script in the source archive computes file identifiers only and is not a dependency of any mathematical claim.

```
from pathlib import Path
import json,time,argparse
from math import comb
import numpy as np
import verify_dimension_families as v

def invseries(b,n,p):
    g=[pow(b[0],-1,p)]
    while len(g)<n:
        k=min(n,2*len(g));bg=v.mul(b[:k],g,p)[:k]
        h=[(-x)%p for x in bg];h[0]=(h[0]+2)%p
        g=v.mul(g,h,p)[:k];g+= [0]*(k-len(g))
    return g

def divide(a,b,p=3):
    a=v.trim([x%p for x in a]);b=v.trim([x%p for x in b])
    if len(a)<len(b):return [0],a
    count=len(a)-len(b)+1
    if count<=24:
        rem=a[:];q=[0]*count;iv=pow(b[-1],-1,p)
        for k in range(count-1,-1,-1):
            c=rem[k+len(b)-1]*iv%p;q[k]=c
            if c:
                for j,x in enumerate(b):rem[k+j]=(rem[k+j]-c*x)%p
        return v.trim(q),v.trim(rem)
    revb=list(reversed(b));reva=list(reversed(a))[:count]
    qrev=v.mul(reva,invseries(revb,count,p),p)[:count];qrev +=[0]*(count-len(qrev))
    q=v.trim(list(reversed(qrev)));rem=v.add(a,v.scale(v.mul(b,q,p),-1,p),p)
    assert len(rem)<len(b)
    return q,rem

def gcd(a,b,p=3):
    while b!=[0]:a,b=b,divide(a,b,p)[1]
    return v.scale(a,pow(a[-1],-1,p),p)

def reduce_fraction(a,b,p=3):
    a=v.trim([x%p for x in a]);b=v.trim([x%p for x in b])
    if a==[0]:return [0],[1]
    g=gcd(a,b,p);aa,ar=divide(a,g,p);bb,br=divide(b,g,p)
    assert ar==[0] and br==[0] and bb[0]
    iv=pow(bb[0],-1,p)
    return v.scale(aa,iv,p),v.scale(bb,iv,p)

def root(qs,p,k,verbose=False):
    d=len(qs)-1;A,B=reduce_fraction(v.scale(qs[0],-1,p),qs[1],p)
    for j in range(1,k):
        mod=p**(j+1);S=[0]
        for r,q in enumerate(qs):S=v.add(S,v.mul(v.mul(q,v.power(A,r,mod),mod),v.power(B,d-r,mod),mod),mod)
        assert all(c%(p**j)==0 for c in S)
        E=v.trim([(-c//p**j)%p for c in S]);D=v.mul(v.power(B,d,p),qs[1],p)
        C,H=reduce_fraction(E,D,p)
        A=v.add(v.mul(A,H,mod),v.scale(v.mul(C,B,mod),p**j,mod),mod);B=v.mul(B,H,mod)
        if verbose:print(json.dumps({'digit':j+1,'numdeg':len(A)-1,'dendeg':len(B)-1}),flush=True)
    mod=p**k;S=[0]
    for r,q in enumerate(qs):S=v.add(S,v.mul(v.mul(q,v.power(A,r,mod),mod),v.power(B,d-r,mod),mod),mod)
    assert S==[0]
    return A,B

def inputs(s,maxn):
    p=3;N=p**(2*s-2);M=p**(2*s);length=p*N
    assert M*M*length<2**63
    fu=[1]*(2*length);fv=[0]*(2*length)
    for j in range(1,2*length):
        w=j;val=0
        while w%p==0:w//=p;val+=1
```

```
        fu[j]=fu[j-1]*w%M;fv[j]=fv[j-1]+val
    inv=np.array([pow(w,-1,M) for w in fu],dtype=np.int64);fu=np.array(fu,dtype=np.int64);fv=np.array(fv,
     dtype=np.int64)
    ps=np.array([p**e if e<2*s else 0 for e in range(2*int(fv[-1])+1)],dtype=np.int64)
    choose=np.zeros((length,length),dtype=np.int64)
    for j in range(length):
        ii=np.arange(j+1);vv=2*(fv[j]-fv[ii]-fv[j-ii]);uu=fu[j]*inv[ii]%M;uu=uu*inv[j-ii]%M;choose[j,:j+1]=(
     uu*uu%M)*ps[vv]%M
    b=[np.zeros(length,dtype=np.int64)];b[0][0]=1
    for _ in range(maxn):b.append(choose.dot(b[-1])%M)
    mats={}
    for K in [N,p*N]:
        mat=np.zeros((K,K),dtype=np.int64)
        for j in range(K):
            ii=np.arange(j+1);aa=2*K-j+ii-1;bb=2*K-2*j-1;ell=j-ii;vv=fv[aa]-fv[bb]-fv[ell]-2*fv[ii]
            assert min(vv)>=0
            uu=fu[aa]*inv[bb]%M;uu=uu*inv[ell]%M;uu=uu*(inv[ii]*inv[ii]%M)%M;row=uu*ps[vv]%M;mat[j,:j+1]=np.
     where(ell%2,(-row)%M,row)
        mats[K]=mat
    return b,mats

def main(s,dimensions):
    start=time.time();base=Path(__file__).resolve().parent
    c=json.loads((base/f'newton_s{s}_model_checkpoint.json').read_text());p=3;k=2*s-1;M=p**(2*s);N=p**(2*s
     -2);d=len(c['input_lift_coefficients'][0])-1
    b,mats=inputs(s,max(dimensions));reports=[]
    for n in dimensions:
        qs=c['input_lift_coefficients'][n-2];Y,Z=root(qs,p,k,True)
        av={K:v.trim((mat.dot(b[n-1][:K])%M).tolist()) for K,mat in mats.items()}
        ar=[v.taylor(av[N],r,p,M) for r in range(2*s)];f=[comb(2*j,j)*int(b[n][j])%M for j in range(p**s)];
     fs=f[:p**(s-1)];target=[0];bound=0;L=[]
        for r in range(2*s):
            lhs=v.taylor(fs,r,p,M)
            if r:lhs=v.add(lhs,v.scale(v.taylor(fs,r-1,p,M),N,M),M)
            lr=v.add(v.mul(av[p*N],lhs,M),v.scale(v.mul(f,ar[r],M),-1,M),M);L.append(lr)
            term=v.mul(v.mul(lr,v.power(v.scale(Y,p,M),r,M),M),v.power(Z,2*s-1-r,M),M);bound=max(bound,len(
     term)-1);target=v.add(target,term,M)
        assert target==[0],('target',n,[(i,x) for i,x in enumerate(target) if x][:10])
        rep={'dimension':n,'p':p,'s':s,'m':1,'target_modulus':M,'root_identity_zero':True,'
     target_identity_zero':True,'target_degree_bound':bound,'Y_degree':len(Y)-1,'Z_degree':len(Z)-1}
        payload={'report':rep,'Q':qs,'Y':Y,'Z':Z,'L':L}
        (base/f'compact_octa{n}_s{s}_certificate.json').write_text(json.dumps(payload,separators=(',',':'))+
     '\n');reports.append(rep)
        (base/f'compact_s{s}_reports.json').write_text(json.dumps(reports,indent=2)+'\n')
        print(json.dumps({'stage':'target_passed',**rep,'elapsed_seconds':round(time.time()-start,3)}),flush
     =True)
if __name__=='__main__':
    parser=argparse.ArgumentParser();parser.add_argument('--s',type=int,required=True);parser.add_argument('
     --n',type=int,nargs='+',required=True);a=parser.parse_args();main(a.s,a.n)
```

J Complete fifth-level certificate source and independent evaluator

These programs reproduce the independent fifth-level certificate of Section 30. They are not premises of the symbolic all-level proof. Python 3, NumPy, and the system GMP shared library are required; no `gmpy2` package is used. The polynomial module `verify_dimension_families.py` and the rational Hensel module `compact_hensel.py` are printed in Appendices F and I.

Save those two modules in `research/octa_new_certificates/`. Save the first four programs below in the sibling `research/octa_level5/` directory. Save the independent evaluator in `research/octa_all_level_closure/`. This relative directory layout matches the imports in the printed sources. From the `octa_level5` directory run

```
python3 scaled_convolution.py
python3 linear_reduce.py
python3 direct_certificate.py --test
python3 uniform_level.py --s 5 --stage all --recompute
```

Then execute `verify_saved_level5.py` from its own directory. The generator recomputes the 39 input dimensions, the ordinary dimension model, and the 24 target certificates. The independent evaluator reconstructs the target polynomials and sums all homogeneous terms separately from the generator's root solver and Horner routine. The complete 24-target independent re-evaluation passed in the preceding audit. This edition preserves the exact sources used in that audit; it does not represent that run as a new external review. The compressed coefficient arrays and JSON payloads are generated outputs and are not needed to compile this document.

The final short listing generates the fourth-level summary and its archived identifiers. Save `summarize_level4_certificate.py` in `research/octa_new_certificates/`.

J.1 scaled_convolution.py

Complete source; 109 lines.

```
"""Exact p=3 Bessel/negative-vertex transforms by ordinary convolution.

GMP is used only for exact nonnegative integer multiplication.  The
Kronecker base strictly exceeds every possible convolution coefficient.
"""
from pathlib import Path
from math import comb
import ctypes,ctypes.util,sys,time,json
sys.path.insert(0,str(Path(__file__).resolve().parents[1]/'octa_new_certificates'))
import verify_dimension_families as v

class MPZ(ctypes.Structure):
    _fields_=[('alloc',ctypes.c_int),('size',ctypes.c_int),('limbs',ctypes.c_void_p)]
gmp=ctypes.CDLL(ctypes.util.find_library('gmp'))
zinit=getattr(gmp,'__gmpz_init');zclear=getattr(gmp,'__gmpz_clear')
zimport=getattr(gmp,'__gmpz_import');zexport=getattr(gmp,'__gmpz_export');zmul=getattr(gmp,'__gmpz_mul')
for fn in [zinit,zclear]:fn.argtypes=[ctypes.POINTER(MPZ)]
zmul.argtypes=[ctypes.POINTER(MPZ)]*3
zimport.argtypes=[ctypes.POINTER(MPZ),ctypes.c_size_t,ctypes.c_int,ctypes.c_size_t,ctypes.c_int,ctypes.
    c_size_t,ctypes.c_void_p]
zexport.argtypes=[ctypes.c_void_p,ctypes.POINTER(ctypes.c_size_t),ctypes.c_int,ctypes.c_size_t,ctypes.c_int,
    ctypes.c_size_t,ctypes.POINTER(MPZ)]
zexport.restype=ctypes.c_void_p

def mul(a,b,mod,limit=None):
    a=v.trim([int(x)%mod for x in a]);b=v.trim([int(x)%mod for x in b])
    if a==[0] or b==[0]:return [0]
    width=(min(len(a),len(b)).bit_length()+2*mod.bit_length()+7)//8
    assert (1<<(8*width))>min(len(a),len(b))*(mod-1)**2
    aa=b''.join(x.to_bytes(width,'little') for x in a);bb=b''.join(x.to_bytes(width,'little') for x in b)
    za,zb,zc=MPZ(),MPZ(),MPZ()
```

```
    for z in [za,zb,zc]:zinit(ctypes.byref(z))
    try:
        zimport(ctypes.byref(za),len(aa),-1,1,0,0,aa);zimport(ctypes.byref(zb),len(bb),-1,1,0,0,bb)
        zmul(ctypes.byref(zc),ctypes.byref(za),ctypes.byref(zb))
        length=len(a)+len(b)-1;raw=ctypes.create_string_buffer(length*width);count=ctypes.c_size_t()
        zexport(raw,ctypes.byref(count),-1,1,0,0,ctypes.byref(zc))
        data=raw.raw
        if limit is not None:length=min(length,limit)
        return v.trim([int.from_bytes(data[i*width:(i+1)*width],'little')%mod for i in range(length)])
    finally:
        for z in [za,zb,zc]:zclear(ctypes.byref(z))

class Inputs:
    def __init__(self,s=None,length=None,exponent=None,indices=None):
        if s is not None:
            N=3**(2*s-2);length=6*N;exponent=4*s-2;indices=[N,2*N,3*N,6*N]
        self.length=length;self.exponent=exponent;self.work=3**exponent;self.indices=indices
        self.val=[0]*(2*length)
        for j in range(1,2*length):
            w=j;vv=0
            while w%3==0:w//=3;vv+=1
            self.val[j]=self.val[j-1]+vv
        self.h={K:max(K-l-self.val[2*K-l-1]+self.val[l] for l in range(K)) for K in indices}
        self.H=max(self.h.values());self.mod=3**(exponent+self.H)
        self.unit=[1]*(2*length)
        for j in range(1,2*length):
            w=j
            while w%3==0:w//=3
            self.unit[j]=self.unit[j-1]*w%self.mod
        self.inv=[pow(w,-1,self.mod) for w in self.unit]
        self.J=[pow(3,j-2*self.val[j],self.mod)*self.inv[j]**2%self.mod for j in range(length)]
        self.alpha={}
        for K in indices:
            h=self.h[K];arr=[]
            for l in range(K):
                ev=l-K+h+self.val[2*K-l-1]-self.val[l]
                assert ev>=0
                arr.append(((-1 if l%2 else 1)*pow(3,ev,self.mod)*self.unit[2*K-l-1]*self.inv[l])%self.mod)
            self.alpha[K]=arr

    def vertices(self,Jpower):
        result={}
        for K in self.indices:
            vals=mul(self.alpha[K],Jpower[:K],self.mod,K);vals+=[0]*(K-len(vals));out=[]
            for j,c in enumerate(vals):
                ev=K-j-self.h[K]-self.val[2*K-2*j-1]
                if ev<0:
                    den=3**(-ev);assert c%den==0,(K,j,ev,c);c//=den
                else:c*=3**ev
                out.append(c*self.inv[2*K-2*j-1]%self.work)
            assert out[0]==1
            result[K]=v.trim(out)
        return result

    def composition(self,Jpower,length=None,mod=None):
        length=length or self.length;mod=mod or self.work;out=[]
        for j in range(length):
            den=3**(j-2*self.val[j]);c=Jpower[j] if j<len(Jpower) else 0
            assert c%den==0
            assert self.mod% (mod*den)==0
            out.append((c//den)*self.unit[j]**2%mod)
        return out

def test():
    import random
    for mod in [3,81,3**18,3**29]:
        for na,nb in [(3,4),(32,31),(100,153),(1000,700)]:
            rng=random.Random(na+nb);a=[rng.randrange(mod) for _ in range(na)];b=[rng.randrange(mod) for _
     in range(nb)]
            assert mul(a,b,mod)==v.mul(a,b,mod)
    ins=Inputs(length=100,exponent=14,indices=[1,2,3,7,9,17,27,54])
    current=[1]
    for dim in range(1,8):
        current=mul(current,ins.J,ins.mod,ins.length)
```

```
        exact=v.composition_squares(dim,100)
        assert ins.composition(current,length=30,mod=3**8)==[a%3**8 for a in exact[:30]]
        av=ins.vertices(current)
        for K in ins.indices:assert av[K]==v.vertex(K,exact,ins.work),(dim,K)
    print('scaled convolution verified against exact original formulas')

if __name__=='__main__':test()
```

J.2 linear_reduce.py

Complete source; 85 lines.

```
"""Exact fast gcd reduction for a denominator equal to a linear power in F_3[u]."""
import numpy as np
from math import comb
from pathlib import Path
import sys
sys.path.insert(0,str(Path(__file__).resolve().parents[1]/'octa_new_certificates'))
import compact_hensel as ch
import scaled_convolution as sc
v=sc.v

def linear_power(c,e):
    a=np.zeros(e+1,dtype=np.int64);a[0]=1;step=1;end=1
    while e:
        digit=e%3;e//=3
        if digit:
            old=a[:end].copy()
            a[step:step+end]=(digit*c*old)%3
            if digit==2:a[2*step:2*step+end]=(c*c*old)%3
            end+=digit*step
        step*=3
    return a.tolist()

def translate3(a,t):
    t%=3
    if not t:return list(a)
    length=1
    while length<len(a):length*=3
    out=np.zeros(length,dtype=np.int64);out[:len(a)]=a
    step=1
    while step<length:
        blocks=out.reshape(-1,3,step)
        b1=blocks[:,1,:].copy();b2=blocks[:,2,:].copy()
        blocks[:,0,:]=(blocks[:,0,:]+t*b1+t*t*b2)%3
        blocks[:,1,:]=(b1+2*t*b2)%3
        step*=3
    return v.trim(out.tolist())

def reduce_fraction(a,b,p=3):
    if p!=3:return ch.reduce_fraction(a,b,p)
    a=v.trim([int(x)%p for x in a]);b=v.trim([int(x)%p for x in b])
    if a==[0]:return [0],[1]
    assert b[0]
    iv=pow(b[0],-1,p);a=v.scale(a,iv,p);b=v.scale(b,iv,p)
    if b==[1]:return a,b
    deg=len(b)-1
    multiplicities={}
    for c in [1,2]:
        t=(-pow(c,-1,3))%3
        bt=translate3(b,t)
        multiplicities[c]=next((i for i,x in enumerate(bt) if x),len(bt))
    if sum(multiplicities.values())!=deg:return ch.reduce_fraction(a,b,p)
    C=a;H=[1]
    for c,hb in multiplicities.items():
        if not hb:continue
        root=(-pow(c,-1,3))%3
        at=translate3(C,root);ha=next((i for i,x in enumerate(at) if x),len(at));h=min(ha,hb)
        C=v.scale(translate3(at[h:],-root),pow(pow(c,-1,3),h,3),3)
        H=v.mul(H,linear_power(c,hb-h),3)
    return C,H
```

```
def test():
    import random
    rng=random.Random(0)
    for size in [1,2,5,10,50,100,270]:
        a=[rng.randrange(3) for _ in range(size)]
        for t in [1,2]:
            expected=[sum(a[j]*comb(j,k)*pow(t,j-k,3) for j in range(k,size))%3 for k in range(size)]
            assert translate3(a,t)==v.trim(expected)
    for c in [1,2]:
        for e in range(300):assert linear_power(c,e)==v.power([1,c],e,3)
        for e in [10,40,200,1000]:
            b=linear_power(c,e)
            for h in [0,5,e//2,e]:
                a=v.mul([1,0,2,1],linear_power(c,h),3)
                C,H=reduce_fraction(a,b)
                assert v.mul(C,b,3)==v.mul(a,H,3)
                assert ch.gcd(C,H)==[1]
    for e1,e2,h1,h2 in [(10,20,5,10),(1000,500,555,499),(120,140,130,0)]:
        b=v.mul(linear_power(1,e1),linear_power(2,e2),3)
        a=v.mul(v.mul(linear_power(1,h1),linear_power(2,h2),3),[1,0,1],3)
        C,H=reduce_fraction(a,b)
        assert v.mul(C,b,3)==v.mul(a,H,3) and ch.gcd(C,H)==[1]
    print('ternary translation and exact linear-power rational reduction verified')

if __name__=='__main__':test()
```

J.3 direct_certificate.py

Complete source; 106 lines.

```
#!/usr/bin/env python3
"""Exact negative-vertex certificates using a directly cleared lift equation."""
from pathlib import Path
from math import comb
import sys,json,time,argparse,hashlib
import scaled_convolution as sc
v=sc.v
v.mul=sc.mul
import compact_hensel as ch
import linear_reduce as lr
BASE=Path(__file__).resolve().parent
OLD=BASE.parent/'octa_new_certificates'

def degree_for(k):
    return max(r for r in range(1,3*k+1) if r-1-(len_digits(r,3)-1)<k)

def len_digits(n,p):
    z=0
    while n:n//=p;z+=1
    return z

def equation(av,s):
    p=3;k=2*s-1;N=p**(2*s-2);work=p**(4*s-2);ymod=p**k;d=degree_for(k)
    ar=[v.taylor(av[N],r,p,work) for r in range(d+1)]
    br=[v.taylor(av[2*N],r,p,work) for r in range(d+1)]
    out=[]
    for r in range(d+1):
        aa=[0]
        for i in range(r+1):aa=v.add(aa,v.scale(ar[i],comb(N,r-i),work),work)
        num=v.add(v.mul(av[6*N],aa,work),v.scale(v.mul(av[3*N],br[r],work),-1,work),work)
        num=v.scale(num,p**r,work)
        assert all(c%(p*N)==0 for c in num),('direct integrality',r)
        out.append(v.trim([c//(p*N)%ymod for c in num]))
    assert out[0][0]==0 and out[1][0]==1
    assert all(all(c%3==0 for c in q) for q in out[2:])
    return out

def cleared_eval(qs,Y,Z,mod):
    # Homogeneous Horner form of sum Q_r Y^r Z^(d-r).
    d=len(qs)-1;out=qs[d];zp=[1]
```

```
    for r in range(d-1,-1,-1):
        zp=v.mul(zp,Z,mod)
        out=v.add(v.mul(out,Y,mod),v.mul(qs[r],zp,mod),mod)
    return v.trim([c%mod for c in out])

def root(qs,p,k,verbose=False):
    d=len(qs)-1;A,B=lr.reduce_fraction(v.scale(qs[0],-1,p),qs[1],p)
    if verbose:print(json.dumps({'stage':'root_digit','digit':1,'numdeg':len(A)-1,'dendeg':len(B)-1}),flush=
     True)
    for j in range(1,k):
        mod=p**(j+1);S=cleared_eval(qs,A,B,mod)
        assert all(c%p**j==0 for c in S)
        E=v.trim([(-c//p**j)%p for c in S]);D=v.mul(v.power(B,d,p),qs[1],p)
        C,H=lr.reduce_fraction(E,D,p)
        A=v.add(v.mul(A,H,mod),v.scale(v.mul(C,B,mod),p**j,mod),mod);B=v.mul(B,H,mod)
        if verbose:print(json.dumps({'stage':'root_digit','digit':j+1,'numdeg':len(A)-1,'dendeg':len(B)-1}),
     flush=True)
    assert cleared_eval(qs,A,B,p**k)==[0]
    return A,B

def target_polynomials(av,f,s):
    p=3;N=p**(2*s-2);M=p**(2*s);fs=f[:p**(s-1)];L=[]
    for r in range(2*s):
        lhs=v.taylor(fs,r,p,M)
        if r:lhs=v.add(lhs,v.scale(v.taylor(fs,r-1,p,M),N,M),M)
        lr=v.add(v.mul(av[3*N],lhs,M),v.scale(v.mul(f,v.taylor(av[N],r,p,M),M),-1,M),M)
        L.append(lr)
    return L

def run(s,dimension):
    start=time.time();p=3;k=2*s-1;M=p**(2*s);ins=sc.Inputs(s=s);Jpower=[1]
    print(json.dumps({'stage':'setup','level':s,'dimension':dimension,'length':ins.length,'extra_precision':
     ins.H,'lift_degree':degree_for(k)}),flush=True)
    for j in range(dimension-1):Jpower=sc.mul(Jpower,ins.J,ins.mod,ins.length)
    av=ins.vertices(Jpower)
    print(json.dumps({'stage':'vertices_ready','seconds':round(time.time()-start,3)}),flush=True)
    qs=equation(av,s)
    checkpoint={'s':s,'dimension':dimension,'Q':qs,'vertices':av}
    (BASE/f'direct_octa{dimension}_s{s}_inputs.json').write_text(json.dumps(checkpoint,separators=(',',':'))
     +'\n')
    print(json.dumps({'stage':'equation_ready','degrees':[len(q)-1 for q in qs],'seconds':round(time.time()-
     start,3)}),flush=True)
    Y,Z=root(qs,p,k,True)
    Jnext=sc.mul(Jpower[:3**s],ins.J[:3**s],ins.mod,3**s)
    b=ins.composition(Jnext,3**s,M);f=[comb(2*j,j)*b[j]%M for j in range(3**s)]
    L=target_polynomials(av,f,s);zero=cleared_eval(L,v.scale(Y,p,M),Z,M)
    assert zero==[0],('target',[(i,c) for i,c in enumerate(zero) if c][:10])
    rep={'dimension':dimension,'p':p,'s':s,'m':1,'target_modulus':M,'root_identity_zero':True,'
     target_identity_zero':True,'target_degree_bound':max(len(q)-1+r*(len(Y)-1)+(2*s-1-r)*(len(Z)-1) for r,q
      in enumerate(L)),'Y_degree':len(Y)-1,'Z_degree':len(Z)-1,'elapsed_seconds':round(time.time()-start,3)}
    payload={'report':rep,'Q':qs,'Y':Y,'Z':Z,'L':L}
    packed=json.dumps(payload,separators=(',',':'));rep['certificate_sha256']=hashlib.sha256(packed.encode()
     ).hexdigest()
    (BASE/f'direct_octa{dimension}_s{s}_certificate.json').write_text(packed+'\n')
    (BASE/f'direct_octa{dimension}_s{s}_report.json').write_text(json.dumps(rep,indent=2)+'\n')
    print(json.dumps({'stage':'target_passed',**rep}),flush=True)
    return rep

def test():
    for n,s,filename in [(4,2,'certificate_octa4_p3_s2_m1.json'),(3,3,'certificate_octa3_p3_s3.json')]:
        old=json.loads((OLD/filename).read_text());ins=sc.Inputs(s=s);jp=[1]
        for j in range(n-1):jp=sc.mul(jp,ins.J,ins.mod,ins.length)
        av=ins.vertices(jp)
        assert {str(k):a for k,a in av.items()}==old['vertex_polynomials'],('vertex',n,s)
        qs=equation(av,s);Y,Z=old['lift_y_numerator'],old['lift_y_denominator']
        assert cleared_eval(qs,Y,Z,3**(2*s-1))==[0],('root',n,s)
        y,z=root(qs,3,2*s-1)
        assert v.add(v.mul(Y,z,3**(2*s-1)),v.scale(v.mul(y,Z,3**(2*s-1)),-1,3**(2*s-1)),3**(2*s-1))==[0]
        print(json.dumps({'validated_against_existing_certificate':filename,'all_vertex_coefficients_equal':
     True,'root_rational_functions_equal':True}),flush=True)

if __name__=='__main__':
    pa=argparse.ArgumentParser();pa.add_argument('--s',type=int,default=5);pa.add_argument('--n',type=int,
     default=4);pa.add_argument('--test',action='store_true');a=pa.parse_args()
```

```
    if a.test:test()
    else:run(a.s,a.n)
```

J.4 uniform_level.py

Complete source; 107 lines.

```
#!/usr/bin/env python3
"""Exact all-dimension certificate for one fixed level, using the direct equation."""
from pathlib import Path
from math import comb,factorial
import json,time,hashlib,argparse
import numpy as np
import scaled_convolution as sc
import direct_certificate as dc
v=sc.v
BASE=Path(__file__).resolve().parent

def threshold(e):
    i=z=0
    while z<e:
        i+=1;z+=v.vp(i,3)
    return i

def padded(qs,width):
    a=np.zeros((len(qs),width),dtype=np.int32)
    for r,q in enumerate(qs):a[r,:len(q)]=q
    return a

def samples(s,force=False):
    start=time.time();ins=sc.Inputs(s=s);N=3**(2*s-2);k=2*s-1;M=3**(2*s);d=dc.degree_for(k);ni=threshold(2*k
    );jp=ins.J
    directory=BASE/f's{s}_samples';directory.mkdir(exist_ok=True)
    print(json.dumps({'stage':'sample_setup','s':s,'ninput':ni,'ntarget':threshold(2*s),'degree':d}),flush=
    True)
    for n in range(2,2+ni):
        path=directory/f'n{n}.npz'
        if path.exists() and not force:
            print(json.dumps({'stage':'sample_reused','n':n}),flush=True)
        else:
            av=ins.vertices(jp);qs=dc.equation(av,s)
            jnext=sc.mul(jp[:3**s],ins.J[:3**s],ins.mod,3**s)
            b=ins.composition(jnext,3**s,M);f=[comb(2*j,j)*b[j]%M for j in range(3**s)]
            arr=padded(qs,9*N-3)
            np.savez_compressed(path,Q=arr,A_N=np.array(av[N],dtype=np.int64)%M,A_3N=np.array(av[3*N],dtype=
    np.int64)%M,F=np.array(f,dtype=np.int64))
            print(json.dumps({'stage':'sample_saved','n':n,'seconds':round(time.time()-start,3)}),flush=True
    )
        if n<1+ni:jp=sc.mul(jp,ins.J,ins.mod,ins.length)
    return directory

def model(s):
    start=time.time();N=3**(2*s-2);k=2*s-1;M=3**(2*s);ymod=3**k;d=dc.degree_for(k);ni=threshold(2*k);nm=
    threshold(k);width=9*N-3
    directory=BASE/f's{s}_samples'
    rows=np.stack([np.load(directory/f'n{n}.npz')['Q'] for n in range(2,2+ni)])
    original=rows.copy();newton=np.zeros_like(rows);ordinary=np.zeros((nm,d+1,width),dtype=np.int64)
    checks=[]
    for r in range(ni):
        dr=rows[0];val=v.vp(factorial(r),3);need=min(k,val)
        bad=np.argwhere(dr%3**need)
        assert len(bad)==0,('Newton integrality',r,bad[:5].tolist())
        newton[r]=dr
        if r<nm:
            unit=factorial(r)//3**val;iv=pow(unit,-1,M)
            ordinary[r]=(dr.astype(np.int64)//3**val*iv)%M
        checks.append({'difference_order':r,'required_valuation':need,'passed':True})
        rows=(rows[1:]-rows[:-1])%ymod
    print(json.dumps({'stage':'ordinary_model_integrality_passed','degree':nm-1,'seconds':round(time.time()-
    start,3)}),flush=True)
    for i,n in enumerate(range(2,2+ni)):
```

```
        actual=np.zeros((d+1,width),dtype=np.int64);fall=1
        for r in range(nm):
            if r:fall=fall*(n-2-r+1)%M
            actual=(actual+fall*ordinary[r])%M
        assert np.array_equal(actual%ymod,original[i]),('model evaluation',n)
    output=BASE/f's{s}_ordinary_dimension_model.npz'
    np.savez_compressed(output,newton=newton,ordinary=ordinary)
    report={'s':s,'p':3,'input_dimensions':list(range(2,2+ni)),'input_dimension_count':ni,'model_degree':nm
     -1,'lift_degree':d,'newton_valuation_checks':checks,'all_input_evaluations_match':True,'sha256':hashlib.
     sha256(output.read_bytes()).hexdigest(),'elapsed_seconds':round(time.time()-start,3)}
    (BASE/f's{s}_model_report.json').write_text(json.dumps(report,indent=2)+'\n')
    print(json.dumps({'stage':'model_certified',**{key:report[key] for key in ['s','input_dimension_count','
     model_degree','sha256','elapsed_seconds']}}),flush=True)
    return report

def targets(s,dimensions=None,force=False):
    start=time.time();p=3;k=2*s-1;N=p**(2*s-2);M=p**(2*s);dimensions=dimensions or list(range(2,2+threshold
     (2*s)));reports=[]
    for n in dimensions:
        certificate_path=BASE/f'direct_octa{n}_s{s}_certificate.json';report_path=BASE/f'direct_octa{n}_s{s}
     _report.json'
        a=np.load(BASE/f's{s}_samples'/f'n{n}.npz');qs=[v.trim(row.tolist()) for row in a['Q']]
        if certificate_path.exists() and report_path.exists() and not force:
            payload=json.loads(certificate_path.read_text());assert qs==payload['Q'];rep=json.loads(
     report_path.read_text())
            reports.append(rep);print(json.dumps({'stage':'target_reused','n':n}),flush=True);continue
        Y,Z=dc.root(qs,p,k,True)
        av={N:a['A_N'].tolist(),3*N:a['A_3N'].tolist()};f=a['F'].tolist();L=dc.target_polynomials(av,f,s)
        result=dc.cleared_eval(L,v.scale(Y,p,M),Z,M)
        assert result==[0],('target',n,[(i,c) for i,c in enumerate(result) if c][:10])
        rep={'dimension':n,'p':p,'s':s,'m':1,'target_modulus':M,'root_identity_zero':True,'
     target_identity_zero':True,'target_degree_bound':max(len(q)-1+r*(len(Y)-1)+(2*s-1-r)*(len(Z)-1) for r,q
      in enumerate(L)),'Y_degree':len(Y)-1,'Z_degree':len(Z)-1}
        payload={'report':rep,'Q':qs,'Y':Y,'Z':Z,'L':L};packed=json.dumps(payload,separators=(',',':'))
        certificate_path.write_text(packed+'\n');rep['certificate_sha256']=hashlib.sha256(packed.encode()).
     hexdigest();report_path.write_text(json.dumps(rep,indent=2)+'\n')
        reports.append(rep)
        (BASE/f's{s}_target_reports.json').write_text(json.dumps(reports,indent=2)+'\n')
        print(json.dumps({'stage':'target_passed',**rep,'elapsed_seconds':round(time.time()-start,3)}),flush
     =True)
    return reports

def summary(s):
    mr=json.loads((BASE/f's{s}_model_report.json').read_text());reports=[]
    for n in range(2,2+threshold(2*s)):
        rep=json.loads((BASE/f'direct_octa{n}_s{s}_report.json').read_text());assert rep['root_identity_zero
     '] and rep['target_identity_zero'];reports.append(rep)
    sr={'theorem':f'For all octahedral dimensions n>=2, every coefficient of E_(3,{s},1) is divisible by
     3^{2*s}.','prime':3,'level':s,'multiplier':1,'target_modulus':3**(2*s),'work_modulus':3**(4*s-2),'
     input_dimension_count':threshold(4*s-2),'ordinary_model_degree':threshold(2*s-1)-1,'
     target_dimension_count':threshold(2*s),'lift_degree':dc.degree_for(2*s-1),'
     all_input_difference_integrality_checked':True,'all_polynomial_coefficients_checked':True,'
     power_series_cutoff':None,'dimension_model_report':mr,'target_reports':reports}
    (BASE/f'all_dimensions_s{s}_summary.json').write_text(json.dumps(sr,indent=2)+'\n')
    manifest={'level':s,'model':mr['sha256'],'certificates':[{'n':x['dimension'],'sha256':x['
     certificate_sha256']} for x in reports]}
    manifest['aggregate_sha256']=hashlib.sha256(json.dumps(manifest,sort_keys=True,separators=(',',':')).
     encode()).hexdigest()
    (BASE/f's{s}_manifest.json').write_text(json.dumps(manifest,indent=2)+'\n')
    print(json.dumps({key:sr[key] for key in ['theorem','input_dimension_count','ordinary_model_degree','
     target_dimension_count','all_polynomial_coefficients_checked','power_series_cutoff']},indent=2),flush=
     True)

if __name__=='__main__':
    pa=argparse.ArgumentParser();pa.add_argument('--s',type=int,default=5);pa.add_argument('--stage',choices
     =['samples','model','targets','summary','all'],default='all');pa.add_argument('--n',type=int,nargs='+');
     pa.add_argument('--recompute',action='store_true');a=pa.parse_args()
    if a.stage in ['samples','all']:samples(a.s,a.recompute)
    if a.stage in ['model','all']:model(a.s)
    if a.stage in ['targets','all']:targets(a.s,a.n,a.recompute)
    if a.stage in ['summary','all']:summary(a.s)
```

J.5 verify_saved_level5.py

Complete source; 80 lines.

```
"""Re-read and independently evaluate every archived level-five identity.

This verifier does not invoke the certificate generator, its root solver,
or its homogeneous Horner evaluator.  It builds powers and sums all terms.
Exact modular convolution is the tested packed-integer backend.
"""
from pathlib import Path
from math import comb
import sys, json, time
import numpy as np

ROOT = Path(__file__).resolve().parents[2]
DATA = ROOT / 'research/octa_level5'
sys.path.insert(0, str(DATA))
import scaled_convolution as sc

def trim(a):
    while len(a)>1 and not a[-1]:
        a.pop()
    return a

def add(a,b,m):
    z=[0]*max(len(a),len(b))
    for i,c in enumerate(a): z[i]=c
    for i,c in enumerate(b): z[i]=(z[i]+c)%m
    return trim(z)

def scale(a,c,m):
    return trim([(x*c)%m for x in a])

def powers(a,d,m):
    out=[[1]]
    for _ in range(d): out.append(sc.mul(out[-1],a,m))
    return out

def homogeneous(qs,y,z,m):
    d=len(qs)-1
    yp,zp=powers(y,d,m),powers(z,d,m)
    total=[0]
    for r,q in enumerate(qs):
        term=sc.mul(q,sc.mul(yp[r],zp[d-r],m),m)
        total=add(total,term,m)
    return total

def taylor(a,r,m):
    out=[0]*(3*(len(a)-1)+1)
    for j in range(r,len(a)):
        out[3*j]=int(a[j])*comb(j,r)%m
    return trim(out)

def main():
    started=time.time()
    results=[]
    N=6561; mod=59049
    for n in range(2,26):
        data=json.loads((DATA/f'direct_octa{n}_s5_certificate.json').read_text())
        sample=np.load(DATA/'s5_samples'/f'n{n}.npz')
        qs,y,z,L=data['Q'],data['Y'],data['Z'],data['L']
        assert z[0]%3
        assert qs==[trim(row.tolist()) for row in sample['Q']]
        assert qs[0][0]==0 and qs[1][0]==1
        assert all(all(c%3==0 for c in q) for q in qs[2:])
        assert homogeneous(qs,y,z,19683)==[0],('root',n)
        f=sample['F'].tolist(); small=f[:81]
        a=sample['A_N'].tolist(); ap=sample['A_3N'].tolist()
        rebuilt=[]
        for r in range(10):
            left=taylor(small,r,mod)
            if r: left=add(left,scale(taylor(small,r-1,mod),N,mod),mod)
            lr=add(sc.mul(ap,left,mod),scale(sc.mul(f,taylor(a,r,mod),mod),-1,mod),mod)
```

```
            rebuilt.append(lr)
        assert rebuilt==L,('target construction',n)
        assert homogeneous(L,scale(y,3,mod),z,mod)==[0],('target',n)
        results.append({'dimension':n,'root_verified':True,'target_reconstructed':True,'target_verified':
    True})
        print(json.dumps({'dimension':n,'verified':True,'elapsed_seconds':round(time.time()-started,3)}),
    flush=True)
    report={'method':'independent sum-of-powers evaluation of saved identities','dimensions':results,'
     all_24_passed':True,'elapsed_seconds':round(time.time()-started,3)}
    (Path(__file__).resolve().parent/'independent_level5_audit.json').write_text(json.dumps(report,indent=2)
     +'\n')
    print(json.dumps({'all_24_passed':True,'elapsed_seconds':report['elapsed_seconds']}),flush=True)

if __name__=='__main__': main()
```

J.6 summarize_level4_certificate.py

Complete source; 29 lines.

```
#!/usr/bin/env python3
"""Assemble deterministic identifiers for the already verified level-four data."""
from pathlib import Path
import json,hashlib
base=Path(__file__).resolve().parent

def sha(data):return hashlib.sha256(json.dumps(data,sort_keys=True,separators=(',',':')).encode()).hexdigest
    ()
model=json.loads((base/'newton_s4_model_checkpoint.json').read_text())
reports=[];certs=[]
for n in range(2,20):
    data=json.loads((base/f'compact_octa{n}_s4_certificate.json').read_text())
    report=data['report']
    assert report['dimension']==n and report['s']==4 and report['p']==3 and report['m']==1
    assert report['root_identity_zero'] and report['target_identity_zero']
    reports.append(report);certs.append({'dimension':n,'sha256':sha(data)})
assert model['input_dimension_count']==30 and model['model_degree']==17
assert model['fixed_denominator_exponent']==19
manifest={'prime':3,'level':4,'multiplier':1,'model_checkpoint_sha256':sha(model),'
    compact_certificate_sha256s':certs}
summary={'theorem':'For every octahedral dimension n>=2, every coefficient of E_(3,4,1) is divisible by 3^8.
    ',
 'prime':3,'level':4,'multiplier':1,'target_modulus':6561,'lift_y_modulus':2187,'work_modulus':4782969,
 'input_dimension_count':30,'input_dimension_representatives':[2,31],'ordinary_model_degree':17,
 'target_dimension_count':18,'target_dimension_representatives':[2,19],'lift_equation_degree':9,
 'universal_lagrange_denominator_exponent':19,'maximum_compact_target_degree':max(r['target_degree_bound']
    for r in reports),
 'all_input_difference_integrality_checked':True,'all_polynomial_coefficients_checked':True,
 'formal_power_series_cutoff':None,
 'universal_dimension_justification':'Ordinary dimension-polynomial model and fixed universal Lagrange
    numerator/denominator; compact roots agree by uniqueness; 18 consecutive target values suffice by
    factorial finite differences.',
 'certificate_sha256':sha(manifest),'manifest':manifest,'target_reports':reports}
(base/'all_dimensions_newton_s4_summary.json').write_text(json.dumps(summary,indent=2)+'\n')
print(json.dumps({k:v for k,v in summary.items() if k not in ['manifest','target_reports']},indent=2))
```

K Source-to-section preservation map

The letters P, B, E, T identify the original four source manuscripts: P, positive vertex interpolation; B, the revised base manuscript on Hecke traces; E, exceptional/elliptic and sharpness arguments; T, two-cusp descent and auxiliary Cartier methods. Each source mathematical section, including its unnumbered proofs, is retained at the destination below. Section numbers refer to this unified edition. Source introductions have been synchronized with the final theorem scope; bibliography entries are merged by work. Historical program listings and their recorded output are explicitly identified as such.

Source	Mathematical section	Here
P	The target and the resulting scope	2
P	Positive vertex interpolation: complete hypercubic and simplicial congruences	3
P	A matrix block argument for the hyperoctahedral family	4
P	Verification, sharpness, and scope	5
P App.	A complete coefficientwise certificate in hyperoctahedral dimension four	A
B	Modular trace methods and their conclusions	32
B	Modular families of cusp type	33
B	The Hecke trace	34
B	Proof of the Main Theorem	35
B	Proofs of the corollaries	36
B	Families with an elliptic zero	37
B	Zagier's case D	38
B	Limits of the modular method and an elliptic extension	39
B	Computational verification	40
B App.	The verification script	B
B App.	Output of the script	C
E	Statements, normalization, and scope	41
E	Elliptic zeros: a rational trace estimate	42
E	Zagier D at primes congruent to ± 1 modulo five	43
E	Exact coefficient obstructions to larger exponents	44
E	Relation to the cited literature	45
E	Exact verification and proof dependencies	46
T	Scope and exact statements	47
T	The complete Zagier D congruence	48
T	The first level in every dimension	49
T	A canonical negative-power basis in all three families	50
T	The three-dimensional hyperoctahedral family: a weighted Cartier reduction	51
T	Exact verification and the role of the auxiliary arguments	52

K.1 Explicit proof and statement preservation

The original source proof environments total $8 + 20 + 13 + 9 = 50$ in the order P, B, E, T. The three supplements contribute $9 + 6 + 13 = 28$ additional proof environments, so all 78 source proof environments are retained. The global combination and analytic-ring transfer in the introduction add their own proofs. Their mathematical bodies are preserved; local cross-reference and citation keys are renamed consistently. Theorem and lemma statements are retained with their assumptions. The low-dimensional modular corollary is worded explicitly to keep its $3s$ claim confined to the three-dimensional hypercubic and simplicial families. The elliptic extension remains open.

The following unnumbered arguments are also retained: the positive Cartier coefficient extraction and low-dimensional reductions; the entire independent four-dimensional certificate derivation; elliptic factorial-carry estimates, including the $108t$ boundary repair; the Zagier D residual and both cusp expansions; all three negative-vertex formulas and their completion proviso; the octahedral symmetric-square and Bol identities, telescoping endpoints, primitive integrality, and exact Domb transport. Thus the preservation check is not based only on counts of theorem or proof environments.

K.2 Editorial reconciliation

Statements that Zagier D at $p \equiv \pm 2 \pmod 5$, the three-dimensional hyperoctahedral case, or dimensions $n \geq 4$ were globally unresolved have been replaced by references to their complete proofs. Statements explaining why a particular auxiliary method alone is insufficient have been retained as method limitations. No ordinary-truncation theorem has been inferred merely from a weighted differential congruence, a negative-power basis congruence, or a finite numerical sample. The historical verification logs retain their original labels with an explicit status annotation.

The bibliography is consolidated, Beukers's modular-forms paper is cited under one key, and the long quotation from its introduction is paraphrased while preserving the mathematical question. Fu's work is retained as contextual literature, not a proof input. The original computational claims are described according to what the supplied programs actually check.

K.3 The three supplements and the complete-edition additions

SP denotes the all-odd-prime small-prime supplement, OP the arbitrary-dimensional hyperoctahedral supplement, and AP the all-level three-adic supplement. Every mathematical section is retained, including unnumbered arguments and failed auxiliary estimates that explain the need for the subsequent construction.

Source	Retained material	Here
SP	Scope and the all-odd-prime theorem	6
SP	Three-family orbit module and normalization at $p \mid n+1$	7
SP	Complete scalar cube and simplex descent	8
SP	Independent geometric first level	9
SP	Three exact entire-series certificates	10
SP	Sharp obstruction for the uncorrected period functional	11
SP	Interpolation obstruction and verification	12, 13
SP	Complete exact certificate program	E
OP	All-level $p \geq 5$ theorem and finite-level theorem	14
OP	Formal input and exact factorial blocks	15, 16
OP	Support-weighted symmetric reduction	17
OP	Mixed-precision proof when $3 \mid n$	18
OP	First level and finite polynomial framework	19, 20
OP	Dimension-uniform levels two and three	21
OP	Compact fourth-level Hensel certificates	22
OP	Exact-row obstruction and limits	23
OP	Four complete certificate programs	F–I
AP	All-level theorem and formal input	24, 25
AP	Positive vertices and exact tails	26
AP	Integral cutoff kernel and quadratic divisibility	27
AP	Universal weak descent and dimension Frobenius ideal	28
AP	All-level formal completion	29
AP	Independent dimension-uniform fifth-level certificate	30
AP	Verification and precise limits	31
New	Global combination and analytic-ring transfer	1
New	Complete fifth-level generator and independent evaluator	J

K.4 Reconciliation in this complete edition

The previous all-level exclusions for small primes are replaced by cross-references to their proofs. The restricted assumptions of an individual older proof remain unchanged. In particular, no $p > n$ hypothesis has simply been erased from the original symmetric reduction. The new support-weighted and dimension-polynomial arguments supply the extensions. The prime 2, general elliptic-zero extension, and unrestricted numerical specialization remain outside the global three-family theorem.

References

[1] A. O. L. Atkin and J. Lehner, *Hecke operators on* $\Gamma_0(m)$, Math. Ann. **185** (1970), 134–160.

[2] B. Alinquant and R. Osburn, *On sporadic sequences*, arXiv:2312.07134 (2023; revised 2024).

[3] G. Almkvist and W. Zudilin, *Differential equations, mirror maps and zeta values*, in: Mirror Symmetry V, AMS/IP Stud. Adv. Math. **38**, Amer. Math. Soc., 2006, 481–515.

[4] J. M. Borwein, P. B. Borwein and F. G. Garvan, *Some cubic modular identities of Ramanujan*, Trans. Amer. Math. Soc. **343** (1994), 35–47.

[5] F. Beukers and M. Vlasenko, *Dwork crystals I*, Int. Math. Res. Not. IMRN 2021, 8807–8844.

[6] F. Beukers and M. Vlasenko, *Dwork crystals II*, International Mathematics Research Notices **2021** (2021), 4427–4444. arXiv:1907.10390v3; doi:10.1093/imrn/rnaa120.

[7] F. Beukers and M. Vlasenko, *Dwork crystals III: from excellent Frobenius lifts towards supercongruences*, International Mathematics Research Notices **2023** (2023), 20433–20483. arXiv:2105.14841; doi:10.1093/imrn/rnad101. The theorem and numbering used here are those of version 3.

[8] F. Beukers, *Supercongruences using modular forms*, Algebra & Number Theory 20 (2026), 1727–1768. arXiv:2403.03301.

[9] F. Beukers, *Some congruences for the Apéry numbers*, J. Number Theory **21** (1985), 141–155.

[10] F. Beukers, *Irrationality proofs using modular forms*, Astérisque **147–148** (1987), 271–283.

[11] S. Cooper, *Ramanujan's Theta Functions*, Springer, 2017.

[12] M. J. Coster, *Supercongruences*, Ph.D. thesis, Universiteit Leiden, 1988.

[13] F. Diamond and J. Shurman, *A First Course in Modular Forms*, Grad. Texts in Math. **228**, Springer, 2005.

[14] W. Duke, *Continued fractions and modular functions*, Bull. Amer. Math. Soc. (N.S.) **42** (2005), 137–162.

[15] B. Dwork, *p-adic cycles*, Publ. Math. IHÉS **37** (1969), 27–115.

[16] Y. Fu, *p-adic rigidity and supercongruences at rank two attractors*, arXiv:2609.36045 (2026).

[17] A. Mellit and M. Vlasenko, *Dwork's congruences for the constant terms of powers of a Laurent polynomial*, Int. J. Number Theory **12** (2016), 313–321.

[18] R. Osburn and B. Sahu, *A supercongruence for generalized Domb numbers*, arXiv:1201.6195 (2012).

[19] R. Osburn, B. Sahu and A. Straub, *Supercongruences for sporadic sequences*, Proc. Edinb. Math. Soc. **59** (2016), 503–518.

[20] K. Ono, *The Web of Modularity: Arithmetic of the Coefficients of Modular Forms and q-series*, CBMS Regional Conf. Ser. Math. **102**, Amer. Math. Soc., 2004.

[21] A. Straub, *Gessel–Lucas congruences for sporadic sequences*, arXiv:2301.12248 (2023).

[22] M. Vlasenko, *Congruences and cohomology*, arXiv:2412.13313 (2024).

[23] D. Zagier, *Integral solutions of Apéry-like recurrence equations*, in: Groups and Symmetries, CRM Proc. Lecture Notes **47**, Amer. Math. Soc., 2009, 349–366.